Python 程序设计讲义

> 一份为「零基础大学生 · Python 公共课」写的完整讲义：九章正文 + 20 道课后作业详解 + 期末模拟卷 + 三份速查附录。

这份讲义是怎么来的

本讲义由课程讲义《02_基础知识》至《10_模块和包》九份 PDF 整合梳理而成，并把课后作业的 20 道题按知识点归到了对应章节的末尾。在此基础上，又参照了当前最主流的几套公开教学资源做了**精修**：

资源	特点	适合什么时候看
黑马程序员《Python 从入门到精通》	集数多、单点讲得极细，节奏慢	课上完全没听懂，需要从零逐行重学
尚硅谷《0 基础入门学习 Python》	节奏快一些，偏工程化，会顺带讲 IDE 和调试	想快点过一遍，顺便学怎么用工具
嵩天《Python 语言程序设计》（北理工 · 中国大学 MOOC）	国家级慕课，章节顺序和本课程最接近	想跟着一门课系统学
廖雪峰 Python 教程	极简图文，正文里大量"注意"框	当字典查，不适合通读
菜鸟教程 Python3	词条式速查，带在线运行	忘了某个方法怎么拼，30 秒查一下
Python 官方教程（中文版）	最权威、版本最新	想知道"官方到底怎么定义这个行为"

怎么配合用：本讲义负责"系统 + 考点 + 作业"；视频负责"听不懂的地方再听一遍"；速查站负责"忘记语法时随手查"。**不要**用视频替代动手敲代码。

本讲义怎么用

每个知识点都标了重要程度，看到标记就知道该花多少力气：

标记	含义	看到它该怎么做
【核心】	必须记住，后面每一章都要用	合上讲义能自己复述出来
【易错】	初学者最常踩的坑	停下来，亲手在电脑上验证一次
【注意】	一般性提醒	读一遍，心里有数即可
【了解】	知道有这回事即可	需要时回来查表
【速查】	表格型资料，不必背	做题时对照使用

另有两种常驻栏目：

- **【常见错误】** 用 `✗ 错误写法` / `✓ 正确写法` 并列对照，把新手最常踩的坑直接摊开写在你眼前。
- **本章方法总结与易错清单**（每章最后一节）——一章学完后的复习卡，含**知识地图、速查表、易错清单、必须能默写的代码片段**四块。

还有两条约定：

1. 每章的**例题**讲"怎么做"，编号形如"例题 3-2"。
2. 每章的**作业题**对应你截图里的 OJ 原题，题号与"作业题 N"一一对应；**题面数据一字未改**，答案统一放在全书最后（含思路、代码、测试数据、易错点）。

> **强烈建议**：读讲义时手边开着 Python，把每一个代码块都敲一遍。看懂和敲出来是两回事。

本次精修做了什么

在原有整合版的基础上，这一版做了四类改动。

一、删重复

同一个知识点只在一处详细讲，其它地方压缩成 2~3 行 + 交叉引用（写成"详见 4.5 节"这种形式）。典型的有：切片与不可变性（第 4 章内部三处重复）、闰年判断与素数判断（第 2 章例题 vs 第 3 章例题）、`split` / `join`（第 4 章 vs 第 7 章）、`math` 与 `round`（第 1 章 vs 第 6 章）。

二、增补

- **每章新增一节「本章方法总结与易错清单」**（位置在本章自测之前），固定包含四块：**知识地图 • 方法速查表 • 易错清单 • 必须能默写的代码片段**。考前只翻这几节就能过一遍全书。
- **新增 60 多处【常见错误】✗/✓ 对照**，把新手最容易踩的坑直接摊开写在对应知识点旁边，多数附真实报错原文。
- **新增数十道带完整输入输出的例题**，全部实机跑过，输出与讲义所写一致。
- **各章自测题扩充**（例如第 1 章 12 → 16 道、第 4 章 12 → 17 道、第 8 章 10 → 15 道），答案同步补齐。
- **作业题答案整节重写**：每道题给出「思路分析 → 参考代码 → 多组测试数据 → 易错点」，并附一张"这 20 道题到底在考什么"的总表。
- **期末模拟卷扩充到 36 题**，覆盖全书九章，并加了「失分自查表」。
- **新增三份附录**：A 常用速查表、B 30 个新手报错速查（报错 → 原因 → 处方）、C 在线判题输入输出模板。

三、纠错

- 20 道作业题的**全部样例与边界数据都重新实跑验证过**，答案按实际运行结果校对。
- 用脚本检查了全书每一段 Python 代码能否通过语法编译（故意示错的 ✗ 代码块除外）。

四、体例统一

- 新增 **【注意】** 标记；所有交叉引用都带章节号；代码里不出现中文标点。

- 表格统一为「方法 | 作用 | 是否原地修改 | 返回值 | 典型例子」这类有信息量的列。

> **【注意】** 因为新增了一节，各章的「本章自测」「本章作业题」的**节号顺延了 1 位**（例如第 1 章的自测由 1.15 变成 1.16）。**题目内容和题号没有任何变化**，作业题 1~20 的题干、输入输出格式与样例都逐字保留了原样。

目录

全书共九章，外加一套期末模拟卷、全部答案与三份附录。章节顺序与课程进度一致：

章	标题	对应讲义	本章作业题
第 1 章	基础知识	02_基础知识	1 ~ 2
第 2 章	流程控制	03_流程控制	3 ~ 14
第 3 章	函数	04_函数	15 ~ 16
第 4 章	内置组合数据类型	05_内置组合数据类型	17 ~ 20
第 5 章	输入输出与文件处理	06_输入输出和文件处理	—
第 6 章	数值处理与计算	07_数值处理与计算	—
第 7 章	字符串与文本处理	08_字符串和文本处理	—
第 8 章	面向对象程序设计基础	09_面向对象程序设计基础	—
第 9 章	模块与包	10_模块和包	—
期末模拟测试	—	—	—
答案与解析	各章自测题 + 20 道作业题 + 模拟卷	—	—
附录 A	Python 常用速查表	—	—
附录 B	30 个新手报错速查（报错 → 原因 → 处方）	—	—
附录 C	在线判题（OJ）输入输出模板	—	—

开始之前：给零基础同学的三条建议

1. **手要比眼快。**看懂了 ≠ 会写了。每段代码都亲手敲一遍，敲错了再改，改对了才算学会。
2. **报错是朋友，不是敌人。**看到红色报错先读**最后一行**，那里写着异常类型和原因；再看它给出的行号，问题通常就在那一行或上一行。附录 B 把最常见的 30 个报错整理成了对照表。
3. **每天 30 分钟，比周末 5 小时管用。**编程是肌肉记忆，连续小剂量练习的效果远好于集中突击。

第 1 章 基础知识

> 对应课程讲义：《02_基础知识》。这一章是全书的地基，后面每一章都建立在它之上。请务必把"赋值号不是等号""input() 拿到的是字符串""浮点数不精确"这三件事搞清楚。
>
> **本章怎么学**：先把 1.1、1.2、1.3、1.4、1.8 这五节看透（想清楚 → 读进来 → 存起来 → 算出来 → 送出去），其余小节可以先当资料查。例题编号接着老版往下排，所以题目出现的先后与编号不完全一致（比如 1.3 节里会先看到例题 1-9），这不影响阅读。

1.1 程序的骨架：IPO 模型

写程序之前，先想清楚三件事，这个思路叫 IPO 模型：

阶段	英文	做什么
输入	Input	程序从哪里拿到数据？键盘输入、文件、参数……
处理	Process	对数据做什么计算或判断？
输出	Output	结果以什么形式给用户看？

> **【核心】** 把代码按"输入 → 处理 → 输出"分成三段写，段与段之间空一行。这样条理清晰，出错时也容易定位是哪一段的问题。

做 OJ 题时 IPO 还有一层现实含义：先看"输入格式"决定怎么读，再看"输出格式"决定怎么写，最后才想怎么算。很多 WA 不是算错，而是这两步没看清。

1.2 第一个程序与输出 print()

```
print("Hello world!")
```

`print()` 的作用就是把括号里的内容显示到屏幕上。括号里带引号的那段内容叫 **字符串**：用一对引号包围起来的字符序列。

> **【易错】** 引号必须成对，且不能混用。一对引号可以是一对单引号 `'...'`，也可以是一对双引号 `"..."`，但不能左边单引号、右边双引号。

`print()` 可以一次输出多个对象，对象之间用逗号分隔：

```
print('hello', 'world', '!')          # hello world !
print('hello', 'world', '!', sep='*') # hello*world*!
print('hello', 'world', '!', sep='')  # helloworld!
```

> **【核心】** 逗号会自动加一个空格，加号不会。这是初学者最容易混淆的一对写法。

```
print("Hello", "world")   # Hello world    ← 逗号，自动加空格
print("Hello" + "world")  # Helloworld     ← 加号，直接连接，不加空格
```

sep 参数：控制多个输出对象之间的分隔符，默认是一个空格。

end 参数：控制本次输出以什么字符结尾，默认是换行符 `\n`。

```
print('hello', end=' ')
print('world', end='')
print(' ', end='!')
print()
print('New line')
# hello world !
# New line
```

> **【了解】** sep 与 end 的先后顺序没有要求，但都必须放在要显示的内容之后。单独一个 `print()`（括号里什么都没有）会输出一个空行。

输出"保留两位小数"的最小写法（OJ 最常用）

```
x = 3.14159
print(f"{x:.2f}")      # 3.14
```

字符串前加一个字母 `f`，大括号里写变量或表达式，冒号后面的 `.2f` 表示"定点格式，保留 2 位小数"。

> **【注意】** 平时记住这一句就够用：`print(f"{变量:.2f}")`。要保留 3 位就把 2 改成 3。宽度对齐、千分位、百分比等完整的格式说明符见 7.10 节，那里会系统讲。
>
> **【常见错误】** 中文输入法下的全角标点（，。：；（）），Python 一个都不认识。
>
> ```
> # X 错误写法
> print（"你好"）
> print("你好"，"世界")
> ```
>
> ```
> # √ 正确写法
> print("你好")
> print("你好", "世界")
> ```
>
> 报错是 `SyntaxError: invalid character '（' (U+FF08)`。只要看到报错指向一个你觉着"很正常"的符号，先检查输入法是不是还在中文状态。

1.3 输入 input()

```
name = input("What's your name? ")
print("Hello", name)
```

`input()` 的执行过程是：先给出提示信息，然后等待用户输入。输入的**信息永远是字符串类型**，保存在赋值号左边的变量中。

> **【核心】** input() 拿到的永远是字符串。如果输入的是数字、并且打算让它参与运算，必须先转换成数字类型：`int()` 转整数，`float()` 转浮点数，`eval()` 把整个字符串当表达式计算。

```
radius = float(input())   # 正确：转成浮点数后可以参与运算
radius = input()          # 危险：radius 是字符串，"5" 不能做乘法
```

> **【注意】** 提示语会真的输出到屏幕上。本地练习很方便，但交到 OJ 上就是多余的输出，会被判错——提交前把括号里清空，写成 `input()`。

一行读入多个数：map(int, input().split())（核心）

```
a, b = map(int, input().split())          # 一行两个整数
nums = list(map(int, input().split()))    # 一行 n 个整数，存成列表
total = sum(map(int, input().split()))    # 一行 n 个整数，只要和
```

拆开看这串东西在干什么：`input()` 读进一整行字符串 → `.split()` 按空白（空格、制表符）切成若干段 → `map(int, ...)` 把每一段都转成整数。

> **【核心】** 这是 OJ 题里出现频率最高的一句。左边的变量个数要和输入的数字个数一致，否则会报 `ValueError`（not enough values to unpack）。
>
> **【注意】** `list(...)` 把结果变成列表，列表在第 4 章讲，这里先照着写；有了它就能用 `len(nums)` 求个数。

输入形式	读入写法
一行一个整数	`n = int(input())`
一行一个实数	`x = float(input())`
一行两个整数（空格分隔）	`a, b = map(int, input().split())`
一行 n 个整数、要逐个用	`nums = list(map(int, input().split()))`
一行 n 个整数、只要总和	`total = sum(map(int, input().split()))`
一整行原样读进来	`s = input()`

> **【注意】** `int()` 和 `float()` 会自动忽略字符串两端的空白，所以 `int(" 5 ")` 得到 5；但 `int("5 5")` 会报错——一行里有两个数，必须先用 `split()` 拆开。

例题 1-9：一行读入多个数求总分与平均分

题目：第一行输入整数 n，第二行输入 n 个整数（空格分隔）。输出它们的总和，以及平均值（保留 2 位小数）。

```
n = int(input())
nums = list(map(int, input().split()))
total = sum(nums)

print(total)
print(f"{total / n:.2f}")
```

输入：

```
3
80 90 100
```

输出：

```
270
90.00
```

要点：① 一行多个数用 `map(int, input().split())`；② `sum()` 对一列数求和；③ 平均值一定用 `/` 而不是 `//`（用 `//` 会得到 90，OJ 直接判错）；④ 保留 2 位小数用 `f"{...:.2f}"`。

【常见错误】 以为 `input()` 拿到的是数字，直接拿来算。

```
# ✗ 错误写法
a = input()
b = input()
print(a + b)          # 输入 3 和 5，输出 35
```

```
# ✓ 正确写法
a = int(input())
b = int(input())
print(a + b)          # 输入 3 和 5，输出 8
```

`input()` 返回的永远是字符串，两个字符串之间的 `+` 是"连接"不是加法。顺带记住：`'10' < '9'` 是 True，因为字符串是从左往右逐个字符比大小的。

1.4 变量与赋值

变量是计算机语言中能储存计算结果或能表示值的抽象概念，可以通过变量名访问。一个很贴切的比喻：**变量名像是门牌号码**，它是引用而不是存储本身。

创建变量的语句叫赋值语句：

```
变量名 = 变量值
```

其中 `=` 叫做 **赋值运算符**。

【易错•全章最重要】 `=` 不是数学中的等号。

```
x = 10
x = x + 2
```

从数学上看 `x = x + 2` 永远不成立，但在程序里完全合法，执行过程分两步：

1. 先计算右侧的表达式 `x + 2`，得到 `12`；
2. 再把 `12` 赋给变量 `x`，原来的 `10` 被覆盖。

也正因为 `=` 是"把右边放进左边"，赋值号的**左边只能是变量**：

```
1 = x            # 错误：左边是数字，不是变量
2 * x = 3 + y    # 错误：左边是表达式，不是变量
```

几种赋值写法

连续赋值：从右往左依次赋值。

```
i = j = k = 1     # 等价于 k = 1; j = k; i = j
```

同时赋值：右侧先全部算完，再一起赋给左边。

```
x, y = 2, 3
x, y = x + y, x * y   # 此时 x=5, y=6
```

> **【易错】** 同时赋值两边的个数必须一致：`a, b = 1, 2, 3` 报 `ValueError`（too many values to unpack），`a, b, c = 1, 2` 同样报错。一行读入多个整数时最容易撞上这个坑。

交换两个变量的值：常规写法要借助临时变量。

```
x, y = 2, 3
t = x
x = y
y = t
```

因为 Python 支持同时赋值，交换可以简写成一行，这是 Python 很优雅的地方：

```
x, y = y, x
```

> **【核心】** `x, y = y, x` 写排序、写交换时会反复用到，必须条件反射般写出来。它的原理正是上面那句"右侧先全部算完，再一起赋给左边"。
>
> **【常见错误】** `=` 与 `==` 混淆。
>
> ```
> # X 错误写法：这一行只是一个比较，什么也没发生
> x = 1
> x == 2
> print(x)        # 1
> ```

```
# ✓ 正确写法
x = 1
x = 2
print(x)        # 2
```

`=` 是赋值（把右边放进左边），`==` 是比较（两边相等吗）。条件里写成 `if x = 1:` 会直接报 `SyntaxError: invalid syntax`（if 在第 2 章讲），到时候第一时间想到这对符号。

1.5 变量命名规范

标识符：作为变量、函数等名字的有效字符串集合。

关键字（保留字）：Python 语言保留使用的标识符，不能用于其它用途。

```
import keyword
print(keyword.kwlist)
```

输出包含 `False, None, True, and, as, assert, async, await, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield`。

> **【了解】** 不必背这个列表。真拿不准某个名字能不能用，敲一遍看报不报错就行。

三条硬性规则（必须遵守）：

1. 变量名只能含有数字、字母以及下划线；
2. 变量名不能以数字开头；
3. 不能使用关键字作变量名。

> **【注意】** Python **区分大小写**：`Score`、`score`、`SCORE` 是三个互不相干的变量。命名风格不统一时，最容易出现"明明定义过却说没定义"的 `NameError`。

命名风格：

风格	示例	说明
下划线命名法	`high_score`	Python 官方推荐
小驼峰命名法	`highScore`	可以用（多受 Java 影响）
大驼峰命名法	`HighScore`	一般不用于变量名

八条命名建议：

1. 选择具有描述性的名字，做到望名识义，用 `score` 而不是 `s`；
2. 风格保持一致，两种风格都可以，但不要在一个程序里混用；
3. 大驼峰一般不得用于变量命名（它另有用途，如类名或异常名）；
4. 一般不得使用全大写字母做变量名，如 `NUMBER_OF_THE_RESULT`，这种写法一般代表常量；

5. 不用下划线开头或结尾（这类变量往往有特殊用途）；
6. 变量名不要太长，最好在 15 个字符以内；
7. 除特殊情况外（如循环控制变量习惯用 `i`），一般不用单字母作变量名；
8. 一般用英文单词命名。

> **【易错】拼音与汉字的三个禁区**：汉语环境里很难对应翻译成英文的，可以用拼音，但不允许英拼混用（错误示例：`RunYear` 代表闰年）；更不允许使用拼音首字母缩写（错误示例：`ncwssmys`）；不允许在变量名中出现汉字。

1.6 数据类型总览

计算机处理的数据是要区分类型的，类型决定了这份数据能做什么运算。

类别	包含的类型
基本数据类型	整数 int、浮点数 float、复数 complex、布尔型 bool
内置容器类型	字符串 str、元组 tuple、列表 list、字典 dict、集合 set
自定义类型	自定义类、第三方类等

本章只用到 int、float、bool、str 四种，容器类型从第 4 章开始讲。

> **【速查】** 拿不准一个变量是什么类型时，用 `type()` 打印出来，这比盯着代码猜快得多。

```
print(type(3))       # <class 'int'>
print(type(3.0))     # <class 'float'>
print(type("3"))     # <class 'str'>
print(type(True))    # <class 'bool'>
```

运算符与表达式

运算符用于对操作数进行运算。按操作数个数分：

- **二元运算符**：大部分运算符属于这一类。`a + b` 中 `+` 的操作数是 a 和 b，二元运算符一般位于两个操作数之间。
- **一元运算符**：如 `-a`，只有 a 一个操作数。若 a 是 5，则 `-a` 是 -5；若 a 是 -3，则 `-a` 是 3。

表达式一般由运算符和操作数组成。`1 + 2` 中 `+` 是二元运算符，1 和 2 是操作数。

> **【核心】** 仅操作数也能构成表达式，比如单独一个 `1` 本身就是合法表达式。

1.7 整数、浮点数与进制

类型	说明	举例
整数 int	没有小数部分的数字，理论上可以处理任意大小的整数	1, 27, -100, 0

类型	说明	举例
浮点数 float	带有小数点的数字，并不等价于数学中的小数	2.376, -99.1, 1.0
复数 complex	形如 a+bj 的数字	2+3j

> **【核心】混合运算的类型规则**：整数与整数运算结果还是整数；只要有一个浮点数参与，结果就是浮点数。`1 + 2` 得 3，`1 + 2.0` 得 3.0，`4 / 2` 得 2.0。想让结果变成整数，得自己用 `int()` 或 `//` 处理。

进制（了解即可）：人类习惯十进制，计算机天然二进制，为方便阅读又产生八进制与十六进制，其中十六进制借用 A~F 代表 10~15。

进制	前缀	转换函数	示例
十进制	无	`int()`	`int(0x61)` → 97
二进制	`0b`	`bin()`	`bin(97)` → `0b1100001`
八进制	`0o`	`oct()`	`oct(97)` → `0o141`
十六进制	`0x`	`hex()`	`hex(97)` → `0x61`

> **【了解】** 进制这一块只要记住"前缀 + 函数名"就够了。把整数按其它进制**格式化输出**（例如 `f"{255:#x}"` 得到 `0xff`）属于格式化内容，见 7.10 节。

浮点数的两个限制

> **【核心】** 浮点数有范围限制，也有精度限制。

- **大小有限制**：绝对值最大的浮点数是 `1.7976931348623157e+308`，绝对值最小的是 `2.2250738585072014e-308`。
- **精度有限制**：浮点数只有 15 位有效数字，超长的部分没有意义。

科学计数法记作 `aen` 或 `aEn`，a 是尾数，n 是指数，e 是固定的分隔字符，含义是 a 乘以 10 的 n 次方。之所以叫浮点数，是因为按科学记数法表示时小数点位置可变：1.23×10^9 和 12.3×10^8 完全相等。很大或很小的数必须用科学计数法：`1.23*10**9` 写成 `1.23e9`，`0.000012` 写成 `1.2e-5`。

著名问题：0.1 + 0.2 不等于 0.3

```
print(0.1 + 0.2)   # 0.30000000000000004
```

> **【易错】** 这不是 Python 出错了，是大多数计算机程序都会得到的结果。根源是浮点数在计算机中以二进制小数保存。整数运算永远是精确的，而浮点数运算则可能会有误差。

对于两个浮点数，直接比较是否相等是很容易出问题的，正确做法是比较它们的差是否足够小：

```
a = 0.1 + 0.2
b = 0.3
eps = 0.000001
```

```
print(a == b)             # False，a 等于 b 吗？
print(abs(a - b) < eps)   # True，a 近似等于 b 吗？
```

【核心】两种解决办法：一是尽量使用整数，例如处理与金钱相关的数值时，以"分"为单位；二是使用 `decimal` 模块。

```
from decimal import Decimal
a = Decimal('0.1')
b = Decimal('0.2')
print(a + b)   # 0.3
```

注意：`Decimal()` 的参数要写成字符串 `'0.1'`，不能写数字 `0.1`。

【常见错误】 用 `==` 直接比较两个浮点数。

```
# ✗ 错误写法
a = 0.1 + 0.2
print(a == 0.3)           # False
```

```
# ✓ 正确写法：比较两个数的差是否足够小
a = 0.1 + 0.2
print(abs(a - 0.3) < 1e-9)  # True
```

0.1、0.2、0.3 这类小数在二进制里都是无限循环小数，存进去就已经被截断了。判断浮点数"相等"永远用差值，涉及金额则改用整数（以分为单位）或 `decimal`。

1.8 算术运算

用于数字的算术运算符与数学中基本一致，只有除法分成了三种，这是本章的重点。顺带记住两个记法差异：Python 不认识 `×` 和 `÷`，乘法用 `*`，除法用 `/`。

算术运算符总表（速查）

运算符	描述	结果类型	举例
`+`	一元加，得到操作数本身	同操作数	a=-10 时 `+a` 仍是 -10
`-`	一元减，变换操作数的符号	同操作数	a=-10 时 `-a` 是 10
`+`	二元加	int 或 float	`10 + 20` 结果 30
`-`	二元减	int 或 float	`10 - 20` 结果 -10
`*`	乘法	int 或 float	`10 * 20` 结果 200
`/`	算术除法 truediv，**结果总是浮点数**	一定是 float	`15 / 2` 结果 7.5
`//`	地板除法 floordiv，**向下取整**	int 或 float	`10 // 3` 结果 3
`%`	取模（取余），返回除法的余数	int 或 float	`10 % 3` 结果 1

运算符	描述	结果类型	举例
`**`	乘方（幂运算）	int 或 float	`2 ** 3` 结果 8

优先级

与数学中"先乘除、后加减"一样，从高到低为：幂运算 `**` → 一元 `+` `-` → 乘除 `* / // %` → 加减 `+ -`。完整的优先级总表见 1.13 节。

> **【易错】** 调整运算优先级时永远使用小括号，无论有多少级。中括号和大括号在 Python 中另有用途（列表、字典等），不能用来分组运算。

三种除法的区别（核心）

```
print(5 / 2)     # 2.5
print(6 / 2)     # 3.0    ← 结果本应是整数，也要写成浮点形式
print(10 / 3)    # 3.3333333333333335

print(10 // 3)   # 3
print(9 // 3)    # 3
print(-9 // 3)   # -3
print(-10 // 3)  # -4     ← 不是 -3!
print(10 // 3.5) # 2.0    ← 有浮点数参与，结果是浮点数
```

地板数：指不大于此数的最大整数。注意它和"截断"完全不同。

```
import math
print(math.floor(-3.5))  # -4  ← 不是 -3!
print(math.floor(3.5))   # 3
```

取余运算 %（核心：负数很反直觉）

核心公式：设 `x // y = z`，则 `x % y = x - y * z`。

```
print(10 % 3)      # 1
print(-10 % 3)     # 2   ← 不是 -1!
print(-10 % -3)    # -1
print(-10 % -3.5)  # -3.0
```

验算 `-10 % 3`：因为 `-10 // 3 = -4`，代入公式得 `-10 - 3 * (-4) = -10 + 12 = 2`，与输出一致。

> **【核心】** Python 的 `%` 结果符号跟**除数**一致。取余最重要的用途是判断整除与处理循环问题：`n % 2 == 0` 判断偶数，`n % 7` 处理"星期几"这类问题。

负数下的 // 与 %（本章最难、最容易考）

表达式	结果	说明
`7 // 2`	3	普通情况

表达式	结果	说明
`-7 // 2`	-4	向下取整：-3.5 的地板数是 -4，**不是 -3**
`7 // -2`	-4	商的符号由两数符号决定，取整方向不变
`-7 // -2`	3	3.5 的地板数是 3
`7 % 2`	1	符号跟除数（+2）
`-7 % 2`	1	符号跟除数（+2），所以是正数
`7 % -2`	-1	符号跟除数（-2），所以是负数
`-7 % -2`	-1	同上

> **【核心】三句话记住**：① `//` 是"向下取整"（往数轴左边走），不是"向零截断"；② `%` 的结果符号与**除数**一致；③ 一切都能用恒等式 `x == (x // y) * y + (x % y)` 验算。取余常用于把结果限制在 `0 ~ y-1`（如"星期几""第几个"）。
>
> **【注意】** C、Java 的整除是"向零截断"，同样的式子 `-7 / 2` 在那边得 -3。跨语言时不要凭直觉搬。

```
print(divmod(7, 2))    # (3, 1)    一次拿到商和余数
print(divmod(-7, 2))   # (-4, 1)   等价于 (-7 // 2, -7 % 2)
```

> **【速查】** `divmod(a, b)` 等价于 `(a // b, a % b)`，处理"几分几秒""几时几分"时很好用，元组在第 4 章讲。

例题 1-3：求 n 天后是周几

题目：已知今天是周四，求 n 天后是周几？用 1 代表周一，2 代表周二，……，6 代表周六，0 代表周日。

```
n = int(input())
print((4 + n) % 7)
```

验证：n=1 时 `(4+1)%7 = 5`，是周五；n=3 时 `(4+3)%7 = 0`，是周日。

例题 1-4：反转输出一个四位整数

题目：输入一个四位（1000~9999 之间）整数，反转各位数字后显示。反转后如果最高位为 0，不需要显示。

```
n = eval(input())
a = n // 1000        # 千位
b = n // 100 % 10    # 百位
c = n // 10 % 10     # 十位
d = n % 10           # 个位
m = d * 1000 + c * 100 + b * 10 + a
print(m)
```

输入	输出
3125	5213
8670	768

> **【核心】拆分位数的固定套路**：要取从右往左第 k 位，用 `n // 10**(k-1) % 10`，例如百位是 `n // 100 % 10`；要去掉最低位就写 `n // 10`。这个套路在进制转换、数字统计类题目里反复出现。

至于 8670 为什么输出 768 而不是 0768：因为 `0 * 1000` 就是 0，数字 768 本来就是 768，print 出来自然没有前导零。

> **【常见错误】** 该输出整数的地方用了 `/`。
>
> ```
> # ✗ 错误写法
> print(6 / 2)       # 3.0，结果是整数也带小数点，OJ 会判错
> print(7 / 2)       # 3.5，想要整数商却得到小数
> ```
>
> ```
> # ✓ 正确写法
> print(6 // 2)      # 3
> print(7 // 2)      # 3
> print(-7 // 2)     # -4，注意是向下取整，不是 -3
> ```
>
> 一句话选运算符：要小数用 `/`，要整数商用 `//`，要余数用 `%`。`/` 的结果**一定是浮点数**，这正是"本地运行没问题、交上去就错"的高频原因。

例题 1-10：负数也要算对（求商与余数）

题目：输入一个整数 n（可正可负），输出 n 除以 7 的商和余数，要求余数始终满足 0 ≤ 余数 < 7（先输出商，再输出余数，各占一行）。

```
n = int(input())
print(n // 7)
print(n % 7)
```

输入 `-10`，输出：

```
-2
4
```

验算：`-10 // 7 = -2`（-1.428… 向下取整得到 -2），`-10 % 7 = -10 - 7 × (-2) = 4`。再代入恒等式：`(-2) * 7 + 4 = -10`，正确。

> **要点**：Python 的 `//` 是"向下取整"、`%` 的符号跟除数，所以除数为正 7 时，余数一定落在 0~6，被除数是负数也不例外。如果按"向零截断"的直觉，会算成商 -1、余数 -3，那就错了。输入 `10` 时应输出 `1` 和 `3`。

1.9 常用内置函数与 math 库

内置数学函数（速查）

函数	描述	举例
`abs(x)`	返回 x 的绝对值，类型不变	`abs(-2)` = 2
`max(x1,x2,...)`	返回最大值	`max(1,5,2)` = 5
`min(x1,x2,...)`	返回最小值	`min(1,5,2)` = 1
`pow(a,b)`	返回 a 的 b 次方，等价于 `a**b`	`pow(2,3)` = 8
`round(x)`	返回与 x 最接近的整数；正好居中时返回**偶数**	`round(4.5)` = 4
`round(x,n)`	保留小数点后 n 位小数，返回 float	`round(5.466,2)` = 5.47
`sum(xs)`	对一列数求和	`sum([1,2,3])` = 6

> **【注意】** `abs()` 保持原类型（`abs(-2)` 得整数 2），而 `math.fabs()` 一律返回浮点数（得 2.0）。OJ 上要求输出整数时，这两个不能随便换。

round() 的"银行家舍入"（易错，这里讲透）

```
round(0.5)   # 0    ← 0 是偶数
round(1.5)   # 2
round(2.5)   # 2    ← 不是 3!
round(3.5)   # 4
round(-0.5)  # 0
round(-1.5)  # -2
round(-2.5)  # -2   ← 不是 -3!
round(-3.5)  # -4
```

这套规则叫"四舍六入五取偶"：不是正好 .5 时按常规四舍五入；正好是 .5 时，向**偶数**那边靠。这样设计是为了让大量数据求和时向上舍与向下舍的次数大致相当，误差不会单向累积（IEEE 754 标准就规定用这种舍入方式）。

对比 `int(x)` 是**截断**（向零取整），不是四舍五入：`int(4.5)` = 4，`int(-3.5)` = -3。

> **【核心】想要数学上的四舍五入，这样写**：x ≥ 0 时 `int(x + 0.5)`；x < 0 时 `int(x - 0.5)`。合并成一个表达式：`int(x + 0.5) if x >= 0 else int(x - 0.5)`。

`round(x, n)` 的 n 还可以是负数，代表向小数点**前**保留 -n 位：

```
round(1234.5678, 2)    # 1234.57
round(1234.5678, -2)   # 1200.0
```

> **【核心】返回值类型不一样**：`round(x)` 返回 `int`，`round(x, n)` 返回 `float`。所以 `round(1234.5678, -2)` 是 `1200.0` 而不是 `1200`。

> **【易错】`round(x, n)` 在 n > 0 时并不总是可靠**：`round(2.675, 2)` 得到 `2.67`。因为 2.675 存进计算机后实际是 2.67499999...，它比 2.675 小，怎么舍都到不了 2.68；`round(1.335, 2)` 得到 `1.33` 是同样的原因。

涉及金额等对精度敏感的场景，用 1.7 节讲过的 `decimal`，并显式指定"四舍五入"：

```
from decimal import Decimal, ROUND_HALF_UP
print(Decimal("2.675").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))   # 2.68
```

> **【注意】** `f"{x:.2f}"` 遵循的是同一套舍入规则（`f"{2.5:.0f}"` 得到 `'2'`），也一样躲不开二进制误差。OJ 题的数据极少卡在"正好一半"上，所以放心用 `f"{x:.2f}"`；只有做金额计算才需要动用 `Decimal`。

math 库常用函数与常量（速查）

使用前需要先 `import math`。

函数/常量	描述	举例
`math.pi`	圆周率常数	3.141592653589793
`math.e`	自然对数的底数	2.718281828459045
`math.fabs(x)`	把 x 看作浮点数，返回绝对值	`math.fabs(-2)` = 2.0
`math.ceil(x)`	向上取最近的整数（不小于 x 的最小整数）	`math.ceil(-2.1)` = -2
`math.floor(x)`	向下取最近的整数（不大于 x 的最大整数）	`math.floor(-2.1)` = -3
`math.exp(x)`	e 的 x 次方	`math.exp(1)` = 2.718281828459045
`math.log(x)`	x 的自然对数	`math.log(math.e)` = 1.0
`math.log(x, base)`	x 以 base 为底的对数	`math.log(100,10)` = 2.0
`math.sqrt(x)`	x 的平方根	`math.sqrt(4)` = 2.0
`math.hypot(x, y)`	直角三角形斜边长，即 $\sqrt{(x^2+y^2)}$	`math.hypot(3,4)` = 5.0
`math.sin/cos/tan(x)`	三角函数，**x 为弧度**	`math.sin(math.pi/2)` = 1.0
`math.asin/acos/atan(x)`	反三角函数，**返回弧度**	`math.acos(1.0)` = 0.0
`math.degrees(x)`	弧度转角度	`math.degrees(math.pi)` = 180.0
`math.radians(x)`	角度转弧度	`math.radians(90)` = 1.5707963267948966

> **【了解】** 三角函数的参数是弧度不是角度。手上的角度要先 `math.radians(角度)` 转换；反过来 asin/acos/atan 返回的也是弧度，需要 `math.degrees()` 转回角度。漏掉这一步是最常见的错误。

> 【注意】 没有 `math.round()`。取整四件套 `int` / `floor` / `ceil` / `round` 的对比表见 1.10 节。

1.10 类型转换

数字之间互转

转换	行为	举例
浮点 → 整数	`int(x)` 直接把小数部分截断丢弃	`int(-3.5)` = -3
整数 → 浮点	`float(x)` 在 x 后面补上 `.0`	`float(4)` = 4.0

五个"取整"函数的区别（核心易错）

写法	行为	4.5	-3.5
`int(x)`	把小数部分直接截断丢弃	4	-3
`math.floor(x)`	向下取最近的整数	4	-4
`math.ceil(x)`	向上取最近的整数	5	-3
`round(x)`	与 x 最接近的整数；正好居中时取偶数	4	-4
`int(x+0.5)` / `int(x-0.5)`	数学意义上的四舍五入	5	-4

> 【核心】**一句话记住**：x 为正数时 int 和 floor 一样；x 为负数时 int 和 ceil 一样。int 是"向零截断"，floor 是"向下"，ceil 是"向上"，三者只有在 x 为负数时才分道扬镳。

字符串与数字互转（实践中极其常用）

函数	行为	举例与注意
`int(x)`	把形为整数的字符串转为整数	`int("45")` = 45；`int("4.5")` **会报错**
`float(x)`	把形为浮点数或整数的字符串转为浮点数	`float("45.6")` = 45.6
`eval(x)`	把字符串 x 视为表达式计算其值	`eval("1+2")` = 3
`str(x)`	把整数或浮点数转为字符串	字符串与数字连接时必须用它

> 【易错】`int("4.5")` **会报错**，这几乎每个初学者都会撞一次。想保留小数就用 `float("4.5")`；想取整要先转换再截断：`int(float("4.5"))`。
>
> 【注意】`int()` 与 `float()` 会自动忽略两端的空白：`int(" 5 ")` = 5。但字符串中间有空格（`"5 5"`）照样报错。

```
a = 2
b = 3
s = "The result of " + str(a) + "+" + str(b) + " is " + str(a + b)
print(s)   # The result of 2+3 is 5
```

【易错】区分数字与数字字符：数字类型可以进行数学运算，`1 + 2 = 3`；而 `'1'`、`'2'` 本质上也是字符，只能用来组成字符串，不能进行数学运算。`print('1' + '2')` 得到的是 `12`，这是字符串"连接"，不是加法。

【常见错误】 用 `int()` 直接转换带小数点的输入。

```
# ✗ 错误写法
n = int(input())      # 输入 3.5
# ValueError: invalid literal for int() with base 10: '3.5'
```

```
# ✓ 正确写法
n = float(input())          # 需要小数：得到 3.5
n = int(float(input()))     # 需要整数：得到 3（截断，不是四舍五入）
```

`int()` 只认"长得像整数"的字符串，`"3.5"` 这种要先 `float()` 再 `int()`。做题前先确认输入格式里写的是"整数"还是"实数"，再决定用哪个。

1.11 字符类型与编码

计算机只认识 0 和 1，要把字符编码之后计算机才能使用。最早只能处理英文的编码字符集叫 **ASCII**，共收录 128 个字符。Python 3 缺省使用 **UTF-8** 编码，ASCII 中的字符在 ASCII 与 UTF-8 中编码保持一致。UTF-8 让 Python 很容易处理包括中文在内的多国语言文字。

编码	字符	编码	字符	编码	字符
48	0	65	A	97	a
49	1	66	B	98	b
50	2	67	C	99	c
……	……	……	……	……	……
55	7	88	X	120	x
56	8	89	Y	121	y
57	9	90	Z	122	z

【核心】三个有用的结论：① `ord(char)` 得到字符的编码，`chr(number)` 得到编码为 number 的字符；② 字符比较大小的依据是它的编码；③ 大写字母的编码比小写字母小（A=65，a=97），所以 `'A' < 'a'` 是 True。

```
print(ord('a'))       # 97
print(ord('中'))      # 20013
print(chr(97))        # 'a'
print(chr(22269))     # '国'
```

【注意】 数字字符 `'0'` 的编码是 48，所以"数字字符 → 数字"还可以写成 `ord(ch) - ord('0')`；反过来 `chr(ord('a') + 1)` 就是 `'b'`。字母表内的运算（大小写转换、第几个字母）都靠编码差值，见 7.3 节例题 7-1。

【了解】 Python 没有独立的字符类型，它并不区分字符和字符串，即使只是一个字符，也被当作字符串处理。

1.12 比较运算与逻辑运算

比较运算

比较运算的结果为布尔值，只有 True 和 False 两个值。

运算符	描述	举例
`==`	等于	`10 == 20` 为 False
`!=`	不等于	`10 != 20` 为 True
`<`	小于	`10 < 20` 为 True
`<=`	小于等于	`10 <= 20` 为 True
`>`	大于	`10 > 20` 为 False
`>=`	大于等于	`10 >= 20` 为 False

【易错】 有两个符号的运算符（`==`、`!=`、`<=`、`>=`）是一个整体，中间不可插入空格。`==` 相当于数学上的等于号，而 `=` 是赋值号，二者完全不同。

链式比较（Python 的特色，区间判断必用）

```
x = 50
print(0 <= x < 100)          # True   ← Python 允许这样连写
print(0 <= x and x < 100)    # True   ← 完全等价的写法
```

【核心】 `0 <= x < 100` 叫链式比较，等价于 `0 <= x and x < 100`，但 x 只被求值一次。判断"在不在某个区间里"时优先用链式写法，读起来和数学一样。

【注意】 端点开闭差一个数：`0 <= x <= 100` 和 `0 <= x < 100` 不是一回事，这类"边界差一"是 OJ 上最常见的 WA 原因之一。写条件时把边界值代进去念一遍。

【注意】 C、Java 里写 `0 <= x < 100` 会被算成 `(0 <= x) < 100`，结果恒为真。这是 Python 与其它语言少见的差异，别把两边的习惯混着用。

例题 1-11：判断时刻是否合法（链式比较）

题目：输入两个整数 h 和 m（空格分隔），判断 h 时 m 分是不是一个合法的 24 小时制时刻（小时在 0~23 之间，分钟在 0~59 之间）。合法输出 True，否则输出 False。

```
h, m = map(int, input().split())
print(0 <= h <= 23 and 0 <= m <= 59)
```

输入 `23 59`，输出：

```
True
```

输入 `24 0`，输出：

```
False
```

要点：① 链式比较把"两个边界"写在一个式子里，比 `h >= 0 and h <= 23` 更接近数学写法；② 一行读两个整数用 `map(int, input().split())`；③ 直接用 `print()` 打印布尔表达式，输出刚好是 `True` / `False`（首字母大写），OJ 上很常用。

逻辑值

其它类型的数据参与逻辑运算时，按下面的规则折算：

结果	哪些值
视同 False	数字 0、0.0、0j；空字符串、空元组、空列表、空字典、空集合；None
视同 True	其它数字；上述数据类型的非空值

```
print(bool(0))        # False
print(bool(1))        # True
print(bool(''))       # False  ← 空字符串
print(bool('False'))  # True   ← 注意!
```

【易错】`bool('False')` 是 True，这是很经典的一个坑。`'False'` 是一个非空字符串，非空即 True，跟字符串内容是不是"False"毫无关系。要判断内容得写 `s == 'False'`。

【了解】 布尔值在需要时会当成 1 和 0 参与运算：`True + True` 得 2，`True * 5` 得 5。第 2 章统计"满足条件的个数"时会用到这个特性。

三种逻辑运算（核心）

运算优先级：`not` > `and` > `or`。

运算	名称	真值表	结果
`not A`	逻辑非（一元）	True→False，False→True	只有 True/False
`A and B`	逻辑与	两个都为 True 才是 True	返回"最早确定表达式值的操作数"
`A or B`	逻辑或	两个都为 False 才是 False	返回"最早确定表达式值的操作数"

```
print(not 1)          # False，1 视同 True，取反为 False
print(not '')         # True
```

```
print(6 and 'lucky')  # 'lucky'，6 为真，返回第二个
print(0 and '')       # 0，0 为假，直接返回第一个

print(6 or 'lucky')   # 6，6 为真，返回第一个
print(0 or '')        # ''，0 为假，返回第二个
```

【核心】and 和 or 有"短路求值"：and 左边为假时右边不再计算，or 左边为真时右边不再计算。这带来两个实用写法——`name = user_input or "匿名"` 用作默认值；`n != 0 and 10/n > 1` 用作前置检查，正因为短路，n 为 0 时右边根本不会被求值，不会报除零错误。

1.13 复合赋值与运算符优先级

复合赋值运算

设初始值 `a = 10`：

运算符	描述	举例
`+=`	`a = a + b`	`a += 1`，a 变为 11
`-=`	`a = a - b`	`a -= 1`，a 变为 9
`*=`	`a = a * b`	`a *= 3`，a 变为 30
`/=`	`a = a / b`	`a /= 3`，a 变为 3.3333333333333335
`//=`	`a = a // b`	`a //= 3`，a 变为 3
`%=`	`a = a % b`	`a %= 3`，a 变为 1
`**=`	`a = a ** b`	`a **= 2`，a 变为 100

【易错】 复合赋值运算符是一个整体，两个符号中间不可插入空格，`a + = 1` 是错误的写法。另外 `/=` 与 `/` 一样会得到浮点数：`a = 10; a /= 2` 之后 a 是 5.0。

运算符优先级总表（速查）

从上向下，优先级由高到低：

运算符	描述	操作数个数
`**`	幂运算符	二元
`+ -`	一元加、一元减	一元
`* / // %`	乘法、算术除法、地板除法、取模	二元
`+ -`	二元加、二元减	二元
`<= < > >= == !=`	比较运算符	二元
`not`	逻辑非	一元

运算符	描述	操作数个数
`and`	逻辑与	二元
`or`	逻辑或	二元
`= += -= *= /= //= %= **=`	赋值运算符	二元

> **【核心】记不住就加小括号**：`(a + b) * c` 远比让别人去回忆优先级表可靠。另外注意 `=` 的优先级最低且"先算右边"，所以 `x = 1 + 2 * 3` 会先算出 7 再赋值。

例题 1-5：显示当前时间

思路：用 `//` 层层取整得到时和分，用 `%` 取出余数得到当前这一位。

```
import time

currentTime = time.time()
totalSeconds = int(currentTime)
currentSecond = totalSeconds % 60
totalMinutes = totalSeconds // 60
currentMinute = totalMinutes % 60
totalHours = totalMinutes // 60
currentHour = totalHours % 24

print("Current time is " + str(currentHour) + ":"
      + str(currentMinute) + ":" + str(currentSecond) + " GMT")
```

> **【核心】时间/单位换算的通用套路**："先 `//` 降级、再 `%` 取当前位"。秒数转几时几分几秒、毫秒转时分秒，都是同一套写法。

例题 1-6：房贷计算器

题目：输入贷款年利率、贷款年限、总贷款额，输出月供和总还款额。

公式：月利率 = 年利率 / 12；月供 = 总贷款额 × 月利率 ÷ (1 − 1/(1+月利率)^(贷款年限×12))；总还款额 = 月供 × 贷款年限 × 12。

```
annualInterestRate = eval(input("Enter annual interest rate, e.g., 8.25: "))
monthlyInterestRate = annualInterestRate / 1200
numberOfYears = eval(input("Enter number of years as an integer, e.g., 5: "))
loanAmount = eval(input("Enter loan amount, e.g., 120000.95: "))

monthlyPayment = loanAmount * monthlyInterestRate / (
    1 - 1 / (1 + monthlyInterestRate) ** (numberOfYears * 12))
totalPayment = monthlyPayment * numberOfYears * 12

print("The monthly payment is " + str(int(monthlyPayment * 100) / 100))
print("The total payment is " + str(int(totalPayment * 100) / 100))
```

> **【核心】** `int(monthlyPayment * 100) / 100` 是"保留两位小数"的一个实用技巧：先乘 100、截断成整数、再除 100，避开了 `round()` 在浮点数上的不确定性。处理金额时这个写法很常见；不过 OJ 上更省事的是 1.2 节那句 `print(f"{x:.2f}")`。

例题 1-7：计算两点间距离

题目：输入两个点的坐标，输出两点间距离。公式是 $\sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$。

```
x1, y1 = eval(input("Enter x1 and y1: "))
x2, y2 = eval(input("Enter x2 and y2: "))
distance = ((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)) ** 0.5
print("The distance between the two points is", distance)
```

> **【了解】** 用户输入 `1, 2` 时，`eval("1, 2")` 得到元组 `(1, 2)`，于是可以用 `x1, y1 = ...` 一次拆开赋给两个变量，这是 1.4 节"同时赋值"的实际应用。求距离也可以直接写 `math.hypot(x1 - x2, y1 - y2)`。

例题 1-8：计算三角形内角

思路三步：输入三个顶点坐标 → 用两点距离公式求出三边 a、b、c → 用余弦定理求得三个内角。

```
import math

x1, y1, x2, y2, x3, y3 = eval(input("Enter three points: "))
a = math.sqrt((x2 - x3) ** 2 + (y2 - y3) ** 2)
b = math.sqrt((x1 - x3) ** 2 + (y1 - y3) ** 2)
c = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
A = math.degrees(math.acos((a * a - b * b - c * c) / (-2 * b * c)))
B = math.degrees(math.acos((b * b - a * a - c * c) / (-2 * a * c)))
C = math.degrees(math.acos((c * c - b * b - a * a) / (-2 * a * b)))
print("The three angles are", round(A * 100) / 100.0,
      round(B * 100) / 100.0, round(C * 100) / 100.0)
```

> **【易错】** `math.acos()` 返回的是弧度，必须用 `math.degrees()` 转成角度，漏掉这一步结果会变成 0~3.14 之间的一个怪数。

1.14 注释

注释的作用

> **【核心】注释的两大用途**：① 提高程序的可读性，让今后阅读这段代码的人（包括一段时间以后的你自己）能够理解你的思路；② 调试时临时移除代码，把可能有问题的代码注释掉，让解释器忽略它，再运行调试。

合理的代码注释应该占源代码的 1/3 左右。

单行注释

Python 使用 `#` 号实现单行注释，`#` 可以在一行中从头开始，也可以从中间开始。

```
# 使用 print 输出字符串
print("Hello World!")

print(100)
print(3 + 100 * 2)
print("Hello World!")   # 输出 Python 欢迎信息
print(36.7 * 14.5)      # 输出乘积
print(100 % 7)          # 输出余数
```

多行注释

Python 使用三个连续的单引号 `'''` 或者三个连续的双引号 `"""` 注释多行内容，通常用来为文件、模块、类或者函数添加版权或功能描述信息。

```
'''
使用 3 个单引号分别作为注释的开头和结尾
可以一次性注释多行内容
'''
```

> **【易错】** 多行注释**不支持嵌套**。Python 遇到第一个 `'''` 就开始注释，遇到下一组 `'''` 就结束，不会去管你的"嵌套"意图。另外，当注释符作为字符串的一部分出现时，就不能视为注释标记，而应看做正常代码的一部分：

```
print('''Hello,World!''')      # 这里是字符串，不是注释
print("#是单行注释的开始")      # # 在引号内，只是普通字符
```

> **【了解】** 这也顺便说明了一件事：三引号本身是字符串语法，"多行注释"只是把一段没人接收的字符串放在那里而已。
>
> **【注意】** 注释里写中文完全没问题（源文件默认 UTF-8），注释里的全角标点也不影响运行——但**代码**里的标点必须半角。

1.15 本章方法总结与易错清单

> 这一节是全章的"复习卡"：考前只翻这一节，能自己复述出来就说明知识点都到手了。

一、本章知识地图

- **入口：数据怎么进来**
  - 1.1 IPO 模型决定程序骨架，1.3 `input()` 负责把数据读进来，1.10 类型转换负责把字符串变成能算的数
  - 这一层没做对，后面写得再漂亮也是 WA：`input()` 拿到的永远是字符串

- **存放：数据存在哪**
  - 1.4 变量与赋值（`=` 是赋值，不是等号）、1.5 命名规范、1.6 类型总览、1.7 整数与浮点数
  - 1.7 的两个结论是全书计算的底色：整数运算精确，浮点数运算可能不精确
- **处理：数据怎么算**（本章分量最重的一块，建立在上面两层之上）
  - 1.8 算术运算：`+ - * / // % **`，重点是三种除法、负数的地板除与取模
  - 1.9 内置函数与 math 库：`abs` `max` `min` `pow` `round` `sum` 以及 `math.*`
  - 1.12 比较与逻辑运算：`==` `!=` `<` `<=`、链式比较、`and` / `or` / `not` 与短路
  - 1.13 复合赋值与运算符优先级：`+=` 一类写法，以及"记不住就加括号"
- **出口：结果怎么出去**
  - 1.2 `print()` 的 `sep` / `end`，以及 OJ 最常用的 `print(f"{x:.2f}")`（完整格式见 7.10 节）
- **写得像人写的**：1.11 字符与编码、1.14 注释
- **考试与 OJ 最常考的五件事**：① `//` 与 `%`（尤其是负数）② `int(input())` 与 `map(int, input().split())` ③ `print` 的 `sep` / `end` 与 `f"{x:.2f}"` ④ `int` / `floor` / `ceil` / `round` 四个取整的区别 ⑤ `and` / `or` 的返回值与短路

二、方法总结（速查表）

函数/语法	作用	返回类型	典型例子
`input()`	从键盘读入一整行	str	`s = input()`
`print(a, b, sep=..., end=...)`	输出，逗号自动加空格	None	`print(1, 2, sep="-")` → `1-2`
`int(x)`	转整数，小数部分向零截断	int	`int("12")` → 12；`int(-3.5)` → -3
`float(x)`	转浮点数	float	`float("3.5")` → 3.5
`str(x)`	转字符串	str	`str(12) + "kg"` → `'12kg'`
`eval(x)`	把字符串当表达式求值	看表达式	`eval("3+4")` → 7
`type(x)`	查看类型，调试必备	type	`type(3.0)` → `<class 'float'>`
`abs(x)`	绝对值，类型不变	同 x	`abs(-2)` → 2
`max(...)` / `min(...)`	最大值 / 最小值	同元素	`max(1, 5, 2)` → 5
`pow(a, b)`	幂，等价于 `a ** b`	int 或 float	`pow(2, 3)` → 8
`round(x)`	取最近的整数，.5 取偶	int	`round(2.5)` → 2
`round(x, n)`	保留 n 位小数	float	`round(3.14159, 2)` → 3.14
`sum(xs)`	对一列数求和	int 或 float	`sum(map(int, "1 2 3".split()))` → 6
`math.floor(x)` / `math.ceil(x)`	向下 / 向上取整	int	`math.floor(-3.5)` → -4
`math.sqrt(x)`	平方根	float	`math.sqrt(9)` → 3.0

函数/语法	作用	返回类型	典型例子
`math.radians(x)` / `math.degrees(x)`	角度 ↔ 弧度	float	`math.radians(180)` → 3.141592653589793
`/`	算术除，结果一定是浮点	float	`7 / 2` → 3.5
`//`	地板除，向下取整	int 或 float	`-7 // 2` → -4
`%`	取模，符号跟除数	int 或 float	`-7 % 2` → 1；`7 % -2` → -1
`divmod(a, b)`	一次得到商和余数	tuple	`divmod(7, 2)` → (3, 1)
`ord(c)` / `chr(n)`	字符 ↔ 编码	int / str	`ord('a')` → 97
`bool(x)`	转布尔值（空、0 为 False）	bool	`bool('')` → False
`0 <= x < 100`	链式比较，区间判断	bool	`0 <= 50 < 100` → True

> **【速查】** 取整四件套 `int` / `math.floor` / `math.ceil` / `round` 的逐项对比见 1.10 节；读入的各种写法见 1.3 节的表格；格式化的完整说明符见 7.10 节。

三、易错清单

1. **错误现象**：程序一运行就报 `SyntaxError: invalid character ',' (U+FF0C)` → **原因**：用了中文全角标点 → **正确写法**：`print("你好")`，写代码前先确认输入法是英文状态。
2. **错误现象**：输入 3 和 5，`print(a + b)` 输出 `35` → **原因**：`input()` 返回字符串，`+` 变成了拼接 → **正确写法**：`a = int(input())`，一行多个数用 `map(int, input().split())`。
3. **错误现象**：该输出 `3` 的地方输出 `3.0`，OJ 判错 → **原因**：用了 `/` → **正确写法**：整除写 `6 // 2`，要小数才用 `/`。
4. **错误现象**：`-7 // 2` 算成 `-3`、`7 % -2` 算成 `1` → **原因**：用"向零截断"的直觉算 Python 的除法 → **正确写法**：记住 `//` 向下取整、`%` 符号跟除数，用 `x == (x // y) * y + (x % y)` 验算，例如 `-7 // 2` 是 `-4`、`7 % -2` 是 `-1`。
5. **错误现象**：`0.1 + 0.2 == 0.3` 得到 False → **原因**：二进制浮点表示误差 → **正确写法**：`abs(a - b) < 1e-9`；金额计算改用整数分或 `decimal`。
6. **错误现象**：`round(2.5)` 得 2、`round(2.675, 2)` 得 2.67 → **原因**：银行家舍入（.5 取偶）加上浮点误差 → **正确写法**：要传统四舍五入用 `int(x + 0.5)`，要精确到分用 `Decimal("2.675").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)`。
7. **错误现象**：`ValueError: invalid literal for int() with base 10: '3.5'` → **原因**：`int()` 不接受小数形式的字符串 → **正确写法**：需要小数用 `float(input())`，需要整数用 `int(float(input()))`。
8. **错误现象**：`if x = 1:` 报 `SyntaxError: invalid syntax` → **原因**：`=` 与 `==` 混淆 → **正确写法**：条件里比较用 `==`，赋值用 `=`。

9. **错误现象**：本地运行正确，交到 OJ 上判格式错误 → **原因**：`input()` 里写了提示语，或行尾多出空格、小数位数不对 → **正确写法**：提示语删掉（写成 `input()`），小数用 `print(f"{x:.2f}")`，行尾不要手工加空格。
10. **错误现象**：`bool("False")` 得到 True → **原因**：非空字符串一律为真，与内容无关 → **正确写法**：判断内容要写 `s == "False"`；判断"非空"才用 `if s:`。

四、必须能默写的 10 个片段

1. 读一个整数、一个实数：`n = int(input())`、`x = float(input())`
2. 一行读两个整数：`a, b = map(int, input().split())`
3. 一行读 n 个整数：`nums = list(map(int, input().split()))`
4. 输出保留 2 位小数：`print(f"{x:.2f}")`
5. 交换两个变量：`a, b = b, a`
6. 取末位、去掉末位：`d = n % 10` 与 `n //= 10`
7. 秒数换时分秒：`h, m, s = t // 3600, t % 3600 // 60, t % 60`
8. 负数除法的验算恒等式：`x == (x // y) * y + (x % y)`
9. 区间判断（链式比较）：`0 <= x < 100`
10. 浮点数近似相等：`abs(a - b) < 1e-9`

把上面这些连起来，就是本章所有 OJ 题的最小工具箱：

```
n = int(input())                                # 读一个整数
x = float(input())                              # 读一个实数
a, b = map(int, input().split())                # 一行两个整数
nums = list(map(int, input().split()))          # 一行 n 个整数
print(f"{x:.2f}")                               # 保留 2 位小数
a, b = b, a                                     # 交换两个变量
d = n % 10                                      # 取末位
n //= 10                                        # 去掉末位
t = 3661
h, m, s = t // 3600, t % 3600 // 60, t % 60     # 1 2 1
print(-7 // 2, -7 % 2, 7 % -2)                  # -4 1 -1
print(0 <= 50 < 100)                            # True
```

1.16 本章自测（16 道）

第 1 章 自测题 1　执行 `x = 5; x = x * 2 + 1` 后 x 的值是多少？为什么这个写法合法？

第 1 章 自测题 2　写出 `print(7 // 2)`、`print(7 / 2)`、`print(7 % 2)` 的输出。

第 1 章 自测题 3　`-9 % 4` 等于多少？用公式说明。

第 1 章 自测题 4　下面哪几行会报错？

```
a = int("12")
b = int("12.5")
```

```
c = float("12")
d = eval("3+4")
```

第 1 章 自测题 5　`print(bool(''))` 和 `print(bool('0'))` 分别输出什么？

第 1 章 自测题 6　表达式 `0 or "hello"` 和 `6 and "hello"` 的值分别是什么？

第 1 章 自测题 7　写出把 `x = 3.14159` 保留两位小数输出的两种写法。

第 1 章 自测题 8　为什么 `math.sin(90)` 不等于 1？怎么改？

第 1 章 自测题 9　下列哪些变量名合法？说出理由。

```
2nd_score    _name    class    totalPrice    总价    run-year    high_score
```

第 1 章 自测题 10　下面代码的执行结果是什么？

```
x = 0.1 + 0.2
print(x)
print(x == 0.3)
print(abs(x - 0.3) < 1e-9)
```

第 1 章 自测题 11　写出 `print(round(4.5))`、`print(round(-2.5))`、`print(int(-3.5))`、`print(math.floor(-3.5))` 的输出。

第 1 章 自测题 12　输入一个秒数，输出"几时几分几秒"（例如 3661 → 1:1:1），请写出完整程序。

第 1 章 自测题 13　写出 `print(-7 // 2)`、`print(7 % -2)`、`print(-7 % 2)` 的输出，并用恒等式 `x == (x // y) * y + (x % y)` 验算其中任意一个。

第 1 章 自测题 14　一行输入两个用空格隔开的整数，输出它们的和。写出完整的读入语句和输出语句。

第 1 章 自测题 15　要表示"x 在 0 到 100 之间（含 0，不含 100）"，用链式比较怎么写？用 `and` 又怎么写？两者等价吗？

第 1 章 自测题 16　`print(round(2.675, 2))` 和 `print(f"{2.675:.2f}")` 分别输出什么？为什么都不是 2.68？怎样才能得到 2.68？

1.17 本章作业题

> 以下两题来自你的作业截图（第 1、2 张），是"基础知识"章的配套练习。答案见全书最后"作业题答案"部分。

作业题 1｜三角形的周长和面积

题目描述：一个三角形的三边长分别是 a、b、c，那么它的面积为 √(p(p−a)(p−b)(p−c))，其中 p = (a+b+c)/2。

输入这三个数字，计算三角形的面积，**四舍五入精确到 2 位小数**。数据保证能构成三角形，0 ≤ a, b, c ≤ 1000，每个边长输入时不超过 2 位小数。

输入格式：输入三个实数 a、b、c，一个数字一行。

输出格式：三角形的周长和面积，一个数字一行。

输入输出样例

输入 #1	输出 #1
3	12.00
4	6.00
5	（面积）

说明/提示：Python 中输出 x 时保留 2 位小数，可以使用 `print(f"{x:.2f}")`。

> **考点**：本章第 1.3 节（读入实数）、第 1.8 节（算术运算与开方）、第 1.2 节（用 `f"{x:.2f}"` 保留 2 位小数）。
>
> **输入怎么读**：三个实数**各占一行**，所以写三次 `float(input())`，不要用 `map(int, input().split())`——那是一次读一行里的多个数，而且会把小数丢掉。周长是 `a + b + c`，面积用海伦公式配 `math.sqrt()`，最后两个结果都按 2 位小数输出。

作业题 2｜经过一段时间后的时刻

题目背景：简单数字运算。

题目描述：输入一个开始时刻，再输入一个时长（单位：分钟），计算经过这段时间后的结束时刻。测试输入的数据可以保证两个时刻是**同一天**，程序不必考虑超过 24 点的情况；时刻使用 24 小时制，即小时在 0 到 23 之间，分钟在 0 到 59 之间。

输入格式：输入 3 行整数，第一行为开始时刻的小时，第二行为开始时刻的分钟，第三行为从开始时刻到结束时刻之间有多少分钟。

输出格式：输出 2 行，第一行为结束时刻的小时，第二行为结束时刻的分钟。

输入输出样例

输入 #1	输出 #1
9	11
5	55
170	（结束分钟）

说明/提示：注意输入的第 3 个整数**可正可负**。计算结果不会早于 0 点 0 分，也不会达到或超过 24 点 0 分。

> **考点**：本章第 1.8 节（`//` 与 `%` 做时分换算，含负数的行为）、第 1.3 节（三次读入整数）、第 1.1 节（先统一单位再计算）。
>
> **输入怎么读**：三个整数**各占一行**，写三次 `int(input())`，不要用 `map`。先把开始时刻换算成总分钟数 `total = h * 60 + m + d`，再换回来：小时是 `total // 60`，分钟是 `total % 60`。第三个整数可能是负数，一定要按"`//` 向下取整、`%` 符号跟除数"来理解，别用"向零截断"的直觉；本题保证结果不早于 0 点，所以 `total` 不会为负。

> 提示思路：把开始时刻换算成"从 0 点起经过的分钟数"，加上时长（可能为负），再换回小时和分钟。这正是 1.8 节"`//` 与 `%` 换算"的典型应用。

第 2 章 流程控制

> 对应课程讲义：《03_流程控制》。这一章解决两个问题："程序怎么按不同情况走不同路线"和"怎么重复做同一件事"。它是全课程题目量最大的一章，第 3 章及以后的题目基本都要用到本章的三个模板：**闰年判断、素数判断、数字逐位拆解**。

2.1 程序的三种结构

结构	含义	生活里的例子
顺序结构	代码自上而下，逐行执行	泡面：烧水 → 下面 → 等 3 分钟
分支结构	根据条件，分情况执行	下雨就带伞，不下雨就不带
循环结构	根据条件，重复执行	抄单词：抄到记住为止

> **【核心】** 再复杂的程序，拆开看都是这三种结构的组合。写代码的建议顺序：先搭顺序结构的骨架，遇到"看情况"就加 `if`，遇到"重复"就加循环，最后填细节。
>
> **【速查】** 本章末尾的 2.9 节是"方法总结与易错清单"，写作业前先扫一遍，写完再对一遍，能省下大量调试时间。

2.2 导入模块

Python 把大量现成功能放在**模块**里，用之前先 `import`。常见写法有五种：

```
import math                 # 导入整个模块，调用时写 math.sqrt(9)
import math as m            # 起别名，调用时写 m.sqrt(9)
from math import sqrt       # 只导入某个函数，调用时直接写 sqrt(9)
from math import sqrt, pi   # 导入多个函数
from math import *          # 导入所有函数（不推荐）
```

> **【易错】** 推荐统一用 `import 模块名`。`from ... import *` 会把模块里的所有名字倒进当前文件，容易和你的变量重名，这叫"命名空间污染"；别人读代码时也看不出 `sqrt` 到底从哪来。
>
> **【注意】** 所有 `import` 语句统一写在文件最上面，不要写在循环或判断里面。本章只需要两个模块：`math`（开方、取整）和 `random`（随机数）。

生成随机数

```
import random

random.random()          # [0.0, 1.0) 之间的随机浮点数，没有参数
random.randint(a, b)     # [a, b] 之间的随机整数，两端都包含
random.randrange(a, b)   # [a, b) 之间的随机整数，不包含 b
random.uniform(a, b)     # a 和 b 之间的随机浮点数
```

```
random.choice(seq)      # 从序列里随机取一个元素
random.shuffle(lst)     # 就地打乱列表顺序
```

本章最常用的就是下面三个：

函数	返回值	取值范围	典型用法
`random.random()`	浮点数	[0.0, 1.0)，不含 1	概率判断 `if random.random() < 0.5:`
`random.randint(a, b)`	整数	[a, b]，**两端都含**	掷骰子 `random.randint(1, 6)`
`random.randrange(a, b)`	整数	[a, b)，**不含 b**	随机下标 `random.randrange(0, n)`

> **【易错】** `randint` 和 `randrange` 只差几个字母，端点规则却相反：`random.randint(1, 6)` 能出 6，`random.randrange(1, 6)` 永远出不了 6。`randrange` 用的就是 `range()` 那套"含头不含尾"的规则。
>
> **【了解】** 想让每次运行产生同一串随机数（方便核对输出），可以在最前面加一句 `random.seed(1)`，同一个种子会产生同一串随机数。

例题 2-1：加法题

题目：随机出两个 0~9 的一位数加法题，读入答案，判断答得对不对。

```
import random

number1 = random.randint(0, 9)
number2 = random.randint(0, 9)
answer = eval(input("What is " + str(number1) + " + " + str(number2) + "? "))
print(number1, "+", number2, "=", answer, "is", number1 + number2 == answer)
```

某次运行输入 `6`，输出：

```
3 + 3 = 6 is True
```

> **要点**：`str()` 把数字拼进提示字符串（数字不能直接和字符串相加，详见 1.10 节）；最后输出的 `True` 来自比较表达式 `number1 + number2 == answer` 的结果。题目是随机生成的，每次运行的题面和结果都不一样。
>
> **【注意】** 讲义这里用 `eval(input(...))` 是为了少写一次类型转换。本课程的作业和 OJ 统一用 `int(input())` / `float(input())`：`eval()` 会把输入当表达式执行，输入非法内容时直接报错，工程上并不安全。

2.3 分支结构 if

`if` 语句有三种形式：`if`、`if...else`、`if...elif...else`。

```
cond1 = cond2 = False

if cond1:           # 单分支：条件成立才做事
```

```
    pass
else:               # 双分支：两条路必走一条
    pass

if cond1:           # 多分支：从上往下判断，最多只执行其中一个分支
    pass
elif cond2:
    pass
else:
    pass
```

【核心】 `if` 语句的三条铁律：① 条件后面必须有冒号 `:`；② 同一个分支里的语句缩进必须完全相同，官方推荐 4 个空格；③ **缩进决定"谁属于谁"**——缩进相同的语句是一层，缩进多一层的语句属于上面那一行，跟"离得近不近"没关系。

【了解】 `pass` 是空语句，什么也不做，只用来占位、避免语法错误。

【常见错误】 `if`、`for`、`while` 行尾漏写冒号，程序根本跑不起来。

```
# X 错误写法
score = 85
if score >= 60
    print("及格")
```

```
# ✓ 正确写法
score = 85
if score >= 60:
    print("及格")
```

报错信息是 `SyntaxError: expected ':'`。**看到 SyntaxError，先看报错行号的上一行**，九成是冒号、括号或引号的问题。

【常见错误】 缩进不一致，或者该缩进的没缩进、不该缩进的多缩进了。

```
# X 错误写法：同一个分支里缩进不一致
score = 85
if score >= 60:
    print("及格")
      print("恭喜")        # IndentationError: unexpected indent
```

```
# ✓ 正确写法：同一层的语句左边缘对齐，每一层 4 个空格
score = 85
if score >= 60:
    print("及格")
    print("恭喜")
```

报错信息是 `IndentationError` 或 `TabError`。**不要用 Tab 键缩进**，把编辑器设成"按 Tab 插入 4 个空格"，就能避开 Tab 和空格混用这个坑。

复合条件

```
score = 85

print(score >= 80 and score < 90)   # True，用 and 连接两个比较
print(80 <= score < 90)             # True，Python 允许这样连写
```

【核心】 `80 <= score < 90` 在 Python 里合法，等价于 `80 <= score and score < 90`（而且 `score` 只求值一次）；C、Java 里不能这样写。比较运算和逻辑运算的细节见 1.12 节。

例题 2-2：成绩等级

题目：输入百分制成绩，按 90/80/70/60 分成 A、B、C、D、F 五档输出。

```
score = eval(input("Enter the score: "))

if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
elif score >= 60:
    grade = 'D'
else:
    grade = 'F'

print(grade)
```

输入 `85`，输出：

```
B
```

【常见错误】 多分支的条件必须**从严格到宽松**排列，写反了前面的分支会"抢答"。

```
# X 错误写法：>= 60 先命中，85 分也被判成 D
if score >= 60:
    grade = 'D'
elif score >= 70:
    grade = 'C'
elif score >= 80:
    grade = 'B'
elif score >= 90:
    grade = 'A'
else:
    grade = 'F'
```

```
# √ 正确写法：严格的条件写在前面
if score >= 90:
```

```
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
elif score >= 60:
    grade = 'D'
else:
    grade = 'F'
```

原因：`elif` 只有在前面所有条件都不成立时才会被判断。85 分在第一句 `score >= 60` 就成立了，后面的分支根本没机会执行。**写多分支时先问自己："哪个条件的范围最窄？"**

例题 2-3：判断某年是否为闰年

题目：输入一个年份，判断它是不是闰年。

置闰规则（格里高利历）：一般年份（非世纪年）能被 4 整除则置闰；世纪年（最后两位是 00）能被 400 整除则置闰。

```
year = eval(input("Enter a year: "))

# 方法 1：嵌套 if，逻辑和规则原文一一对应
isLeapYear = False
if year % 100 == 0:
    if year % 400 == 0:
        isLeapYear = True
else:
    if year % 4 == 0:
        isLeapYear = True

# 方法 2：一行表达式（推荐）
isLeapYear = (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)

print(year, "is a leap year?", isLeapYear)
```

输入 `2000`，输出：

```
2000 is a leap year? True
```

【核心】闰年模板（必须背下来）

```
isLeapYear = (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
```

口诀："四年一闰，百年不闰，四百年再闰"。两种写法都只是这三句话的不同翻译。

- `and` 的优先级比 `or` 高，所以最外层的括号可以省，但**建议留着**，看代码的人一眼就知道是哪两句在配对。

- 判断闰年是本课程复用次数最多的子问题：第 3 章 3.11 节的例题 3-4 会把它写成函数 `isLeapYear(year)`，本章作业题 11、作业题 12 都要用它，后面的日期类题目还会再遇到。
- 自检用的四个年份：1900 平年、2000 闰年、2024 闰年、2023 平年。代码写完先用这四个数试一遍。

方法总结：怎样把一段文字规则翻译成条件表达式？

1. 先找"或者"：每一句"……则……"是一条独立规则，规则之间用 `or` 连接（满足任意一条就行）；
2. 每条规则内部的"并且"用 `and` 连接（两个条件要同时满足）；
3. 用 `%` 表达整除关系：`year % 4 == 0` 就是"能被 4 整除"；
4. 最后拿**边界数据**手工验证。边界数据永远是三样：刚好满足的、刚好不满足的、特殊值（比如 4、100、400、1900、2000）。

【了解】为什么要置闰：一回归年约 365.2422 天。按 365 天算，每年差 0.2422 天；按 365.25 天算差 0.0078 天；加上"百年不闰、四百年再闰"之后误差只剩 0.0003 天。

例题 2-4：求一个月的天数

题目：输入年份和月份，输出这个月有多少天。

```
year = eval(input())
month = eval(input())

if month == 2:
    if year % 400 == 0 or (year % 100 != 0 and year % 4 == 0):
        days = 29
    else:
        days = 28
elif month in (1, 3, 5, 7, 8, 10, 12):
    days = 31
elif month == 4 or month == 6 or month == 9 or month == 11:
    days = 30

print(days)
```

输入 `2023`、`2`，输出：

```
28
```

【易错】 `month == 4 or 6 or 9 or 11` 是**错误**写法！`or` 连接的是四个"真值"：`(month == 4) or 6 or 9 or 11`，而 6、9、11 都是非零数字（视同 True），整个表达式永远为真，于是所有月份都被判成 30 天。正确写法是每个比较都写全，或者干脆用 `month in (4, 6, 9, 11)`。

```
# ✗ 错误写法              month == 4 or 6 or 9 or 11
# ✓ 正确写法一            month == 4 or month == 6 or month == 9 or month == 11
# ✓ 正确写法二（推荐）    month in (4, 6, 9, 11)
```

> **【注意】** 判断 2 月的天数时直接复用了例题 2-3 的闰年模板，写成 `year % 400 == 0 or (year % 100 != 0 and year % 4 == 0)` 一行就够了，不必再套一层 `if`。作业题 11、作业题 12 都可以直接抄这一行的思路。

条件表达式（三元表达式）

格式：`表达式1 if 条件 else 表达式2`。条件成立取表达式 1，否则取表达式 2。它只能得到一个**值**，不能写多条语句。

```
x = eval(input())
y = 1 if x >= 0 else -1       # 等价于 if x >= 0: y = 1  else: y = -1

number = eval(input())
print("number is even" if number % 2 == 0 else "number is odd")
```

> **【注意】** 条件表达式适合"二选一取值"的场合，嵌套超过一层就该改回 `if...elif...else`，否则没人看得懂。

2.4 循环结构 while

`while` 是"条件成立就一直做"的循环，适合**循环次数事先不知道**的场合。

```
i = 1
sum = 0
while i <= 100:
    sum += i
    i += 1
print(sum)      # 5050
```

> **【核心】循环三要素**——写循环前先把这三样想清楚： ① **初值**：循环变量在哪里、从几开始（写在循环之前）； ② **条件**：循环什么时候继续（用循环变量写）； ③ **更新**：循环变量每一轮怎么变（写在循环体内）。 三样缺一，循环要么一次都不执行，要么永远停不下来。**写完循环先检查第 ③ 条。**

死循环怎么排查

死循环（程序卡住不动，只能按 Ctrl+C 强停）是初学者第一大事故。按下面四步查：

1. **查更新**：循环条件里用到的变量，在循环体里被改过吗？`i += 1` 是不是漏写了，或者写到循环体外面去了？
2. **查方向**：变量是在朝条件需要的方向走吗？（条件写 `i <= 100`，更新却写成 `i -= 1`。）
3. **查初值**：条件第一次判断成立吗？不成立就一次都不执行——注意"没输出"和"死循环"是两种相反的毛病。
4. **打印观察**：在循环体第一行临时加 `print(i)`，跑几步看变量怎么变，找到问题后再删掉这行。这是最有效的调试手段。

【注意】 死循环不一定是 bug。`while True:` 配 `break` 是很常见的写法（见 2.6 节）。判断标准是："人能不能一眼看出它在什么情况下退出？"能，就是好代码。

【常见错误】 `while` 循环忘记更新循环变量，条件永远成立。

```
# X 错误写法：i 永远是 1，死循环
i = 1
while i <= 100:
    print(i)
```

```
# √ 正确写法：把更新放进循环体
i = 1
while i <= 100:
    print(i)
    i += 1
```

症状：程序不报错、也不结束，一直刷屏。按 Ctrl+C 停下，然后回到"死循环四步排查"的第 1 步。

例题 2-5：猜数字

题目：程序随机想一个 1~100 的整数，用户反复猜，程序提示"太大""太小"，猜中结束。

```
import random

number = random.randint(1, 100)
print("Guess a magic number between 1 and 100")
guess = -1
while guess != number:
    guess = eval(input("Enter your guess: "))
    if guess == number:
        print("Yes, the number is", number)
    elif guess > number:
        print("Your guess is too high")
    else:
        print("Your guess is too low")
```

某次运行（答案是 42）：

```
Guess a magic number between 1 and 100
Enter your guess: 50
Your guess is too high
Enter your guess: 25
Your guess is too low
Enter your guess: 42
Yes, the number is 42
```

要点：`guess` 先赋一个**不可能等于答案**的初值（-1），保证循环至少能进来一次——这就是"循环三要素"里的初值。如果把 `guess = -1` 删掉，第一次执行 `while guess != number` 时 `guess` 还不存在，会

> 报 `NameError`（详见 2.9 节易错清单）。

2.5 for...in 循环与 range()

`for...in` 一般用于**已知循环次数**的循环。它的执行过程是：从序列里依次取出元素，每取一个就把循环变量赋成它，然后执行一次循环体；取完了循环就结束。

```
for c in 'hello':        # 循环 5 次，依次输出 h e l l o
    print(c)

for i in (2, 3, 5, 7):   # 循环 4 次
    print(i)
```

> **【核心】** `for` 循环的次数**不是由循环变量决定的，而是由序列里元素的个数决定的**。所以在 `for i in range(1, 5)` 里，不管你怎样改 `i` 都不影响循环次数（改的只是这一轮的值）。

range() 函数

写法	产生的序列	说明
`range(n)`	0, 1, 2, ..., n−1	从 0 开始，共 n 个；n ≤ 0 时为空
`range(a,b)`	a, a+1, ..., b−1	**包含 a，不包含 b**；b ≤ a 时为空
`range(a,b,k)`	a, a+k, a+2k, ...	k>0 时不超过 b；k<0 时从大往小；k 不能为 0，省略时相当于 1

```
list(range(5))          # [0, 1, 2, 3, 4]
list(range(1, 10))      # [1, 2, 3, 4, 5, 6, 7, 8, 9]
list(range(0, 10, 2))   # [0, 2, 4, 6, 8]        步长为 2 的偶数
list(range(10, 1, -1))  # [10, 9, 8, 7, 6, 5, 4, 3, 2]   倒序
list(range(5, 0, -1))   # [5, 4, 3, 2, 1]
list(range(0))          # []  空序列，循环体一次都不执行
```

> **【核心】三参数口诀**：`range(起点，终点，步长)`——"**起点算、终点不算、步长可正可负**"。要输出 1 到 5，必须写 `range(1, 6)`；要倒着数，步长写 `-1`。
>
> 用 `list(range(...))` 把序列打印出来看，是记住 `range` 最快的办法。（`list()` 的细节见第 4 章。）
>
> **【常见错误】** `range(1, 5)` 只循环到 4，不是到 5。

```
# ✗ 错误写法：想让 i 取 1~5，结果只到 4
for i in range(1, 5):
    print(i)          # 输出 1 2 3 4
```

```
# ✓ 正确写法：右端写"想要的最后一个数 + 1"
for i in range(1, 6):
    print(i)          # 输出 1 2 3 4 5
```

原因：`range` 是"含头不含尾"的，右端参数的含义是"到哪里为止（不含）"。**只要循环次数带"到 N 为止"的字样，就检查一遍右端是不是 N + 1。**

【易错】循环变量在循环结束后仍然存在，它的值是最后一次取到的那个。

```
for i in range(3):
    pass
print(i)        # 2，循环结束后 i 还能用
```

Python 的 `for` 不会新建作用域（这点和 C、Java 不同），所以循环结束后循环变量还在，保留最后一个值；但如果循环体一次都没执行（例如 `range(0)`），这个变量就从来没被赋过值，用它必然 `NameError`。

实践建议：循环结束后不要再复用同名变量（比如又拿 `i` 当计数器的初值），很容易接错值。需要保留的值，循环里就存到专门的累加器 / 计数器里。

【注意】 循环变量用不上时，习惯写成 `_`：`for _ in range(n):`，读代码的人马上知道"这个变量不重要"。

OJ 里怎么读数据（模板速查）

【核心】 本课程 OJ 的输入格式基本只有下面四种，把这四个模板背下来，后面所有作业都不会卡在"读数据"上。

① 一行一个数

```
n = int(input())            # 整数
x = float(input())          # 小数
```

② 一行多个数（最常用）

```
a, b = map(int, input().split())            # 恰好两个整数
x, y, r = map(float, input().split())       # 恰好三个小数
nums = list(map(int, input().split()))      # 一行不定个数，存成列表（列表见第 4 章）
```

`input().split()` 的含义：先把整行读成字符串，再按**空白**（空格、Tab）切成若干段，得到字符串列表；`map(int, ...)` 再把每一段转成整数。

③ 行数已知：先读行数，再循环读

```
n = int(input())
for _ in range(n):
    a, b = map(int, input().split())
    print(a + b)
```

④ 读到文件结束（EOF）——两种等价模板

```
# 模板 A：sys.stdin 逐行迭代（推荐，写得最短）
import sys
```

```
for line in sys.stdin:
    if not line.strip():        # 跳过空行
        continue
    a, b = map(int, line.split())
    print(a + b)
```

```
# 模板 B：input() + EOFError（本地手输时按 Ctrl+Z 再回车 表示结束）
while True:
    try:
        a, b = map(int, input().split())
    except EOFError:
        break
    print(a + b)
```

【易错】 `split()` 和 `split(' ')` 不一样：`split()` 按任意空白切分并丢掉空串（`"a b".split()` 得到 `['a', 'b']`），`split(' ')` 按单个空格切分、保留空串。**读 OJ 数据一律用 `split()`**，它能自动处理行首行尾和连续空格。

【易错】 `input()` 返回的永远是字符串。`'10' < '9'` 结果是 True（字符串比大小是逐字符比的），必须先 `int()` 转换再比较。

【注意】 忘了 EOF 模板，是多组测试数据"只过了第一组"的最常见原因（详见 2.9 节易错清单第 12 条）。

例题 2-6：两个年份之间有多少天

题目：输入两个年份，统计从第一个年份到第二个年份（含两端）一共有多少天。

```
year1 = int(input())
year2 = int(input())
days = 0
for i in range(year1, year2 + 1):
    if i % 400 == 0 or (i % 100 != 0 and i % 4 == 0):
        days += 366
    else:
        days += 365
print(days)
```

输入 `2023`、`2024`，输出：

```
731
```

要点：`range(year1, year2 + 1)` 里的 `+ 1` 就是为了把 `year2` 这一年也算进去——这是"含头不含尾"最常见的应用场景。循环里的闰年判断直接套用例题 2-3 的模板。

while 和 for 的转换

同一个循环常常两种写法都行，转换规则是：**把"初值"挪到 `for` 的起点，把"更新"变成步长。**

```
# while 写法
i = 1
while i < n:
    ...
    i += 1
```

```
# for 写法
for i in range(1, n):
    ...
```

> **【核心】** 已知循环次数时优先用 `for`：不用写初值和更新，少两行代码、少两个出错机会；循环次数未知（靠条件退出）时才用 `while`。**能写 `for` 就不写 `while`。**

易错：不要用 == 比较浮点数

```
item = 1
sum = 0
while item != 0:       # 死循环，无法退出！
    sum += item
    item -= 0.1
```

由于浮点误差，`item` 会变成 `1.3877787807814457e-16`、`-0.09999999999999987`……永远不会恰好等于 0，循环无法退出。

再比如求 `0.01 + 0.02 + … + 0.99 + 1`：

```
sum = 0
i = 0.01
while i <= 1.0:        # 结果 49.50000000000003，比预期的 50.5 少
    sum += i
    i = i + 0.01
```

原因：循环变量最后变成 `1.0000000000000007`，已经大于 1，最后一轮没有执行，所以少了 1。

> **【核心】浮点数的两条铁律**（背景见 1.7 节）：① **绝不用 `==` / `!=` 判断两个浮点数相等**，要写 `abs(a - b) < 1e-9`；② **需要精确计数的循环，控制变量一律用整数。**

```
sum = 0
i = 0.01
for count in range(100):       # 用整数控制循环 100 次
    sum += i
    i = i + 0.01
print("The sum is", sum)       # 50.50000000000003
```

例题 2-7：打印九九乘法表

```
print(" Multiplication Table")
print(" |", end='')
for j in range(1, 10):
    print(" ", j, end='')
print()
print("----------------------------------------")

for i in range(1, 10):
    print(i, "|", end='')
    for j in range(1, 10):
        print(format(i * j, '4d'), end='')
    print()
```

输出：

```
 Multiplication Table
 |  1  2  3  4  5  6  7  8  9
----------------------------------------
1 |   1   2   3   4   5   6   7   8   9
2 |   2   4   6   8  10  12  14  16  18
3 |   3   6   9  12  15  18  21  24  27
4 |   4   8  12  16  20  24  28  32  36
5 |   5  10  15  20  25  30  35  40  45
6 |   6  12  18  24  30  36  42  48  54
7 |   7  14  21  28  35  42  49  56  63
8 |   8  16  24  32  40  48  56  64  72
9 |   9  18  27  36  45  54  63  72  81
```

> **【核心】** `print()` 加 `end` 参数可以输出后不换行（`end=''` 表示什么都不加），单独一个 `print()` 用来换行。打印表格、图形时这两个技巧反复使用。（`sep` 和 `end` 的用法见 1.2 节，`format()` 的格式化输出也见 1.2 节。）

嵌套循环怎么写？ 记住两句话：

- **外层管行，内层管列**：外层循环变量控制"第几行"，内层循环变量控制"这一行里第几个"；
- **内层跑完一整圈，外层才走一步。**所以总循环次数是"外层次数 × 内层次数"（这道题是 9 × 9 = 81 次）。

例题 2-8：打印金字塔

题目：输入一个不大于 9 的正整数 n，输出一个 n 行金字塔。

```
n = int(input())
for i in range(1, n + 1):
    for j in range((n - i) * 2):
        print(" ", end="")
    for j in range(1, i + 1):
        print(j, end=' ')
    for j in range(i - 1, 0, -1):
```

```
        print(j, end=' ')
    print()
```

输入 `3`，输出：

```
    1
  1 2 1
1 2 3 2 1
```

要点：这道题把"循环三要素"用全了——第一部分打印每行开头的空格（`(n - i) * 2` 个，随行号递减），第二部分正着数 `1..i`，第三部分倒着数 `i-1..1`（用 `range(i - 1, 0, -1)`，步长为 −1）。**图形题的通法**：先在草稿纸上写出前 3 行，标出"第 i 行要打印几个空格、几个数字"，再把规律翻译成 `range()`。

2.6 break 与 continue

- `break`：立即结束循环，跳到循环后面的语句；
- `continue`：结束**本轮**，直接进入下一轮的条件判断。

```
# 循环输入数字，输出其平方，直到输入 0 为止
while True:
    x = int(input())
    if x == 0:
        break
    print(x ** 2)
```

```
# 打印 100 以内不被 7 整除、且数字中不含 7 的整数
for i in range(1, 101):
    if i % 7 == 0:
        continue
    if '7' in str(i):
        continue
    print(i)
```

【核心】`break` 只能跳出"当前这一层"循环。 嵌套循环里，`break` 只结束最内层，外层照常继续：

```
外层 i = 1  ─┬─ 内层 j = 1
             ├─ 内层 j = 2 → break ──► 内层结束
             └─ 回到外层 i = 2  ──────► 外层不受影响，继续下一轮
```

【常见错误】 以为嵌套循环里写一个 `break` 就能一次跳出两层。

```
# X 错误写法：只跳出内层，外层还会继续
for i in range(5):
    for j in range(5):
        if i * j == 6:
```

```
            break           # 只跳出 j 循环
    print(i, "这一行还会执行")
```

```
# ✓ 正确写法：用标志变量，跳出内层后再判断一次
found = False
for i in range(5):
    for j in range(5):
        if i * j == 6:
            found = True
            break               # 先跳出内层
    if found:
        break                   # 再跳出外层
```

另外两种跳出多层循环的办法：① 把循环写进函数，用 `return` 直接结束整个函数（见第 3 章 3.3 节）；② 改进外层循环条件，让它自己变假。

【易错】 `while` 循环里用 `continue`，很容易把"更新变量"那一步跳过去，造成死循环。

```
# ✗ 死循环：i 等于 2 时 continue 掉，i += 1 永远执行不到
i = 0
while i < 5:
    if i == 2:
        continue
    print(i)
    i += 1
```

```
# ✓ 正确写法一：把更新放在 continue 之前
i = 0
while i < 5:
    i += 1
    if i == 3:
        continue
    print(i)

# ✓ 正确写法二（更推荐）：改用 for 循环，更新由 range 负责
for i in range(5):
    if i == 2:
        continue
    print(i)
```

【速查】四个"提前结束"的语句

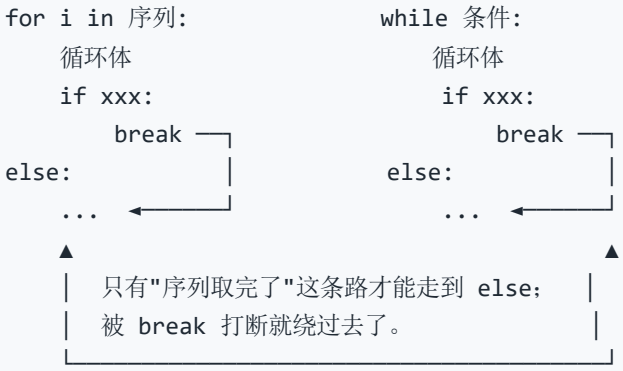
语句	作用范围	之后会发生什么
`break`	最内层的那一个循环	执行循环后面的语句
`continue`	当前这一轮循环	直接开始下一轮
`return`	整个函数	函数结束，返回调用处（见 3.3 节）

语句	作用范围	之后会发生什么
`sys.exit()`	整个程序	立即退出（【了解】知道即可）

2.7 循环中的 else

`else` 与 `while` 或 `for` 配合使用时，意思是：**当循环正常退出（不是遇到 `break` 退出）时，执行 `else` 子句。**

把判断过程拆成三句话，就不会记错了：



- 循环**没有**遇到 `break`，正常结束（序列取完 / 条件变假）→ **执行 else**；
- 循环遇到 `break` → **跳过 else**；
- 循环体一次都没执行（例如 `range` 为空）→ 也算"正常结束"，**执行 else**（这点最容易错）。

【了解】 这是 Python 特有的语法，其它语言一般没有把 else 用在循环上的写法。

```
number = int(input())
for i in range(2, number):
    if number % i == 0:
        break
else:
    print(number, "is a prime number")
```

【易错】 `else` 的缩进写错，它就会从"循环的 else"变成"`if` 的 else"，意思完全不同。

```
# X 错误写法：else 缩进到循环体里，成了 if 的 else，每一轮都会打印
for i in range(2, n):
    if n % i == 0:
        break
    else:
        print(n, "是素数")
```

```
# √ 正确写法：else 和 for 左边缘对齐
for i in range(2, n):
    if n % i == 0:
```

```
        break
else:
    print(n, "是素数")
```

自查法：`else` 前面那个关键字是 `for` / `while`，还是 `if`？看它和谁左边缘对齐。

例题 2-9：判断素数（三种写法递进）

```
# 写法 1：用标志变量
number = int(input())
isPrime = True
for i in range(2, number):
    if number % i == 0:
        isPrime = False
        break
if isPrime:
    print(number, "is a prime number")
else:
    print(number, "is not a prime number")
```

```
# 写法 2：用循环 else
number = int(input())
for i in range(2, number):
    if number % i == 0:
        break
else:
    print(number, "is a prime number")
```

```
# 写法 3：优化上限，只需要试到 √n
import math
number = int(input())
isPrime = True
for i in range(2, math.ceil(number ** 0.5) + 1):
    if number % i == 0:
        isPrime = False
        break
if isPrime:
    print(number, "is a prime number")
else:
    print(number, "is not a prime number")
```

输入 `97`，输出：

```
97 is a prime number
```

【核心】为什么只需要试到 √n：如果 n 是合数，必存在一个因数 a ≤ √n。设 m = ⌊√n⌋，若存在 a ≤ b 使 a×b = n，那么必然 a ≤ m。反证：假设 a > m，因为 a ≤ b，所以 b > m，于是 a×b > m² = n，与 n = a×b 矛盾。

【核心】素数判断模板（背下来）

```
number = int(input())
isPrime = True
if number < 2:                                  # 0 和 1 都不是素数，这一步最容易漏
    isPrime = False
else:
    for i in range(2, int(number ** 0.5) + 1):
        if number % i == 0:
            isPrime = False
            break
print("是素数" if isPrime else "不是素数")
```

- `int(number ** 0.5) + 1` 和例题里的 `math.ceil(number ** 0.5) + 1` 效果一样，前者不用导入 `math`。
- **边界一定要试**：1（不是素数）、2（是素数，`range(2, 2)` 为空，循环一次都不走）、9（不是素数）。
- 判断素数是复用次数第二多的子问题：本章作业题 8、作业题 9 都要用；第 3 章 3.11 节的例题 3-5 会把同一套逻辑写成函数 `isPrime(number)`，那时你只需要调用它。

例题 2-10：求两个正整数的最大公约数

```
# 写法 1：从小到大试
n1 = eval(input()); n2 = eval(input())
gcd = 1
k = 2
while k <= n1 and k <= n2:
    if n1 % k == 0 and n2 % k == 0:
        gcd = k
    k += 1
print(gcd)
```

```
# 写法 2：从大往小找，找到的第一个就是最大公约数
a = int(input()); b = int(input())
if a < b:
    a, b = b, a
for i in range(b, 0, -1):
    if a % i == 0 and b % i == 0:
        print(i)
        break
```

```
# 写法 3：辗转相除法（欧几里得算法），最快
a = int(input()); b = int(input())
if a < b:
    a, b = b, a
c = a % b
while c != 0:
```

```
    a = b
    b = c
    c = a % b
print(b)
```

输入 `24`、`36`，输出：

```
12
```

【核心】 写法 3 的依据只有一句：`gcd(a, b) = gcd(b, a % b)`，当 `b == 0` 时 `a` 就是答案。**推荐写法 3**。第 3 章 3.11 节的例题 3-6 会把它写成函数。

例题 2-11：判断回文数

题目：回文数即倒过来读与原数字相同的数，例如 12321。

```
number = int(input('请输入一个整数：'))
n = number
m = 0
while n != 0:
    m = m * 10 + n % 10    # 取出个位，接到新数的末尾
    n //= 10               # 去掉个位
if m == number:
    print(number, '是回文数')
else:
    print(number, '不是回文数')
```

输入 `12321`，输出：

```
12321 是回文数
```

【核心】"逐位取出、反向拼接"模板

```
m = 0
while n != 0:
    m = m * 10 + n % 10     # n % 10 取个位
    n //= 10                # n // 10 去掉个位
```

循环结束时 `m` 就是 `number` 的反转数。这和 1.8 节的"逐位拆解"是同一套思想的两种用法，务必练熟。作业题 3（水仙花数）、作业题 4（四位回文数）、作业题 9（反素数）都要用它。

注意：判断回文数也可以用字符串反转 `str(n) == str(n)[::-1]`（第 5 章），但考试和作业按数字写法给分。

综合例题（分支 + 循环 + 输入）

下面两道例题把本章的分支、循环和输入读法串起来用，难度和作业题 6~14 相当。

例题 2-12：求一行若干个整数的最大值、最小值与平均值

题目：一行输入若干个整数（用空格隔开，至少一个），输出它们的最大值、最小值和平均值，各占一行（平均值保留 2 位小数）。

输入：

```
3 7 -2 9 4
```

输出：

```
9
-2
4.20
```

```
nums = list(map(int, input().split()))   # 一行多个数，见 2.5 节的输入模板 ②

max_num = min_num = nums[0]              # 关键：用第一个数起手，不要用 0 起手
total = 0

for x in nums:
    if x > max_num:
        max_num = x
    if x < min_num:
        min_num = x
    total += x

print(max_num)
print(min_num)
print(format(total / len(nums), '.2f'))
```

要点：求最大值 / 最小值的标准套路是"**先拿第一个当擂主，后面的逐个挑战**"。千万不要写 `max_num = 0`：万一输入全是负数（比如 −5 −8 −3），0 就永远守擂成功，输出 0，答案全错。作业题 14 要用同一套写法。

例题 2-13：百钱百鸡（穷举法）

题目：公鸡 5 钱一只，母鸡 3 钱一只，小鸡 1 钱三只。用 100 钱买 100 只鸡，问公鸡、母鸡、小鸡各多少只？输出所有方案，每行三个整数。

输出：

```
0 25 75
4 18 78
8 11 81
12 4 84
```

```
for rooster in range(0, 21):             # 公鸡最多 20 只（100 // 5）
    for hen in range(0, 34):             # 母鸡最多 33 只（100 // 3）
```

```
        chick = 100 - rooster - hen      # 鸡的总数被限定为 100，小鸡不用再循环
        if chick % 3 != 0:               # 小鸡按 3 只一组，必须能被 3 整除
            continue
        if 5 * rooster + 3 * hen + chick // 3 == 100:
            print(rooster, hen, chick)
```

要点（穷举法 / 枚举法三步走）： ① **定范围**：每个变量的取值范围由题目条件算出来（100 钱最多买 20 只公鸡），范围越小越快； ② **少一层循环**：能由已知量推出来的量（小鸡 = 100 − 公鸡 − 母鸡）就不要单独循环，三层循环变两层； ③ **逐个验证**：用 `if` 把题目给的约束一条条写下来（钱数正好 100）。

穷举法是本课程解决"数学味很浓"的题目的万能方法：作业题 3（水仙花数）和自测题 9（完数）都能用。**只要数据范围不大（几千以内），穷举永远不会错，先写对再想快。**

2.8 异常处理

Python 运行出错会停止并报错，这叫**引发异常**。通过异常处理机制可以检查并处理异常，让程序不至于突然结束。

```
try:
    num = float(input("Enter a number: "))
except ValueError:
    print("That was not a number!")
```

【核心】 `try` 用来圈出可能引发异常的代码，`except` 用来标记异常出现时执行的代码。**指定异常类型是好习惯**，有用户交互的地方通常容易出错。

常见异常类型（速查）

异常名称	描述
`Exception`	常规错误的基类
`ZeroDivisionError`	除（或取模）零
`ValueError`	传入无效的参数（如 `float('hi')`）
`TypeError`	对类型无效的操作（如 `None + 1`）
`IndexError`	序列中没有此索引（越界）
`KeyError`	字典中没有这个键
`NameError`	使用了未声明/未初始化的对象
`ImportError`	导入模块/对象失败
`AttributeError`	对象没有这个属性
`SyntaxError`	Python 语法错误
`IndentationError`	缩进错误

异常名称	描述
`TabError`	Tab 和空格混用
`KeyboardInterrupt`	用户中断执行（通常是 ^C）

> **【了解】** 不知道类型名时，可以直接在解释器里故意试一次，报错信息里就写着异常名字。

处理多种异常

```
# 方法一：一个 except 捕获多种异常
for value in (None, "Hi!"):
    try:
        print("Attempting to convert", value, "-->", end=" ")
        print(float(value))
    except (TypeError, ValueError):
        print("Something went wrong!")
```

```
# 方法二：分别处理，可以给出有针对性的解决办法
for value in (None, "Hi!"):
    try:
        print(float(value))
    except TypeError:
        print("I can only convert a string or a number!")
    except ValueError:
        print("I can only convert a string of digits!")
    except:
        print("Something is wrong, but I don't know why.")
```

> **【易错】** 最后那个光秃秃的 `except:` 能捕获**所有**异常，包括你按 Ctrl+C 产生的 `KeyboardInterrupt`——程序会变得"停不下来"，真正的 bug 也被悄悄吞掉。**永远写清楚异常类型**；确实要兜底就写 `except Exception:`。

获取异常参数

```
try:
    num = float(input("\nEnter a number: "))
except ValueError as e:
    print("That was not a number! Or as Python would say...", e)
```

else 与 finally

```
try:
    num = float(input("\nEnter a number: "))
except ValueError as e:
    print("That was not a number!", e)
else:
    print("You entered the number", num)   # 没有异常时执行
```

```
finally:
    print("I'm always here!")              # 无论如何都执行
```

> **【核心】四个子句的执行时机**：`try` 总是执行；`except` 只在发生**匹配的**异常时执行；`else` 只在**没有发生异常**时执行；`finally` **无论有没有异常都执行**（常用于关闭文件、释放资源，见第 7 章）。

抛出异常 raise 与断言 assert

```
try:
    number = float(input("Enter a positive number: "))
    if number <= 0:
        raise ValueError("Not a positive number!")
except Exception as e:
    print("Something is wrong. ", e)
```

断言：`assert 条件, "条件为 False 时显示的提示信息"`。

```
x = 0.1 + 0.2
assert x == 0.3, "Ooop, x is wrong"   # 抛 AssertionError
```

> **【核心】** 断言针对的是**程序员的错误**，而不是用户的错误。对于可以恢复的错误，请抛出异常，而不是用 assert 检测它。运行 Python 时使用 `-O` 选项可以禁用断言。

日志 logging

> **【核心】** 不要用 `print()` 调试，会与正常的输出混淆。日志是给程序员看的，不是给用户看的。

日志级别	日志函数	描述
DEBUG	`logging.debug()`	调试，最低级别
INFO	`logging.info()`	一般事件的信息
WARNING	`logging.warning()`	警告，可能存在问题但不影响运行
ERROR	`logging.error()`	错误，会导致程序失败
CRITICAL	`logging.critical()`	严重，最高级别，致命错误

```
import logging
logging.basicConfig(level=logging.DEBUG,
                    format='%(asctime)s - %(levelname)s - %(message)s')
x = 1
logging.debug("x=" + str(x))
# 2023-03-09 15:28:14,204 - DEBUG - x=1

logging.disable(logging.CRITICAL)   # 禁用所有日志
```

> 【了解】处理异常有两种风格：**EAFP**（先做，出错再处理，用 try）和 **LBYL**（先检查，再操作，用 if）。Python 提倡 EAFP：`try: n = int(s) except ValueError:` 通常比 `if s.isdigit():` 更省事，也不容易漏掉边界情况。

2.9 本章方法总结与易错清单

一、本章知识地图

- **第 1 层：三种结构**（2.1 节）——顺序、分支、循环。所有程序都是这三样的组合。
- **第 2 层：分支**
  - `if / elif / else`（2.3 节）：多选一，条件**从严格到宽松**排列；
  - 条件表达式 `A if 条件 else B`（2.3 节）：只需要一个值时的简写；
  - 条件本身由比较运算和 `and / or / not` 拼成（1.12 节）。
- **第 3 层：循环**
  - `while`（2.4 节）：次数未知时用，靠"初值 + 条件 + 更新"三要素维持；
  - `for ... in range(...)`（2.5 节）：次数已知时用，右端**取不到**；
  - `break / continue`（2.6 节）：提前结束整个循环 / 跳过本轮；
  - `for ... else`（2.7 节）：只有"正常结束"才执行，被 `break` 打断就不执行。
- **第 4 层：用循环 + 分支解决经典子问题**（2.7 节例题）
  - 素数判断（例题 2-9）、最大公约数（例题 2-10）、数字反转与回文（例题 2-11）、穷举法（例题 2-13）。
- **第 5 层：出错了怎么办**（2.8 节）——`try / except / else / finally`、`raise`、`assert`、`logging`。
- **考试与作业最常考的五件事**：① `range` 的边界与步长；② 多分支的顺序；③ `break` 与 `for...else` 的配合；④ 素数 / 闰年 / 数字拆解三大模板；⑤ OJ 的输入读法。

二、方法总结（速查表）

语法 / 函数	作用	关键点（最容易错的地方）	典型例子
`if 条件:`	单分支	行尾冒号不能少，缩进 4 空格	`if n % 2 == 0:`
`if / elif / else`	多选一	条件从严格到宽松；最多执行一个分支	例题 2-2
`A if 条件 else B`	二选一取值	只能得到值，不能写多条语句	`y = 1 if x >= 0 else -1`
`条件1 and 条件2`	两个都要满足	`and` 优先级高于 `or`，配对建议加括号	`year % 4 == 0 and year % 100 != 0`
`in`	判断是否属于某个集合	比一串 `or` 更短、更不易错	`month in (4, 6, 9, 11)`
`while 条件:`	次数未知的循环	三要素：初值、条件、更新	例题 2-5

语法 / 函数	作用	关键点（最容易错的地方）	典型例子
`for i in range(n):`	循环 n 次	`i` 取 0 ~ n−1	累加 1~100
`range(a, b, k)`	等差序列	含 a 不含 b；k 可负（倒序）；k 不能为 0	`range(10, 1, -1)`
`break`	结束整个循环	**只跳出当前这一层**	找到即停
`continue`	跳过本轮	`while` 里注意别把"更新"跳过去	跳过 7 的倍数
`for / while ... else`	正常结束时执行	遇 `break` 不执行；空循环也会执行	例题 2-9 写法 2
`input().split()`	一次读一行多个值	返回字符串列表，要 `map(int, ...)` 转换	`a, b = map(int, input().split())`
`while True: ... break`	明确的"死循环"	必须有一条能走到的 `break`	读到 0 结束
`try / except`	出错不中断	写清楚异常类型，别用裸 `except:`	输入校验
`random.randint(a, b)`	随机整数	两端都含	`random.randint(1, 6)`
`random.randrange(a, b)`	随机整数	不含 b，和 `range` 规则一致	`random.randrange(0, n)`

三、易错清单

1. **错误现象**：`SyntaxError: expected ':'` → **原因**：`if / for / while` 行尾漏了冒号 → **正确写法**：`for i in range(5):`。看到 SyntaxError，先看报错行号的**上一行**。
2. **错误现象**：`IndentationError: unexpected indent` 或 `TabError` → **原因**：缩进层数不一致，或 Tab 和空格混用 → **正确写法**：同一层语句左对齐，每层统一 4 个空格，编辑器设"Tab 转空格"。
3. **错误现象**：`range(1, 5)` 只跑到 4 → **原因**：`range` 含头不含尾 → **正确写法**：`range(1, 6)`。凡是"到 N 为止"，右端写 N + 1。
4. **错误现象**：循环结束后用循环变量，拿到了奇怪的值 → **原因**：`for` 不产生新作用域，循环变量保留最后一次的值；一次都没循环时它根本不存在 → **正确写法**：需要保留的值在循环里存到专门的变量里，循环后不要复用同名变量。
5. **错误现象**：程序不报错、不结束，一直刷屏 → **原因**：`while` 的循环体里没有更新循环变量 → **正确写法**：把 `i += 1` 放进循环体，缩进和 `while` 同层。
6. **错误现象**：`while item != 0`（item 每次减 0.1）永远不退出 → **原因**：浮点误差，item 永远不会**恰好**等于 0 → **正确写法**：用整数控制循环次数，或改成 `while item > 1e-9:`；判断浮点相等一律用 `abs(a - b) < 1e-9`。
7. **错误现象**：`month == 4 or 6 or 9 or 11` 恒为真 → **原因**：`or` 连接的是四个"真值"，6、9、11 都视同 True → **正确写法**：`month in (4, 6, 9, 11)`。
8. **错误现象**：85 分被判定成 D → **原因**：多分支条件从宽到严排列，前面的分支"抢答"了 → **正确写法**：条件从严格到宽松，范围最窄的写在最前面。

9. **错误现象**：嵌套循环里 `break` 之后外层还在跑 → **原因**：`break` 只跳出当前这一层循环 → **正确写法**：加标志变量跳出两层，或把循环写进函数用 `return`（见 3.3 节）。
10. **错误现象**：`while` 里用 `continue` 之后程序卡死 → **原因**：`continue` 跳过了后面的 `i += 1` → **正确写法**：把更新写在 `continue` 之前，或改用 `for`。
11. **错误现象**：`for...else` 里的语句每轮都打印 → **原因**：`else` 缩进到了循环体内，和 `if` 配对了 → **正确写法**：`else` 与 `for` 左边缘对齐。
12. **错误现象**：多组测试数据只处理了第一组就结束 → **原因**：没写读 EOF 的循环 → **正确写法**：`for line in sys.stdin:`，或 `while True: try: ... except EOFError: break`（见 2.5 节模板 ④）。
13. **错误现象**：`'10' < '9'` 竟然是 True，比较大小全乱 → **原因**：`input()` 返回字符串，字符串按字符逐个比较 → **正确写法**：先 `int()` / `float()` 转换再比较。
14. **错误现象**：求最大值的程序在"输入全是负数"时输出 0 → **原因**：`max_num = 0` 起手，0 比所有负数都大 → **正确写法**：用第一个数据当擂主 `max_num = nums[0]`（见例题 2-12）。
15. **错误现象**：`except:` 之后连 Ctrl+C 都停不下来，bug 也被吞掉 → **原因**：裸 `except` 捕获所有异常，包括 `KeyboardInterrupt` → **正确写法**：写清楚异常类型，兜底用 `except Exception:`。

四、必须能默写的 10 个片段

1. 判断闰年

```
isLeapYear = (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
```

"四年一闰、百年不闰、四百年再闰"三句话的翻译，作业题 11、作业题 12 直接用。

2. 判断素数

```
isPrime = True
if number < 2:
    isPrime = False
else:
    for i in range(2, int(number ** 0.5) + 1):
        if number % i == 0:
            isPrime = False
            break
```

交作业前把边界 1、2、9 各试一遍。

3. 累加与计数

```
total = 0
count = 0
for i in range(1, n + 1):
    total += i
    if i % 2 == 0:
        count += 1
```

累加器初值 0、计数器初值 0、连乘初值 1，这是三个固定套路。

4. 求最大值（最小值）

```
max_num = nums[0]          # 用第一个数据当擂主，不要用 0
for x in nums:
    if x > max_num:
        max_num = x
```

5. 数字逐位拆解与反转

```
m = 0
while n != 0:
    m = m * 10 + n % 10     # 取个位接到末尾
    n //= 10                # 去掉个位
```

循环结束时 `m` 是 `n` 的反转数；只要每一位，把 `m = ...` 换成 `print(n % 10)` 即可。

6. 辗转相除求最大公约数

```
a, b = 24, 36
while b != 0:
    a, b = b, a % b
print(a)          # 12
```

依据是 `gcd(a, b) = gcd(b, a % b)`，比逐个试快得多。

7. 一行读入多个数

```
a, b = map(int, input().split())          # 恰好两个
nums = list(map(int, input().split()))    # 不定个数
```

8. 读到 EOF（多组数据）

```
import sys
for line in sys.stdin:
    if not line.strip():
        continue
    a, b = map(int, line.split())
    print(a + b)
```

```
while True:
    try:
        a, b = map(int, input().split())
    except EOFError:
        break
    print(a + b)
```

9. `while True` + `break`

```
while True:
    x = int(input())
```

```
    if x == 0:          # 出口必须写在最前面
        break
    print(x ** 2)
```

10. `for...else`

```
for i in range(2, number):
    if number % i == 0:
        print("不是素数")
        break
else:
    print("是素数")      # 只有"没被 break"才会执行到这里
```

`else` 与 `for` 必须左对齐。

2.10 本章自测（15 道）

第 2 章 自测题 1 写出下面程序的输出。

```
score = 85
if score >= 60:
    grade = 'D'
elif score >= 70:
    grade = 'C'
elif score >= 80:
    grade = 'B'
else:
    grade = 'A'
print(grade)
```

第 2 章 自测题 2 `range(2, 10, 3)` 产生的序列是什么？循环执行几次？

第 2 章 自测题 3 下面循环会输出什么？为什么？

```
for i in range(1, 5):
    if i == 3:
        continue
    print(i, end=' ')
```

第 2 章 自测题 4 下面两段代码分别输出什么？

```
# 代码 A
for i in range(2, 5):
    if i == 3:
        break
else:
    print("正常结束")

# 代码 B
for i in range(2, 5):
```

```
    if i == 9:
        break
else:
    print("正常结束")
```

第 2 章 自测题 5　为什么 `while item != 0`（item 每次减 0.1）会变成死循环？请给出两种改法。

第 2 章 自测题 6　用一行条件表达式把下面代码改写。

```
if x % 2 == 0:
    s = "偶"
else:
    s = "奇"
```

第 2 章 自测题 7　下面哪个判断月的写法是正确的？为什么？

```
month == 4 or 6 or 9 or 11
month == 4 or month == 6 or month == 9 or month == 11
month in (4, 6, 9, 11)
```

第 2 章 自测题 8　`try / except / else / finally` 四个子句分别在什么时候执行？

第 2 章 自测题 9　编程：输出 1~1000 之间的所有完数。完数是指一个数恰好等于它的所有真因子之和，例如 6 = 1 + 2 + 3。

第 2 章 自测题 10　编程：打印如下图形（n 行，n 由用户输入）。

```
*
**
***
****
*****
```

第 2 章 自测题 11　写出下面程序的输出，并解释最后打印出来的 `i` 为什么是 4 而不是 5。

```
s = 0
for i in range(1, 5):
    s += i
print(i, s)
```

第 2 章 自测题 12　下面程序的输出是什么？其中的 `break` 跳出了几层循环？

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            break
        print(i, j)
```

第 2 章 自测题 13　下面程序想统计 1~100 中 7 的倍数的个数，结果却一直不结束，问题出在哪里？请改正，并写出正确的结果。

```
i = 1
count = 0
while i <= 100:
    if i % 7 == 0:
        count += 1
print(count)
```

第 2 章 自测题 14　编程：一行输入若干个整数（用空格隔开，至少一个），输出其中的最大值与最小值，各占一行。

输入样例：`3 7 -2 9 4`；输出样例：`9` 和 `-2`（各一行）。

第 2 章 自测题 15　编程：输入正整数 n，求 1! + 2! + … + n! 的值（例如 n = 5 时，结果是 1 + 2 + 6 + 24 + 120 = 153）。

2.11 本章作业题

> 以下十二题来自你的作业截图（第 3~14 张），都属于"分支 + 循环"的练习。难题我会附上提示。

作业题 3｜水仙花数

题目描述：输入一个三位数，判断它是否为"水仙花数"。所谓"水仙花数"是指一个三位数，其各位数字立方和等于该数本身。

输入格式：一个三位（100~999 之间）整数。

输出格式：如果输入的是水仙花数则输出 `True`，不是水仙花数输出 `False`。

输入 #1	输出 #1	输入 #2	输出 #2
153	True	255	False

说明/提示：输出的首字母是大写。

> **考点**：本章第 2.7 节例题 2-11 的"逐位拆解"模板（`n % 10` 取个位、`n // 10` 去掉个位），配合 2.3 节的多分支。 **输入怎么读**：`n = int(input())`，一个整数一行。

作业题 4｜四位回文数

题目描述：输入一个四位的整数，判断这个数是不是回文数。回文数即一个数如果从左向右和从右向左读是一样的。

输入格式：一个四位（1000~9999 之间）整数。

输出格式：是回文数输出 `True`，不是回文数输出 `False`。

输入 #1	输出 #1	输入 #2	输出 #2
4386	False	1331	True

说明/提示：注意输出单词首字母大写。

> **考点**：本章第 2.7 节例题 2-11 的数字反转模板（四位数也可以直接拆成千、百、十、个位再拼回去）。
> **输入怎么读**：`n = int(input())`。

作业题 5｜判断两个直角三角形是否相似

题目描述：判断两个直角三角形是否相似。如果第一个直角三角形的两个直角边 (a,b) 比值等于另一个直角三角形两个直角边 (c,d) 的比值，则视为相似。注意两条边的输入顺序不能影响结果的判定，即判断条件为：a : b = c : d 或者 a : b = d : c。

输入格式：先后四次输入正整数，前两个为第一个直角三角形的两个直角边，后两个为第二个直角三角形的直角边。

输出格式：相似输出 `True`，否则输出 `False`。

输入 #1	输出 #1	输入 #2	输出 #2	输入 #3	输出 #3
1 2 3 6	True	1 2 6 3	True	1 2 3 4	False

说明/提示：为计算准确，能用乘法就不要用除法。判断条件转化为：`a*d == b*c` 或者 `a*c == b*d`。

> **考点**：本章第 2.3 节的复合条件（`and` / `or` 的组合）；"能用乘法就不用除法"是为了避开 1.7 节的浮点误差。 **输入怎么读**：样例是一行四个数，写成 `a, b, c, d = map(int, input().split())`；若 OJ 的数据是一行一个，就分四次 `int(input())`。

作业题 6｜符号交替求和

题目描述：按规则计算出 1 到 N 之间的结果，计算形如：1 + 2 − 3 + 4 − 5 + 6 − 7 ...，符号交替。

输入格式：程序运行时先输入一个正整数 N（N ≥ 1），计算时包含输入的 N。

输出格式：输出计算结果。

输入 #1	输出 #1	输入 #2	输出 #2
2	3	5	-1

说明/提示：前两项是加法，从第 3 项开始"减、加..."交替。可以自己先手算验证：1 + 2 = 3；1 + 2 − 3 + 4 − 5 = −1。

> **考点**：本章第 2.5 节的 `for` + `range()`，配上 2.9 节速查表里的累加器模板（`s = 0` 起手，循环里 `s += ...`）。 **输入怎么读**：`n = int(input())`。

作业题 7｜a + aa + aaa + ...

题目描述：求这样的一组数据和，s = a + aa + aaa + aaaa + ... 的值，其中 a 是一个数字，共有 n 个数字相加。例如 a = 2、n = 5，则 s = 2 + 22 + 222 + 2222 + 22222。

输入格式：两个数字，每个一行，第一个数字记录在 a 中（a 是 1~9 之间的数字），第二个数字记录在 n 中（n 是正整数）。

输出格式：符合上述公式的累加和值。

输入 #1	输出 #1
2	24690
5	（和值）

说明/提示：设当前项为 `t`，下一项就是 `t * 10 + a`，边生成边累加即可。

> **考点**：本章第 2.5 节的 `for` 循环 + 2.9 节必背片段第 3 条（累加器）；"边生成边累加"是这类题的通用姿势。 **输入怎么读**：两行两个数，注意顺序是先 a 后 n：先 `a = int(input())`，再 `n = int(input())`。

作业题 8｜显示一个整数的所有素因子

题目描述：显示一个大于 1 的整数的所有素因子。素因子即该因子必须是素数。若此数为素数，则仅显示此数本身即可。

输入格式：一个大于 1 的整数。

输出格式：升序显示该整数的所有素因子，即较小素因子的应该先输出，一个素因子一行。

输入 #1	输出 #1
120	2 / 2 / 2 / 3 / 5（每个一行）

说明/提示：从最小的因子 2 开始试除，除得尽就输出并继续除同一个因子；除不尽再试下一个因子。

> **考点**：本章第 2.7 节例题 2-9 的素数模板 + 2.4 节 `while` 循环（同一个因子要反复除）。 **输入怎么读**：`n = int(input())`。

作业题 9｜反素数

题目描述：反素数（逆向拼写的素数）是指一个将其逆向拼写后也是一个素数的**非回文数**（所谓回文数，就是说一个数字从左边读和从右边读的结果是一模一样的，例如 12321）。例如：17 和 71 都是反素数。

输入格式：两个大于 1 的整数，一个一行，前者不大于后者。求它们之间（包含这两个整数）的所有反素数。

输出格式：输出两个整数之间的所有反素数，每个反素数一行。

输入 #1	输出 #1
2	13
50	17 / 31 / 37（每个一行）

说明/提示：三步走——① 判断 n 是否为回文数，是则跳过；② 判断 n 是否为素数；③ 把 n 反转后判断是否也是素数。

> **考点**：本章第 2.7 节三件套——素数模板（例题 2-9）+ 数字反转模板（例题 2-11）+ `break` / `continue` 控制循环。 **输入怎么读**：两个数一行一个：先 `left = int(input())`，再 `right =`

> `int(input())`。

作业题 10｜点是否在圆内

题目描述：已知一个点的坐标 (x, y) 和一个圆（圆心固定为原点 (0,0)，半径 r），判断点是否在圆内。比较时，当两个浮点数之差的绝对值小于 0.000001 时可认为二者相等。两点坐标计算距离的公式为 d = $\sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$。

输入格式：输入三个数字（整数或小数），代表点的 x 坐标、y 坐标、圆的半径 r，一个数字一行。

输出格式：若在圆内输出 1；在圆上输出 0；在圆外输出 −1。

输入 #1	输出 #1	输入 #2	输出 #2
4 5 10	1	9 9 10	-1

说明/提示：本题考查的就是 1.7 节"浮点数不能直接用 == 比较"这一知识点，判断"在圆上"要用差的绝对值小于 1e-6。

> **考点**：本章第 2.3 节的多分支（先判"在圆上"，再判内、外）；浮点比较见 2.5 节和 2.9 节易错清单第 6 条。 **输入怎么读**：三个数一行一个，用 `x = float(input())` 这样读三次（有小数必须用 `float`）。

作业题 11｜距当月月底还有几天

题目描述：输入某日的日期，计算此日距当月月底还有几天。

输入格式：输入三个整数，分别为年、月、日，一行一个。年份输入的范围是 1793~9999 年。输入的日期为合法的日期。

输出格式：这一天距离当月月底还有多少天，**计算时不含输入的这一天**。

输入 #1	输出 #1	输入 #2	输出 #2
2023 2 26	2	2022 3 3	28

说明/提示：2023 是平年，2 月 28 天，26 日到月底（28 日）还有 2 天。别忘了闰年 2 月是 29 天。

> **考点**：本章第 2.3 节例题 2-3 的闰年模板 + 例题 2-4 的"月份天数"写法。 **输入怎么读**：三个整数一行一个：`year = int(input())`、`month = int(input())`、`day = int(input())`。

作业题 12｜这一天是当年的第几天

题目描述：找出任意一天是一年中的第几天。注意闰年的判断：世纪年（年份最后两位是 00 的）需要除得尽 400 才是闰年，非世纪年需要除得尽 4 才是闰年。注意：1 月 1 日为第 1 天。

输入格式：输入一个日期的年、月、日，一行一个。

输出格式：这一天是当年的第几天。

输入 #1	输出 #1	输入 #2	输出 #2
2000 8 31	244	2022 3 31	90

说明/提示：先把 1~12 月的天数放进一个元组，累加前 m−1 个月，再加上当月天数；闰年时给 2 月加 1。

> **考点**：本章第 2.3 节例题 2-3 的闰年模板 + 2.5 节的 `for` 累加（提示里的元组见第 4 章）。 **输入怎么读**：三个整数一行一个：`year = int(input())`、`month = int(input())`、`day = int(input())`。

作业题 13 | 三个整数升序输出

题目描述：用户输入三个整数，然后按其值从小到大（升序）显示它们。

输入格式：三个整数，一行一个。

输出格式：以升序显示三个整数，一行一个。

输入 #1	输出 #1
99	-8
-8	0
0	99

说明/提示：可以用 1.4 节的"同时赋值"做交换，比引入临时变量干净得多。

> **考点**：本章第 2.3 节的复合条件（`and` 连接两个比较），交换用 1.4 节的同时赋值 `a, b = b, a`。 **输入怎么读**：三个整数一行一个；也可以一行读：`a, b, c = map(int, input().split())`。

作业题 14 | 最大值及其出现次数

题目描述：读取若干个（至少 1 个）整数（正负都有可能，但不包含 0），找出其中的最大值，并计算它的出现次数。

输入格式：依次输入若干整数，一个一行，以输入数字 0 结束（0 仅表示结束，不作为要处理的数字）。

输出格式：输入的数字中的最大值，然后是该最大值出现的次数，一个一行。

输入 #1	输出 #1
3 / 4 / -2 / 5 / 2 / 5 / 0	5
（每行一个，0 结束）	2

说明/提示：典型的"先读一个当作当前最大值，之后每读一个就比一次"的写法；注意每读到一个新的更大值，计数要重置为 1。

> **考点**：本章第 2.4 节 `while` + 2.6 节 `break`；"先读一个当擂主"是 2.9 节必背片段第 4 条。 **输入怎么读**：`while True:` 里写 `x = int(input())`，读到 0 就 `break`（最大值和计数要在循环之前初始化）。

第 3 章 函数

> 对应课程讲义：《04_函数》。本章的核心思想是“不要重复自己”（DRY）。语法本身不难，难点是三个概念：**参数怎么传、变量在哪一层起作用、递归什么时候该用**。把这三件事想清楚，函数就算过关了。

3.1 为什么要使用函数

如果在开发程序时需要某块代码多次运行，为了提高编写效率和代码重用，就把具有独立功能的代码块组织为一个小模块，这就是函数。

先看不用函数的版本：

```
# 不使用函数：同一套逻辑被复制了三遍，改一处就得改三处
total = 0
for i in range(1, 11):
    total += i
print("Sum from 1 to 10 is", total)

total = 0
for i in range(28, 38):
    total += i
print("Sum from 28 to 37 is", total)

total = 0
for i in range(35, 50):
    total += i
print("Sum from 35 to 49 is", total)
```

```
# 使用函数：只写一次，反复调用
def add_range(i1, i2):
    result = 0
    for i in range(i1, i2 + 1):
        result += i
    return result

def main():
    print("Sum from 1 to 10 is", add_range(1, 10))
    print("Sum from 28 to 37 is", add_range(28, 37))
    print("Sum from 35 to 49 is", add_range(35, 49))

main()
```

> **【核心】DRY 原则**：Don't Repeat Yourself！停止复制 + 粘贴。函数带来的好处可以总结成四条： ① **复用**：写一次，到处调用； ② **可读**：`add_range(1, 10)` 一眼能看懂意图，不用逐行读循环； ③ **可改**：逻辑要改只动函数体一处，所有调用点同时生效； ④ **可测**：每个函数能单独验证对错，出错的排查范围小。

> **【注意】** 什么时候该把一段代码抽成函数？出现下面任意一条就可以动手：同一段逻辑写了第二遍；一段代码超过一屏；这段代码需要单独验证对错。
>
> **【易错】** 不要用内置函数名当自己的函数名或变量名。上面第一版把累加变量叫 `sum`，后面若再写 `def sum(...)`，就会把内置的 `sum()` 覆盖掉，之后再用 `sum([1, 2, 3])` 会报 `TypeError`。常见的名字黑名单：`sum` `max` `min` `list` `dict` `str` `len` `type` `input` `print`。

3.2 函数的定义与调用

```
def functionName(参数列表):
    # 函数体
```

```
def larger(num1, num2):
    if num1 > num2:
        result = num1
    else:
        result = num2
    return result

def main():
    i = 5
    j = 2
    k = larger(i, j)
    print("The larger number of", i, "and", j, "is", k)

main()
```

这段代码里有几个术语要分清：

术语	在例子中指	说明
函数头	`def larger(num1, num2):`	函数定义的第一行，末尾必须有冒号
函数名	`larger`	遵循 PEP 8，用小写加下划线
形参	`num1`、`num2`	定义时写的“占位变量”
实参	`i`、`j`	调用时真正传进去的值
返回值	`result`	`return` 交回调用处的值
函数体	缩进的那几行	函数被调用时才执行

调用一个函数时，Python 依次做四件事：

1. 先算出实参 `i`、`j` 的值；
2. 把值按顺序绑定给形参 `num1`、`num2`（相当于做了两次赋值）；
3. 执行函数体，遇到 `return` 立刻结束，把值交回；
4. 回到调用处，用返回值接着往下算。

> **【易错】** 函数必须先定义后调用。在 `def` 之前调用会报 `NameError: name 'f' is not defined`。
>
> **【注意】** 上面这个函数在教材里叫 `max`，这里改成了 `larger`。因为 `max` 是 Python 内置函数名，覆盖它只会给自己添麻烦。

文档字符串

函数的特别机制：文档字符串必须写在函数的**第一行**，用来描述函数功能，可以多行，也可以没有。

```
def ask_yes_no(question):
    """Ask a yes or no question."""
    response = None
    while response not in ("y", "n"):
        response = input(question).lower()
    return response

ask_yes_no("Are you kidding? ")
```

> **【速查】** 三引号文档字符串的标准长法，以及两种查看方式：

```
def add(a, b):
    """返回 a 与 b 的和。

    a、b 均为数字。
    """
    return a + b

help(add)          # 交互环境里用 help() 查看
print(add.__doc__) # 也可以直接读 __doc__
```

> **【注意】** PEP 8 命名规范：函数名、变量名用小写字母加下划线（`snake_case`，如 `count_vowels`、`is_leap_year`）；驼峰写法 `MaxValue` 一般留给类名（见第 8 章）。

3.3 函数的返回值

函数分为有返回值和无返回值两种。

- `input()` 有返回值，所以要写成 `s = input("Enter a word")`；
- `print()` 没有返回值，所以写成 `s = print('Hello')` 之后，s 里只会得到一个 `None`。

> **【核心】** 所有的函数都将返回一个值。如果没有写返回值，函数将返回一个特殊值：`None`。

```
def add(a, b):
    result = a + b      # 算完就结束了，没有 return

print(add(1, 2))        # None
```

函数会在运行结束后退出，也会在遇到 `return` 语句时退出（类似循环中的 break）。语法是 `return` 或 `return None`。

```
def printGrade(score):
    if score < 0 or score > 100:
        print("Invalid score")
        return              # 提前结束，等同于 return None
    if score >= 90.0:
        print('A')
    elif score >= 80.0:
        print('B')
    else:
        print('F')

printGrade(95)      # A
printGrade(200)     # Invalid score
```

return 与 print 的区别（初学者最糊涂的一点）

一句话记住：**print 是给用户看的，return 是给程序用的。**

对比项	`print(x)`	`return x`
干的事	把值显示到屏幕上	把值交回给调用者
屏幕上有输出吗	有	没有
函数自己的值	仍然是 `None`	就是 x
能不能接着参与运算	不能	能，`y = f(1) * 10`
记忆口诀	“到此一游”	“交货走人”

```
def add_print(a, b):
    print(a + b)          # 只打印，什么也没交回

def add_return(a, b):
    return a + b          # 只交回结果，屏幕上什么也没有

add_print(1, 2)           # 屏幕上出现 3
x = add_print(1, 2)       # 屏幕上出现 3，但 x 是 None
y = add_return(1, 2)      # 屏幕上什么也没有，y 是 3
print(x, y, y * 10)       # None 3 30
# print(x * 10)           # TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'
```

【核心】到底该 print 还是 return：这个结果后面还要用（参与运算、再传给别的函数、拼进别的字符串），就必须 `return`；只有“最后一步给人看”才用 `print`。**绝大多数函数应该只 return，不 print。**

上机时最常见的“数算对了，交上去却是 None”，病根就是函数里写的是 `print(result)`，调用处却写 `s = f(x)`。

【常见错误】 算完了却忘记 `return`，函数的值是 `None`。

```
# X 错误写法：只算了一下，没有交回去
def add(a, b):
    a + b

print(add(1, 2))     # None
```

```
# ✓ 正确写法
def add(a, b):
    return a + b

print(add(1, 2))     # 3
```

`a + b` 只是算了一个表达式就把结果丢掉；函数走到最后一行也没碰到 `return`，于是返回 `None`。

【常见错误】 `return` 后面的代码永远不会执行（死代码）。

```
# X 错误写法：想先打印再返回，顺序写反了
def check(n):
    return n > 0
    print("检查完毕")       # 永远执行不到

print(check(5))           # 只输出 True，提示不出现
```

```
# ✓ 正确写法：要打印的语句放在 return 之前
def check(n):
    print("检查完毕")
    return n > 0

print(check(5))           # 先打印"检查完毕"，再打印 True
```

`return` 一执行就立刻结束整个函数（比循环里的 `break` 更彻底），它后面的语句是死代码。写函数时发现某行不生效，先看它上面有没有 `return`。

有返回值与多个返回值

```
def getGrade(score):
    if score >= 90.0:
        return 'A'
    elif score >= 80.0:
        return 'B'
    else:
        return 'F'

print(getGrade(85))     # B
```

```
def sort2(number1, number2):
    if number1 < number2:
        return number1, number2
```

```
    return number2, number1

n1, n2 = sort2(3, 2)            # 用两个变量接：解包
numbers = sort2(5, 6)           # 用一个变量接：拿到元组
print(n1, n2)                   # 2 3
print(numbers, type(numbers))   # (5, 6) <class 'tuple'>
```

【核心】 `return a, b` 等价于 `return (a, b)`——所谓“函数返回多个值”，**其实返回的是一个元组。**接收时要么用同样个数的变量解包，要么用一个变量接住整个元组。

个数对不上就报错：

```
def three():
    return 1, 2, 3

a, b = three()      # ValueError: too many values to unpack (expected 2)
```

【易错】 用一个变量接多个返回值，拿到的是元组而不是列表，元组不能修改。想按位置取其中一个，用下标：`sort2(5, 6)[1]` 得到 6。元组与解包的完整规则见 4.4 节、4.8 节。

3.4 函数传参

位置参数与关键字参数

```
def birthday1(name, age):
    print("Happy birthday,", name, "!", " I hear you're", age, "today.\n")

birthday1("Jackson", 18)           # 位置参数：按顺序传
birthday1(18, "Jackson")           # 语法没错，但结果荒谬
birthday1(name="Jackson", age=18) # 关键字参数：按名字传
birthday1(age=18, name="Jackson") # 关键字参数可以任意顺序
```

【易错】 位置实参和关键字实参可以混用，但**位置参数不能出现在任何关键字参数之后**：

```
def func1(x, y, z):
    print("x =", x, "y =", y, "z =", z)

func1(3, 1, 4)          # x = 3 y = 1 z = 4
func1(3, z=4, y=1)      # x = 3 y = 1 z = 4
func1(3, z=4, 1)        # SyntaxError: positional argument follows keyword argument
```

【速查】 调用时的三种传参方式：

写法	规则	例子	什么时候用
全位置	按形参顺序一个个对	`f(1, 2, 3)`	参数少、含义明显
全关键字	顺序随便，名字要对	`f(z=3, x=1, y=2)`	参数多、含义容易混

写法	规则	例子	什么时候用
混合	位置在前，关键字在后	`f(1, 2, z=3)`	最常用的实际写法

默认值参数

```
def birthday2(name="Jackson", age=18):
    print("Happy birthday,", name, "!", " I hear you're", age, "today.\n")

birthday2()                         # 全部用默认值
birthday2(name="Katherine")
birthday2(age=20)
birthday2("Katherine", 20)
```

```
def printArea(width=1, height=2):
    area = width * height
    print("width:", width, "\theight:", height, "\tarea:", area)

printArea()                     # width: 1  height: 2   area: 2
printArea(4, 2.5)               # width: 4  height: 2.5 area: 10.0
printArea(height=5, width=3)    # width: 3  height: 5   area: 15
printArea(width=1.2)            # width: 1.2 height: 2  area: 2.4
```

【易错】 定义中设置默认值的参数**后面**的所有参数也必须设默认值。所以 `def func3(x=3, y=1, z):` 是语法错误。

【核心】 默认值只在 `def` 这一行**执行的那一次**求值，之后每次调用都复用同一个对象。这一点对数字、字符串没影响，对列表、字典就是大坑——见下面的【常见错误】。

【常见错误】 可变默认参数 `def f(lst=[])`：多次调用会共享同一个列表。

```
# ✗ 错误写法：三次调用追加进了同一个列表
def add_item(item, lst=[]):
    lst.append(item)
    return lst

print(add_item("a"))    # ['a']
print(add_item("b"))    # ['a', 'b']   ← 上一次的数据还在!
```

```
# ✓ 正确写法：默认值写 None，函数内部再新建列表
def add_item(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst

print(add_item("a"))    # ['a']
print(add_item("b"))    # ['b']
```

`def add_item(item, lst=[])` 里的 `[]` 在定义时就创建好了，之后每次调用只要不传 `lst`，用的都是这同一个列表。默认值写成 `None`、进函数后判断再新建，是“参数可选又互不影响”的标准写法。可变对象做参数的坑，4.12 节还会从列表中拷贝的角度再讲一次。

限制使用位置参数和关键字参数（了解）

```
def f1(a, b, /):
    return a + b

f1(2, 3)      # 正确
f1(a=2, b=3)  # 错误：/ 之前的参数只能按位置传

def f2(a, *, b, c):
    return a + b + c

f2(1, b=4, c=5)   # 正确
f2(1, 4, 5)       # 错误：* 之后的参数只能按关键字传

def f3(a, b, /, c, *, d, e):
    print(a, b, c, d, e)

f3(1, 2, c=3, d=4, e=5)
```

【了解】 `/` 左边的参数只能按位置传，`*` 右边的参数只能按关键字传。这是给自己写的库加“使用说明”用的，课程作业里几乎用不到，看到别人的代码里出现 `/` 和 `*` 不要慌。

参数的引用传递（重要）

```
def main():
    x = 1
    print("Before the call, x is", x)
    increment(x)
    print("After the call, x is", x)

def increment(n):
    n += 1
    print("\tn inside the function is", n)

main()
# Before the call, x is 1
# n inside the function is 2
# After the call, x is 1
```

【核心】 形参与实参之间是**引用传递**。如果实参是**不可变**数据类型（数字、字符串、元组），函数内不会修改实参的值；如果实参是**可变**数据类型（列表、字典、集合），函数内**可能修改**实参的内容。这个问题非常重要，第 4 章 4.12 节会重点展开。

【速查】 一眼判断“外面的变量会不会被函数改掉”：

函数里的写法	外面实参变不变	原因
`n += 1`（n 是数字、字符串、元组）	不变	不可变对象，形参改指向了新对象
`lst.append(x)` / `lst.sort()` / `lst.clear()`	**变**	原地修改同一个列表对象
`d["k"] = v`（d 是字典）	**变**	原地修改同一个字典
`lst = []` / `lst = lst + [x]`	不变	形参重新绑定到新对象
`return 新对象` 且调用处 `a = f(a)`	变（靠赋值）	不是参数改的，是返回值覆盖的

【常见错误】 以为在函数里写 `lst = []` 就能把外面的列表清空。

```
# ✗ 错误写法：外面一点没变
def reset(lst):
    lst = []

nums = [1, 2, 3]
reset(nums)
print(nums)          # [1, 2, 3]
```

```
# ✓ 正确写法一：原地清空
def reset(lst):
    lst.clear()

# ✓ 正确写法二：不改实参，把新对象交回去
def reset(lst):
    return []

nums = [1, 2, 3]
nums = reset(nums)   # 用返回值覆盖
print(nums)          # []
```

判据很简单：**形参出现在 `=` 左边，就是重新绑定**，和外面的实参再无关系；只有 `append` / `clear` / `sort` / `d[k] = v` 这类写法才是原地修改。

3.5 变量作用域

全局变量与局部变量

```
globalVar = 1

def func1(argVar):
    localVar = 2
    print(globalVar)   # 1
    print(localVar)    # 2
    print(argVar)      # 1

func1(globalVar)
```

```
print(globalVar)       # 1
print(localVar)        # ERROR: 函数外部访问不到局部变量
```

> **【了解】** 局部变量的作用域从创建变量的地方开始，直到包含该变量的函数结束为止。

从函数内部读写全局变量

可以从程序任何作用域中**读取**全局变量，但函数中**不可以直接修改**全局变量。

```
x = 1
def f():
    print(x)      # 1，读取没问题
f()

x = 1
def f():
    x = 2         # 这只是创建了一个同名的局部变量，屏蔽了全局变量
    print(x)      # 2
f()
print(x)          # 1

x = 1
def f():
    global x      # 用 global 才能真正修改全局变量
    x = 2
f()
print(x)          # 2
```

> **【核心】** 全局变量并不是好技术，尽量不要使用。对在多个函数中均需要使用的变量，可以使用“全局常量”（赋值后不再修改）。Python 并没有专门语法来指定常量，只能由程序员自行约束。

同名局部变量的屏蔽效应（易错）

```
x = 1
def f1():
    print(x)      # UnboundLocalError
    x = 2
f1()
```

> **【易错】** 只要函数体里出现了对 `x` 的赋值，Python 就把 `x` 当成局部变量，于是在赋值之前读取它就报错。这是初学者极容易踩的坑。
>
> **【常见错误】** `UnboundLocalError`：函数里又打印又赋值。
>
> ```
> # X 错误写法
> count = 0
>
> def show():
> ```

```
    print(count)     # UnboundLocalError: 局部变量 count 还没赋值
    count = 1
```

```
# ✓ 正确写法一：只读就别赋值
count = 0

def show():
    print(count)     # 0

# ✓ 正确写法二：靠参数和返回值传（推荐）
def show(count):
    return count + 1
```

函数体里只要出现 `count = 1`，Python 在**编译这个函数时**就把 count 登记成本函数的局部变量；执行到 `print(count)` 时这个局部变量还没赋值，于是报 `UnboundLocalError: cannot access local variable 'count' where it is not associated with a value`。老版本 Python 的提示是 `local variable 'count' referenced before assignment`，看到这句是同一个毛病。

嵌套函数与 nonlocal

```
x = 1
def outer():
    x = 2
    def inner():
        nonlocal x    # 声明"我要用外层的 x"
        x = 3
    inner()
    print("outer ==>", x)     # outer ==> 3

outer()
print("global ==>", x)        # global ==> 1
```

【了解】 `nonlocal` 声明的变量是向上一层变量域中查找并绑定；如果上一层不存在此变量，则继续向上查找，直到最外层的函数。

LEGB 规则（核心）

Python 的作用域按照从内到外的顺序分为四种，变量查找遵循 LEGB 规则：

1. **L（Local）** 局部作用域，函数内部定义的变量；
2. **E（Enclosing）** 嵌套作用域，外层函数的变量；
3. **G（Global）** 全局作用域，模块级别的变量；
4. **B（Built-in）** 内置作用域，Python 内置的变量和函数，如 `print`、`len`。

查找顺序就是 L → E → G → B，先找到谁就用谁；四层都没有，抛 `NameError`。

```
x = "global"                # G
```

```
def outer():
    x = "enclosing"          # E
    def inner():
        x = "local"          # L
        print(x)             # local
    inner()

outer()

def outer2():
    x = "enclosing"
    def inner():
        print(x)             # enclosing（本层没有，去 E 找）
    inner()

outer2()

print(len("abc"))            # 3（len 来自 B：内置作用域）
```

【核心】“赋值即声明”：函数里出现 `x = ...`，Python 就把 x 登记为**本函数的局部变量**，查找时第一站 L 里已经有这个名字（只是还没赋值），所以永远不会跑到 G 去取全局的 x。这就是 `UnboundLocalError` 的由来。想改外层的变量，必须显式声明 `global`（改模块级）或 `nonlocal`（改外层函数级）。

【速查】global 与 nonlocal 对比：

关键字	写在哪	让名字指向	典型场景
`global x`	函数内	模块级变量	函数里累加一个全局计数器（不推荐）
`nonlocal x`	内层函数内	外层函数的变量	闭包里的计数器
什么都不写	——	本函数的局部变量	默认情况，也是最好的写法

【核心】 只有模块、类、函数（def、lambda）以及推导式中才会引入新的作用域；`if/elif/else`、`try/except`、`for/while` 这些代码块**不会**引入新作用域，也就是说，这些语句内定义的变量在外部也可以访问。

```
sum = 0
for i in range(10):
    sum += i
print(i)      # 9，循环变量在循环外仍然存在

def test():
    a = 1
print(a)      # NameError，函数内的局部变量外部访问不到

lst = [x for x in 'python']
```

```
print(lst)    # ['p', 'y', 't', 'h', 'o', 'n']
print(x)      # NameError，推导式里的变量外部访问不到
```

> **【了解】** `globals()` 返回模块级变量的字典，`locals()` 返回当前作用域变量的字典，排查脚本时查看变量挺方便（作业里用不到）。

3.6 递归函数

递归函数就是直接或间接调用自身的函数。对某些用其它方法很难解决的问题，递归能给出很自然、直观、简单的解决方案。

```
def countdown(n):
    if n == 0:            # 停止条件
        print("Liftoff!")
        return
    print(n)
    countdown(n - 1)      # 自己调用自己

countdown(3)              # 3 2 1 Liftoff!
```

例题 3-1：计算阶乘

递归定义：`0! = 1`，`n! = n * (n-1)!`（n > 0）。

```
def factorial(n):
    if n == 0:                    # 基准情形（Base case）
        return 1
    else:
        return n * factorial(n - 1)   # 递归调用
```

```
# 用循环实现更简单、更高效
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

例题 3-2：斐波那契数列

数列从 0 和 1 开始，之后的每个数都是前两个数字之和。

```
def fib(index):
    if index == 0:            # 基准情形
        return 0
    elif index == 1:          # 基准情形
        return 1
    else:                     # 递推
        return fib(index - 1) + fib(index - 2)
```

递归写法看着优雅，代价却极大：

n	2	3	4	10	20	30	40	50
递归调用次数	3	5	9	177	21891	2692537	331160281	2075316483

```
# 用循环实现更加简单而且高效（这也是作业题 15 要求的写法）
def fib(index):
    a = 0
    b = 1
    for i in range(2, index + 1):
        a, b = b, a + b
    return b
```

例题 3-3：汉诺塔问题

有些问题的本质就是递归，不用递归很难解决。汉诺塔问题就是个典型：把 n 个大小互不相同的盘子从塔 A 移到塔 C，任何时候大盘都不能放在小盘上面，每次只能移动一个盘子且必须移动塔顶的盘子。

```
def moveDisks(n, fromTower, toTower, auxTower):
    if n == 1:                                  # 停止条件
        print("Move disk", n, "from", fromTower, "to", toTower)
    else:
        moveDisks(n - 1, fromTower, auxTower, toTower)
        print("Move disk", n, "from", fromTower, "to", toTower)
        moveDisks(n - 1, auxTower, toTower, fromTower)

moveDisks(3, 'A', 'B', 'C')
```

【核心】递归三要素：① 基准情形（什么时候不再递归）；② 递推关系（大问题怎么化小）；③ 必须用选择语句控制是否继续递归调用。三条缺一条，程序不是报错就是死循环。

【核心】递归深度有硬上限：Python 默认只允许 1000 层递归，超过就抛 `RecursionError: maximum recursion depth exceeded`。

```
import sys

print(sys.getrecursionlimit())   # 1000，当前上限
sys.setrecursionlimit(5000)      # 可以调大，但一般不要这么做
```

【注意】 `sys.setrecursionlimit()` 只是把“允许的层数”调大，实际的调用栈空间并没有变多；调得太大，程序可能不是报错而是直接崩溃。正确做法是**把递归改成循环**，而不是调大上限。

【注意】能用循环就别用递归：任何用递归解决的问题都可以用迭代（循环）解决。递归的好处是代码短、贴着数学定义；代价是每次调用都要压栈，又慢又占内存，还有深度限制。只有“问题本身就是递归的”（汉诺塔、树/图的遍历、分治）才值得用递归。

对比项	递归	循环
代码长度	短，贴近数学定义	稍长
运行速度	慢（每次调用都有开销）	快
内存	每层都占栈空间，深度受限	常数
出错风险	忘写基准情形就 `RecursionError`	忘记更新循环变量会死循环
适合	汉诺塔、树、分治、回溯	绝大多数重复计算
OJ 建议	题目要求时再用	**首选**

> **【了解】** 递归的执行过程可以理解成“先递推、后回归”：调用 `factorial(3)` 会一路压栈到 `factorial(0)` 得到 1，再一层层把结果乘回来。理解了这张“调用栈”的图，递归就不再神秘。如果递归会重复计算同一个子问题（如斐波那契），可以加缓存优化，见 3.11 节的例题 3-10。

3.7 匿名函数与高阶函数

lambda 表达式

```
def add1(x, y):
    return x + y

add2 = lambda x, y: x + y

print(add1(2, 3))   # 5
print(add2(4, 5))   # 9
```

lambda 的语法是 `lambda 参数列表: 表达式`：冒号后面只能写**一个表达式**，不能写语句（不能有 `return`、不能有赋值、不能有 `if` 语句，但可以用条件表达式）。

```
f = lambda x: x * 2
print(f(3))                     # 6
print(f.__name__)               # <lambda>，没有正式名字

g = lambda x: "正数" if x > 0 else "非正数"   # 条件表达式可以
print(g(-5))                    # 非正数
```

> **【核心】lambda 最实用的场合，是给排序的 `key` 参数当“临时小函数”**（`sorted` 和列表的 `sort` 详见 4.6 节）：

```
words = ["banana", "kiwi", "apple", "fig"]
print(sorted(words, key=len))                 # ['fig', 'kiwi', 'apple', 'banana']
print(sorted(words, key=lambda w: w[-1]))     # ['banana', 'apple', 'fig', 'kiwi']

students = [("Zhang", 92), ("Li", 85), ("Wang", 92)]
```

```
students.sort(key=lambda t: (-t[1], t[0]))    # 分数降序，姓名升序
print(students)   # [('Wang', 92), ('Zhang', 92), ('Li', 85)]
```

【注意】 `key` 接收的是一个**函数对象**，所以要写函数名（不加括号），或者写一个 lambda。

【常见错误】 把函数调用写进了 `key`。

```
# X 错误写法：len() 被当场调用，缺少参数
words = ["banana", "kiwi", "apple"]
print(sorted(words, key=len()))       # TypeError: len() takes exactly one argument (0
given)
```

```
# √ 正确写法：传函数名本身
words = ["banana", "kiwi", "apple"]
print(sorted(words, key=len))         # ['kiwi', 'apple', 'banana']
print(sorted(words, key=lambda w: len(w)))   # 等价写法
```

Python 把 `key=` 后面的东西当作“待会儿要调用的函数”存起来，所以不能先加括号把它调用掉。

高阶函数

在 Python 中，变量可以指向函数：

```
f = abs
print(f(-123))    # 123
```

既然变量可以指向函数，而函数的参数能接收变量，那么一个函数就可以接收另一个函数作为参数，这种函数就称之为**高阶函数**。

```
def add(a, b, func):
    return func(a) + func(b)

print(add(-10, 10, abs))                  # 20
print(add(-10, 10, lambda x: x ** 2))     # 200
```

【了解】 常用的高阶函数还有 `map`、`filter`、`sorted`、`max(key=...)`，简要用法见 4.6 节的速查表；这一节只要记住“函数也是对象，可以当参数传、可以当返回值”就够了。

3.8 变长参数与解包

接收任意数量的位置参数

```
def make_pizza(size, *toppings):
    """概述要制作的披萨"""
    print("Making a " + str(size) + "-inch pizza with the following toppings:")
    for topping in toppings:
        print("- " + topping)
```

```
make_pizza(16, 'pepperoni')
make_pizza(12, 'mushrooms', 'green peppers', 'extra cheese')
```

接收任意数量的关键字参数

```
def build_profile(first, last, **user_info):
    profile = {}
    profile['first_name'] = first
    profile['last_name'] = last
    for key, value in user_info.items():
        profile[key] = value
    return profile

user_profile = build_profile('Albert', 'Einstein',
                             location='Princeton', field='physics')
print(user_profile)
# {'first_name': 'Albert', 'last_name': 'Einstein',
#  'location': 'Princeton', 'field': 'physics'}
```

【核心】小结：`*args` 收集所有未匹配的位置参数组成一个 tuple；`**kwargs` 收集所有未匹配的关键字参数组成一个 dict。二者同时存在时，`*args` 一定在 `**kwargs` 之前。

星号才是关键，名字可以随便取（`*nums`、`**options` 都行），但惯例是写成 `args` 和 `kwargs`。

【速查】 形参的完整摆放顺序，从左到右不能乱：

顺序	写法	例子
1	普通位置参数	`def f(a, b)`
2	带默认值的参数	`def f(a, b=0)`
3	收集位置参数	`def f(a, *args)`
4	只能用关键字的参数	`def f(a, *, k)`
5	收集关键字参数	`def f(a, **kwargs)`
组合	全部放一起	`def f(a, b=0, *args, k=1, **kwargs)`

【易错】 `*args` 后面再写普通参数，那个参数就必须按关键字传：

```
def f(a, *args, k):
    return a, args, k

print(f(1, 2, 3, k=4))    # (1, (2, 3), 4)
# f(1, 2, 3, 4)           # TypeError: missing 1 required keyword-only argument: 'k'
```

调用时解包

```
numbers_tuple = ("1", "2")
numbers_dict = {'second': "22", 'first': "11"}

def print_str(first, second):
    print(first, second)

print_str(*numbers_tuple)   # 1 2
print_str(**numbers_dict)   # 11 22
```

> **【核心】** 定义时的 `*` 是“打包”，调用时的 `*` 是“拆包”，方向正好相反：
>
> - `f(*lst)` 把列表/元组拆成一个个位置参数；
> - `f(**d)` 把字典拆成 `键=值` 形式的关键字参数，**字典的键必须是字符串，而且要能和形参名对上**，否则报 `TypeError: f() got an unexpected keyword argument`。

3.9 闭包与装饰器

闭包

在一个外函数中定义了一个内函数，内函数里使用了外函数的局部变量，并且外函数的返回值是内函数的引用，这样就构成了一个**闭包**。

```
def outer(a):
    b = 10
    def inner():
        return a + b      # 用到了外函数的临时变量
    return inner          # 返回内函数的引用

demo = outer(5)
print(demo())             # 15
demo2 = outer(7)
print(demo2())            # 17
```

> **【了解】** 一般情况下函数结束时内部所有东西都会释放；但闭包是特殊情况，外函数发现自己的局部变量将来会被内部函数用到，就把这个局部变量绑定给了内部函数。

函数装饰器

装饰器是修改其它函数功能的函数，写在函数定义的前面，像一顶“帽子”。调用函数时，第一件事不是执行这个函数，而是把它作为参数传入它头顶上这顶“帽子”。

```
import time

def timeit(f):
    def inner(*args, **kwargs):
        t = time.time()
```

```
        r = f(*args, **kwargs)
        print(time.time() - t)
        return r
    return inner

@timeit
def f1(n):
    for i in range(n + 1):
        j = i
    return j

print(f1(1000000))
```

带参数的装饰器需要两层嵌套（了解即可）：

```
def say_hello(country):
    def wrapper(func):
        def deco(*args, **kwargs):
            if country == "China":
                print("你好!")
            elif country == "America":
                print("Hello!")
            return func(*args, **kwargs)
        return deco
    return wrapper

@say_hello("China")
def xiaoming():
    return "我是小明。"
```

【了解】 装饰器就是“把函数交给另一个函数包一层”的语法糖，本质是闭包 + `*args/**kwargs` + 一行 `@名字`。看懂上面两个例子的执行顺序即可，自己写装饰器是后续课程的内容。顺便注意 `inner` 里先 `f(*args, **kwargs)` 拿到结果、最后 `return r`——又一次用到了“print 只给人看、return 才交回值”。

3.10 生成器

在 Python 中，一边循环一边计算的机制称为**生成器（Generator）**。函数体中包含 `yield` 关键字的函数就是生成器函数。

```
def add():
    for i in range(10):
        yield i

g = add()
print(g)          # <generator object add at 0x...>
print(next(g))    # 0
print(next(g))    # 1
```

```
def gen():
    print('111111'); yield '111111'
    print('222222'); yield '222222'
    print('333333'); yield '333333'

g = gen()
next(g)           # 111111
next(g)           # 222222
next(g)           # 333333
next(g, 'over')   # 没有元素了，返回默认值 'over'
```

【核心】 每次 `next()` 只返回一次 yield 的值然后**暂停**，下一次 next() 从当前位置继续。如果没有元素可以迭代了还调用 next()，需给定默认值，否则报 `StopIteration`；如果在生成器函数中使用 `return`，则终止迭代且不能得到返回值。

```
# 生成器里的死循环也不会真的死，执行多少次 next 就返回多少个值
def way():
    i = 0
    while True:
        i += 1
        yield i

# 语法糖：yield from
def way2():
    yield from range(5)
```

【注意】 生成器是“一次性”的：遍历完就空了，第二次遍历拿不到任何元素。

```
g = (i for i in range(3))
print(list(g))    # [0, 1, 2]
print(list(g))    # []   ← 已经用完了
```

需要反复使用就先用 `list(g)` 存成列表，或者干脆写成普通函数返回列表。

【了解】 生成器最大的好处是省内存：`(x * x for x in range(10 ** 8))` 不会真的生成一亿个数，而是用一个算一个。数据量大又只需汇总时（如 `sum(x * x for x in data)`），它比列表推导式更合适。

3.11 函数应用示例

前面三个老例题（闰年、素数、最大公约数）在第 2 章都做过，这里换一个视角：**同一个问题，封装成函数之后好在哪、和循环版差在哪。**算法本身的推导请看第 2 章对应例题，本节不再重复。

例题 3-4：判断某年是否为闰年

要求：函数名 `isLeapYear`，形参 `year` 为整数，返回 True/False。置闰规则与循环版写法见 2.3 节例题 2-3。

```
def isLeapYear(year):
    return year % 400 == 0 or (year % 100 != 0 and year % 4 == 0)

print(isLeapYear(2000))    # True
print(isLeapYear(1900))    # False
print(isLeapYear(2024))    # True
```

输入 `2024`，输出：

```
True
```

> **要点**：循环版把“怎么算”和“怎么用”揉在一起；函数版把判断逻辑收进 `return` 一行里，以后凡是需要“某年有多少天”（2.4 节例题 2-4）、“两个日期差多少天”（2.5 节例题 2-6）的地方，直接调用 `isLeapYear(year)` 就行，不用再抄一遍判断条件。
>
> **易错**：函数里只 `print(True)` 而不 `return`，调用处拿到的是 `None`，`if isLeapYear(y):` 永远不成立。

例题 3-5：判断一个正整数是否为素数

要求：函数名 `isPrime`，参数 `number` 为大于等于 2 的整数，返回 True/False。循环版的三种写法见 2.7 节例题 2-9。

```
def isPrime(number):
    isPrime = True
    for i in range(2, number):
        if number % i == 0:
            isPrime = False
            break
    return isPrime
```

```
# 对照版：把 break 换成 return，代码更短
def isPrime(number):
    for i in range(2, number):
        if number % i == 0:
            return False
    return True
```

```
# 实用版：处理 n < 2 的情况，上界收到平方根
def isPrime(number):
    if number < 2:
        return False
    for i in range(2, int(number ** 0.5) + 1):
        if number % i == 0:
            return False
    return True

print(isPrime(2), isPrime(9), isPrime(97))    # True False True
```

> **要点**：**函数版最大的好处是能用 `return` 提前收工。**循环版要先用标志变量记下结论、`break` 跳出、循环结束后再判断；函数版一发现因数就 `return False` 走人，少一个变量、少一层缩进，也少一次“循环结束后忘记判断标志”的错误。
>
> **易错**：上界写成 `int(number ** 0.5) + 1`。如果写成 `math.ceil(number ** 0.5) + 1`，`range(2, 3)` 里正好有 2，素数 2 会被判成合数；`number < 2` 的兜底也别丢，否则 `isPrime(1)` 会返回 True。

例题 3-6：求两个正整数的最大公约数

要求：函数名 `gcd`，返回两个正整数的最大公约数。循环版三种写法见 2.7 节例题 2-10，作业题 16 的分数约分就要用它。

```
def gcd(num1, num2):
    i = min(num1, num2)
    while i > 0:
        if num1 % i == 0 and num2 % i == 0:
            return i
        i -= 1
```

```
# 写法 2：辗转相除法（欧几里得算法），快得多（见 2.7 节例题 2-10 写法 3）
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

print(gcd(12, 18), gcd(7, 13))    # 6 1
```

```
# 写法 3：同样的算法写成递归，最贴近数学定义
def gcd(a, b):
    return a if b == 0 else gcd(b, a % b)
```

> **要点**：同一个算法，循环版和递归版怎么选见 3.6 节的对比表。`gcd` 的递归深度是 O(log n)（参数每次至少减半），比阶乘、斐波那契温和得多，一般不会 `RecursionError`；但习惯上仍推荐写法 2 的循环版。
>
> **易错**：`gcd` 是典型的“给别的函数用”的工具函数——把它单独写出来，约分、判断互质、分数运算都能调用；如果把它塞进四则运算的 `if` 分支里，代码会立刻乱成一团。

例题 3-7：成绩统计（返回多个值其实是元组）

题目：输入第一行一个整数 n（n ≥ 1），接下来 n 行每行一个 0~100 的整数分数。输出最高分、最低分、平均分，用空格分隔，平均分保留两位小数。

```
def stat(nums):
    """返回 (最高分, 最低分, 平均分) 三元组."""
    return max(nums), min(nums), sum(nums) / len(nums)

n = int(input())
nums = [int(input()) for _ in range(n)]
```

```
hi, lo, avg = stat(nums)      # 用三个变量解包
print(hi, lo, f"{avg:.2f}")
print(stat(nums))             # 用一个变量接，看到的是元组
```

输入：

```
5
92
85
78
96
88
```

输出：

```
96 78 87.80
(96, 78, 87.8)
```

要点：`return a, b, c` 返回的是一个元组，`hi, lo, avg = stat(nums)` 是元组解包（4.8 节）。函数只负责“算”，输出格式交给调用者：想改成一行一个、想加百分号，都只动 `print` 那一行，统计逻辑一个字不用改。

易错：`max`、`min`、`sum` 都是内置函数，别把自己的函数也起名叫 `max` 或 `sum`。

例题 3-8：个数不定的数据统计（*args 与调用时解包）

题目：输入一行若干个整数（个数不定，至少一个），输出它们的和与平均值，平均值保留两位小数。要求统计函数用 `*args` 接收参数。

```
def total(*nums):
    """nums 是一个元组，传多少个数字都行."""
    return sum(nums), sum(nums) / len(nums)

print(total(1, 2, 3))          # 直接一个个传：(6, 2.0)

data = list(map(int, input().split()))
s, avg = total(*data)          # 调用时用 * 拆开
print(s, f"{avg:.2f}")
```

输入：

```
1 2 3 4 5
```

输出：

```
(6, 2.0)
15 3.00
```

> **要点**：定义处的 `*nums` 把散着的实参**打包**成元组，调用处的 `*data` 把列表**拆开**成一个个实参，两个星号方向相反，配起来很灵活——同一份数据，放在列表里能算，一个个传也能算。
>
> **易错**：`input().split()` 得到的是字符串，必须先 `map(int, ...)` 转成整数再求和，否则 `sum` 会对字符串报 `TypeError`。

例题 3-9：按分数排名（lambda 配 key）

题目：输入第一行一个整数 n，接下来 n 行每行“姓名 分数”。按分数从高到低排序后输出；分数相同时按姓名字典序升序。姓名只含英文字母，分数为整数。

```
n = int(input())
students = []
for _ in range(n):
    name, score = input().split()
    students.append((name, int(score)))

students.sort(key=lambda t: (-t[1], t[0]))    # 分数降序，姓名升序
for name, score in students:
    print(name, score)
```

输入：

```
3
Zhang 92
Li 85
Wang 92
```

输出：

```
Wang 92
Zhang 92
Li 85
```

> **要点**：`key` 接收的是一个函数，`lambda t: (-t[1], t[0])` 返回元组，Python 按元组从左到右比较：先比 `-分数`（取负号就等于降序），相同再比姓名。全部都要降序时可以直接写 `reverse=True`；只要有一列要升序，就得用这种“取负号 + 组元组”的技巧。
>
> **易错**：`students.sort(...)` 是原地排序、返回 `None`，别写 `students = students.sort(...)`；要新列表就用 `sorted(students, key=...)`（区别见 4.5 节、4.6 节）。

例题 3-10：记忆化递归求斐波那契的第 n 项

题目：输入一个整数 n（1 ≤ n ≤ 90），输出斐波那契数列的第 n 项（F(1) = 1，F(2) = 1）。

```
memo = {}                    # 缓存：算过的项存在这里

def fib(n):
```

```
    if n in memo:            # 算过就直接取，不再往下递归
        return memo[n]
    if n <= 2:
        result = 1
    else:
        result = fib(n - 1) + fib(n - 2)
    memo[n] = result
    return result

print(fib(int(input())))
```

输入 `10`，输出：

```
55
```

> **要点**：朴素递归的 `fib(40)` 要算三亿多次（见 3.6 节的表），加上缓存后每个 n 只算一次，`fib(90)` 也瞬间出结果。这种“把算过的结果存起来”的技巧叫**记忆化**，是递归优化的第一步；标准库的 `functools.lru_cache` 用一行装饰器就能做到同样的事（了解即可）。
>
> **注意**：作业题 15 明确要求用循环，不能用这个写法。递归即使加了缓存，仍有 O(n) 的调用深度和函数开销，n 一大就 `RecursionError`。

3.12 本章方法总结与易错清单

一、本章知识地图

- **3.2 定义与调用**（本章地基）：`def 函数名(形参):` → 函数体 → 调用 → 实参绑定形参；文档字符串、PEP 8 命名。
  - **3.3 返回值**：`return` 把值交回；不写就是 `None`；`return a, b` 是元组。**考试第一高频。**
  - **3.4 传参**：位置 → 关键字 → 默认值 → 引用传递；可变对象会被函数改掉，可变默认值只求值一次。
  - **3.5 作用域**：LEGB 查找顺序；函数内赋值 = 声明局部变量；`global` / `nonlocal`。
- **3.6 递归**：建立在“函数能调用自己”之上；三要素；深度上限 1000；能用循环就用循环。
- **3.7 lambda 与高阶函数**：函数也是对象，可以赋值、可以当参数传、可以当返回值。
- **3.8 变长参数与解包**：定义处 `*args` 打包、调用处 `*lst` 拆包，两个星号方向相反。
- **3.9 闭包与装饰器**（了解）：闭包 = 内层函数 + 外层变量；装饰器 = 闭包 + 语法糖。
- **3.10 生成器**（了解）：`yield` 一次产出一个值，省内存，但只能遍历一次。
- **3.11 应用示例**：把上面的东西组合起来解决 OJ 题（闰年、素数、最大公约数、成绩统计、排名、递归优化）。

考试最常考的五个点：① `return` 与 `print` 的区别；② 默认参数、关键字参数的写法规则；③ 可变对象作参数 / 可变默认值陷阱；④ 作用域与 `UnboundLocalError`；⑤ 递归三要素与“递归改循环”。

二、方法总结（速查表）

表 1：语法速查

写法	作用	返回值 / 效果	典型例子
`def f(a, b):`	定义函数	——	`def add(a, b): return a + b`
`f(1, 2)`	按位置调用	函数的返回值	`add(1, 2)` → 3
`f(b=2, a=1)`	按关键字调用	同上	顺序随便，名字要对
`return x`	交回结果并结束函数	x	`return a + b`
`return a, b`	返回元组	`(a, b)`	`lo, hi = sort2(3, 2)`
`return` / 不写	提前结束 / 自然结束	`None`	校验失败时直接 `return`
`def f(a, b=1):`	默认参数	不传就用 1	`printArea(4)`
`def f(*args):`	收集位置参数	元组	`total(1, 2, 3)`
`def f(**kw):`	收集关键字参数	字典	`build_profile('A', 'B', city='X')`
`f(*lst)` / `f(**d)`	调用时解包	——	`print_str(*numbers_tuple)`
`lambda x: x + 1`	匿名小函数	函数对象	`sorted(xs, key=lambda t: -t[1])`
`global x`	声明用模块级变量	——	函数里改全局变量（不推荐）
`nonlocal x`	声明用外层函数变量	——	闭包计数器
`yield x`	生成器的产出	生成器对象	`for i in gen(): ...`
`sys.getrecursionlimit()`	查看递归深度上限	默认 1000	`sys.setrecursionlimit(5000)` 可改

表 2：易混概念 PK

A	B	区别一句话	记忆例
`print(x)`	`return x`	给人看 vs 给程序用	`x = f()` 拿到的是 return 的值
位置参数	关键字参数	按顺序 vs 按名字	`f(1, b=2)` 合法，`f(a=1, 2)` 报错
`lst = []`（重新绑定）	`lst.clear()`（原地修改）	换个名字指向 vs 改同一个对象	外面的列表只对后者有反应
`global`	`nonlocal`	改模块级 vs 改外层函数级	只读时两个都不用写
递归	循环	优雅但慢、有深度上限 vs 快且稳	`fib(40)` 朴素递归要 3 亿多次调用
定义处的 `*args`	调用处的 `*lst`	打包成元组 vs 拆开成实参	方向相反，配着用
`sort()`	`sorted()`	原地改自己 vs 返回新列表	`sort` 返回 `None`

三、易错清单

1. **错误现象**：函数返回 `None`。→ **原因**：忘了写 `return`，函数体最后只有一个表达式。→ **正确写法**：补上 `return 表达式`，用 `print(f(1))` 自查。

2. **错误现象**：屏幕上明明打印出了结果，`x = f()` 里 x 却是 `None`。→ **原因**：把 `print` 当成了 `return`。→ **正确写法**：函数里用 `return`，需要显示时在调用处写 `print(f(...))`。
3. **错误现象**：`UnboundLocalError: cannot access local variable 'x' ...`（老版本提示 `local variable 'x' referenced before assignment`）。→ **原因**：函数体里出现了 `x = ...`，Python 就把 x 登记为局部变量，赋值之前读取就报错。→ **正确写法**：只读就别赋值；要靠参数和返回值传递；万不得已才 `global x`。
4. **错误现象**：同一个函数调用几次，列表越来越长。→ **原因**：可变默认参数 `def f(lst=[])` 的默认值只在定义时创建一次。→ **正确写法**：默认值写 `None`，函数内 `if lst is None: lst = []`。
5. **错误现象**：函数里写了 `lst = []`，外面的列表一点没变。→ **原因**：这只是让形参重新绑定到新列表，与实参无关。→ **正确写法**：要清空用 `lst.clear()`，要换新对象就 `return` 出去让调用处 `a = f(a)`。
6. **错误现象**：`return` 后面的语句不执行，提示语没打印。→ **原因**：`return` 立即结束整个函数。→ **正确写法**：把要在返回前做的事移到 `return` 之前。
7. **错误现象**：`f(1, y=2, 3)` 报 `SyntaxError: positional argument follows keyword argument`。→ **原因**：位置参数写在了关键字参数后面。→ **正确写法**：位置参数全部写在前面，关键字参数写在后面。
8. **错误现象**：`a, b = f()` 报 `ValueError: too many values to unpack (expected 2)`。→ **原因**："返回多个值"其实是返回元组，解包变量个数必须一致。→ **正确写法**：变量个数对齐，或者用一个变量接住整个元组。
9. **错误现象**：`RecursionError: maximum recursion depth exceeded`。→ **原因**：没写基准情形，或数据规模超过默认上限 1000 层。→ **正确写法**：补终止条件；优先改成循环；确实需要时才 `sys.setrecursionlimit(...)`（调大仍有崩溃风险）。
10. **错误现象**：函数里改了数字、字符串实参，外面没变。→ **原因**：数字、字符串、元组不可变，`n += 1` 只是让形参指向新对象。→ **正确写法**：用返回值，`n = f(n)`。
11. **错误现象**：生成器第二次遍历没有元素。→ **原因**：生成器是一次性的，遍历完就空了。→ **正确写法**：需要重复使用就先 `list(g)` 转成列表。
12. **错误现象**：`sorted(words, key=len())` 报 `TypeError`。→ **原因**：`key` 要的是函数对象，加括号等于先调用。→ **正确写法**：写 `key=len` 或 `key=lambda w: len(w)`。

四、必须能默写的 8 个片段

1. 最小函数骨架（定义 → 返回 → 调用）：

```
def add(a, b):
    """返回两数之和."""
    return a + b

print(add(1, 2))    # 3
```

1. 返回多个值 + 元组解包：

```
def min_max(nums):
    return min(nums), max(nums)
```

```
lo, hi = min_max([3, 1, 4])    # lo = 1, hi = 4
```

1. 可变默认值的安全写法：

```
def collect(item, box=None):
    if box is None:
        box = []
    box.append(item)
    return box
```

1. 判断型函数的“提前 return”模板：

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
```

1. 递归三要素模板：

```
def fact(n):
    if n == 0:                  # ① 基准情形
        return 1
    return n * fact(n - 1)      # ② 递推关系 ③ if 控制是否继续
```

1. `*args` / `**kwargs` 的标准签名：

```
def func(a, b=0, *args, **kwargs):
    print(a, b, args, kwargs)

func(1, 2, 3, 4, x=5)     # 1 2 (3, 4) {'x': 5}
```

1. lambda 配 `key` 的多关键字排序：

```
students = [("Wang", 92), ("Li", 85), ("Zhang", 92)]
students.sort(key=lambda t: (-t[1], t[0]))
print(students)    # [('Wang', 92), ('Zhang', 92), ('Li', 85)]
```

3.13 本章自测（14 道）

第 3 章 自测题 1　下面函数调用后输出什么？为什么 x 的值没有变？

```
def increment(n):
    n += 1
    return n
```

```
x = 1
increment(x)
print(x)
```

第 3 章 自测题 2　下列哪一行会报错？为什么？

```
def f(x, y, z=3):
    return x + y + z

f(1, 2)
f(1, 2, 3)
f(x=1, y=2)
f(1, y=2, 3)
```

第 3 章 自测题 3　写出下面程序的输出。

```
x = 10
def f():
    x = 20
    print(x)
f()
print(x)
```

第 3 章 自测题 4　下面程序执行时会报什么错？怎么改？

```
x = 1
def f():
    print(x)
    x = 2
f()
```

第 3 章 自测题 5　写一个函数 `count_vowels(s)`，返回字符串 s 中元音字母（aeiou，不区分大小写）的个数。

第 3 章 自测题 6　写一个函数，接收任意多个数字，返回它们的最大值；若没有参数则返回 None。

第 3 章 自测题 7　用递归写一个函数求 1+2+...+n；再用循环写一遍，比较两者的写法。

第 3 章 自测题 8　解释 `fib(40)` 用递归写为什么会非常慢，而用循环写却瞬间完成。

第 3 章 自测题 9　下面代码的输出是什么（注意 LEGB 规则）？

```
def outer():
    x = 1
    def inner():
        nonlocal x
        x += 1
        return x
    print(inner(), inner(), x)
outer()
```

第 3 章 自测题 10　写一个函数 `stat(nums)`，接收一个数字列表，返回三个值：最大值、最小值、平均值。

第 3 章 自测题 11　下面程序输出什么？为什么？

```
def add(item, lst=[]):
    lst.append(item)
    return lst

print(add(1))
print(add(2))
print(add(3))
```

第 3 章 自测题 12　下面程序输出什么？变量 x 的值是什么？

```
def f():
    print("hello")

x = f()
print(x)
```

第 3 章 自测题 13　已知 `scores = [("Zhang", 92), ("Li", 85), ("Wang", 92)]`，用一行代码得到“按分数从高到低、分数相同按姓名升序”的**新列表**，并写出结果。

第 3 章 自测题 14　写一个递归函数 `to_bin(n)`，把十进制正整数 n 转成二进制字符串（要求 `to_bin(13)` 得到 `'1101'`）；再说明：如果 n 非常大（例如 `n = 10 ** 5000`），这段递归会出什么问题，怎么改。

3.14 本章作业题

作业题 15 | 斐波那契数列（第 m 项到第 n 项）

题目背景：斐波那契数列由意大利中世纪数学家斐波那契（Fibonacci，公元 1175—1240）在 1202 年著作的《算盘书》（Liber Abaci）中以兔子繁殖为例引出。书中提出问题：假设一对刚出生的小兔一个月后就能长成大兔，再过一个月就能生下一对小兔，并且此后每个月都生一对小兔，一年内没有发生死亡，那么一对刚出生的兔子在一年内繁殖成多少对兔子？由该繁衍规律得到的每个月兔子的数量组成了斐波那契数列，因此也称为"兔子数列"。

题目描述：斐波那契数列：1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, … 如果设 F(n) 为该数列的第 n 项（n∈N*），那么数列可以写成如下形式：F(n) = F(n−1) + F(n−2)，初始条件 F(1) = 1，F(2) = 1。

输入格式：整数 m 和 n（2 ≤ m ≤ n），一行一个。

输出格式：斐波那契数列从第 m 项到第 n 项，一行一个。

输入 #1	输出 #1
6	8
10	13 / 21 / 34 / 55（每个一行）

说明/提示：**本题不可以使用递归函数实现**，应该使用循环实现，否则有可能导致递归过深、时间超时或内存溢出。

> **考点**：本章 3.6 节“递归与循环的取舍”对比表、3.12 节易错清单第 9 条；本题明确禁用递归，必须用循环实现。 **输入怎么读**：m 和 n 各占一行，用两次 `int(input())` 读入；从第 m 项算到第 n 项，每算出一项就 `print` 一次。

作业题 16 | 两个分数的四则运算

题目描述：计算二个分数的四则运算结果。

1. 输出结果应该是约分过的最简分数，如 10/12 应该输出为 5/6。提示：正分数约分为最简分数时，分子分母同时除以它们的最大公约数即可。
2. 如果输出结果的分数为负值，则负号标在分子上，如 −3/5，输出 −3 和 5。
3. 如果要输出结果为 0 时，分母输出 1。
4. 如果要结果是个整数时，分母输出 1。
5. 输入的两个分数也遵守上述规则。

输入格式：共输入 5 个数字，分别是：第 1 个分数（分别输入分子和分母）、运算符（1 代表加法，2 代表减法，3 代表乘法，4 代表除法）、第 2 个分数（分别输入分子和分母），每个数字一行。

输出格式：计算两个分数的四则运算结果，以分数形式输出，分别输出分子分母，一个一行。

输入 #1	输出 #1
2	5
3	12
2	（分母）
1	
4	

说明/提示：本例含义为 2/3 − 1/4，结果输出 5/12。

> 提示思路：约分需要最大公约数，所以先写一个 `gcd(a, b)` 函数；四则运算的公式分别是 a/b + c/d = (a·d + c·b)/(b·d)，减法把加号换成减号，乘法是 (a·c)/(b·d)，除法是 (a·d)/(b·c)。运算完再约分，并处理"分母为 1""分子为 0""负号在分子"这三种情况。
>
> **考点**：本章 3.11 节例题 3-6（把最大公约数封装成 `gcd()` 函数）、3.3 节的 `return` 与 3.4 节的参数传递。 **输入怎么读**：5 个数字每个占一行，用 5 次 `int(input())` 依次读入；注意第 3 个数是运算符（1 加、2 减、3 乘、4 除）。

第 4 章 内置组合数据类型

> 对应课程讲义：《05_内置组合数据类型》。这一章内容多、方法多，是期末考试的绝对重点。请重点掌握：**序列的索引与切片、列表的增删改、字典的键值对操作、推导式。**

本章的方法特别多，硬背是背不完的。给你一个统一的自查法：遇到任何一个方法，先问三句话。

1. 它是**原地修改**原来的对象，还是**返回一个新对象**？
2. 它有没有**返回值**？（很多方法返回的是 `None`）
3. 这个类型是**可变**的还是**不可变**的？

把这三句话挂在嘴边，本章一半的坑都能自己绕开。本章末尾 4.13 节有完整的方法速查表和易错清单，做作业前先翻一翻。

4.1 总览：五种内置组合类型

类型	是否可变	是否序列	说明
字符串 str	不可变	是	以字符组成的序列
元组 tuple	不可变	是	以任意数据类型组成的序列
列表 list	**可变**	是	以任意数据类型组成的序列
字典 dict	**可变**	否	键值对集合
集合 set	**可变**	否	不重复且无序的元素集合

三种序列（字符串、元组、列表）共享很多共性操作：

- 加法运算符 `+`、乘法运算符 `*`、成员运算符 `in` / `not in`；
- 内置函数 `len()`、`max()`、`min()`、`sum()`（sum 不适用于字符串）；
- 索引（index）与切片（slice）。

> **【核心】** 判断"可变"只问一句话：能不能**原地**改动其中一个元素？列表、字典、集合能；字符串、元组不能。

"可变 / 不可变"是本章的总纲，它决定了三件事：能不能按索引赋值、切片拿到的是副本还是别名、传进函数之后会不会被改掉。详见 4.3 节和 4.12 节。

本章的学习路线：序列通用操作（4.3 节）→ 元组（4.4 节）→ 列表（4.5 节，最重）→ 遍历与统计的惯用法（4.6 节）→ 推导式（4.7 节）→ 解包（4.8 节）→ 嵌套序列（4.9 节）→ 字典（4.10 节）→ 集合（4.11 节）→ 函数与可变对象（4.12 节）。

> **【注意】** 字符串是第 7 章的主角，本章只讲它作为"序列"的那一部分（构造、转义、比较、索引与切片）；`split`、`join`、`strip`、`find`、格式化等字符串方法的主场在第 7 章，本章只在 4.6 节放一张速查表。

4.2 字符串基础

Python 并不区分字符和字符串，即使只是一个字符，也被当做字符串处理。用一对单引号或双引号即可构造字符串。注意引号本身不是字符串的一部分，所以 `'abc'` 只有 a、b、c 三个字符。

转义字符（速查）

转义字符	说明
`\n`	换行符，光标移到下一行开头
`\r`	回车符，光标移到本行开头
`\t`	水平制表符（Tab 键），一般相当于四个空格
`\\`	反斜线本身
`\'`	单引号
`\"`	双引号
`\ooo`	3 位八进制数对应的字符，如 `'\141'` 就是 `'a'`
`\xhh`	2 位十六进制数对应的字符，如 `'\x61'` 就是 `'a'`
`\uhhhh`	4 位十六进制数表示的 Unicode 字符，如 `'\u4e2d'` 就是 `'中'`

换行符的跨平台差异：

操作系统	换行符	转义字符	数值
Unix/Linux	LF	`\n`	0x0A
Mac（旧）	CR	`\r`	0x0D
Windows/DOS	CR/LF	`\r\n`	0x0D0A

可以用 `os.linesep` 查询当前操作系统使用的换行符。

原始字符串与多行字符串

```
print("hello\nworld")     # 输出两行
print(r"hello\nworld")    # hello\nworld，加了 r 就不转义
```

```
print('''
Python 的三引号允许字符串跨多行，
包括换行符、制表符和其它特殊字符。
''')
```

字符串的比较

字符串用 `==` 判断是否相等、`!=` 判断是否不等。比大小时，两个字符串的字符逐一对齐比较，依据是字符的 Unicode 编码；如果前面部分一样，则较长者较大。

```
'apple' < 'banana'        # True，第一个字符 'a' < 'b'
'apple' < 'application'   # True，第五个字符 'e' < 'i'
'apple' < 'applepie'      # True，前五个字符相同，后者更长
```

【易错】 字符串比大小比的是**字符的 Unicode 编码**，不是数值大小，所以 `'10' < '9'` 是 `True`。OJ 题里 `input()` 读进来的永远是字符串，想按数值比较必须先转换：

```
a, b = input(), input()      # 依次输入 10 和 9
print(a > b)                 # False  字符串比较，先比第一个字符 '1' < '9'
print(int(a) > int(b))       # True   转成整数比较才是你要的结果
```

4.3 序列通用操作

加法、乘法与成员运算

```
"hello" + "world"     # 'helloworld'，连接，不会自动加空格
"hello" * 3           # 'hellohellohello'，重复 3 遍

s = 'world'
print('wo' in s)      # True
print('wd' in s)      # False
print('e' not in s)   # True
```

【注意】 `+` 只能用于**同类型**的序列：`"ab" + "cd"`、`[1] + [2]`、`(1,) + (2,)` 都可以，但 `[1] + (2,)` 会报 `TypeError`，要先转换。重复用 `*`，`[0] * 5` 得到 `[0, 0, 0, 0, 0]`，这是初始化列表最常用的写法。

长度、最大最小

```
message = "Hello world!\n你好，世界！"
print(len(message))   # 19，一个汉字算一个字符

s = 'world'
print(max(s), min(s)) # w d
```

`len()` 数的是元素个数（汉字、空格、标点都算 1 个）。`max()` / `min()` 用在字符串上，返回的是编码最大 / 最小的**字符**，不是最长的单词；`sum()` 只能用于数字序列。

索引

序列中的每个元素都有两个位置编号：从左向右是 `0, 1, …, n-1`；从右向左是 `-1, -2, …, -n`。

```
+---+---+---+---+---+---+
| p | y | t | h | o | n |
+---+---+---+---+---+---+
```

```
  0   1   2   3   4   5
 -6  -5  -4  -3  -2  -1
```

```
s = 'python'
for c in s:                             # 只要元素
    print(c)
for i in range(len(s)):                 # 只要下标（写法偏旧）
    print(s[i])
for i, c in enumerate(s, start=1):      # 下标和元素都要（推荐，详见 4.6 节）
    print(i, c)
for c in reversed(s):                   # 反向遍历
    print(c)
```

> **【易错】** 下标从 **0** 开始，最后一个元素是 `a[-1]` 或 `a[len(a) - 1]`。写成 `a[len(a)]` 一定报 `IndexError: list index out of range`。题目里说"第 1 个"时，先想清楚要不要减 1。

切片（核心）

格式：`序列[起点:终点:步长]`，即 `s[a:b:k]`。步长 k 和第 2 个冒号可省略，默认 k=1；起点 a 和终点 b 也可省略，但**第 1 个冒号不可省略**。

写法	含义
`s[a:b]`	取下标大于等于 a 但小于 b 的元素（从 a 到 b−1）
`s[a:]`	从下标 a 到结尾
`s[:b]`	从下标 0 到 b−1
`s[:]`	全部元素，即副本
`s[a:b:k]`	k>0 时按间隔 k 正序取；k<0 时按间隔 k 倒序取
`s[::-1]`	反转整个序列

```
s = 'python'
print(s[1:4])     # yth
print(s[-4:4])    # th
print(s[-2:-4])   # ''   空字符串
print(s[3:])      # hon
print(s[:-3])     # pyt
print(s[5:10])    # n    越界不会报错
print(s[1:5:2])   # yh
print(s[:])       # python
print(s[::-1])    # nohtyp
```

> **【核心】** 切片是**含头不含尾**的，即 `s[a:b]` 包含 a 不包含 b。**索引越界报错，切片越界不报错**（自动截断），所以 `s[5:10]` 这种"取到末尾"的写法是安全的，也是最常用的简写。`s[::-1]` 反转序列更是必须记牢的写法。

反转一个序列有 4 种写法，作业和考试都会遇到：

写法	是否原地修改	适用范围	说明
`s[::-1]`	否，返回新序列	字符串 / 元组 / 列表	最短，最常用
`list(reversed(s))`	否，返回新列表	任何序列	`reversed` 返回的是迭代器
`s.reverse()`	**是**	只有列表	没有返回值
`''.join(reversed(s))`	否	只有字符串	得到反转后的字符串

```
s = 'python'
print(s[::-1])                  # nohtyp
print(''.join(reversed(s)))     # nohtyp
print(s[::-1] == s)             # False，回文判断就写 s == s[::-1]

a = [1, 2, 3]
a.reverse()                     # 原地反转，不能写成 a = a.reverse()
print(a)                        # [3, 2, 1]
```

不可变性：整体替换 vs 部分修改（核心）

不可变性不是说变量不能换值。**整体替换**（让变量指向一个新对象）永远允许；被禁止的是**部分修改**（通过索引或切片改动对象内部）。

```
s = 'python'
s = 'java'          # 正确：整体替换，s 指向了新字符串
s[0] = 'j'          # 错误：TypeError，字符串不支持按索引赋值

s = 'python'
r = ''
for c in s:
    r = c + r       # 正确：拼接出新字符串
print(r)            # nohtyp
```

> **【核心】** 记住这条总纲，元组和字符串的"不可变"就不难理解：`t = (7, 8, 9)` 是整体替换，允许；`t[0] = 7` 是部分修改，禁止（详见 4.4 节）。列表、字典、集合三种类型两种都允许，所以在 4.5 节要额外小心"别名"和"拷贝"的问题。

4.4 元组 tuple

元组以小括号为限界标志，可以含有任意类型的元素，例如 `(1, "hello", True)`。

```
t = (1, "hello", True)
t = tuple('hello')          # ('h', 'e', 'l', 'l', 'o')
t = tuple(range(1, 10))     # (1, 2, 3, 4, 5, 6, 7, 8, 9)
t = ()                      # 空元组
t = tuple()                 # 空元组
```

```
t = (1,)    # 只有一个元素的元组，逗号不可省略
t = (1)     # 这只是一个整数 1，不是元组！
```

> **【易错】** `t = (1)` 得到的是整数 1 而不是元组，必须写成 `t = (1,)`。空元组写 `()` 或 `tuple()` 都行。

元组的常用操作与字符串一致：`len()`、`max()`、`min()`、`sum()`、`in`、`not in`、索引、切片、`+`、`*`、比较运算。切片规则与 4.3 节完全相同，这里不再重复。

```
t = (1, -2, 0, 3, -4)
print(len(t), max(t), min(t), sum(t))   # 5 3 -4 -2
print(t[1])                             # -2
print(t[1:4])                           # (-2, 0, 3)
print(t[::-1])                          # (-4, 3, 0, -2, 1)
print((1, 3, 5) + (2, 4, 6))            # (1, 3, 5, 2, 4, 6)
```

两个元组内置方法：

```
fruits = ('orange', 'apple', 'pear', 'banana', 'kiwi', 'apple', 'banana')
print(fruits.index('banana'))      # 3
print(fruits.index('banana', 4))   # 6，从下标 4 开始找
print(fruits.count('apple'))       # 2
print(fruits.count('tangerine'))   # 0
```

> **【易错】** `index()` 在找不到元素时会抛出 `ValueError` 异常，`count()` 找不到则返回 0。
>
> **【注意】** 元组不可变，说的同样是"不能部分修改"（详见 4.3 节）。整体替换、以及内部可变元素本身被修改，都是允许的：

```
t = ([1, 3], 2, 4)
t[0][1] = 5          # 正确：元组里的列表本身是可变的
print(t)             # ([1, 5], 2, 4)
t[1] = 9             # 错误：TypeError，元组本身不可变
```

既然列表功能更多，为什么还要用元组？

1. 元组比列表快，占内存更小；
2. 元组表示"不该被改动"的数据，比如坐标、日期、RGB 颜色；
3. 有些地方**必须**用元组：字典的键（4.10 节）、函数一次返回多个值（4.8 节）。

4.5 列表 list

列表和元组一样都是序列，区别在于列表是**可变**数据类型。列表是本章的重中之重。

```
list1 = []                                          # 空列表
list2 = [1]                                         # 一个元素不必加逗号
list3 = [2, 3, 4]
list4 = ['python', 3.14, 666, False, (3, 6, 9), [4, 6, 8]]
```

```
list1 = list()                # 空列表
list2 = list([2, 4, 6])       # [2, 4, 6]
list3 = list(('red', 'green', 'blue'))
list4 = list(range(3, 6))     # [3, 4, 5]
list5 = list('python')        # ['p', 'y', 't', 'h', 'o', 'n']
list6 = list(1, 2, 3)         # 错误：list() 的参数必须是可迭代对象
```

【易错】 列表和元组不能直接相加：`[1,2,3] + (4,5,6)` 会报 `TypeError`。需要先转换：`a + list(b)`。

别名、浅拷贝与深拷贝（核心·易错）

这是全章最容易出错的地方。先看清楚一个等号做了什么：

```
a = [1, 2, 3]
b = a              # 没有复制！b 只是 a 的另一个名字（别名）
b.append(4)
print(a)           # [1, 2, 3, 4]   a 也变了
```

写法	名称	说明
`b = a`	别名（引用赋值）	a、b 是同一个列表，改谁都一样
`b = a.copy()`	浅拷贝	新的外层列表，元素与原列表共享
`b = a[:]`	浅拷贝	等价写法
`b = list(a)`	浅拷贝	等价写法
`b = copy.copy(a)`	浅拷贝	需要 `import copy`
`b = copy.deepcopy(a)`	深拷贝	递归复制每一层，代价最高

```
import copy
a = [[1, 2], [3, 4]]
b = a.copy()               # 浅拷贝：只复制第一层
b[0].append(9)             # 改的是内层列表，而内层列表是共享的
print(a)                   # [[1, 2, 9], [3, 4]]   a 被带坏了！
print(b)                   # [[1, 2, 9], [3, 4]]

c = copy.deepcopy(a)       # 深拷贝：每一层都是新的
c[0].append(100)
print(a)                   # [[1, 2, 9], [3, 4]]   a 不变
print(c)                   # [[1, 2, 9, 100], [3, 4]]
```

【核心】 一句话记牢：**`=` 不复制，`copy()` 只复制一层，嵌套结构要 `deepcopy`**。元素全是数字、字符串的"平铺"列表，浅拷贝就够了，不必用 `deepcopy`。

【常见错误】 以为 `b = a` 是"复制一份"，结果改 b 却把 a 也改了。

```
# ✗ 错误写法
a = [1, 2, 3]
b = a
b.append(4)
print(a)      # [1, 2, 3, 4]  a 被意外修改
```

```
# ✓ 正确写法
a = [1, 2, 3]
b = a.copy()  # 浅拷贝，也可以写 b = a[:] 或 b = list(a)
b.append(4)
print(a)      # [1, 2, 3]
print(b)      # [1, 2, 3, 4]
```

列表、字典、集合赋值给变量时存的都是"引用"。如果元素本身又是列表（嵌套），`.copy()` 还不够，要 `import copy` 后用 `copy.deepcopy(a)`，详见 4.13 节易错清单第 3、4 条。

理解列表的可变性

列表可以通过索引、切片、列表方法**增加、删除或替换**部分元素，这是它和字符串、元组的根本区别（不可变性的总纲见 4.3 节）。

```
a = [1, 3, 5, 7, 9]
a[1] = 6
print(a)          # [1, 6, 5, 7, 9]

a = [1, 3, 5, 7, 9]
a[1:3] = [6]      # 用 1 个元素替换 2 个元素，列表变短
print(a)          # [1, 6, 7, 9]

a[1:1] = [0, 0]   # 切片赋值还能"插入"，长度可以变
print(a)          # [1, 0, 0, 6, 7, 9]
```

【易错】 索引赋值只能修改**已有**的元素，`a[10] = 1` 会报 `IndexError`；而切片赋值可以改变列表长度，两者不要混为一谈。空切片 `a[1:1]` 赋值就是"在下标 1 处插入"。

增加元素：append / insert / extend

```
list1 = [1, 2, 3]
list1.append(6)          # [1, 2, 3, 6]                末尾追加一个元素
list1.insert(1, 8)       # [1, 8, 2, 3, 6]             在下标 1 处插入
list1.extend([5, 7, 9])  # [1, 8, 2, 3, 6, 5, 7, 9]    把另一个可迭代对象逐个追加
```

【易错】 `append` 把参数**当成一个元素**整个加进去，`extend` 把参数**拆开**逐个加进去，`+` 则生成一个新列表：

```
a = [1, 2]
a.append([3, 4])     # [1, 2, [3, 4]]      嵌套了一个列表
```

```
b = [1, 2]
b.extend([3, 4])      # [1, 2, 3, 4]        拆开追加
c = [1, 2] + [3, 4]   # [1, 2, 3, 4]，c 是新列表，原来的两个列表都没变
```

删除元素：remove / pop / del / clear

```
list1 = [1, 2, 3, 8, 4, 5, 6, 8, 7]
if 9 in list1:
    list1.remove(9)          # 先判断再删，否则报错
while 8 in list1:
    list1.remove(8)          # 配合循环才能删干净
print(list1)                 # [1, 2, 3, 4, 5, 6, 7]
```

```
list1 = [1, 2, 3, 8, 4, 5, 6, 8, 7]
x = list1.pop()      # 删除并返回最后一个元素，x = 7
y = list1.pop(3)     # 删除并返回下标 3 的元素，y = 8
print(list1)         # [1, 2, 3, 4, 5, 6, 8]

del list1[3]         # del 是独立的语句，不是函数
del list1            # 甚至可以直接删除整个列表
```

写法	删除谁	返回值	找不到时
`a.remove(x)`	第一个**值为 x** 的元素	无（None）	`ValueError`
`a.pop()`	最后一个元素	被删的元素	空列表时 `IndexError`
`a.pop(i)`	下标 i 的元素	被删的元素	`IndexError`
`del a[i]`	下标 i 的元素	语句，无返回值	`IndexError`
`a.clear()`	全部元素	无（None）	不会出错

【常见错误】 一边 for 遍历一边 remove，结果"跳着删、删不干净"。

```
# ✗ 错误写法
a = [1, 2, 2, 3, 2]
for x in a:
    if x == 2:
        a.remove(x)
print(a)          # [1, 3, 2]  还有一个 2 没删掉
```

```
# ✓ 正确写法 1：遍历副本
a = [1, 2, 2, 3, 2]
for x in a[:]:        # a[:] 是拷贝，remove 删的是原列表
    if x == 2:
        a.remove(x)
print(a)          # [1, 3]

# ✓ 正确写法 2：用推导式重建（推荐）
a = [1, 2, 2, 3, 2]
```

```
a = [x for x in a if x != 2]
print(a)          # [1, 3]
```

删除元素会让后面的元素整体前移，而循环的下标还在往前走，于是刚好跳过一个元素。

翻转与排序

```
list1 = [3, 5, 7]
list2 = list(reversed(list1))   # list1 不变，翻转结果放 list2
print(list1)                    # [3, 5, 7]
print(list2)                    # [7, 5, 3]
list1.reverse()                 # list1 自身被翻转
print(list1)                    # [7, 5, 3]
```

```
list1 = [5, 3, 7]
list1.sort()                    # 原地排序，函数无返回值
print(list1)                    # [3, 5, 7]
list1.sort(reverse=True)        # 降序
print(list1)                    # [7, 5, 3]
```

```
list1 = [5, 3, 7]
list2 = sorted(list1)                # 不改原列表，返回新列表
list3 = sorted(list1, reverse=True)  # 降序
print(list1, list2, list3)           # [5, 3, 7] [3, 5, 7] [7, 5, 3]
```

【核心】`sort` 与 `sorted` 的区别：`list.sort()` 是**列表的方法**，**原地修改**列表且没有返回值（得到 `None`）；`sorted()` 是**内置函数**，可以作用于任何可迭代对象，**不修改原对象**而是返回新列表。这两个用混是常见错误。

【常见错误】 把"原地方法"的返回值赋给变量，结果拿到 `None`。

```
# X 错误写法
a = [3, 1, 2]
b = a.sort()          # a 排好了，但 b 是 None
print(b)              # None
c = [1, 2]
c = c.append(3)       # c 变成 None，原来的列表丢了！
print(c)              # None
```

```
# √ 正确写法
a = [3, 1, 2]
a.sort()              # 原地方法单独一行调用，直接用 a
print(a)              # [1, 2, 3]
b = sorted(a)         # 要新列表就用内置函数 sorted
print(b)              # [1, 2, 3]
c = [1, 2]
c.append(3)           # 原地追加，不要接收返回值
print(c)              # [1, 2, 3]
```

> 规律：**列表的 `append`、`extend`、`insert`、`remove`、`sort`、`reverse`、`clear` 全都是原地修改、返回 `None`**。凡是"原地方法"，都不要写 `x = x.方法()`。

排序的 3 个实用参数：

- `reverse=True`：从大到小；
- `key=函数`：按什么排。key 收到一个元素，返回用来比较的"排序依据"；
- 排序是**稳定**的：key 相同的元素保持原来的相对顺序，所以多关键字排序可以一次写进 key 里。

```
words = ['banana', 'apple', 'pear', 'fig']
print(sorted(words))                                # 按字母序
print(sorted(words, key=len))                       # 按长度：['fig', 'pear', 'apple', 'banana']
print(sorted(words, key=len, reverse=True))         # 长的在前
print(sorted(words, key=lambda w: (len(w), w)))     # 先按长度，长度相同按字母序

# 多关键字排序的经典写法：先按分数降序，分数相同按姓名升序
students = [('Tom', 88), ('Ann', 95), ('Bob', 88)]
print(sorted(students, key=lambda t: (-t[1], t[0])))
# [('Ann', 95), ('Bob', 88), ('Tom', 88)]
```

> **【速查】** `key=lambda t: (-t[1], t[0])` 这个写法必须记住：用负号实现"降序"，用元组实现"多关键字"。它也是 4.6 节综合示例和 4.10 节例题 4-6 的核心。

例题 4-1：按指定规则排序

`sorted` 的 `key` 参数接收一个函数，用来指定排序逻辑。

```
a = [5, -3, -7]
print(sorted(a, key=abs))      # [-3, 5, -7]   按绝对值排序

a = [('Two', 2), ('One', 1), ('Three', 3), ('Four', 4)]
print(sorted(a))                          # 按元组第一项排
print(sorted(a, key=lambda x: x[1]))      # 按元组第二项排

print(sorted(['apple', 'ant', 'duck'], key=lambda s: len(s)))
# ['ant', 'duck', 'apple']   按字符串长度排序
```

> **要点**：`key=abs` 可以直接传内置函数名；需要自己算的时候用 `lambda`。`sorted(a)` 对元组列表默认按第一项排，第一项相同再比第二项，这就是"字典序"。

例题 4-2：统计字母出现次数

题目：输入一段文字（没有换行），忽略其中非英文字母的其它字符，统计 26 个字母出现的次数（大小写视为同一个字母）。输出出现次数最多的字母的出现次数，然后是这些字母（小写形式，按字母表顺序）。

```
text = input().lower()
freq = [0] * 26
for c in text:
```

```
    if 'a' <= c <= 'z':
        freq[ord(c) - ord('a')] += 1

m = max(freq)
print(m)
max_index = -1
for i in range(freq.count(m)):
    max_index = freq.index(m, max_index + 1)
    print(chr(max_index + ord('a')))
```

要点：`freq = [0] * 26` 是"用列表当计数器"的经典写法：26 个格子对应 26 个字母，`ord(c) - ord('a')` 把字母映射成 0~25 的下标。`freq.count(m)` 数出有几个字母达到最大值，`freq.index(m, start)` 从指定位置往后找下一个。同样的题目用字典做会更好写（详见 4.10 节）。

例题 4-3：打印杨辉三角形

题目：输入一个正整数 n，输出杨辉三角形的前 n 行，每行用一个 Python 列表表示。

```
n = int(input())
row = [1]
print(row)
for _ in range(n - 1):
    nxt = [1]                       # 每行的第一个 1
    for i in range(len(row) - 1):
        nxt.append(row[i] + row[i + 1])
    nxt.append(1)                   # 每行的最后一个 1
    row = nxt                       # 整体替换：row 指向新列表，不影响旧列表
    print(row)
```

输入 `4`，输出：

```
[1]
[1, 1]
[1, 2, 1]
[1, 3, 3, 1]
```

要点：原写法里 `a = b` 之后又对 `a` 做 `insert` / `append`，改的其实是同一个列表，虽然结果碰巧是对的，但极易看错（别名问题详见 4.5 节）。这里改成"每行新建一个列表 `nxt`，最后整体替换 `row`"，逻辑清晰、不会踩坑。下一行 = 上一行相邻两项之和、两端补 1，也可以用推导式一行写完：

```
n = int(input())
row = [1]
for _ in range(n):
    print(row)
    row = [1] + [row[i] + row[i + 1] for i in range(len(row) - 1)] + [1]
```

4.6 列表相关的高级函数（速查）

本节解决三件事：**怎么遍历得更聪明**（`enumerate`、`zip`）、**怎么统计和判断更省事**（`sum`、`any`、`all`、`max/min` 的 key）、**常用函数速查**（见本节末尾的两张表）。

enumerate：下标和元素一起要

```
names = ['Tom', 'Ann', 'Bob']
for i, name in enumerate(names):           # 下标默认从 0 开始
    print(i, name)                         # 0 Tom / 1 Ann / 2 Bob

for i, name in enumerate(names, start=1):  # 想让编号从 1 开始就写 start=1
    print(i, name)                         # 1 Tom / 2 Ann / 3 Bob

print(list(enumerate(['a', 'b'])))         # [(0, 'a'), (1, 'b')]
```

> 【核心】需要下标时**不要**再写 `for i in range(len(a))` 然后 `a[i]`，写 `for i, x in enumerate(a)` 更短、更安全，也不容易越界。`enumerate` 返回的是迭代器，想变成列表要 `list(enumerate(a))`。

zip：把多个序列一一配对

```
names = ['Tom', 'Ann', 'Bob']
scores = [88, 95, 70]
for n, s in zip(names, scores):
    print(n, s)                    # Tom 88 / Ann 95 / Bob 70

print(list(zip(names, scores)))    # [('Tom', 88), ('Ann', 95), ('Bob', 70)]
print(dict(zip(names, scores)))    # {'Tom': 88, 'Ann': 95, 'Bob': 70}   建字典常用写法

a = [1, 2, 3]
b = [4, 5, 6, 7, 8]
print(list(zip(a, b)))             # [(1, 4), (2, 5), (3, 6)]   长度不等按短的截断

# 一步完成矩阵转置（详见 4.7 节）
matrix = [[1, 2, 3], [4, 5, 6]]
print(list(map(list, zip(*matrix))))   # [[1, 4], [2, 5], [3, 6]]
```

> 【易错】`zip` 在长度不等时**按最短的截断**，多余的元素被悄悄丢掉，并不报错。两个列表长度不一样时先检查，否则数据会丢。

统计与判断：sum / any / all

```
nums = [3, -1, 4, -1, 5]
print(sum(nums))                       # 10
print(sum(nums, 100))                  # 110，第二个参数是起始值
print(max(nums), min(nums))            # 5 -1
print(any(x < 0 for x in nums))        # True   只要有一个满足就是 True
```

```
print(all(x > 0 for x in nums))        # False  必须全部满足才是 True
print(sum(1 for x in nums if x < 0))   # 2      统计负数个数，不必先造列表
```

> **【注意】** 空序列上 `any([])` 是 `False`，`all([])` 是 `True`（"没有一个不满足"）。而 `sum`、`max`、`min` 用在空序列上会报 `ValueError`，用之前先判断序列非空。
>
> **【核心】** `any` / `all` / `sum` 里面写的是**生成器表达式**（没有方括号），不需要先造一个列表，省内存（详见 4.7 节）。

map / filter / reduce（速查）

函数	作用	示例
`random.shuffle(list)`	原地打乱列表顺序	`random.shuffle(list1)`
`enumerate(seq)`	同时取索引和值	`for i, v in enumerate(['tic','tac','toe'])`
`filter(f, it)`	过滤出使 f 为 True 的元素	`list(filter(lambda x: x%2==1, [1,2,3]))`
`map(f, it)`	对每个元素应用 f	`list(map(lambda x: x**2, [1,2,3]))`
`reduce(f, it)`	累积计算，返回单个值，需 `from functools import reduce`	`reduce(lambda x,y: x*10+y, [3,4,5,6])` 得到 3456
`zip(a, b)`	把多个序列的元素一一配对	`list(zip([1,2,3],[4,5,6]))`

```
# filter 过滤出列表中的所有奇数
def is_odd(n):
    return n % 2 == 1
print(list(filter(is_odd, [1, 2, 3, 4, 5, 6, 7, 8, 9, 10])))  # [1, 3, 5, 7, 9]

# map 映射
print(list(map(lambda x: x ** 2, [1, 2, 3, 4, 5])))            # [1, 4, 9, 16, 25]

# zip 一一配对，元素个数与最短的列表一致
a = [1, 2, 3]
c = [4, 5, 6, 7, 8]
print(list(zip(a, c)))    # [(1, 4), (2, 5), (3, 6)]
```

> **【了解】** `filter` 和 `map` 返回的都是迭代器，要看内容得用 `list()` 包一层。列表推导式能完全替代它们，而且更好读，所以实际写题时优先用推导式（详见 4.7 节）。

一行读入与一行输出（OJ 必备）

```
# OJ 里最常见的三种读法（split 的详细用法见 7.5 节，join 见 7.6 节）
a, b = map(int, input().split())            # 一行两个整数
nums = list(map(int, input().split()))      # 一行 n 个整数
```

```
t = int(input())                                # 多组数据：先读组数
for _ in range(t):
    n = int(input())
    nums = list(map(int, input().split()))

print(" ".join(map(str, nums)))                 # 输出：数字列表转成一行，行尾无空格
```

> **【注意】** `input()` 读到的**永远是字符串**，`"10 9".split()` 得到的是 `['10', '9']`，想要数字必须 `map(int, ...)` 或逐个 `int()`。输出时反过来，`" ".join(...)` 里的元素必须都是字符串，所以要 `map(str, ...)`。

综合示例：一次遍历搞定统计与排序

题目：第一行一个正整数 n（1 ≤ n ≤ 1000），第二行是 n 个整数（用空格分隔，可能是负数）。请输出 4 行：① 所有数的和；② 最大值和它**第一次出现的位置**（位置从 1 开始，中间一个空格）；③ 是否所有数都大于 0，是则输出 `Yes`，否则输出 `No`；④ 排序后的序列，规则是"先按绝对值从小到大，绝对值相同按原值从小到大"，用空格分隔，行尾不能有多余空格。

```
n = int(input())
nums = list(map(int, input().split()))

print(sum(nums))                            # ① 求和

m = max(nums)                               # ② 最大值及其第一次出现的位置
pos = 0
for i, x in enumerate(nums, start=1):       # 位置从 1 开始编号
    if x == m:
        pos = i
        break
print(m, pos)

print("Yes" if all(x > 0 for x in nums) else "No")   # ③ 全部大于 0 才输出 Yes

result = sorted(nums, key=lambda x: (abs(x), x))     # ④ 多关键字排序
print(" ".join(map(str, result)))
```

输入：

```
5
3 -1 4 -1 5
```

输出：

```
10
5 5
No
-1 -1 3 4 5
```

> **要点**：① `enumerate(nums, start=1)` 是"边遍历边要位置"的标准写法；其实一行 `print(m, nums.index(m) + 1)` 也行，但 `enumerate` 更能表达"第一次出现"的意思。② `all()` 直接吃生成器，不必额外建列表。③ `key=lambda x: (abs(x), x)` 是**多关键字排序**的通用套路，元组逐个比较。④ 输出用 `" ".join(map(str, ...))` 保证行尾没有多余空格——这是 OJ 最常见的扣分点。

序列常用方法总结（速查）

方法	作用	字符串	元组	列表
`+`	连接	是	是	是
`*`	重复	是	是	是
`in` / `not in`	成员判定	是	是	是
`[i]`	索引	只可读	只可读	是（可读写）
`[a:b]`	切片	只可读	只可读	是（可读写）
`del`	删除	只能删整个	只能删整个	是
`len/max/min`	长度、最值	是	是	是
`sum`	求和	否	是	是
`reversed/sorted/map/zip`	翻转、排序、映射、配对	是	是	是
`index(x)` / `count(x)`	查找 / 计数	是	是	是
`copy/append/insert/extend/remove/pop/clear/reverse/sort`	列表专有	否	否	是

本章知识点各自的"主场"（重复的内容都集中在一处，避免来回翻）：

知识点	详细讲解位置
索引、切片、反转	4.3 节
别名、浅拷贝、深拷贝	4.5 节
排序 `sort` / `sorted` / `key`	4.5 节（4.6 节补充多关键字）
`enumerate` / `zip` / `any` / `all` / `sum`	4.6 节（本节）
推导式、生成器表达式	4.7 节
序列解包	4.8 节
字典的 `get` / `setdefault`	4.10 节
集合运算	4.11 节

知识点	详细讲解位置
函数与可变对象	4.12 节
字符串方法 `split` / `join` / `strip` / 格式化	第 7 章（7.5、7.6、7.10 节）
方法速查表、易错清单、必背片段	4.13 节

4.7 推导式（解析式）

推导式是从一个数据序列构建另一个新数据序列的 Python 特性，分为列表、字典、集合推导式三种。**没有元组推导式**，类似的语法生成的是生成器表达式。

列表推导式

语法形式：`[表达式 for 变量 in 序列 if 条件]`

```
# 推导式写法
alist = [x * x for x in range(10)]
print(alist)   # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# 等价的循环写法
alist = []
for x in range(10):
    alist.append(x * x)
```

> **【核心】** 列表推导式与 for 循环逻辑上等价，只是更短、通常也更快（循环在 C 层执行）。遇到"筛出满足条件的元素""对每个元素做同样的运算""查找所有满足条件的位置"，先想推导式。

```
# 多层循环：第一个 for 是外循环
vec = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
numbers = [num for ele in vec for num in ele]
print(numbers)   # [1, 2, 3, 4, 5, 6, 7, 8, 9]

# 过滤：只保留大于 0 的元素
alist = [-1, -2, 1, 2, 3, -4, -5, -6]
blist = [i for i in alist if i > 0]
print(blist)     # [1, 2, 3]

# if-else 要写在 for 前面，它是"表达式"而不是"过滤"
print([x if x > 0 else 0 for x in alist])   # [0, 0, 1, 2, 3, 0, 0, 0]
```

> **【易错】** 嵌套推导式的书写顺序是"**从外到内**"，求值顺序是"**从左到右**"：`for ele in vec` 在前是外层循环，`for num in ele` 在后是内层循环。另外，`if` 只能写在末尾做过滤，想要"二选一"必须把 `if-else` 写在最前面。

```
# 在字典中查找所有最高分的同学
scores = {'Zhang': 45, 'Li': 78, 'Wang': 40, 'Zhou': 96, 'Zhao': 65, 'Sun': 90}
highest = max(scores.values())
```

```
highest_person = [name for name, score in scores.items() if score == highest]
print(highest, highest_person)     # 96 ['Zhou']
```

```
# 查找列表中最大元素的所有位置
from random import randint
x = [randint(1, 10) for i in range(10)]
m = max(x)
max_index = [index for index, value in enumerate(x) if value == m]
print(x, m, max_index)     # 例如 [3, 9, 9, 1, ...] 9 [1, 2]
```

列表推导式 vs 生成器表达式（一句话记住内存差别）

```
a = [i * i for i in range(10)]    # 列表推导式：立刻把所有结果算出来放进内存
b = (i * i for i in range(10))    # 生成器表达式：惰性求值，用到才算，几乎不占内存
print(a)          # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
print(b)          # <generator object <genexpr> at 0x...>
print(tuple(b))   # (0, 1, 4, 9, 16, 25, 36, 49, 64, 81)
```

> **【核心】** 一句话：**只要一个最终结果（求和、求最值、判断存在）就用生成器表达式；要反复用、要看全部内容就用列表推导式。** `sum(x * x for x in range(10 ** 8))` 用生成器几乎不占内存，先建列表则可能直接爆内存。
>
> **【注意】** 生成器只能**消费一次**：上面的 `b` 被 `tuple(b)` 取空之后，再 `list(b)` 得到的是空列表。

矩阵转置（经典例题）

```
matrix = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]

# 方法一：嵌套的列表推导式
result = [[row[i] for row in matrix] for i in range(4)]

# 方法二：使用内置函数 zip() 和 list()
result = list(map(list, zip(*matrix)))
# [[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

字典推导式与集合推导式

```
# 字典推导式：快速交换键值位置
mca = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
dicts = {v: k for k, v in mca.items()}
print(dicts)     # {1: 'a', 2: 'b', 3: 'c', 4: 'd'}

# 集合推导式：结果自动去重
squared = {i * 2 for i in [1, 1, 2]}
print(squared)   # {2, 4}
```

> **【易错】** 四种括号对应四种结果，别记混：`[k for ...]` 是**列表**，`(k for ...)` 是**生成器**，`{k for ...}` 是**集合**，`{k: v for ...}` 是**字典**（冒号不能丢）。

> **【注意】** 别拿推导式做"副作用"：`[print(x) for x in a]` 虽然能跑，但会顺手造一个没用的列表，直接写 `for x in a: print(x)`。

4.8 序列解包

序列解包把一个序列（或任何可迭代对象）解开，并将得到的值存储到一系列变量中。

```
x, y, z = 1, 2, 3
x, y = y, x                          # 交换两个变量，不需要中间变量

scoundrel = {'name': 'Robin', 'girlfriend': 'Marion'}
key, value = scoundrel.popitem()   # 直接解包字典的返回值
```

> **【核心】** `a, b = b, a` 是必须会的写法；`for k, v in d.items()` 和 `for i, x in enumerate(...)` 用的也是解包（详见 4.10 节、4.6 节）。
>
> **【易错】** 解包的序列元素个数必须与等号左边列出的目标个数**相同**，否则会报 `ValueError`。想接收多余的值，可以用通配符 `*`：

```
a, b, *rest = (1, 2, 3, 4)     # rest = [3, 4]
a, *rest, b = [1, 2, 3, 4]     # rest = [2, 3]
a, *rest, b = [1, 4]           # rest = []
```

带星号的变量被赋予一个**列表**，可以出现在任意位置，序列长度不足时被赋予空列表。

```
# 嵌套解包
(a, b), (c, d) = (1, 2), (3, 4)
print(a, b, c, d)      # 1 2 3 4

# 函数一次返回多个值，本质是返回元组再解包（返回值的规则见 3.3 节）
def min_max(nums):
    return min(nums), max(nums)

lo, hi = min_max([3, 1, 4, 1, 5])
print(lo, hi)          # 1 5
```

4.9 嵌套序列与二维列表

嵌套序列就是序列中有序列，例如全班期末成绩单：包含所有学生，每个学生包含所有成绩。

```
nested = ['first', ('second', 'third'), ['fourth', 'fifth', 'sixth']]
scores = [('Moe', 1000), ('Larry', 1500), ('Curly', 3000)]

print(scores[0])        # ('Moe', 1000)
a_score = scores[2]
print(a_score[0])       # Curly
print([name for name, score in scores if score > 1200])   # ['Larry', 'Curly']
```

> **【了解】** 嵌套超过两层就不好读了。能用"统一的模式"（比如统一用列表）就别混着用。

二维列表（矩阵）

初始化是二维列表最容易出错的地方：

```
m, n = 3, 4
grid = [[0] * n for _ in range(m)]   # 正确：3 行，每行是各自独立的列表
grid[0][0] = 1
print(grid)      # [[1, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```

> **【常见错误】** 用 `[[0] * n] * m` 初始化二维列表，改一个格子，整列都跟着变。
>
> ```
> # ✗ 错误写法
> grid = [[0] * 3] * 3      # 三行其实是同一个列表的三个别名
> grid[0][0] = 1
> print(grid)               # [[1, 0, 0], [1, 0, 0], [1, 0, 0]]
> ```
>
> ```
> # ✓ 正确写法
> grid = [[0] * 3 for _ in range(3)]   # 每次循环都新建一行
> grid[0][0] = 1
> print(grid)               # [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
> ```
>
> `*` 复制的是**同一个对象的引用**，这是 4.5 节"别名"问题的另一种表现。

读入一个矩阵（n 行，每行 m 个数）：

```
n, m = map(int, input().split())
matrix = []
for _ in range(n):
    matrix.append(list(map(int, input().split())))
```

按行打印一个矩阵：

```
for row in matrix:
    for value in row:
        print(value, end=' ')
    print()
# 更短的写法：print(*row) 会把 row 里的元素用空格隔开打印
```

> **【注意】** 遍历二维列表时，`for row in matrix` 拿到的是**每一行（一个列表）**，还要再来一层循环才能取到具体数字。另外 `eval(input(...))` 虽然能自动识别输入的类型，但用在 `input()` 上并不安全，课程作业里一律用 `int()` / `map(int, ...)`。

例题 4-4：给选择题评分

```
answers = [
    ['A', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'],
    ['D', 'B', 'A', 'B', 'C', 'A', 'E', 'E', 'A', 'D'],
    ['E', 'D', 'D', 'A', 'C', 'B', 'E', 'E', 'A', 'D']]
keys = ['D', 'B', 'D', 'C', 'C', 'D', 'A', 'E', 'A', 'D']

for i in range(len(answers)):
    correctCount = 0
    for j in range(len(answers[i])):
        if answers[i][j] == keys[j]:
            correctCount += 1
    print("Student", i, "'s correct count is", correctCount)
```

要点：外层循环遍历每个学生，内层循环遍历每道题，`answers[i][j]` 读作"第 i 个学生的第 j 题"。如果还想打印学生姓名，就把数据结构改成 `(姓名, 答案列表)` 的嵌套结构，用 `for name, ans in answers:` 解包（详见 4.8 节）。

4.10 字典 dict

列表和元组把信息组织成序列，而字典是把信息**成对存储**，就像字典中单词和解释成对使用。每对数据由"键 key"和"值 value"组成，称为键值对。

【核心】 一个字典中不能有相同的键，而且**键必须是不可变类型**（数字、字符串、元组等），不能是列表、字典或集合；值可以是任意类型，也可以重复。字典本身是可变数据类型。

【了解】 Python 3.7 以后字典会**记住键的插入顺序**，遍历时按插入的先后输出；但字典仍然不靠下标访问，"有顺序"不等于"能下标"。

创建字典

```
dict1 = {}                              # 空字典
dict2 = dict()                          # 空字典
tel1 = dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
tel2 = dict(sape=4139, guido=4127, jack=4098)
tel3 = dict(zip(['sape', 'guido', 'jack'], [4139, 4127, 4098]))   # zip 配对建字典（详见 4.6 节）
```

访问字典中的值：get 与 setdefault

```
students = {'3021666666': 'Zhang San', '3022888888': 'Li Si'}
print(students['3021666666'])           # Zhang San
print(students['3021000000'])           # KeyError!
```

```
# 正确做法 1：先用 in 测试（in 测试的是键，不是值）
if '3021000000' in students:
    print(students['3021000000'])
```

```
else:
    print("I don't know.")

# 正确做法 2：用 get()，键不存在时返回 None 或指定的默认值
print(students.get('3022888888'))                    # Li Si
print(students.get('3022000000'))                    # None
print(students.get('3022000000', "I don't know."))  # I don't know.
```

【易错】 直接用一个不存在的键访问字典会抛出 `KeyError`。不确定键在不在的时候用 `get()`，它不会报错。

`get` 和 `setdefault` 的区别必须分清：

写法	键存在时	键不存在时	是否修改字典
`d[key]`	返回对应的值	**报 KeyError**	否
`d.get(key)`	返回对应的值	返回 `None`	否
`d.get(key, 默认值)`	返回对应的值	返回默认值	否
`d.setdefault(key, 默认值)`	返回对应的值	把 key 写进字典（值为默认值）并返回它	**是**

```
# 词频统计的三种写法，效果完全一样
counts = {}
for ch in 'hello':
    if ch in counts:            # ① 先判断再改（最啰嗦，但最好懂）
        counts[ch] += 1
    else:
        counts[ch] = 1

counts = {}
for ch in 'hello':
    counts[ch] = counts.get(ch, 0) + 1          # ② get：推荐

counts = {}
for ch in 'hello':
    counts[ch] = counts.setdefault(ch, 0) + 1   # ③ setdefault：同样常用
print(counts)      # {'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

【核心】 `get` 只读不改，`setdefault` 不存在就**写进去**。想按"类别"往字典里装列表时（比如把学生按班级分组），`d.setdefault(cls, []).append(name)` 是标准写法：

```
# 把学生按班级分组
students = ['1班-张三', '2班-李四', '1班-王五']
groups = {}
for s in students:
    cls, name = s.split('-')
    groups.setdefault(cls, []).append(name)
print(groups)      # {'1班': ['张三', '王五'], '2班': ['李四']}
```

三个重要的"视图"方法

```
students = {'3021666666': 'Zhang San', '3022888888': 'Li Si'}

print(tuple(students.keys()))     # ('3021666666', '3022888888')
print(tuple(students.values()))   # ('Zhang San', 'Li Si')
print(tuple(students.items()))
# (('3021666666', 'Zhang San'), ('3022888888', 'Li Si'))
```

```
# 四种遍历方式
for k in students:                      # 默认遍历键
    print(k)
for k in students.keys():               # 等价写法
    print(k)
for v in students.values():             # 只遍历值
    print(v)
for k, v in students.items():           # 键和值都要（最常用）
    print('The key', k, 'has the value of', v)
```

【注意】 `keys()`、`values()`、`items()` 返回的是**视图**，不是列表：它跟着字典动态变化，想要真正的列表就 `list(d.keys())`。

【易错】 遍历字典的过程中**不能增加或删除键**，否则会报 `RuntimeError: dictionary changed size during iteration`。要改就先遍历副本：

```
d = {'a': 1, 'b': 2, 'c': 3}
for k in list(d):           # list(d) 把键复制成一个列表
    if d[k] < 2:
        del d[k]
print(d)                    # {'b': 2, 'c': 3}
```

【常见错误】 用 `d[key]` 读一个可能不存在的键，程序直接崩在 `KeyError`。

```
# ✗ 错误写法
d = {'a': 1}
print(d['b'])               # KeyError: 'b'
print(d.get('b') + 1)       # TypeError: 返回的是 None，不能和数字相加
```

```
# ✓ 正确写法
d = {'a': 1}
print(d.get('b', 0))        # 0   给一个默认值，才能参与计算
print(d.get('b', 0) + 1)    # 1
```

统计、累加、计数时一律写 `d[k] = d.get(k, 0) + 1`，不要先取 `d[k]` 再判断。

增、改、删

```
students['3021666666'] = 'Wang Wu'    # 键存在则修改；键不存在则新增

del students['3021666666']            # 删除指定键值对
students.pop('3022888888')            # 等价写法，并返回被删的值
students.popitem()                    # 删除最后加入的一个键值对，以元组形式返回
students.clear()                      # 清空字典
```

【易错】 新增和修改用的是**同一句代码** `dict[key] = value`，所以要先判断键是否存在，否则会不小心覆盖已有数据。删除时键必须存在：`del d[k]` 和 `d.pop(k)` 在键不存在时都会报 `KeyError`，`pop` 可以给第二个参数当默认值（`d.pop(k, None)`）。

字典常用方法总结（速查）

方法	作用	是否原地修改	返回值
`in` / `not in`	对**键**进行成员判定	否	True / False
`d[key]`	由 key 取 value	否	value（键不存在报 KeyError）
`d.get(key, 默认值)`	安全取值	否	value 或默认值（默认 None）
`d.setdefault(key, 默认值)`	安全取值，不存在就写入	是	value 或默认值
`d[key] = value`	键存在则改，不存在则增	是	无
`del d[key]`	删除指定键值对	是	无（语句）
`d.pop(key, 默认值)`	删除并返回 value	是	被删的 value
`d.popitem()`	删除最后加入的一个键值对	是	(key, value)
`d.clear()`	清空字典	是	None
`d.keys()` / `values()` / `items()`	取键 / 值 / 键值对视图	否	视图对象
`len(d)`	键值对个数	否	int
`max(d)` / `min(d)`	最大 / 最小的**键**	否	key

【注意】 `popitem()` 在 Python 3.7 以后删除的是**最后一个加入**的键值对（旧版本才是"随机删除"），返回的形式是元组 `(key, value)`，可以解包。

例题 4-5：找到出现次数最多的单词

题目：输入一段文字，所有非英文字母均视为单词之间的分隔符，大小写字母视为同一个字母，统计其中所有单词出现的次数。输出出现次数最多的单词的出现次数，然后是这些单词（小写形式，按升序一行一个）。

```
wordCounts = {}
line = input().lower()

for ch in line:
```

```
    if not ch.isalpha():
        line = line.replace(ch, " ")      # 非字母统一换成空格

for word in line.split():                 # split() 自动丢弃多余空格（详见 7.5 节）
    wordCounts[word] = wordCounts.get(word, 0) + 1

m = max(wordCounts.values())
print(m)
for word in sorted(w for w, c in wordCounts.items() if c == m):
    print(word)
```

输入 `It was the best of times, it was the worst of times.` 时，输出：

```
2
it
of
the
times
was
```

要点：① `ch.isalpha()` 判断是否为字母，非字母统一换成空格，后面一句 `split()` 就能把单词切干净；② 计数用 `get` 一行搞定，比先 `if word in wordCounts` 再分支更短；③ 最后"次数最多的单词按升序输出"用 `sorted` 带条件生成器一次完成，不必手工排序。

例题 4-6：按分数排序并输出最高分

题目：第一行一个正整数 n（1 ≤ n ≤ 100），表示学生人数；接下来 n 行，每行是 `学号 分数`（学号是不含空格的字符串，分数是 0~100 的整数，学号不重复）。请输出 3 部分：① 分数最高的学号（若多人并列，输出**最先输入**的那一位）；② 全体同学的平均分（保留两位小数）；③ 按分数从高到低输出每个同学的 `学号 分数`，分数相同时保持输入顺序。

```
n = int(input())
scores = {}
for _ in range(n):
    sid, sc = input().split()          # 一行两个字段，直接解包（详见 4.8 节）
    scores[sid] = int(sc)

best = max(scores, key=scores.get)      # 按"值"找"键"：max 作用在键上，比较依据是值
print(best)

print(f"{sum(scores.values()) / len(scores):.2f}")

for sid, sc in sorted(scores.items(), key=lambda kv: -kv[1]):
    print(sid, sc)                      # sorted 是稳定排序，分数相同就保持插入顺序
```

输入：

```
4
1001 88
```

```
1002 95
1003 88
1004 70
```

输出：

```
1002
85.25
1002 95
1001 88
1003 88
1004 70
```

要点：① `max(d, key=d.get)` 是"取字典中值最大的键"的固定写法，考试很爱考；② `sorted(d.items(), key=lambda kv: -kv[1])` 按值降序，靠的是 sorted 的**稳定性**才让并列的同学保持输入顺序；③ `f"{x:.2f}"` 的格式化详见 7.10 节。

4.11 集合 set

集合与列表类似，可以用来存储一组元素。但不同于列表，集合中的元素**不重复且不按任何特定顺序**放置。可以把集合想象成一个只有 key 没有 value 的字典。

```
s1 = set()                              # 注意：{} 是空字典，不是空集合
s2 = {1, 3, 5}                          # 字面量生成集合
s3 = set((2, 4, 6))                     # 从元组生成集合
s4 = set([x * 2 for x in range(1, 10)]) # {2, 4, 6, 8, 10, 12, 14, 16, 18}
s5 = set('python')                      # {'t', 'h', 'o', 'y', 'n', 'p'}
```

【易错】 创建空集合只能写 `set()`，因为 `{}` 是空字典。

常用操作

```
s1 = {1, 3, 5}
s1.add(6)         # {1, 3, 5, 6}
s1.remove(5)      # {1, 3, 6}
print(len(s1), max(s1), min(s1), sum(s1))   # 3 6 1 10
print(3 in s1)                              # True
```

【易错】 `remove(e)` 删除不存在的元素会报 `KeyError`，`discard(e)` 删不存在的元素**不报错**；如果元素已经在集合里，`add(e)` 不会重复增加。

集合运算（速查 • 核心）

运算符	名称	内置函数	说明
`s1 == s2`	相等	——	是否包含相同的元素

运算符	名称	内置函数	说明
`s1 != s2`	不等	——	是否包含不同的元素
`s1 < s2`	真子集	——	s2 包含 s1 所有元素，且还多
`s1 <= s2`	子集	`s1.issubset(s2)`	s2 包含 s1 所有元素
`s1 > s2`	真超集	——	s1 包含 s2 所有元素，且还多
`s1 >= s2`	超集	`s1.issuperset(s2)`	s1 包含 s2 所有元素
`s1 | s2`	并集	`s1.union(s2)`	两个集合所有元素
`s1 & s2`	交集	`s1.intersection(s2)`	两个集合共同的元素
`s1 - s2`	差集	`s1.difference(s2)`	出现在 s1 但不在 s2 的元素
`s1 ^ s2`	对称差	`s1.symmetric_difference(s2)`	除共同元素外的所有元素

```
set1 = {"green", "red", "blue", "red"}   # 重复元素自动去掉
print(set1)                              # {'red', 'blue', 'green'}

print(set1 | {"green", "yellow"})   # 并集
print(set1 - {"green", "yellow"})   # 差集
print(set1 & {"green", "yellow"})   # 交集
print(set1 ^ {"green", "yellow"})   # 对称差
```

【核心】集合最实用的三个用途：① **去重**：`set(列表)`，一行搞定"统计不同数字的个数"；② **交并差**：求两个列表的共同元素、只在某一边的元素；③ **成员判断快**：`x in 集合` 平均只要常数时间，`x in 列表` 要一个一个比，数据量大时差距很明显。

【注意】 集合**没有顺序**，所以不能下标访问（`s[0]` 会报 `TypeError`），输出的顺序也可能和你想的不一样。要按顺序输出就先 `sorted(s)` 变成列表。

【了解】 `frozenset` 是"不可变的集合"，可以当字典的键；平时做题用不到，看到知道是什么即可。

例题 4-7：两批数据的交、差、并

题目：第一行一个正整数 n，第二行 n 个整数（可能有重复）；第三行一个正整数 m，第四行 m 个整数。请输出 3 行：① 两批数据中**共同出现**的不同数字的个数；② 只出现在第一批中的数字，按升序每行输出一个；③ 两批数据合起来一共有多少种不同的数字。

```
n = int(input())
a = set(map(int, input().split()))      # 直接用 set 接收，读入的同时去重
m = int(input())
b = set(map(int, input().split()))

print(len(a & b))          # ① 交集的大小

for x in sorted(a - b):    # ② 差集，sorted 保证升序
    print(x)
```

```
print(len(a | b))          # ③ 并集的大小
```

输入：

```
5
1 2 3 4 5
4
4 5 6 7
```

输出：

```
2
1
2
3
7
```

要点：① 读入后直接 `set(...)`，去重和类型转换一步完成；② `n` 和 `m` 只表示个数，代码里用不到也不会报错（想更规范可以写成 `_ = int(input())`）；③ 集合是无序的，凡是要按顺序输出，一律先 `sorted()`。

4.12 函数与可变数据类型（重要）

前置知识：形参与实参之间是**引用传递**，这一条在 3.4 节已经讲过。本节专门讨论"可变对象被传进函数之后会发生什么"，这也是作业里最容易丢分的地方。

列表作为参数：引用传递

```
def main():
    x = 1               # 不可变
    y = [1, 2, 3]       # 可变
    m(x, y)
    print("x is", x)        # x is 1      没变
    print("y[0] is", y[0])  # y[0] is 5   变了!

def m(number, numbers):
    number = 6              # 只是让局部变量指向新值
    numbers[0] = 5          # 修改了实参指向的那个列表的内容
    numbers = [7, 8, 9]     # 只是让局部变量指向别的列表，不影响实参

main()
```

【核心】理解"引用传递"的关键：函数内对**元素**赋值（`numbers[0] = 5`）会改变外部列表；函数内对**整个变量**重新赋值（`numbers = [7, 8, 9]`、`lst = []`、`lst = lst + [4]`）不会影响外部列表，只是让局部变量改了指向。

【常见错误】 想在函数里"整理"传进来的列表，却写成了重新赋值，外部毫无变化。

```
# X 错误写法
def add_one(lst):
    lst = lst + [1]      # 造了一个新列表，外部列表没变
    return lst

a = [1, 2, 3]
add_one(a)
print(a)                 # [1, 2, 3]   没变
```

```
# √ 正确写法 1：用返回值接收新列表
def add_one(lst):
    return lst + [1]

a = [1, 2, 3]
a = add_one(a)           # 把返回值赋回去
print(a)                 # [1, 2, 3, 1]

# √ 正确写法 2：确实想改原列表，就用原地方法
def add_one_inplace(lst):
    lst.append(1)        # 原地修改，不需要 return

a = [1, 2, 3]
add_one_inplace(a)
print(a)                 # [1, 2, 3, 1]
```

一句话：**想改外部就用原地方法，不想改外部就返回新对象并让调用者接收。**别写"既返回新对象、又指望外部跟着变"的代码。

默认参数为列表的陷阱（易错）

```
def add(x, lst=[]):
    if x not in lst:
        lst.append(x)
    return lst

print(add(1))                # [1]
print(add(2))                # [1, 2]   期望是 [2]!
print(add(3, [11, 12, 13, 14]))   # [11, 12, 13, 14, 3]
print(add(4))                # [1, 2, 4]   期望是 [4]!
```

【易错】 默认参数 `lst=[]` 只在**函数定义时创建一次**，所有不传该参数的调用共享同一个列表，这就是上面"越加越多"的原因。**默认参数不要写可变对象**（列表、字典、集合），要用 `None` 打招呼：

```
# 正确写法：用 None 作默认值
def add(x, lst=None):
    if lst is None:
```

```
        lst = []
    if x not in lst:
        lst.append(x)
    return lst
```

函数中返回列表 vs 修改原列表

```
# 写法一：返回新列表，原列表不变（推荐，副作用最小）
def reverse1(lst):
    result = []
    for element in lst:
        result.insert(0, element)
    return result

list1 = [1, 2, 3, 4, 5, 6]
list2 = reverse1(list1)
print(list1)    # [1, 2, 3, 4, 5, 6]
print(list2)    # [6, 5, 4, 3, 2, 1]
```

```
# 写法二：直接修改原列表，函数不需要返回值
def reverse2(lst):
    for i in range(len(lst) // 2):
        lst[i], lst[-i - 1] = lst[-i - 1], lst[i]   # 用解包交换，不需要中间变量

list1 = [1, 2, 3, 4, 5, 6]
reverse2(list1)
print(list1)    # [6, 5, 4, 3, 2, 1]
```

【注意】 这两种写法都对，但**同一次作业里要保持一致**：要么"传进去、拿返回值"（写法一），要么"传进去、直接改"（写法二）。最怕写成半吊子：

```
def reverse3(lst):
    result = []
    for element in lst:
        result.insert(0, element)
    lst = result       # 只是让局部变量改了指向，外部列表没变

list1 = [1, 2, 3, 4, 5, 6]
reverse3(list1)
print(list1)           # [1, 2, 3, 4, 5, 6]   没变！
```

真想原地换掉内容，用 `lst.clear()` 加 `lst.extend(result)`；更简单的是直接用内置的 `lst.reverse()`，或者写 `lst[:] = reversed(lst)`。

【速查】 函数的参数里，不可变类型（int、str、tuple）不用操心；**可变类型（list、dict、set）一律先想清楚"我是要改它，还是要返回一个新的"**。把列表传进函数之前，也可以先 `.copy()` 一份再改，避免"顺手改坏"调用者的数据（详见 4.5 节）。

例题 4-8：去重并排序（函数与可变对象）

题目：输入一行用空格分隔的整数（可能有重复）。请写两个函数：`dedup_sorted(nums)` 返回"去重且升序"的**新列表**，并且不修改传入的列表；`dedup_inplace(nums)` **原地**把传入的列表改成去重且升序的结果，不需要返回值。最后输出三行：① 第一个函数的返回结果；② 调用第一个函数之后原列表的内容（用来证明它没变）；③ 调用第二个函数之后原列表的内容。

```
def dedup_sorted(nums):
    """返回去重并升序的新列表，不修改传入的列表。"""
    return sorted(set(nums))

def dedup_inplace(nums):
    """原地把 nums 改成去重并升序的结果，没有返回值。"""
    result = sorted(set(nums))    # 先把结果算好
    nums.clear()                  # 再清空原列表
    nums.extend(result)           # 最后把结果填回去

data = list(map(int, input().split()))

print(dedup_sorted(data))     # ① 返回新列表
print(data)                   # ② 原列表没变

dedup_inplace(data)           # ③ 原地修改，不要写成 data = dedup_inplace(data)
print(data)
```

输入 `3 1 2 3 2`，输出：

```
[1, 2, 3]
[3, 1, 2, 3, 2]
[1, 2, 3]
```

要点：① `sorted(set(nums))` 一行同时完成去重和排序，得到的是新列表；② 想改变外部列表的**内容**，只能用 `clear()` + `extend()` 这类原地方法，写成 `nums = sorted(set(nums))` 外部不会变（写法对比见本节前面的错误对照）；③ `dedup_inplace` 里必须先算好 `result` 再 `clear()`，否则 `set(nums)` 拿到的是空集合；④ 函数要么"返回新对象"，要么"原地修改"，不要写"返回 None 又指望外部变化"的半吊子代码。

4.13 本章方法总结与易错清单

一、本章知识地图

- **4.1 总纲**：五种组合类型（str / tuple / list / dict / set）× 可变性。可变性决定了本章的一切。
  - 不可变：str、tuple → 只能整体替换，不能部分修改（4.3 节讲透）
  - 可变：list、dict、set → 能部分修改，因此必须小心**别名**（4.5 节讲透）
- **序列线**（有下标、能切片）：str → tuple → list
  - 4.3 节 序列通用操作：索引、切片、`+`、`*`、`in`、`len/max/min/sum`、反转 ← **考试最高频**

- 4.4 节 元组：不可变的序列，能当字典的键
- 4.5 节 列表：本章最重的一节（增删改 + 拷贝 + 排序）
- 4.6 节 遍历与统计的惯用法：`enumerate`、`zip`、`sum`、`any`、`all`
- 4.7 节 推导式：从序列造序列
- 4.8 节 序列解包
- 4.9 节 嵌套序列与二维列表

- **映射线**：4.10 节 字典（键值对，键必须不可变）
- **散列线**：4.11 节 集合（去重与交并差）
- **综合线**：4.12 节 函数与可变对象（引用传递、默认参数陷阱）

考试最常考的五件事：① 索引与切片（含 `[::-1]`）；② 列表的增删改与 `sort` / `sorted` 的区别；③ 字典的 `get` 与键值对遍历；④ 推导式；⑤ 集合运算。

二、方法总结（速查表）

方法 / 语法	作用	是否原地修改	返回值	典型例子
`a[i]` / `a[-1]`	按下标取值	否	元素	`s[0]`、`s[-1]` 是最后一个
`a[i:j:k]`	切片，含头不含尾，越界不报错	否	新序列	`s[1:4]`、`a[::-1]` 反转
`a[i] = x`	按下标赋值（只有可变序列可以）	是	无	`a[0] = 9`
`a[i:j] = [...]`	切片赋值，长度可以变	是	无	`a[1:3] = [0]`
`len / max / min / sum`	长度、最值、求和	否	数值	`sum([1, 2, 3])`
`a + b` / `a * n`	连接 / 重复	否	新序列	`[0] * 5`
`x in a`	成员判断	否	True / False	`3 in [1, 2, 3]`
`a.append(x)`	末尾追加一个元素	**是**	**None**	`a.append(6)`
`a.extend(it)`	逐个追加另一个可迭代对象	**是**	**None**	`a.extend([5, 7])`
`a.insert(i, x)`	在下标 i 处插入	**是**	**None**	`a.insert(1, 8)`
`a.remove(x)`	删除第一个值为 x 的元素	**是**	**None**	`a.remove(2)`，不存在报 ValueError
`a.pop()` / `a.pop(i)`	删除并返回末尾 / 下标 i 的元素	**是**	被删元素	`x = a.pop()`
`del a[i]` / `a.clear()`	删除某个元素 / 清空	**是**	无	`del a[0]`
`a.sort(key=, reverse=)`	原地排序	**是**	**None**	`a.sort(reverse=True)`

方法 / 语法	作用	是否原地修改	返回值	典型例子
`sorted(a, key=, reverse=)`	排序成新列表，可用于任何可迭代对象	否	新 list	`sorted(a, key=len)`
`a.reverse()`	原地反转	**是**	**None**	`a.reverse()`
`a.copy()` / `a[:]`	浅拷贝	否	新 list	`b = a.copy()`
`copy.deepcopy(a)`	深拷贝（需 `import copy`）	否	新对象	`b = copy.deepcopy(a)`
`a.index(x)` / `a.count(x)`	查位置 / 计数	否	下标 / 个数	`a.count(2)`，index 找不到报 ValueError
`enumerate(a, start=1)`	同时取下标和元素	否	迭代器	`for i, x in enumerate(a)`
`zip(a, b)`	一一配对，按短的截断	否	迭代器	`dict(zip(ks, vs))`
`any(...)` / `all(...)`	存在 / 全都	否	True / False	`all(x > 0 for x in a)`
`d[k]` / `d.get(k, 默认值)`	取值 / 安全取值	否	值	`d.get('x', 0)`
`d.setdefault(k, 默认值)`	取出；不存在就写入	**是**	值	`d.setdefault(c, []).append(x)`
`d[k] = v` / `del d[k]`	增或改 / 删除	**是**	无	`d['a'] = 1`
`d.keys()` / `values()` / `items()`	键 / 值 / 键值对视图	否	视图	`for k, v in d.items()`
`s.add(x)` / `s.discard(x)`	加元素 / 安全删除	**是**	**None**	`s.discard(3)`
`s1 & s2`、`s1 | s2`、`s1 - s2`、`s1 ^ s2`	交、并、差、对称差	否	新 set	`set(a) & set(b)`
`[表达式 for x in it if 条件]`	列表推导式	否	新 list	`[x*x for x in a if x > 0]`
`(表达式 for x in it)`	生成器表达式，惰性省内存	否	生成器	`sum(x*x for x in a)`
`x, y = y, x`	交换 / 解包	——	——	`a, b = b, a`

“是否原地修改”和“返回值”两列是全章最该背下来的东西，判断题、改错题几乎都从这里出。

三、易错清单

1. **错误现象**：以为下标从 1 开始，写 `a[1]` 想取第一个元素，或写 `a[len(a)]` 想取最后一个。→ **原因**：序列下标从 **0** 开始。→ **正确写法**：第一个是 `a[0]`，最后一个是 `a[-1]`，越界会报 `IndexError`。
2. **错误现象**：不敢写 `s[5:10]`，怕越界报错。→ **原因**：混淆了索引和切片。→ **正确写法**：**索引越界报错，切片越界自动截断**，`s[5:10]` 得到 `''`，很安全。
3. **错误现象**：`b = a` 之后改 `b`，`a` 也变了。→ **原因**：赋值传的是引用，两个名字指向同一个列表。→ **正确写法**：`b = a.copy()` / `b = a[:]` / `b = list(a)`。

4. **错误现象**：`b = a.copy()` 之后改内层列表，`a` 还是被带坏。→ **原因**：`copy()` 只是**浅拷贝**，只复制第一层，嵌套子对象仍然共享。→ **正确写法**：`import copy` 后用 `b = copy.deepcopy(a)`，或者逐行重建 `b = [row[:] for row in a]`。
5. **错误现象**：`b = a.sort()` 之后 `b` 是 `None`，`c = c.append(3)` 之后 `c` 直接变成 `None`。→ **原因**：列表的原地方法（`append`、`extend`、`insert`、`remove`、`sort`、`reverse`、`clear`）都返回 `None`。→ **正确写法**：原地方法单独一行调用；要新列表用 `b = sorted(a)`。
6. **错误现象**：`s.strip()` 之后 `s` 没变。→ **原因**：字符串不可变，字符串方法一律返回**新字符串**。→ **正确写法**：`s = s.strip()`、`s = s.lower()` 都要赋值回去（字符串方法详见第 7 章）。
7. **错误现象**：一边 `for x in a` 一边 `a.remove(x)`，结果跳着删、删不干净。→ **原因**：删除会让后面的元素整体前移，而循环下标还在往前走。→ **正确写法**：遍历副本 `for x in a[:]`，或用推导式重建 `a = [x for x in a if 条件]`。
8. **错误现象**：`grid = [[0] * 3] * 3` 之后改一个格子，整列都变了。→ **原因**：`*` 复制的是同一个列表的引用，三行其实是同一行。→ **正确写法**：`grid = [[0] * 3 for _ in range(3)]`。
9. **错误现象**：`t = (1)` 以为是单元素元组。→ **原因**：没有逗号就是一个整数。→ **正确写法**：单元素元组写 `t = (1,)`。
10. **错误现象**：`s = {}` 想建空集合。→ **原因**：`{}` 是空**字典**。→ **正确写法**：空集合写 `s = set()`。
11. **错误现象**：程序报 `KeyError: 'xxx'`。→ **原因**：用 `d[k]` 读了不存在的键。→ **正确写法**：`d.get(k, 默认值)`；计数、累加一律写 `d[k] = d.get(k, 0) + 1`。
12. **错误现象**：报 `RuntimeError: dictionary changed size during iteration`。→ **原因**：遍历字典时增删了键。→ **正确写法**：遍历副本 `for k in list(d):`。
13. **错误现象**：`9 in {'a': 9}` 得到 `False`，以为程序错了。→ **原因**：`in` 判断的是**键**，不是值。→ **正确写法**：判值写 `9 in d.values()`。
14. **错误现象**：`s[0]` 取集合元素时报 `TypeError`。→ **原因**：集合无序，不支持下标和切片。→ **正确写法**：先 `sorted(s)` 得到列表再取。
15. **错误现象**：函数多次调用后列表越来越长。→ **原因**：默认参数 `def f(lst=[])` 只在**定义时求值一次**，所有调用共享同一个列表。→ **正确写法**：默认写 `None`，函数内 `if lst is None: lst = []`。
16. **错误现象**：函数里 `lst = []`、`lst = lst + [4]` 之后，外面的列表毫无变化。→ **原因**：对整个变量重新赋值只是让局部变量改指向。→ **正确写法**：想改外部就用原地方法（`append`、`clear` + `extend`），否则用 `return` 返回新对象并让调用者接收。
17. **错误现象**：想保留原列表却写了 `a.sort()`，原数据被改掉。→ **原因**：`sort()` 是原地方法，`sorted()` 才返回新列表。→ **正确写法**：不改原数据用 `b = sorted(a)`；确实要原地排就用 `a.sort()`。
18. **错误现象**：`'10' < '9'` 得到 `True`，比较结果和数学上相反。→ **原因**：`input()` 读到的永远是字符串，字符串比的是 Unicode 编码。→ **正确写法**：比较前先转换，`n = int(input())` 或 `list(map(int, input().split()))`。
19. **错误现象**：`zip` 之后元素少了一截，程序却不报错。→ **原因**：`zip` 按**最短**的序列截断。→ **正确写法**：先检查两个序列长度是否相等，或者改用下标遍历。
20. **错误现象**：用 `all(nums)` 判断“都是正数”，空列表也返回 `True`。→ **原因**：空序列上 `all([])` 是 `True`、`any([])` 是 `False`。→ **正确写法**：先判断序列非空，再判断条件。

四、必须能默写的 10 个片段

1. 反转与回文判断：

```
s[::-1]                 # 反转任意序列；s == s[::-1] 就是回文判断
```

1. 交换两个变量（不需要中间变量）：

```
a, b = b, a
```

1. 带下标遍历，编号从 1 开始：

```
for i, x in enumerate(nums, start=1):
    print(i, x)
```

1. 并行遍历与配对建字典：

```
for name, score in zip(names, scores):
    print(name, score)
d = dict(zip(names, scores))
```

1. 一行读入 n 个整数：

```
nums = list(map(int, input().split()))
```

1. 多组测试数据（先读组数）：

```
t = int(input())
for _ in range(t):
    n = int(input())
    nums = list(map(int, input().split()))
```

1. 去重后按升序输出：

```
for x in sorted(set(nums)):
    print(x)
```

1. 多关键字排序（分数降序、姓名升序）：

```
sorted(students, key=lambda t: (-t[1], t[0]))
```

1. 词频统计（用 get 计数）：

```
counts[ch] = counts.get(ch, 0) + 1
```

1. 输出一行且行尾无空格：

```
print(" ".join(map(str, ans)))
```

4.14 本章自测（17 道）

第 4 章 自测题 1　写出 `s = 'python'` 时下列表达式的值：`s[2]`、`s[-1]`、`s[1:4]`、`s[::-1]`、`s[10]`（会不会报错？）、`s[10:20]`（会不会报错？）。

第 4 章 自测题 2　`t = (1)` 与 `t = (1,)` 有什么区别？请说明类型。

第 4 章 自测题 3　请说出 `list1.sort()` 和 `sorted(list1)` 的三点区别。

第 4 章 自测题 4　下面代码的输出是什么？

```
a = [1, 3, 5, 7, 9]
a[1:4] = [0]
print(a)
```

第 4 章 自测题 5　解释下面代码为什么“越加越多”，并写出正确写法。

```
def add(x, lst=[]):
    lst.append(x)
    return lst

print(add(1), add(2), add(3))
```

第 4 章 自测题 6　用一行列表推导式生成 1~100 之间所有能被 3 整除的数的平方组成的列表。

第 4 章 自测题 7　写出下面程序的输出。

```
d = {'a': 1, 'b': 2}
d['c'] = 3
d['a'] = 9
print(d)
print('a' in d)
print(9 in d)
```

第 4 章 自测题 8　为什么字典的键不能用列表？请举例说明。

第 4 章 自测题 9　有两个列表 `a = [1,2,3,4,5]` 和 `b = [4,5,6,7]`，用集合运算求：① 两个列表的共同元素；② 只在 a 中出现的元素；③ 两个列表去重后的所有元素。

第 4 章 自测题 10　用列表推导式统计字符串 `"hello world"` 中每个单词的长度（提示：先 `split()`）。

第 4 章 自测题 11　写程序：读取一行用空格分隔的整数，去掉重复后按升序输出。

第 4 章 自测题 12　下面代码的输出是什么？为什么？

```
def f(lst):
    lst = lst + [4]
    return lst

a = [1, 2, 3]
b = f(a)
print(a, b)
```

第 4 章 自测题 13　写出下面三行代码的输出。

```
print(list(enumerate(['a', 'b'])))
print(list(enumerate(['a', 'b'], start=1)))
print(list(zip([1, 2, 3], 'abcd')))
```

第 4 章 自测题 14　已知 `students = [('Tom', 88), ('Ann', 95), ('Bob', 88), ('Cindy', 70)]`，用**一行代码**得到“按分数从高到低、分数相同按姓名从小到大”排序的新列表（提示：`sorted` 的 `key` 里可以放元组）。

第 4 章 自测题 15　下面代码输出什么？如果希望 `a`、`b` 完全独立，应该怎么改？

```
import copy
a = [[1, 2], [3, 4]]
b = a.copy()
b[0].append(9)
print(a)
print(b)
```

第 4 章 自测题 16　用**一行代码**判断列表 `nums` 中是否存在重复元素（提示：`len(set(...))`）。

第 4 章 自测题 17　下面代码想删掉列表中所有的 2，实际结果是什么？为什么？请给出两种正确写法。

```
a = [2, 2, 3, 2, 1]
for x in a:
    if x == 2:
        a.remove(x)
print(a)
```

4.15 本章作业题

作业题 17｜判断序列是否平衡

题目描述：对于一个包含 n 个正整数的序列 a，我们规定此序列是平衡的当且仅当存在一个正整数 i（1 ≤ i < n）使得序列第 1 到第 i 个数字的总和等于第 i+1 到第 n 个数字的总和。现在请你判断序列 a 是否是平衡的。

输入格式：本题单个测试点内包含多组测试数据。第一行是一个正整数 t，表示测试用例组数。接下来是 t 组测试用例，对每组测试用例，一共两行：第一行包含一个正整数 n，表示序列长度；第二行包含 n 个正整数，代表序列 a。

输出格式：对每组测试用例输出一行一个字符串。如果 a 是平衡的，输出 `Yes`，否则输出 `No`。

输入 #1	输出 #1
3	Yes
3	Yes
1 2 3	No
4	（第三行输出）

输入 #1	输出 #1
2 3 1 4	
5	
1 2 3 4 5	

说明/提示：样例解释——① 第一组，令 i = 2，有 1 + 2 = 3，因此序列是平衡的；② 第二组，令 i = 2，有 2 + 3 = 1 + 4，因此序列是平衡的；③ 第三组，不存在满足要求的 i。

数据规模与约定：保证 1 ≤ t ≤ 100，序列元素小于 10000。

> 提示思路：先把总和求出来，再从左往右累加。如果某一步"前缀和 == 总和 − 前缀和"（且不是最后一项），就是平衡的。 **考点**：本章第 4.6 节的 `sum()` 与第 4.3 节的遍历累加（前缀和思想）。 **输入怎么读**：先 `t = int(input())`；每组第一行 `n = int(input())`，第二行 `a = list(map(int, input().split()))`。每组是**两行**，别把 n 和数列读到一起。

作业题 18｜最大值及其出现位置

题目描述：读取若干个（至少 1 个）整数，找出它们中的最大值，并找到它出现的位置（第 1 个整数的位置为 1，后面依次增加）。

输入格式：第一个数字是样本数据的个数，然后后面是一组样本数据，一个一行。

输出格式：输入的数字中的最大值，然后是该最大值出现的位置，一个一行。

输入输出样例 1

输入 #1	输出 #1
4	321
321	1
-58	4
89	（位置）
321	

输入输出样例 2

输入 #2	输出 #2
7	8
5	2
8	5
2	7
4	（位置）
8	

输入 #2	输出 #2
2	
8	

说明/提示：样例 1 中 321 出现在第 1 位和第 4 位；样例 2 中 8 出现在第 2、5、7 位。注意"位置"可能有多个，要按出现的先后顺序全部输出。

> **考点**：本章第 4.6 节的 `enumerate`（要位置就靠它）与第 4.3 节的 `max()`、`index()`。 **输入怎么读**：第一个整数单独一行，后面的数**每行一个**，所以用 `n = int(input())` 再循环 `n` 次读 `int(input())`，不要写 `split()`。

作业题 19 | 最高分和最低分

题目描述：用户输入学生个数以及每个学生的分数，然后显示最高分和最低分的分数。

输入格式：第一个整数是学生的个数，后面的整数为每个学生的成绩，成绩均为 0~100 之间的整数（含 0 和 100）。

输出格式：先显示最高分，再显示最低分，一行一个。

输入 #1	输出 #1
3	96
88	65
96	（最低分）
65	

说明/提示：读完整个列表后直接调用 `max()` 和 `min()` 即可；也可以边读边更新最大值和最小值。

> **考点**：本章第 4.3 节的 `max()` / `min()`，或第 4.5 节“边读边更新最值”的写法。 **输入怎么读**：成绩可能一行一个、也可能写在同一行，最稳的读法是 `import sys` 之后 `data = list(map(int, sys.stdin.read().split()))`，再取 `data[0]` 和 `data[1: 1 + n]`。

作业题 20 | 校门外的树

题目描述：某校大门外长度为 L 的马路上有一排树，每两棵相邻的树之间的间隔都是 1 米。我们可以把马路看成一个数轴，马路的一端在数轴 0 的位置，另一端在 L 的位置；数轴上的每个整数点，即 0, 1, 2, ..., L，都种有一棵树。

由于马路上有一些区域要用来建地铁，这些区域用它们在数轴上的起始点和终止点表示。已知任一区域的起始点和终止点的坐标都是整数，区域之间可能有重合的部分。现在要把这些区域中的树（**包括区域端点处的两棵树**）移走。你的任务是计算将这些树都移走后，马路上还有多少棵树。

输入格式：第一行有两个整数，分别表示马路的长度 L 和区域的数目 m。接下来 m 行，每行两个整数，表示一个区域的起始点和终止点的坐标。

输出格式：输出一行一个整数，表示将这些树都移走后，马路上剩余的树木数量。

输入 #1	输出 #1
500 3	298
150 300	
100 200	
70 71	

说明/提示：对于 20% 的数据，保证区域之间没有重合的部分；对于 100% 的数据，保证 1 ≤ L ≤ 10^4，1 ≤ m ≤ 100。

> 提示思路：这是一道"模拟"题。建立一个长度 L+1 的列表，初值全部为 1（表示有树），每读入一个区间就把区间内所有位置置 0（注意区间是**闭区间**，即包含两个端点），最后统计还有多少个 1。 **考点**：本章第 4.5 节的列表初始化 `[1] * (L + 1)` 与循环改值；第 4.9 节的“标记数组”思想。 **输入怎么读**：第一行 `L, m = map(int, input().split())`；接下来 `m` 行每行两个整数，用 `for _ in range(m):` 配 `a, b = map(int, input().split())`。注意区间是**闭区间**，内层循环写 `range(a, b + 1)`。

第 5 章 输入输出与文件处理

> 对应课程讲义：《06_输入输出和文件处理》。这一章没有配套作业题，学完后请务必完成本章的 13 道自测题，其中 3 道是 OJ 风格的读写文件小练习，要动手写。

前面的 `input()` 和 `print()` 交换的是一次一点点的数据；真正要处理成批数据（几百行成绩、几千条记录）就得靠**文件**。本章只讲三件事：怎么打开文件、怎么读怎么写、怎么用 `os` 管理磁盘上的文件和目录。

5.1 文件类型与打开

文件就是字节流（byte stream），也叫做字符流：

- **文本文件**（字符流文件）：如 `.txt`、`.csv`、`.py`，内容是看得懂的字符，需要指定字符编码（ASCII、UTF-8、GB 等）；
- **二进制格式文件**：如图片、影音文件，内容是按特定格式排布的字节，必须用二进制模式打开。

Python 可以很容易地读写文本文件，优点是跨平台、易于使用。

打开文件的完整语法是：

```
open(file, mode='r', encoding=None)
```

参数	含义	常用取值
`file`	文件名或路径	`"animals.txt"`、`"D:/data/a.txt"`
`mode`	打开模式，默认 `'r'`	`"r"` `"w"` `"a"` `"rb"`
`encoding`	文本编码，只在文本模式下有意义	一律写 `"utf-8"`

> **【核心】** 在 Windows 上读写中文文本文件，永远显式写 `encoding="utf-8"`。不写的话 Python 会用操作系统的默认编码，中文 Windows 上是 GBK：读一个 UTF-8 文件立刻抛 `UnicodeDecodeError`，写出来的文件发给别人就是乱码。

这是校内讲义最容易漏、写作业最容易踩的一个坑，下面用一组对照看清楚。

> **【常见错误】** 打开含中文的文件不写 `encoding`，Windows 下必报错。
>
> ```
> # ✗ 错误写法：Windows 中文环境默认按 GBK 解码
> with open("readme.txt", "r") as f:
>     print(f.read())
> # UnicodeDecodeError: 'gbk' codec can't decode byte 0xe4 in position 0: illegal multibyte
> sequence
> ```
>
> ```
> # ✓ 正确写法：读写都显式指定 utf-8
> with open("readme.txt", "r", encoding="utf-8") as f:
>     print(f.read())
> ```

> 报错信息里出现 `'gbk' codec can't decode` 就是这个问题，**读和写都要写 `encoding="utf-8"`**。

打开之前先确认文件存在

```
import os

if os.path.isfile("read_it.txt"):
    with open("read_it.txt", "r", encoding="utf-8") as text_file:
        print(text_file.read())
else:
    print("文件不存在")
```

> **【核心】** 以读取方式打开一个文件前，应先测试它是否存在。检测路径的几个函数（`os.path.isfile`、`os.path.exists`）在 5.7 节汇总。

如果没检查就打开一个不存在的文件，会得到：

```
FileNotFoundError: [Errno 2] No such file or directory: 'read_it.txt'
```

捕获并处理这个异常的方法见 2.8 节。这里先弄清楚一个更根本的问题：**为什么找不到？**

相对路径与绝对路径

```
import os

print(os.getcwd())                      # 当前工作目录，例如 C:\Users\Lenovo\Desktop
print(os.path.isfile("read_it.txt"))    # 相对路径：从当前工作目录开始找
```

> **【核心】** 相对路径不是相对你的 `.py` 文件，而是相对**当前工作目录**（用 `os.getcwd()` 打印出来）。在 PyCharm 里点运行、在 cmd 里敲 `python 程序.py`、直接双击文件，当前工作目录可能都不是同一个，于是同一个程序会出现"昨天还能打开，今天就报 FileNotFoundError"。

找不到文件时的标准排查动作：

```
import os

print(os.getcwd())                      # 第一步：当前目录到底在哪
print(os.path.abspath("read_it.txt"))   # 第二步：这个相对路径实际指向谁
print(os.path.exists("read_it.txt"))    # 第三步：那里真有这个文件吗
```

> **【注意】** 练手的小程序把数据文件和 `.py` 放在同一个目录、用 IDE 的运行按钮启动，一般不会出问题。一旦报 FileNotFoundError，第一反应就是在程序开头 `print(os.getcwd())`，而不是反复改文件名。绝对路径、相对路径的完整规则见 5.7 节。
>
> **【核心】** 文件打开后一定要关闭，最省心的办法是 `with` 语句，详见 5.5 节。

5.2 文件指针（游标）

文件对象内部记着一个"读到哪儿了"的位置，叫做文件指针或游标。

```
file.tell()           # 检查当前文件指针的位置
file.seek(0)          # 回到文件头部
file.seek(offset, whence=0)   # whence 的取值见下表
```

whence	含义
0（默认）	从文件开头开始算起
1	从当前位置开始算起
2	从文件末尾开始算起

```
# 用一个例子看清游标是怎么往前走的
with open("animals.txt", "r", encoding="utf-8") as f:
    print(f.tell())      # 0      一开始在文件开头
    print(f.read(4))     # Ant
    print(f.tell())      # 4      读走 4 个字符后游标也走了 4 格
    f.seek(0)            # 手动把游标拨回开头
    print(f.tell())      # 0
```

【易错】 读取类函数都是从**当前游标位置**开始的，所以连续调用 `read()` / `readline()` 会依次往后读。搞不清结果为什么不对时，先想想游标在哪。

【常见错误】 `read()` 连着调用两次，第二次读到空字符串。

```
# X 错误写法：第二次 read() 什么也读不到
with open("animals.txt", "r", encoding="utf-8") as f:
    print(f.read())      # Ant Bee / Cat Dog / Eel Fox
    print(f.read())      # 空串！
```

```
# ✓ 正确写法：要重新读就先把游标拨回开头
with open("animals.txt", "r", encoding="utf-8") as f:
    first = f.read()
    f.seek(0)            # 关键的一句
    second = f.read()    # 内容和 first 一样
```

`read()` 会把游标一路推到**文件末尾**。同理，`for line in f:` 把文件遍历完之后，游标也在末尾，此时再 `f.read()` 同样是空串。要重新读，先 `seek(0)`。

【注意】 文本模式下 `seek` 只支持 `whence=0`，或者 `whence=1/2` 且 `offset=0`。写 `f.seek(2, 1)` 会报 `io.UnsupportedOperation: can't do nonzero cur-relative seeks`。

5.3 读取文件

先准备一个练习用的文件（在 Python 里执行一次即可）：

```
with open("animals.txt", "w", encoding="utf-8") as f:
    f.write("Ant Bee\n")
    f.write("Cat Dog\n")
    f.write("Eel Fox\n")
```

假设 `animals.txt` 的内容是：

```
Ant Bee
Cat Dog
Eel Fox
```

函数	行为	返回值
`read()`	从游标处一直读到文件尾	整个文件内容的字符串
`read(n)`	从游标处读取 n 个字符	最多 n 个字符的字符串
`readline()`	从游标处读取一行，遇到 `\n` 停止	一行的字符串（**带行尾的 `\n`**）
`readline(n)`	从游标处读取 n 个字符，遇到 `\n` 也会停止	字符串
`readlines()`	读取所有行，以列表形式返回	列表，每个元素是一行（**带 `\n`**）

```
with open("animals.txt", "r", encoding="utf-8") as infile:
    print(infile.read())
# Ant Bee
# Cat Dog
# Eel Fox
```

```
with open("animals.txt", "r", encoding="utf-8") as infile:
    s1 = infile.read(4)      # 'Ant '
    s2 = infile.read(10)     # 'Bee\nCat Do'
    print(repr(s1), repr(s2))
```

```
with open("animals.txt", "r", encoding="utf-8") as infile:
    print(repr(infile.readline()))   # 'Ant Bee\n'
    print(repr(infile.readline()))   # 'Cat Dog\n'
    print(repr(infile.readline()))   # 'Eel Fox\n'
    print(repr(infile.readline()))   # ''      ← 读完了返回空串
```

```
with open("animals.txt", "r", encoding="utf-8") as infile:
    print(infile.readlines())        # ['Ant Bee\n', 'Cat Dog\n', 'Eel Fox\n']
```

四种读法的区别（重点）

读法	一次读入多少	返回类型	内存占用	什么时候用
`f.read()`	整个文件	`str`	与文件等大	小文件，或需要把全文当成一个字符串处理

读法	一次读入多少	返回类型	内存占用	什么时候用
`f.readline()`	一行	`str`	一行	需要精确控制读取节奏（读一行、跳一行）
`f.readlines()`	整个文件	`list`	比文件还大（每行额外一个对象）	小文件，之后要按下标随机取第几行
`for line in f:`	一行	每次一个 `str`	一行	**推荐**，绝大多数场景都用它

> **【核心】** 一共就四种读法：`read()` 读全部、`readline()` 读一行、`readlines()` 读成列表、`for line in f` 逐行迭代。**平时一律用 `for line in f`**，只有确实需要整块文本时才用 `read()`。
>
> **【注意】** 处理几十 MB 以上的大文件时，绝对不要用 `read()` 或 `readlines()`：它们会把整个文件塞进内存，机子立刻变卡甚至 MemoryError。`for line in f` 无论文件多大都只占一行的内存。

行尾的换行符（重点）

> **【常见错误】** 「逐行读取」拿到的字符串行尾带 `\n`，直接 `print` 会多出一堆空行，拿去比较或拼接也会出错。

```
# ✗ 错误写法：每行之间多出一个空行
with open("animals.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line)          # print 自己会补一个换行，line 末尾还有一个 \n
```

```
# ✓ 正确写法：先去掉行尾的换行符
with open("animals.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.rstrip("\n"))
```

> `readline()`、`readlines()`、`for line in f` 三种方式拿到的每一行**都带行尾的 `\n`**（文件最后一行如果没有换行符就不带）。用 `rstrip("\n")` 只去掉换行；写成 `strip()` 会把行首缩进和行尾空格一起去掉，处理有缩进的数据时反而弄错。

处理大文件

```
# 用 while 循环
with open("animals.txt", "r", encoding="utf-8") as infile:
    line = infile.readline()
    while line != "":            # 读到空串说明到文件末尾了
        print(line.rstrip("\n"))
        line = infile.readline()
```

```
# 用 for 循环，更简洁，推荐
with open("animals.txt", "r", encoding="utf-8") as infile:
```

```
    for line in infile:
        print(line.rstrip("\n"))
```

要带行号，用 `enumerate()`，不要自己维护一个计数器：

```
with open("animals.txt", "r", encoding="utf-8") as infile:
    for i, line in enumerate(infile, start=1):
        print(i, line.rstrip("\n"))
# 1 Ant Bee
# 2 Cat Dog
# 3 Eel Fox
```

例题 5-1：统计成绩文件

题目：文件 `scores.txt` 中每行一个整数成绩，其中可能夹着空行。请输出：有效成绩的个数、总分、平均分（保留两位小数）、最高分，每个占一行。

```
count = 0        # 有效成绩个数
total = 0        # 总分
best = None      # 最高分；None 表示"还没有任何成绩"

with open("scores.txt", "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()          # 去掉行尾换行和多余空白
        if line == "":               # 空行跳过
            continue
        score = int(line)
        count += 1
        total += score
        if best is None or score > best:
            best = score

if count > 0:
    print(count)
    print(total)
    print(f"{total / count:.2f}")
    print(best)
```

`scores.txt` 的内容是：

```
88
92

75
```

输出：

```
3
255
```

```
85.00
92
```

要点：① `line.strip()` 之后判空，是处理"文件里夹空行、末尾有空行"的标准动作；② `total` 初始值取 0，但最大值不能初始化成 0——万一成绩全是负数就错了，所以用 `None` 表示"还没读到任何值"；③ `f"{...:.2f}"` 控制两位小数，格式化语法见 1.6 节。

5.4 写入文件

```
with open("animals.txt", "w", encoding="utf-8") as outfile:
    outfile.write("Ant Bee\n")
    outfile.write("Cat Dog\n")
    outfile.write("Eel Fox\n")
```

```
# 写入字符串列表
lines = ["Line 1\n", "This is line 2\n", "That makes this line 3\n"]
with open("write_it.txt", "w", encoding="utf-8") as text_file:
    text_file.writelines(lines)
```

【注意】 `writelines()` 只会把列表里的字符串**一个接一个原样倒进去**，不会替你加换行。列表里每个字符串要自己带 `\n`，否则全部挤在同一行。

```
# 追加数据
with open("Info.txt", "a", encoding="utf-8") as outfile:
    outfile.write("Python is interpreted\n")
```

【核心】 `write()` 方法**不会自动换行**，需要手工在行末加上换行符 `\n`。文件使用完后一定要 `close()`，用 `with` 最省事。

文件打开模式（速查 • 核心）

模式	含义
`'r'`	只读（默认）。文件指针在文件头。文件不存在会报错
`'w'`	只写。文件存在则**清空内容**；不存在则创建
`'a'`	追加。文件指针在文件尾，新内容追加在已有内容之后；不存在则创建
`'r+'`	读写。指针在文件头。文件不存在会报错
`'w+'`	读写。文件存在则清空内容；不存在则创建
`'a+'`	读与追加。指针在文件尾；不存在则创建

【易错】 `'w'` 会**清空原有内容**！不小心用 `'w'` 打开一个已有的重要文件，内容就没了。只想在末尾加内容请用 `'a'`。

【常见错误】 想往日志末尾追加一行，用了 `"w"`，结果整个日志被清空。

```
# ✗ 错误写法：文件里只剩最后写进去的那一行
with open("log.txt", "w", encoding="utf-8") as f:
    f.write("第 3 次运行\n")
```

```
# ✓ 正确写法：追加用 "a"
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("第 3 次运行\n")
```

`"w"` 打开文件的那一瞬间原内容就被清掉了，和后面写了多少无关。写文件之前先问自己一句：**这份内容是要覆盖，还是要追加？**

【常见错误】 往文件里直接写数字，报 `TypeError`。

```
# ✗ 错误写法
with open("Numbers.txt", "w", encoding="utf-8") as f:
    for i in range(10):
        f.write(i)
# TypeError: write() argument must be str, not int
```

```
# ✓ 正确写法：先转成字符串再加换行
with open("Numbers.txt", "w", encoding="utf-8") as f:
    for i in range(10):
        f.write(str(i) + "\n")
```

`write()` 的参数**只能是字符串**。三种等价写法：`f.write(str(i) + "\n")`、`f.write(f"{i}\n")`、`print(i, file=f)`——`print` 会自己把参数转成字符串并补上换行。

以上模式默认都是文本模式（t），要以二进制模式打开需要加参数 b，例如 `rb`、`wb`、`ab`。

```
# 数字要先转成字符串再写入
from random import randint

with open("Numbers.txt", "w", encoding="utf-8") as outfile:
    for i in range(10):
        outfile.write(str(randint(0, 9)) + " ")

with open("Numbers.txt", "r", encoding="utf-8") as infile:
    s = infile.read()
    numbers = [eval(x) for x in s.split()]
    for number in numbers:
        print(number, end=" ")
```

【注意】 `eval(x)` 会把字符串当表达式求值，功能强但有风险；这里每个数据都是普通整数，写成 `int(x)` 更清楚也更安全。

例题 5-2：写进文件再读回来

题目：输入一个整数 n（1 ≤ n ≤ 9），把 1~n 每个数的平方按 `i: i*i` 的格式写入 `squares.txt`（每行一条），然后重新打开这个文件，把内容原样打印出来。

```
n = int(input())

with open("squares.txt", "w", encoding="utf-8") as f:
    for i in range(1, n + 1):
        f.write(f"{i}: {i * i}\n")

with open("squares.txt", "r", encoding="utf-8") as f:
    print(f.read(), end="")
```

输入 `4`，`squares.txt` 里得到：

```
1: 1
2: 4
3: 9
4: 16
```

屏幕输出与上面完全相同。

要点：① 写在 `with` 块里，出了块才会真正 `close`，「写」和「读」必须分成两个 `with` 块，否则读不到刚写的内容；② `f.read()` 返回的字符串本来就以 `\n` 结尾，再用 `print()` 会多一个空行，所以写 `print(f.read(), end="")`；③ `f.write(f"{i}: {i * i}\n")` 里的 `\n` 不能省，否则所有结果挤成一行。

5.5 with 语句（推荐）

```
# 不使用 with 的写法，出现异常时 close 可能执行不到
file = open("./test.txt", "w", encoding="utf-8")
file.write("hello world !")
file.close()

# 使用 with，退出代码块时系统会自动调用 close()
with open("./test.txt", "w", encoding="utf-8") as file:
    file.write("hello world !")

print(file.closed)     # True
```

【核心】 `with` 的作用等效于 `try...finally`：无论代码块里是否发生异常，退出代码块时都会自动关闭文件。**处理文件时使用 with 是最佳实践**，它能避免忘记 close 造成的资源占用。

【常见错误】 忘记 `close()`，写进去的内容没有落到磁盘上。

```
# X 错误写法：数据还留在缓冲区里
f = open("out.txt", "w", encoding="utf-8")
```

```
f.write("hello")
# 后面如果程序崩溃、提前 return，或者一直不退出，文件里就可能是空的
```

```
# ✓ 正确写法一：用 with，自动关闭
with open("out.txt", "w", encoding="utf-8") as f:
    f.write("hello")

# ✓ 正确写法二：手动关闭（或者用 f.flush() 强制刷盘）
f = open("out.txt", "w", encoding="utf-8")
f.write("hello")
f.close()
```

`write()` 只是把数据写进内存缓冲区，`close()` 才会把缓冲区刷到磁盘。CPython 恰好会在文件对象被回收时顺手帮你关掉，所以小程序里常常看不出问题——但这是实现细节，不能依赖，**写文件一律用 `with`**。

【注意】 `with` 块结束后变量 `file` 还在，但文件已经关闭，再调用 `file.read()` 会报 `ValueError: I/O operation on closed file`。想知道关没关，看 `file.closed`。

一次打开多个文件也是允许的：

```
# 复制文件：一边读一边写
with open("a.txt", "r", encoding="utf-8") as fin, \
     open("b.txt", "w", encoding="utf-8") as fout:
    fout.write(fin.read())
```

【了解】 `with` 的实现原理建立在上下文管理器之上，只有支持上下文管理协议（实现了 `__enter__` 和 `__exit__` 方法）的对象才能使用 `with`。

5.6 CSV 文件

CSV 文件就是一个用逗号分隔值的文本文件。它简单、被广泛支持、可用文本编辑器查看，但没有类型、没有字体颜色、没有多个工作表、不能合并单元格、不能嵌入图像。

【核心】 不要用 `split()` 手工解析 CSV！CSV 有自己的转义字符，允许逗号出现在值内部（例如 `"Hello, world!"`），`split()` 处理不了。请使用 Python 自带的 `csv` 模块。

读取 CSV

```
import csv

with open("example.csv", "r", encoding="utf-8", newline="") as exampleFile:
    exampleReader = csv.reader(exampleFile)
    exampleData = list(exampleReader)

print(exampleData)
# [['4/5/2015 13:34', 'Apples', '73'], ['4/5/2015 3:41', 'Cherries', '85'], ...]
```

```
print(exampleData[0][1])   # 'Apples'   —— [行][列]
```

```
# 大型 CSV 文件：在 for 循环中使用 Reader 对象，避免一次性装入内存
import csv

with open("example.csv", "r", encoding="utf-8", newline="") as exampleFile:
    exampleReader = csv.reader(exampleFile)
    for row in exampleReader:
        print("Row #" + str(exampleReader.line_num) + " " + str(row))
```

【注意】 `csv.reader` 读出来的每个字段**都是字符串**，数字要自己用 `int()` / `float()` 转换；`reader.line_num` 记录的是已经读过的行号。

写入 CSV 与 delimiter 参数

```
import csv

with open("output.csv", "w", encoding="utf-8", newline="") as outputFile:
    outputWriter = csv.writer(outputFile)
    outputWriter.writerow(["spam", "eggs", "bacon", "ham"])
    outputWriter.writerow(["Hello, world!", "eggs", "bacon", "ham"])
    outputWriter.writerow([1, 2, 3.141592, 4])
```

输出文件内容：

```
spam,eggs,bacon,ham
"Hello, world!",eggs,bacon,ham
1,2,3.141592,4
```

【注意】 写 CSV 时 `open` 一定要加 `newline=""`。不加的话，`csv.writer` 写出的 `\r\n` 会被文本模式再翻译一次，结果每两行之间多出一个空行。另外 `writerow()` 的参数可以是数字，它会自动转成字符串——这一点和 `f.write()` 不同。

```
# 用制表符代替逗号（TSV 文件）
csv.reader(exampleFile, delimiter="\t")
csv.writer(outputFile, delimiter="\t")
```

例题 5-3：汇总 CSV 成绩

题目：文件 `students.csv` 第一行是表头，之后每行是「姓名,语文,数学」。请输出每人的总分，最后再输出总分最高的姓名。

```
import csv

best_name = ""      # 总分最高的人
best_total = -1     # 目前见过的最高总分
```

```
with open("students.csv", "r", encoding="utf-8", newline="") as f:
    reader = csv.reader(f)
    next(reader)                        # 跳过表头那一行
    for row in reader:
        name = row[0]
        total = int(row[1]) + int(row[2])
        print(f"{name} {total}")
        if total > best_total:          # 严格大于，分数并列时保留先出现的
            best_total = total
            best_name = name

print(best_name)
```

`students.csv` 的内容是：

```
姓名,语文,数学
张三,88,92
李四,95,79
王五,60,100
```

输出：

```
张三 180
李四 174
王五 160
张三
```

> **要点**：① 表头用 `next(reader)` 跳过，等价于先 `readline()` 一次；② `csv.reader` 给的字段全是字符串，算总分必须 `int()`；③ 求最大值的初始值取 `-1`（总分不可能比它小），然后一路 `>` 比较即可。

5.7 文件与目录操作

要导入的模块：`import os`（提供操作系统基本功能，`os.path` 是它里面的模块，同样用 `import os` 导入）和 `import shutil`（shell utilities，提供复制、移动、改名、删除）。

基本概念

> **【了解】大小写敏感性**：Windows 一般不区分大小写，Linux/UNIX/BSD 区分，MacOS 与 Windows 类似不区分。从一个区分大小写的系统复制文件到不区分大小写的系统时，可能出现"文件被覆盖"的风险（例如 `file1` 与 `File1` 会被 Windows 视为同一个文件）。
>
> **【了解】路径分隔符**：Windows 使用反斜杠 `\`，Linux/UNIX/MacOS 使用正斜杠 `/`。可以用 `os.sep` 得到当前系统的分隔符，用 `os.path.join()` 拼接路径以自动处理差异。另外，反斜杠在 Python 字符串中代表转义，所以必须连写两遍，即 `\\`。

```
import os
```

```
print(os.sep)                              # Windows 上是 '\'
print(os.path.join("usr", "bin", "spam"))   # usr\bin\spam
```

【易错】 写 Windows 路径时直接写 `"C:\Users\Lenovo"` 会报 `SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes`，因为 `\U` 被当成了 Unicode 转义；写 `"C:\new"` 则更隐蔽——`\n` 变成了换行符。三种安全写法：

```
path1 = "C:\\Users\\Lenovo"      # 双反斜杠
path2 = r"C:\Users\Lenovo"       # 原始字符串，推荐
path3 = "C:/Users/Lenovo"        # 正斜杠在 Windows 上也能用
```

当前目录与路径

```
os.getcwd()                    # 取得当前工作目录
os.chdir(path)                 # 改变当前工作目录
os.path.abspath(path)          # 转为绝对路径
os.path.relpath(path, start)   # 转为相对路径
os.path.isabs(path)            # 判断是否为绝对路径
os.path.dirname(path)          # 取最后一个分隔符之前的内容（所在目录）
os.path.basename(path)         # 取最后一个分隔符之后的内容（文件名）
os.path.split(path)            # 分隔成 (目录名, 文件名)
os.path.splitext(path)         # 分离文件名和扩展名
```

```
path = "/home/user1/docs/file1"
print(os.path.dirname(path))    # /home/user1/docs
print(os.path.basename(path))   # file1
print(os.path.split(path))      # ('/home/user1/docs', 'file1')
print(path.split(os.path.sep))  # ['', 'home', 'user1', 'docs', 'file1']
```

【核心】绝对路径与相对路径：绝对路径从根目录开始标识；相对路径不从根目录开始，总是从**当前目录**开始查找。`.` 代表"这个目录本身"，`..` 代表"这个目录的父目录"。（相对路径的坑见 5.1 节。）

【注意】 `os.getcwd()` 拿到的是"程序运行时所在的目录"，不是"这个 .py 文件所在的目录"，这两个值完全可能不同。同目录下的文件互相导入、相互读取时，搞清楚这一点能省下大量调试时间。

查看目录内容和文件大小

```
os.listdir(path)               # 返回目录中所有的内容
os.path.getsize(file)          # 得到文件的字节数
```

```
import os

for name in os.listdir("."):
    full = os.path.join(".", name)
    if os.path.isfile(full):
        print(name, os.path.getsize(full), "字节")
```

遍历目录树

```
import os

for root, dirs, files in os.walk("."):
    for name in files:
        print(os.path.join(root, name))
    for name in dirs:
        print(os.path.join(root, name))
```

【了解】 `root` 表示正在浏览的目录，`dirs` 是该目录下所有的子目录，`files` 是该目录下所有的文件。只取文件名是不够的，取完整路径要 `os.path.join(root, name)`。

检查路径的有效性

```
os.path.exists(path)     # 文件或目录是否存在
os.path.isfile(path)     # 是否为一个文件
os.path.isdir(path)      # 是否为一个目录
```

操作文件和目录（速查）

```
os.mkdir()                      # 创建新目录（上级目录必须已存在）
os.makedirs()                   # 创建新目录，会自动创建所有必要的中间目录
os.rmdir(path)                  # 删除空目录
os.removedirs(path)             # 删除多级空目录

shutil.copy(source, destination)      # 复制文件，返回新文件路径
shutil.copytree(source, destination)  # 复制目录
shutil.move(source, destination)      # 移动或改名

os.remove(path) / os.unlink(path)     # 删除文件
shutil.rmtree(path)                   # 删除任意目录（可以非空）
```

```
# 复制文件
shutil.copy('file1', 'file2')          # 复制为 file2
shutil.copy('file1', 'dir1')           # 复制到目录 dir1 下，仍叫 file1
shutil.copy('file1', 'dir1/file2')     # 复制到 dir1 下，改名为 file2

# 移动与改名
shutil.move('file1', 'file5')          # 改名为 file5
shutil.move('file5', 'dir5')           # 移动到目录 dir5 下
shutil.move('file2', 'dir5/file6')     # 移动并改名
```

【易错】 `shutil.copy` 如果目标已有同名文件会**直接覆盖**；`shutil.copytree` 如果目标已存在（无论文件还是目录）都会报错；`shutil.move` 若源是文件而目标有同名文件会覆盖，若源是目录而目标有同名文件则报错。

> **【了解】安全删除**：`os.remove` 和 `shutil.rmtree` 是永久删除。想删到回收站需要第三方模块 `send2trash`（`pip install send2trash`），用法是 `send2trash.send2trash(path)`。
>
> **【注意】** 练习 5.7 节的函数时，先 `os.mkdir("test")` 建一个临时目录，在里面随便折腾；确认无误后再对真实文件动手。这些操作删掉的东西不进回收站。

快速检索：常用函数与 shell 命令对照（速查）

函数	作用	类似 shell 命令
`os.getcwd()`	获得当前目录	pwd
`os.chdir()`	切换当前目录	cd
`os.path.join()`	拼接完整路径	——
`os.path.abspath()` / `relpath()`	转绝对 / 相对路径	——
`os.listdir()`	列出目录下的内容	ls
`os.walk()`	递归遍历目录	ls -R
`os.path.getsize()`	获得文件大小	——
`os.path.exists/isfile/isdir`	判断存在性	——
`os.mkdir()` / `os.makedirs()`	创建目录	mkdir / mkdir -p
`shutil.copy()` / `copytree()`	复制文件 / 目录	cp / cp -R
`shutil.move()` / `os.rename()`	移动或改名	mv
`os.remove()` / `os.unlink()`	删除文件	rm
`os.rmdir()` / `os.removedirs()`	删除空目录 / 多级空目录	rmdir / rmdir -p
`shutil.rmtree()`	删除任意目录	rm -R

5.8 本章方法总结与易错清单

一、本章知识地图

```
5.1 打开文件 ─┬── 路径：相对路径从「当前工作目录」算起，用 os.getcwd() 打印
              ├── 模式：r / w / a，其中 w 会清空原文件
              └── 编码：encoding="utf-8"，防止 UnicodeDecodeError
                        │
5.2 文件指针 ◄──────────┘   所有读写都从游标当前位置开始
     │
     ├── 5.3 读：read()  /  readline()  /  readlines()  /  for line in f
     └── 5.4 写：write() /  writelines()，写数字先 str()
          │
5.5 with 语句 ──── 自动 close，上面所有读写操作都该套上它
     │
5.6 csv 模块 ──── 别用 split(",") 解析 CSV
5.7 os / shutil ── 路径拼接、目录遍历、复制、移动、删除
```

- **谁建立在谁之上**：5.1 是入口（拿到文件对象），5.2 决定每一次读写从哪儿开始，5.3 和 5.4 是真正的读与写，5.5 给前面全部操作兜底。这五节是一条线：**打开 → 读写 → 关闭。**
- **考试和作业最常考的五个点**：① `encoding="utf-8"`；② `"w"` 与 `"a"` 的区别；③ 四种读法各返回什么；④ `with` 相比手动 `close` 的好处；⑤ 行尾 `\n` 的处理。
- **5.6、5.7 属于"会用 + 会查"的层次**：CSV 记住 `csv.reader` / `csv.writer` 两个对象，`os` 记住 `getcwd`、`path.join`、`listdir`、`walk`、`path.isfile` 这几个高频函数即可，其余现查现用。

二、方法总结（速查表）

方法/语法	作用	是否原地修改	返回值	典型例子
`open(path, "r", encoding="utf-8")`	打开文件	否	文件对象	`with open(p, encoding="utf-8") as f:`
`f.read()`	读整个文件	否（移动游标）	`str`	`text = f.read()`
`f.read(n)`	读 n 个字符	否	`str`	`s = f.read(4)`
`f.readline()`	读一行	否	`str`（带 `\n`）	`line = f.readline()`
`f.readlines()`	读所有行	否	`list`（每项带 `\n`）	`lines = f.readlines()`
`for line in f:`	逐行迭代	否	每次一个 `str`	`for line in f: print(line.rstrip("\n"))`
`f.write(s)`	写字符串	否	写入的字符个数	`f.write(str(i) + "\n")`
`f.writelines(lst)`	写字符串列表	否	`None`	`f.writelines(lines)`
`print(x, file=f)`	把 print 结果写进文件	否	`None`	`print(i, file=f)`
`f.seek(n)`	移动游标	是	新位置	`f.seek(0)`
`f.tell()`	当前游标位置	否	`int`	`pos = f.tell()`
`f.close()`	关闭文件	是	`None`	交给 `with` 即可
`csv.reader(f)`	迭代读 CSV	否	可迭代对象	`for row in csv.reader(f):`
`csv.writer(f)`	写 CSV	否	writer 对象	`w.writerow([1, 2])`
`os.getcwd()`	当前工作目录	否	`str`	`print(os.getcwd())`
`os.path.join(a, b)`	拼接路径	否	`str`	`os.path.join("data", "a.txt")`
`os.path.isfile(p)`	是否是文件	否	`bool`	`if os.path.isfile(p):`
`os.listdir(p)`	列出目录内容	否	`list`	`os.listdir(".")`
`os.walk(p)`	递归遍历目录	否	生成器	`for root, dirs, files in os.walk("."):`

方法/语法	作用	是否原地修改	返回值	典型例子
`shutil.copy(s, d)`	复制文件	否	新路径 `str`	`shutil.copy("a.txt", "b.txt")`

三、易错清单

1. **错误现象**：`UnicodeDecodeError: 'gbk' codec can't decode byte ...` → **原因**：Windows 中文环境默认用 GBK 解码，文件却是 UTF-8 的 → **正确写法**：读写都写全，`open(p, "r", encoding="utf-8")`。
2. **错误现象**：原来的文件内容莫名其妙没了 → **原因**：用 `"w"` 打开，打开瞬间就清空 → **正确写法**：追加用 `"a"`，确实要覆盖再用 `"w"`。
3. **错误现象**：程序跑完了，文件里却是空的（或只有一部分） → **原因**：忘记 `close()`，数据还在缓冲区 → **正确写法**：全部套 `with open(...) as f:`，必要时 `f.flush()`。
4. **错误现象**：打印出来每行之间多一个空行；用 `==` 比较行内容永远不相等 → **原因**：逐行读到的字符串行尾带 `\n` → **正确写法**：`line.rstrip("\n")` 处理后再用。
5. **错误现象**：第二次 `f.read()` 得到空串 → **原因**：游标已经停在文件末尾 → **正确写法**：先 `f.seek(0)` 再读；大文件改用 `for line in f`。
6. **错误现象**：`TypeError: write() argument must be str, not int` → **原因**：`write()` 只收字符串 → **正确写法**：`f.write(str(n) + "\n")` 或 `print(n, file=f)`。
7. **错误现象**：`FileNotFoundError: [Errno 2] No such file or directory` → **原因**：相对路径是按"当前工作目录"找的，不是按 .py 文件所在目录 → **正确写法**：先 `print(os.getcwd())` 和 `os.path.abspath(p)` 看清楚，或用绝对路径、`os.path.join` 拼路径。
8. **错误现象**：`SyntaxError: (unicode error) 'unicodeescape' codec ...` → **原因**：路径里的 `\U`、`\n` 被当成转义符 → **正确写法**：`r"C:\Users\Lenovo"` 或 `"C:/Users/Lenovo"`。
9. **错误现象**：写出的 CSV 每两行之间多一个空行 → **原因**：Windows 文本模式把 `\r\n` 又翻译了一次 → **正确写法**：`open(p, "w", encoding="utf-8", newline="")`。
10. **错误现象**：`ValueError: I/O operation on closed file` → **原因**：在 `with` 块外面继续用这个文件对象 → **正确写法**：读写全部写在 `with` 块内部。
11. **错误现象**：CSV 里某个字段带着引号、或者列数对不上 → **原因**：用 `split(",")` 手工切分，值内部的逗号被切断了 → **正确写法**：用 `csv` 模块的 `csv.reader` / `csv.writer`。

四、必须能默写的 6 个片段

1. 逐行读文件（最常用，任何文件都套这个模板）：

```
with open("data.txt", "r", encoding="utf-8") as f:
    for line in f:
        line = line.rstrip("\n")
        print(line)
```

1. 把一串数字写进文件，每行一个：

```
with open("nums.txt", "w", encoding="utf-8") as f:
    for i in range(1, 101):
        f.write(str(i) + "\n")
```

1. 把文件里的数字读成列表（跳过空行）：

```
nums = []
with open("nums.txt", "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if line != "":
            nums.append(int(line))
```

1. 用 `with` 同时打开两个文件做复制：

```
with open("a.txt", "r", encoding="utf-8") as fin, \
     open("b.txt", "w", encoding="utf-8") as fout:
    fout.write(fin.read())
```

1. 读之前先确认文件在不在：

```
import os

if os.path.isfile("data.txt"):
    with open("data.txt", "r", encoding="utf-8") as f:
        print(f.read())
else:
    print("文件不存在")
```

1. CSV 的读与写：

```
import csv

with open("in.csv", "r", encoding="utf-8", newline="") as f:
    reader = csv.reader(f)
    next(reader)                    # 跳过表头
    for row in reader:
        print(row)

with open("out.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["张三", 90])
```

5.9 本章自测（13 道）

第 5 章 自测题 1　`open('a.txt', 'w')` 和 `open('a.txt', 'a')` 有什么区别？如果 a.txt 里原本有内容，分别会发生什么？

第 5 章 自测题 2　为什么把 `read()` 连续调用两次，第二次会读到空字符串？

第 5 章 自测题 3　`read()`、`readline()`、`readlines()` 三个函数分别返回什么？

第 5 章 自测题 4　编程：把 1~100 写入文件 `num.txt`，每行一个数字。

第 5 章 自测题 5　编程：读取 `num.txt`，计算所有数字的和并输出。

第 5 章 自测题 6　`with open('a.txt') as f:` 相比 `f = open('a.txt')` 有什么好处？

第 5 章 自测题 7　下面的代码哪里有问题？

```
f = open('data.csv')
for line in f:
    print(line.split(','))
```

第 5 章 自测题 8　编程：统计当前目录下所有 `.py` 文件的个数和总字节数（提示：`os.walk()` 配合 `os.path.splitext()`）。

第 5 章 自测题 9（OJ 风格）　文件 `numbers.txt` 每行一个整数，其中可能夹着空行。读入全部有效整数，按下面的顺序各输出一行：有效整数个数、最大值、最小值、平均值（保留两位小数）。`numbers.txt` 的内容如下：

```
5
12

-3
8
```

第 5 章 自测题 10（OJ 风格）　文件 `score.txt` 每行格式为 `姓名 分数`（姓名中不含空格）。输出分数最高的同学姓名；若最高分有多人，输出最先出现的那一位。`score.txt` 的内容如下：

```
张三 87
李四 95
王五 78
```

第 5 章 自测题 11（OJ 风格）　从键盘输入一个整数 n（1 ≤ n ≤ 100），把 1~n 每个整数写入 `nums.txt`（每行一个）；然后重新打开 `nums.txt`，读回这些数并输出它们的平均值（保留一位小数）。输入 `5` 时，`nums.txt` 中应是 1 到 5 各占一行，屏幕输出 `3.0`。

第 5 章 自测题 12　在 Windows 上用 `open("a.txt", "r")` 读一个含中文的 UTF-8 文件，报 `UnicodeDecodeError: 'gbk' codec can't decode byte ...`。原因是什么？应该怎么改？写文件时要不要也加编码？

第 5 章 自测题 13　下面这段代码运行后，`out.txt` 常常是空的。为什么？请给出两种改法。

```
f = open("out.txt", "w", encoding="utf-8")
f.write("hello")
```

第 6 章 数值处理与计算

> 对应课程讲义：《07_数值处理与计算》。这一章的知识点在前面各章都零散出现过，这里集中起来讲透，重点补上 OJ 里最常用的那批函数，以及"随机数到底取哪个区间""时间有哪三种表示法""计时该用哪个函数"这三件事。学完请务必完成本章的 10 道自测题。

6.1 math 模块（速查）

`math` 是标准库，不用安装，但**必须先导入**：`import math`。

> **【注意】** 这是本章与 1.9 节唯一重叠的地方。1.9 节已经讲过的 `sqrt`、`pi`、`degrees` 等这里不再重复，下面只列全表 + 补充 OJ 常用的那几个。

常用函数全表

分类	函数	说明
取整	`ceil(x)`	大于或等于 x 的最小整数
取整	`floor(x)`	小于或等于 x 的最大整数
取整	`trunc(x)`	向 0 截断取整，等价于 `int(x)`
取整	`modf(x)`	返回 `(小数部分, 整数部分)` 的元组
绝对值	`fabs(x)`	返回 x 的绝对值（float）
阶乘	`factorial(x)`	返回 x 的阶乘，x 必须是**非负整数**
取余	`fmod(x, y)`	浮点余数，符号与 x 相同（和 `%` 不一样，见下）
求和	`fsum(iterable)`	迭代器中浮点数的**精确**求和
判断	`isfinite(x)`	既不是无穷大也不是 NaN 时返回 True
判断	`isinf(x)` / `isnan(x)`	是无穷大 / 是 NaN 时返回 True
判断	`isclose(a, b)`	a 与 b 在容差内"足够接近"时返回 True
数论	`gcd(a, b)`	最大公约数，返回整数
数论	`lcm(a, b)`	最小公倍数，返回整数（**Python 3.9 起才有**）
组合数	`comb(n, k)` / `perm(n, k)`	组合数 / 排列数（3.8 起）
连乘	`prod(iterable)`	迭代器中所有数的乘积（3.8 起）
距离	`dist(p, q)`	两点间欧几里得距离（3.8 起）
拆分	`frexp(x)`	返回尾数和指数 `(m, e)`
指数	`exp(x)` / `expm1(x)`	e 的 x 次方 / e 的 x 次方减 1

分类	函数	说明
幂	`pow(x, y)` / `ldexp(x, i)`	x 的 y 次幂 / x * (2 ** i)
对数	`log(x[, base])`	x 的对数，默认底数为 e
对数	`log1p(x)` / `log2(x)` / `log10(x)`	1+x 的自然对数 / 以 2 为底 / 以 10 为底
方根	`sqrt(x)` / `hypot(x, y)`	平方根 / √(x²+y²)
三角	`sin / cos / tan(x)`	参数为**弧度**
反三角	`asin / acos / atan(x)`、`atan2(y, x)`	返回**弧度**
双曲	`sinh / cosh / tanh(x)` 及 `a*` 系列	双曲函数及其反函数
角度	`degrees(x)` / `radians(x)`	弧度转角度 / 角度转弧度
特殊	`erf(x)` / `erfc(x)` / `gamma(x)` / `lgamma(x)`	误差函数、Gamma 函数等
常量	`math.pi` / `math.e` / `math.inf` / `math.nan`	3.141592653589793 / 2.718281828459045 / 无穷大 / 非数

> **【核心】** 优先级从高到低：`sqrt`、`pi`、`floor` / `ceil`、`gcd` / `lcm`、`isclose`、`inf`。这 6 组覆盖了绝大多数题目。
>
> **【易错】** 这几个函数的**返回类型不一样**，判题时错一个类型就可能报错：

```
import math

print(type(math.ceil(2.1)))    # <class 'int'>     取整函数返回整数
print(type(math.trunc(2.9)))   # <class 'int'>
print(type(math.sqrt(4)))      # <class 'float'>   结果是 2.0 而不是 2
print(type(math.pow(2, 3)))    # <class 'float'>   结果是 8.0
print(2 ** 3)                  # 8                 要整数结果用 ** 或内置 pow()
print(math.pow(2, 3))          # 8.0
```

> **【速查】几条冷门但有用的规则**（知道就行，不必背）：

表达式	结果	说明
`math.gcd(-12, 18)`	6	负数按绝对值算
`math.gcd(0, 5)`	5	`gcd(0, x)` 等于 `abs(x)`
`math.lcm(0, 5)`	0	有一个是 0，结果就是 0
`math.factorial(0)`	1	0 的阶乘规定为 1
`math.fmod(-7, 3)`	-1.0	余数符号与**被除数**相同
`-7 % 3`	2	`%` 的余数符号与**除数**相同

表达式	结果	说明
`math.inf > 10 ** 308`	True	无穷大比任何有限浮点数都大
`-math.inf < -10 ** 308`	True	负无穷小同理
`math.sqrt(-1)`	ValueError	报 `math domain error`

6.1.1 浮点数能不能用 == 比？用 math.isclose

1.7 节讲过浮点数的存储误差，第 2 章作业里也出现过"两数之差小于 0.000001 就认为相等"。现在有了更省事的写法：

```
import math

print(0.1 + 0.2 == 0.3)                          # False  浮点误差
print(abs(0.1 + 0.2 - 0.3) < 1e-9)               # True   老办法：比差值
print(math.isclose(0.1 + 0.2, 0.3))              # True   新办法：容差内即相等

print(math.isclose(1.0, 1.0001))                 # False  默认相对容差 1e-9
print(math.isclose(1.0, 1.0001, abs_tol=1e-3))   # True   放宽绝对容差
print(math.isclose(1e10, 1e10 + 1))              # True   相对容差对"大数"很宽松
```

【注意】 `isclose` 的两个容差参数：`rel_tol` 是**相对**容差（默认 1e-9），`abs_tol` 是**绝对**容差（默认 0.0）。判断"接近 0 的两个小数"必须自己给 `abs_tol`，否则 `math.isclose(1e-12, 0)` 是 False。

【注意】 OJ 题里如果写的是"两个浮点数之差的绝对值小于 0.000001 时可认为相等"，照题目给的精度写 `abs(a - b) < 1e-6` 最稳妥；输出时题目要几位小数就用 `f"{x:.3f}"` 保留几位。

6.1.2 floor / ceil / trunc / int 对负数的区别（易错重点）

这是本章和 OJ 里的高频陷阱。记住一句话：**`int()` 和 `trunc()` 是"朝 0 砍"，`floor()` 是"朝下走"。**

x	int(x)	math.trunc(x)	math.floor(x)	math.ceil(x)	round(x)
2.5	2	2	2	3	2
-2.5	**-2**	**-2**	**-3**	-2	-2
7.0	7	7	7	7	7
-7.9	-7	-7	**-8**	-7	-8

【常见错误】 以为 `math.floor(x)` 和 `int(x)` 总是一回事，结果负数取整差了 1。

```
# ✗ 错误写法：-7 除以 2，想向下取整却写成 int()
x = -7
print(int(x / 2))        # -3   int() 朝 0 截断，不是向下取整
```

```
# ✓ 正确写法：向下取整用 // 或 math.floor()
x = -7
print(x // 2)            # -4
```

```
import math
print(math.floor(x / 2)) # -4
```

对正数两者结果相同，所以你很难在自测中发现；一旦数据里有负数，答案就错了。

【核心】`//` 与 `%` 在负数上的规则：Python 的 `//` 是"向下取整除法"，`%` 的结果符号跟**除数**一致，这和 C / Java 的"向 0 截断"不一样。`-7 // 2` 得 `-4`（不是 -3），`-7 % 3` 得 `2`（不是 -1）。写题时如果拿不准，就用 `math.floor()` / `math.fmod()` 把意图写清楚。

例题 6-3：最大公约数与最小公倍数

题目：输入两个正整数 a、b，输出它们的最大公约数和最小公倍数，一行一个。

```
import math

a, b = map(int, input().split())
print(math.gcd(a, b))
print(math.lcm(a, b))
```

输入 `12 18`，输出：

```
6
36
```

要点：如果你用的 Python 版本低于 3.9，没有 `math.lcm`，可以自己写一行：`lcm = a * b // math.gcd(a, b)`。另外注意 `math.gcd` 要求参数是整数，传浮点数会得到 `TypeError`。

例题 6-4：判断两个浮点数是否"相等"

题目：读入三个浮点数 a、b、c，先输出 a + b 的值，再判断 a + b 与 c 是否相等（容差 1e-9），相等输出 `相等`，否则输出 `不相等`。

```
import math

a, b, c = map(float, input().split())
total = a + b
print(total)
if math.isclose(total, c, rel_tol=1e-9, abs_tol=1e-9):
    print("相等")
else:
    print("不相等")
```

输入 `0.1 0.2 0.3`，输出：

```
0.30000000000000004
相等
```

> **要点**：打印出来的 `0.30000000000000004` 是正确的浮点数结果，不是 Python 出错。如果把 `if` 写成 `if total == c:`，这一题会输出"不相等"。

6.2 random 模块（速查 · 核心）

使用前 `import random`。2.2 节已经见过 `randint` 和 `randrange`，这一节把它们和其它常用函数一次列全。

函数	返回值 / 作用	取值范围
`random.random()`	随机浮点数	[0.0, 1.0)，**永远取不到 1.0**
`random.uniform(a, b)`	随机浮点数	[a, b]（a > b 也可以，不报错）
`random.randint(a, b)`	随机整数	**[a, b]，含两端**
`random.randrange(a, b[, step])`	随机整数	**[a, b)，不含右端**
`random.randrange(n)`	随机整数	[0, n)
`random.choice(seq)`	从序列中随机取**一个**元素	序列不能为空
`random.choices(seq, k=n)`	取 n 个元素，组成**列表**	可能取到重复元素
`random.sample(seq, k=n)`	取 n 个**互不相同**的元素，组成列表	seq 不能是集合，n 不能超过长度
`random.shuffle(lst)`	原地打乱列表，**返回 None**	只能传可变序列（列表）
`random.seed(x)`	设定随机种子，让结果可复现	参数可以是整数、字符串等
`random.getrandbits(k)`	返回 k 个二进制位的随机整数	[0, 2**k)

```
import random

random.seed(2)                          # 设定种子，下面每一次结果都固定
dice = [random.randint(1, 6) for _ in range(5)]
print(dice)                             # [1, 1, 1, 3, 2]  换台电脑跑也是这个结果

print(random.random())                  # 0.7359699890685233   小数部分每次不同
print(random.uniform(1, 2))             # 1.669730401440221
print(random.randrange(0, 50, 5))       # 20   取的是 0/5/10/…/45 中的一个
print(random.choice([1, 2, 3, 4, 5]))   # 3    随机取一个元素

lst = list(range(5))
random.shuffle(lst)
print(lst)                              # [3, 0, 2, 1, 4]  原列表被改变
```

> **【注意】** 上面注释里的数值，是把这段代码**从头到尾连着跑一遍**、且种子为 2 时的真实结果。如果你只复制其中一行单独运行，取到的是随机序列里的下一个数，数值会不一样——这正是"随机数按顺序一个个取"的体现。
>
> **【注意】`randrange` 的三种写法**：`randrange(10)` 等价于 `randrange(0, 10)`，取 0~9；`randrange(1, 10)` 取 1~9；`randrange(0, 50, 5)` 只在 0、5、10、…、45 里取。步长 `step` 只能是非零整数。

> 【核心】 `random.seed(x)` 之后，随机数序列就**完全固定**了：同一个种子、同一条代码路径，每次运行得到的都是同一串数。它的用途有两个——① 调试时让"偶发错误"稳定重现；② 老师出题时固定答案。写作业演示程序时，加上 `seed` 能让你的运行结果和同学的截图对得上。

例题 6-1：随机生成一个四则运算题

题目：随机生成一道 100 以内的四则运算题，让用户作答，并判断对错。减法要保证结果非负，除法要保证能整除。

```
import random

n1 = random.randint(1, 9)
n2 = random.randint(1, 9)
method = random.randint(1, 4)

if method == 1:
    print(n1, "+", n2, "=?")
    answer = int(input())
    if answer == n1 + n2:
        print("回答正确")
    else:
        print("回答错误，正确答案是", n1 + n2)
elif method == 2:
    if n1 < n2:
        n1, n2 = n2, n1
    print(n1, "-", n2, "=?")
    answer = int(input())
    if answer == n1 - n2:
        print("回答正确")
    else:
        print("回答错误，正确答案是", n1 - n2)
elif method == 3:
    print(n1, "*", n2, "=?")
    answer = int(input())
    if answer == n1 * n2:
        print("回答正确")
    else:
        print("回答错误，正确答案是", n1 * n2)
else:
    n3 = n1 * n2
    print(n3, "/", n1, "=?")
    answer = int(input())
    if answer == n2:
        print("回答正确")
    else:
        print("回答错误，正确答案是", n2)
```

运行示意（随机数每次不同，这里假设 `n1 = 3`、`n2 = 7`、`method = 2`）：

```
7 - 3 =?
4
回答正确
```

要点：① 用 `method = random.randint(1, 4)` 来"随机挑一种运算"，这是 `if / elif / else` 配随机数的固定套路；② 减法前先 `n1, n2 = n2, n1` 交换，保证被减数不小于减数（这是 1.4 节讲过的解包交换）；③ 除法故意出 `n3 / n1`，这样答案一定是整数 `n2`，不会出现除不尽的情况。

6.2.1 区间到底含不含端点（重灾区）

【常见错误】 把 `randint` 和 `randrange` 记成一样的，取值范围差 1。

```
# X 错误写法：想取 1~10 的随机整数，却写成 randrange(1, 10)
import random
x = random.randrange(1, 10)      # 永远取不到 10
```

```
# √ 正确写法
import random
x = random.randint(1, 10)          # 1~10，含 10
y = random.randrange(1, 11)        # 效果同上
z = random.randrange(1, 10)        # 1~9，不含 10
```

记忆法：**`randint` 的 t 可以想成 "two ends"（两端都含）；`randrange` 和 `range()` 一样不含右端。**另外 `randint(a, b)` 在 a > b 时会抛 `ValueError: empty range in randrange(2, 2)`，而 `randint(a, a)` 合法，永远返回 a。

【注意】 `randint` 底层就是 `randrange`，所以 `randint(a, b)` 等价于 `randrange(a, b + 1)`。用哪个人习惯就好，**同一段代码里别混着写**，否则很容易自己绕晕。

6.2.2 choice / choices / sample / shuffle 的区别

这四个函数都在"从序列里挑东西"，但返回值差别很大：

函数	挑几个	是否可重复	是否改变原序列	返回值
`choice(seq)`	1 个	——	否	单个元素
`choices(seq, k=n)`	n 个	**可以重复**	否	新列表
`sample(seq, k=n)`	n 个	**不可重复**	否	新列表
`shuffle(lst)`	全部	——	**是（原地打乱）**	`None`

```
import random

random.seed(7)
print(random.sample(range(1, 11), 5))      # [6, 3, 7, 10, 1]  五个互不相同的数
print(random.choices(range(1, 11), k=5))   # [1, 6, 4, 1, 6]   有重复，这就是"有放回"
```

【常见错误】 想洗牌，结果写成 `lst = random.shuffle(lst)`，`lst` 变成了 `None`。

```
# X 错误写法
import random
cards = [1, 2, 3, 4, 5]
cards = random.shuffle(cards)
print(cards)          # None     函数原地打乱，不返回新列表
```

```
# √ 正确写法
import random
cards = [1, 2, 3, 4, 5]
random.shuffle(cards)     # 直接调用，不要接返回值
print(cards)              # [4, 1, 5, 2, 3]
```

要"打乱后的新列表、原列表不动"，就写 `new = random.sample(cards, len(cards))`。

【易错】 `random.sample` 不能对集合或字典使用，会报 `TypeError: Population must be a sequence. For dicts or sets, use sorted(d).`（要先 `sorted(s)` 转成列表）；`k` 大于序列长度会报 `ValueError: Sample larger than population or is negative`。想"有放回地抽 n 次"，必须用 `choices`。

6.2.3 seed 放在哪里（OJ 与调试）

【易错】 同一份 `seed` 代码，写在循环**外**和写在循环**内**，结果完全不一样。

```
import random

random.seed(1)                        # 只在开头设一次
print([random.randint(1, 6) for _ in range(3)])   # [2, 5, 1]

random.seed(1)                        # 每次取数前都重设
print([random.randint(1, 6) for _ in range(3)])   # [2, 5, 1]  和上面一样

for _ in range(3):
    random.seed(1)                    # 写在循环里：每次都拿到同一个数
    print(random.randint(1, 6), end=" ")          # 2 2 2
```

原因：随机数是一个一个从序列里取的，重设种子等于把指针拨回开头。**要复现"第 i 次"的结果，就把 `seed` 放在那次之前。**

例题 6-5：可复现的掷骰子统计

题目：先用 `random.seed(2)` 固定随机序列，再掷 5 次骰子（每次 `randint(1, 6)`），输出这 5 次点数以及点数之和。

```
import random

random.seed(2)
```

```
dice = [random.randint(1, 6) for _ in range(5)]
print(dice)
print(sum(dice))
```

输入：无。输出：

```
[1, 1, 1, 3, 2]
8
```

> **要点**：`seed(2)` 决定了这 5 个数一定是 `1 1 1 3 2`。你可以把 `seed(2)` 改成 `seed(42)` 试试，只要种子相同，在任何电脑上跑出来的结果都相同。做题时若题目给了种子，**一定要照抄那个数字**。

6.3 time 模块

`time` 是标准库。它本身不复杂，难的是"同一个时刻有三种表示法"，初学时极容易搞混方向。

6.3.1 三种时间表示法

概念	类型	长什么样	怎么得到
① 时间戳 timestamp	`float`	`1760000000.123456`	`time.time()`
② 时间元组 struct_time	`time.struct_time`	`time.struct_time(tm_year=2026, tm_mon=10, ...)`	`time.localtime()` / `time.gmtime()` / `time.strptime()`
③ 格式化字符串	`str`	`"2026-10-08 15:30:45"`	`time.strftime()` / `time.ctime()`

补充两个前置概念：

概念	含义
UTC	协调世界时，即世界标准时间，采用 24 小时制
Epoch	Unix 纪元，即 1970 年 1 月 1 日 00:00:00 UTC
Unix 时间戳	从 Epoch 开始所经过的秒数，不考虑闰秒

`struct_time` 一共 9 个字段：`tm_year`、`tm_mon`、`tm_mday`、`tm_hour`、`tm_min`、`tm_sec`、`tm_wday`（星期几，**0 表示周一**）、`tm_yday`（当年第几天，1~366）、`tm_isdst`（是否夏令时，一般 -1）。

> **【注意】** `struct_time` 是"带名字的元组"，两种取值方式都行：`t.tm_year` 或 `t[0]`。所以 `t[0]` 是年，不要写成 `t[1]`。

6.3.2 获取当前时间与计时

```
import time

timestamp = time.time()          # 时间戳（秒级浮点数）
local_time = time.localtime()    # 本地时间的 struct_time
```

```
utc_time = time.gmtime()          # UTC 时间的 struct_time

print("当前时间戳:", timestamp)
print("本地时间:", local_time)
print("UTC时间:", utc_time)
```

运行结果示意（具体结果依运行时间而不同）：

```
当前时间戳: 1715833304.1631322
本地时间: time.struct_time(tm_year=2024, tm_mon=5, tm_mday=16, tm_hour=12, ...)
UTC时间: time.struct_time(tm_year=2024, tm_mon=5, tm_mday=16, tm_hour=4, ...)
```

【核心】 `time.time()` 返回的是**秒级浮点数**，它按系统时钟走，可能被系统校时或时区影响，**不适合测量程序跑了多久**。要计时请用 `time.perf_counter()`——它返回一个只能"相减"的高精度单调时钟，永远只增不减。

【常见错误】 计时用 `time.time()`，测出来的时间忽大忽小甚至为负。

```
# ✗ 错误写法：time.time() 可能被系统对时影响，精度也低
import time
start = time.time()
total = sum(range(1000000))
print("耗时", time.time() - start, "秒")
```

```
# ✓ 正确写法：计时专用 perf_counter()
import time
start = time.perf_counter()
total = sum(range(1000000))
elapsed = time.perf_counter() - start
print(f"耗时 {elapsed:.3f} 秒")     # 耗时 0.016 秒
```

记住一句话：**要"现在是几点"用 `time.time()`，要"跑了多久"用 `time.perf_counter()`。** 还有 `time.perf_counter_ns()` 返回纳秒整数，适合测极短的操作；`time.sleep(n)` 是让程序暂停 n 秒，不要拿它当计时用。

【了解】 `time.time()` 的精度取决于操作系统，Windows 上大约 1 毫秒。它有用的一面是"时间戳相减就是秒数"：`int(time.time())` 常用作随机数种子（例如 `random.seed(int(time.time()))`）。

6.3.3 时间格式化：strftime 与 strptime

```
import time

# struct_time 对象 → 字符串
formatted_time = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime())
print("格式化后的本地时间:", formatted_time)      # 2024-05-16 12:22:02

# 字符串 → struct_time 对象
```

```
parsed_time = time.strptime("2021-09-10 10:22:47", "%Y-%m-%d %H:%M:%S")
print("解析后的时间:", parsed_time)
```

【核心】 `strftime` 是"格式化"，`strptime` 是"解析"，一个是 f（format），一个是 p（parse）。名字很像，务必分清：**看到 `str` 在前就是把字符串当"原料"，看到 `f` 在前就是产出字符串。**

【常见错误】 把 `strftime` 和 `strptime` 的参数顺序或方向记反，程序直接报错。

```
# X 错误写法：把字符串喂给 strftime，把 struct_time 喂给 strptime
import time
print(time.strftime("2021-09-10", "%Y-%m-%d"))     # TypeError
print(time.strptime(time.localtime(), "%Y-%m-%d")) # TypeError
```

```
# ✓ 正确写法：格式串永远放在第二个参数
import time
print(time.strftime("%Y-%m-%d", time.localtime()))            # 2026-10-08
print(time.strptime("2021-09-10", "%Y-%m-%d"))                # struct_time
```

报错信息形如 `TypeError: Tuple or struct_time argument required`（strftime 收到的第一个参数位置不对）和 `TypeError: strptime() argument 0 must be str, not <class 'time.struct_time'>`。另外**格式串必须和输入串严格对上**，`"2026/10/08"` 配 `"%Y-%m-%d"` 会报 `ValueError: time data '2026/10/08' does not match format '%Y-%m-%d'`。

常用格式符表（速查）

格式符	含义	例子
`%Y`	四位年份	2026
`%y`	两位年份	26
`%m`	月份（01~12）	10
`%d`	日（01~31）	08
`%H`	小时，24 小时制（00~23）	15
`%I`	小时，12 小时制（01~12）	03
`%M`	分钟（00~59）	30
`%S`	秒（00~59）	45
`%f`	微秒，6 位（仅 datetime 支持）	223290
`%p`	上午 / 下午（AM / PM）	PM
`%j`	当年第几天（001~366）	281
`%w`	星期几，**0 是周日**	4
`%a` / `%A`	星期简写 / 全称	Thu / Thursday
`%b` / `%B`	月份简写 / 全称	Oct / October

格式符	含义	例子
`%%`	一个百分号	%

> **【易错】** 三个最容易记混的：① `%y` 是两位年份，`%Y` 是四位；② `%H` 是 24 小时制，`%I` 是 12 小时制且必须与 `%p` 一起用；③ **星期几有两种编号**——`struct_time.tm_wday` 和 `date.weekday()` 是"周一 = 0"，而 `strftime("%w")` 是"周日 = 0"，`date.isoweekday()` 是"周一 = 1"。
>
> **【速查】常用的三种格式串**：`"%Y-%m-%d"`（只日期）、`"%H:%M:%S"`（只时间）、`"%Y-%m-%d %H:%M:%S"`（日期时间）。跨年跨月的题目基本都用第一个。

例题 6-6：某日期是当年的第几天（用 %j）

题目：输入一行日期，格式为 `YYYY-MM-DD`，输出它所在的年份和它是当年的第几天。

```
import time

s = input()                                          # 例如 2026-03-01
t = time.strptime(s, "%Y-%m-%d")                      # 字符串 -> struct_time
print(t.tm_year, "年的第", int(time.strftime("%j", t)), "天")
```

输入 `2026-03-01`，输出：

```
2026 年的第 60 天
```

> **要点**：`%j` 得到的是 `"060"` 这样带前导零的字符串，所以要套一层 `int()` 才是 60。如果题目要的是"含当天还是不含当天"，用 6.4 节的双日期相减更不容易错。
>
> **【注意】** `time` 模块**不提供日期加减**（没有 `add` 这种东西）。要在日期上加减，就把日期转成时间戳再换算，或者直接用下一节的 `datetime`——后者才是正道。
>
> **【易错】时间戳转换的两个方向**：`time.localtime(ts)` 是"时间戳 → struct_time"，`time.mktime(t)` 是"struct_time → 时间戳"。想看 `1970-01-01` 长什么样，就写 `time.strftime("%Y-%m-%d", time.localtime(0))`。

6.4 datetime 模块

`time` 只能处理"某一时刻"，要**做日期加减、比较大小、算年龄**，就得用 `datetime` 模块。它里面有四个常用类：

类	表示什么	例子
`date`	只有日期	`date(2026, 10, 8)`
`time`	只有时间	`time(15, 30, 45)`
`datetime`	日期 + 时间	`datetime(2026, 10, 8, 15, 30, 45)`
`timedelta`	一段时间间隔	`timedelta(days=100)`

6.4.1 创建对象与取属性

```
import datetime

# 当前日期和时间
datetime_object = datetime.datetime.now()
print(datetime_object)          # 2025-09-27 15:03:59.223290

# 当前日期
date_object = datetime.date.today()
print(date_object)              # 2025-09-27
```

```
from datetime import date, time, datetime, timedelta

d = date(2019, 4, 13)                    # 表示日期
print(d)                                 # 2019-04-13

today = date.today()
print("当前日期:", today)
print("当前年:", today.year, "当前月:", today.month, "当前日:", today.day)

a = time()                               # 00:00:00
b = time(11, 34, 56)                     # 11:34:56
c = time(hour=11, minute=34, second=56)  # 11:34:56
d2 = time(11, 34, 56, 234566)            # 11:34:56.234566

x = datetime(2019, 11, 28)                        # 2019-11-28 00:00:00
y = datetime(2019, 11, 28, 23, 55, 59, 342380)
print(y.year, y.month, y.day, y.hour, y.minute, y.second)
print(y.timestamp())                              # 时间戳
```

【核心】 `datetime()` 构造函数中的前三个参数 year、month、day 是**必需**的，其余可选。参数顺序是"年 月 日 时 分 秒 微秒"，全部用位置参数写最省事：`datetime(2026, 10, 8, 15, 30, 45)`。

【注意】`now()` 与 `today()` 的区别：`datetime.now()` 返回带时分秒的 datetime，`date.today()` 只返回日期。两者都是**本地时间**（"天真时间"，不带时区信息）；要带时区得写 `datetime.now(timezone.utc)`，本课程不要求。

6.4.2 strftime 与 strptime（datetime 版）

`time` 里学的格式符在 `datetime` 里**完全通用**，只是调用方式变成了"对象.方法"：

```
from datetime import datetime

dt = datetime(2026, 10, 8, 15, 30, 45)
print(dt.strftime("%Y-%m-%d %H:%M:%S"))    # 2026-10-08 15:30:45
print(dt.strftime("%Y年%m月%d日"))          # 2026年10月08日

s = "2000-02-29 08:00:00"
```

```
dt2 = datetime.strptime(s, "%Y-%m-%d %H:%M:%S")
print(dt2)                                  # 2000-02-29 08:00:00
print(dt2.date())                           # 2000-02-29    只取出日期部分
```

【常见错误】 想解析日期却写成 `date.strptime(...)`。

```
# ✗ 错误写法：date 类没有 strptime 方法
from datetime import date
d = date.strptime("2026-10-08", "%Y-%m-%d")
# AttributeError: type object 'datetime.date' has no attribute 'strptime'
```

```
# ✓ 正确写法：用 datetime.strptime，再取 .date()
from datetime import datetime
d = datetime.strptime("2026-10-08", "%Y-%m-%d").date()
```

`strptime` 是 `datetime` 类的方法（它只能解析"日期 + 时间"）。反向操作 `strftime` 则三个类都有，`date`、`time`、`datetime` 都能直接调用。

6.4.3 timedelta：两个日期或时间之间的差

```
from datetime import date, datetime, timedelta

t1 = date(year=2018, month=7, day=12)
t2 = date(year=2017, month=12, day=23)
t3 = t1 - t2
print("t3 =", t3)              # 201 days, 0:00:00
print(type(t3))                # <class 'datetime.timedelta'>

a = timedelta(weeks=2, days=5, hours=1, seconds=33)
b = timedelta(days=4, hours=11, minutes=4, seconds=54)
print("a - b =", a - b)        # 14 days, 13:55:39
```

【核心】 `timedelta 的用法就三条**：① 两个日期**相减**得到 timedelta（同类型之间才能相减）；② **日期 ± timedelta** 得到新日期（做加减）；③ 取"差多少天"用属性 **.days**，它是一个整数；要总秒数用 **.total_seconds()`**（返回 float）。

```
from datetime import date, timedelta
print((date(2026, 10, 8) - date(2026, 1, 1)).days)          # 280
print((date(2026, 10, 8) + timedelta(days=100)))            # 2027-01-16
print(timedelta(minutes=90).total_seconds())                # 5400.0
```

【易错】 `timedelta` 的属性叫 `days` 和 `seconds`（`seconds` 只表示**不足一天的秒数**，范围 0~86399），不要和 `total_seconds()` 搞混。另外 `timedelta` 的关键字参数是 `weeks / days / hours / minutes / seconds / milliseconds / microseconds`，**没有 `months` 和 `years`**——因为月份天数不固定，要"加一个月"只能自己判断。

例题 6-7：计算两个日期相差多少天

题目：输入两行日期（每行"年 月 日"三个整数，用空格分隔），输出它们相差的天数（结果取绝对值）。

```
from datetime import date

y1, m1, d1 = map(int, input().split())
y2, m2, d2 = map(int, input().split())
days = (date(y2, m2, d2) - date(y1, m1, d1)).days
print(abs(days))
```

输入：

```
2026 1 1
2026 3 1
```

输出：

```
59
```

要点：两个 `date` 相减直接得到 `timedelta`，`.days` 取出整数天数，闰年、大小月都由 Python 自动处理。输入读法上，`eval(input())` 对多余空格很敏感（会直接报 `SyntaxError`），`map(int, input().split())` 才稳。

【核心】 有了 `datetime` 模块，前面作业题里"算第几天""算距月底还有几天"这一类问题都有了更省事的写法。但考试中通常要求你用**基本原理**（闰年判断 + 月份天数表）手写——两种能力都要会。

例题 6-8：从出生日期算年龄

题目：输入出生日期（一行"年 月 日"三个整数），以 `2026-10-08` 为"今天"，输出活了几天、年龄几岁零几天。

```
from datetime import date

y, m, d = map(int, input().split())
today = date(2026, 10, 8)
days = (today - date(y, m, d)).days
print("活了", days, "天")
print(f"年龄:{days // 365} 岁 {days % 365} 天")
```

输入 `2003 1 20`，输出：

```
活了 8662 天
年龄: 23 岁 267 天
```

要点：这里把"今天"写死成固定日期，是为了让你自己的运行结果和讲义一致；实际写成 `date.today()` 更有意义。至于"几岁零几天"，用 `days // 365` 和 `days % 365` 只是近似算法（没考虑闰年），考试里如果要求精确，就要按年、月逐段减，或者用 `relativedelta`（第三方库，不在本课程范围）。

> **【核心】** 下面这道例题 6-2 和例题 6-7 是同一类题（都是"两个日期差几天"），区别只在**输入格式**：6-2 用的是校内讲义常见的 `eval(input())` 逗号写法，6-7 用的是 OJ 更常见的空格写法。两题的思路完全一样，你会一题就会两题。

例题 6-2：计算两个日期相差多少天

题目：输入两行日期（每行"年 月 日"三个用逗号分隔的整数），输出它们相差的天数。

```
from datetime import date

y1, m1, d1 = eval(input())
y2, m2, d2 = eval(input())
days = (date(y2, m2, d2) - date(y1, m1, d1)).days
print(abs(days))
```

输入：

```
2019, 4, 13
2000, 2, 29
```

输出：

```
6983
```

> **要点**：`eval(input())` 会把 `2019, 4, 13` 当成元组 `(2019, 4, 13)` 来解包，这是校内讲义常用的输入写法；但它对非法输入毫无抵抗力，多打一个空格都可能报 `SyntaxError`，所以**更推荐用** `map(int, input().split())`（写法见例题 6-7）。
>
> **【注意】** `2000-02-29` 是合法的（2000 年是闰年），而 `1900-02-29` 会直接抛 `ValueError: day is out of range for month`——闰年规则由 `datetime` 自动判断，你不用自己写 `% 4` 和 `% 400`。
>
> **【核心】** 顺便回看例题 6-2 的代码：`2019, 4, 13` 这种输入之所以能用 `eval` 解包，是因为 `eval` 把它当成元组 `(2019, 4, 13)`。这个技巧校内讲义很爱用，但只适合"输入完全规矩"的场合。

6.5 本章方法总结与易错清单

一、本章知识地图

本章的四个模块不是并列关系，而是"数字 → 随机 → 时间"三条线：

- **① 单个数字怎么算**
  - 加减乘除与取整：`//`、`%`、`int()`、`round()`（1.8 节、1.10 节）
  - `math` 模块补上"课上没讲全"的数学函数：`gcd` / `lcm` / `isclose` / `inf` / `floor` / `ceil`
  - 底层问题：浮点数不精确 → 所以才需要 `isclose`（1.7 节）
- **② 随机数怎么来**
  - `random.seed()` 决定"可复现"

- 取一个数：`randint`（含两端） / `randrange`（不含右端） / `random` / `uniform`
- 取一组数：`choice`（1 个） / `choices`（可重复） / `sample`（不重复） / `shuffle`（原地打乱）
- `random` 建立在 `math` 之上（内部要算浮点）；做统计题时又要和列表、循环配合（第 4 章）

- **③ 时间怎么表示和计算**
  - `time`：三种表示法（时间戳 / struct_time / 格式化字符串）+ 两种计时器（`time` / `perf_counter`）
  - `datetime`：四个类（`date` / `time` / `datetime` / `timedelta`）
  - 两模块共享同一套**格式符**（`%Y` `%m` `%d` `%H` `%M` `%S`），格式符是本章的"公共地基"

> **考试最常考的五个点**：① `randint` 与 `randrange` 的区间差别；② `math.floor` 与 `//` 在负数上的表现；③ `strftime` 与 `strptime` 的方向；④ `timedelta` 做日期加减；⑤ 浮点数不能直接 `==`。

二、方法总结（速查表）

math

函数	作用	返回类型	例子
`math.floor(x)`	向下取整	int	`math.floor(-2.5)` = -3
`math.ceil(x)`	向上取整	int	`math.ceil(-2.5)` = -2
`math.trunc(x)`	向 0 截断	int	`math.trunc(-2.5)` = -2
`math.fabs(x)`	绝对值	float	`math.fabs(-2)` = 2.0
`math.sqrt(x)`	平方根	float	`math.sqrt(16)` = 4.0
`math.pow(x, y)`	幂	float	`math.pow(2, 3)` = 8.0
`math.gcd(a, b)`	最大公约数	int	`math.gcd(12, 18)` = 6
`math.lcm(a, b)`	最小公倍数（3.9+）	int	`math.lcm(12, 18)` = 36
`math.factorial(n)`	阶乘	int	`math.factorial(5)` = 120
`math.isclose(a, b)`	容差内相等	bool	`math.isclose(0.1+0.2, 0.3)` = True
`math.isinf(x)` / `isnan(x)`	无穷 / 非数	bool	`math.isinf(math.inf)` = True
`math.inf` / `math.pi`	无穷大 / 圆周率	float	`math.inf > 10 ** 308` = True

random

函数	作用	是否原地修改	返回值	例子
`random.seed(x)`	固定随机序列	否	None	`random.seed(2)`
`random.randint(a, b)`	[a, b] 随机整数	否	int	`randint(1, 10)` → 1~10
`random.randrange(a, b)`	[a, b) 随机整数	否	int	`randrange(1, 10)` → 1~9
`random.random()`	[0, 1) 随机浮点	否	float	`random.random()` → 0.586...

函数	作用	是否原地修改	返回值	例子
`random.uniform(a, b)`	[a, b] 随机浮点	否	float	`uniform(1, 2)` → 1.384...
`random.choice(seq)`	随机取 1 个元素	否	元素本身	`choice([1, 2, 3])` → 2
`random.choices(seq, k=n)`	取 n 个（可重复）	否	list	`choices(range(1, 11), k=5)`
`random.sample(seq, k=n)`	取 n 个（不重复）	否	list	`sample(range(1, 34), 6)`
`random.shuffle(lst)`	原地打乱	**是**	**None**	`random.shuffle(lst)`

time

函数	作用	返回类型	例子
`time.time()`	当前时间戳（秒级浮点）	float	`1715833304.1631322`
`time.perf_counter()`	高精度单调时钟，**只用来计时**	float	`perf_counter() - start`
`time.localtime([ts])`	时间戳 → 本地 struct_time	struct_time	`time.localtime(0)`
`time.gmtime([ts])`	时间戳 → UTC struct_time	struct_time	`time.gmtime()`
`time.mktime(t)`	struct_time → 时间戳	float	`time.mktime(t)`
`time.strftime(fmt, t)`	struct_time → 字符串	str	`strftime("%Y-%m-%d", t)`
`time.strptime(s, fmt)`	字符串 → struct_time	struct_time	`strptime(s, "%Y-%m-%d")`
`time.ctime([ts])`	时间戳 → 可读字符串	str	`date` 风格的英文串
`time.sleep(n)`	暂停 n 秒	None	不要拿它计时

datetime

函数 / 属性	作用	返回类型	例子
`date(y, m, d)`	构造日期	date	`date(2026, 10, 8)`
`datetime.now()` / `date.today()`	当前日期时间 / 当前日期	datetime / date	本地时间
`datetime(y, m, d, H, M, S)`	构造日期时间	datetime	前三个参数必需
`dt.strftime(fmt)`	datetime → 字符串	str	`dt.strftime("%Y-%m-%d")`
`datetime.strptime(s, fmt)`	字符串 → datetime	datetime	再 `.date()` 取日期
`d1 - d2`	日期相减	timedelta	用 `.days` 取天数
`d ± timedelta(...)`	日期加减	date / datetime	`d + timedelta(days=100)`
`timedelta(days=n, ...)`	构造时间差	timedelta	`weeks/days/hours/minutes/seconds`
`d.weekday()` / `d.isoweekday()`	星期几	int	`weekday()` 周一 = 0；`isoweekday()` 周一 = 1

三、易错清单

1. **错误现象**：取 1~10 的随机整数时，`randrange(1, 10)` 永远出不了 10。**原因**：`randrange` 和 `range` 一样**不含右端**。**正确写法**：要含两端就用 `randint(1, 10)` 或 `randrange(1, 11)`。
2. **错误现象**：负数向下取整得错。**原因**：`int(x)` 和 `math.trunc(x)` 是"朝 0 截断"，不是向下取整。**正确写法**：向下用 `x // 1` 或 `math.floor(x)`；`-7 // 2` 得 -4，`int(-7 / 2)` 得 -3。
3. **错误现象**：`while abs(x - 0) > 0` 死循环。**原因**：浮点数永远差那么一点点，`0.1 + 0.2 == 0.3` 是 False。**正确写法**：`math.isclose(a, b)` 或 `abs(a - b) < 1e-9`。
4. **错误现象**：计时结果忽大忽小。**原因**：`time.time()` 按系统时钟走，会被校时影响。**正确写法**：计时用 `time.perf_counter()`（要纳秒用 `perf_counter_ns()`）。
5. **错误现象**：`TypeError: strptime() argument 1 must be str`。**原因**：把 struct_time 喂给了 `strptime`，或者把字符串喂给了 `strftime`。**正确写法**：`strf` 产出字符串、`strp` 消耗字符串，格式串永远是第二个参数。
6. **错误现象**：`AttributeError: type object 'datetime.date' has no attribute 'strptime'`。**原因**：`date` 类没有 `strptime`。**正确写法**：`datetime.strptime(s, fmt).date()`。
7. **错误现象**：`lst = random.shuffle(lst)` 之后 `lst` 变成 `None`。**原因**：`shuffle` 原地打乱、返回 None。**正确写法**：单独调用 `random.shuffle(lst)`；要新列表用 `random.sample(lst, len(lst))`。
8. **错误现象**：给两个数求最大公约数，结果报 `TypeError`。**原因**：`math.gcd` 只接受整数，输入里带小数或字符串。**正确写法**：先 `int()` 转换：`a, b = map(int, input().split())`；求最小公倍数用 `math.lcm(a, b)`（低于 3.9 写 `a * b // math.gcd(a, b)`）。

四、必须能默写的 8 个片段

1. **可复现的随机数**

```
import random
random.seed(2)                      # 换一个数字就换一串结果
print([random.randint(1, 6) for _ in range(5)])   # [1, 1, 1, 3, 2]
```

1. **不重复抽样**

```
import random
print(random.sample(range(1, 34), 6))    # 6 个互不相同的数
```

1. **浮点数比较**

```
import math
print(math.isclose(0.1 + 0.2, 0.3))             # True
print(abs(0.1 + 0.2 - 0.3) < 1e-9)              # True（不用 math 时）
```

1. **向下取整 vs 截断**

```
import math
x = -7
print(x // 2, math.floor(-2.5), math.trunc(-2.5), int(-2.5))   # -4 -3 -2 -2
```

1. **计时模板**

```
import time
start = time.perf_counter()
total = sum(range(1000000))
print(f"耗时 {time.perf_counter() - start:.3f} 秒")
```

1. **时间戳 ↔ struct_time**

```
import time
t = time.localtime()                    # 时间戳 -> struct_time
print(time.strftime("%Y-%m-%d %H:%M:%S", t))
print(time.mktime(t))                   # struct_time -> 时间戳
```

1. **字符串 ↔ 时间**

```
import time
t = time.strptime("2026-10-08", "%Y-%m-%d")        # str -> struct_time
s = time.strftime("%Y-%m-%d", t)                   # struct_time -> str
print(s, t.tm_yday)                                # 2026-10-08 281
```

1. **日期加减与求差**

```
from datetime import date, timedelta
d1, d2 = date(2026, 1, 1), date(2026, 3, 1)
print((d2 - d1).days)                   # 59    相差多少天
print(d1 + timedelta(days=100))         # 2026-04-11    过 100 天是哪天
```

6.6 本章自测（10 道）

第 6 章 自测题 1 `random.randint(1, 10)` 和 `random.randrange(1, 10)` 的取值范围分别是什么？

第 6 章 自测题 2 写出用 random 模块模拟"抛 100 次硬币，统计正反面次数"的完整程序。

第 6 章 自测题 3 `time.strftime` 和 `time.strptime` 有什么区别？各举一例。

第 6 章 自测题 4 编程：输入你的出生日期（一行三个整数，年 月 日），计算你从出生到"今天"一共活了多少天（"今天"用 `date.today()`）。

第 6 章 自测题 5 写出 `math.floor(-2.5)`、`math.ceil(-2.5)`、`math.trunc(-2.5)`、`int(-2.5)` 的结果，并把 `math.floor(-7 / 2)` 与 `int(-7 / 2)` 的结果作比较。

第 6 章 自测题 6 用 `random.seed(1)` 之后，随机数的结果会怎样变化？这在调试时有什么用？同一段代码里把 `seed` 写进循环会发生什么？

第 6 章 自测题 7 编程：用 `time.perf_counter()` 测量 `sum(range(1000000))` 的运行时间，输出保留 3 位小数。为什么不推荐用 `time.time()` 来计时？

第 6 章 自测题 8 写出 `-7 // 2`、`-7 % 3`、`math.fmod(-7, 3)` 的结果，并说明 `%` 和 `math.fmod` 在负数上的区别。

第 6 章 自测题 9　编程：先 `random.seed(1)`，再生成 20 个 [1, 20] 之间的随机整数，统计其中落在 [10, 20) 区间内的数有多少个。

第 6 章 自测题 10　编程：输入一行日期（格式 `YYYY-MM-DD`）和一个整数 n，输出该日期之后第 n 天的日期，格式同样是 `YYYY-MM-DD`。

第 7 章 字符串与文本处理

对应课程讲义：《08_字符串和文本处理》。字符串题在课程作业、OJ 判题和实际工作里出现得最多，方法多、坑也多。请重点掌握 **f-string 格式化 / split / join / strip / replace / find / count**，并认识最基础的**正则表达式**。

学完本章你应该能：① 说出哪些字符串方法是"返回新串"（几乎全部），并且记得把结果赋值回去；② 用 `split()` / `join()` 在"字符串"和"列表"之间来回转换；③ 用 f-string 的格式说明符控制宽度、对齐、小数位、千分位、百分比——**这是 OJ 里最容易 WA 的地方**；④ 用 `re.findall` / `re.sub` 做简单的文本提取与替换。

7.1 字符串内置方法的共同特点

【核心】两个前提：① `len(s)`、`max(s)` 这类是**函数**，把字符串放在参数里；而本章的 `s.lower()` 是**字符串的方法**，必须写成"字符串.方法()"。② 字符串是**不可变**的（详见 4.2 节），下面所有方法都**不会改动原字符串**，效果一律通过**返回值**给你——想保留结果就必须赋值回去。

```
str2 = "ITCAST666"
print(str2.lower())     # itcast666    只是"临时"得到一个新串
print(str2)             # ITCAST666    原字符串没变
str2 = str2.lower()     # 想要真的变，必须赋值回去
print(str2)             # itcast666
```

方法可以连着写，这叫**方法链**：上一步返回的新串，直接接着调用下一个方法。

```
s = "  ITCAST666  "
print(s.strip().lower().replace("itcast", "heima"))   # heima666
```

【常见错误】 调用 `s.strip()`、`s.replace(...)` 之后 `s` 纹丝不动，就是这里的原因：方法把结果**返回**给你，不接收返回值等于白干。

```
# ✗ 错误写法：s 并没有变
s = input()
s.strip()
print(s)          # 输入 "  hello  "，输出的还是 "  hello  "
```

```
# ✓ 正确写法：把返回值赋回给 s
s = input()
s = s.strip()
print(s)          # 输入 "  hello  "，输出 hello
```

记忆口诀：**字符串方法一律"返回新串"，从不原地修改。**对比一下，列表的 `append()` / `sort()` 才是原地修改、返回 `None`（详见 4.5 节），两者千万别混。

本章方法可以分成 9 类，先建立一张"地图"，后面按图索骥：

类别	代表方法	详见
状态判断	`isdigit()` `isalpha()` `startswith()`	7.2 节
大小写转换	`lower()` `upper()` `title()`	7.3 节
去空白与填充	`strip()` `lstrip()` `rjust()` `zfill()`	7.4 节
拆分	`split()` `rsplit()` `splitlines()` `partition()`	7.5 节
拼接	`"分隔符".join(序列)`	7.6 节
查找与替换	`find()` `rfind()` `index()` `count()` `replace()`	7.7、7.8 节
批量映射	`str.maketrans()` `translate()`	7.8 节
格式化	`%` / `format()` / f-string	7.10 节
模式匹配	`re` 模块	7.11 节

> **【速查】** 这张表不用背，作用是在脑子里建个索引：拿到一道字符串题，先判断"它属于哪一类"。

7.2 状态判断函数（速查）

方法	判断内容	返回值
`islower()`	是否全小写字母	True / False
`isupper()`	是否全大写字母	True / False
`isdigit()`	是否全是数字	True / False
`isalpha()`	是否全是字母	True / False
`isalnum()`	是否全是数字和字母	True / False
`istitle()`	是否单词首字母大写	True / False
`startswith(str)`	是否以指定字符串开始	True / False
`endswith(str)`	是否以指定字符串结束	True / False

```
print("itcast".islower())      # True
print("ITHEIMA".islower())     # False
print("666".isdigit())         # True
print("itcast666".isdigit())   # False
print("itcast".isalpha())      # True
print("itcast666".isalnum())   # True
print("itcast_666".isalnum())  # False，下划线不算字母数字
print("It Cast".istitle())     # True
print("ItCast".istitle())      # False
print("logo.png".startswith("lo"))    # True
print("logo.png".endswith(".png"))    # True
```

> **【注意】** `is` 开头的判断有两条容易踩的规律：① **空字符串一律返回 False**：`"".isdigit()` 和 `"".islower()` 都是 `False`，所以校验输入时要先确认"非空"。② `isdigit()` 只认"数字字符"：`"-1".isdigit()` 是 `False`（负号不算），`"12.5".isdigit()` 也是 `False`（小数点不算），而全角的 `"１２３".isdigit()` 反而是 `True`。所以 `isdigit()` 只适合判断"非负整数串"，负数和小数要另想办法（详见 1.10 节）。

```
print("".isdigit())                              # False   空串什么都不是
print("-1".isdigit(), "12.5".isdigit())          # False False   符号和小数点不算数字
print("１２３".isdigit())                         # True    全角数字也算数字字符
print("logo.png".endswith((".jpg", ".png")))     # True    可以一次判断多个后缀
```

> **【注意】** `startswith()` / `endswith()` 的参数可以是一个**元组**，一次判断多种情况，比用 `or` 连接清爽得多。

7.3 大小写转换

方法	功能
`lower()`	所有字母转小写（支持英文字母）
`casefold()`	所有字母转小写（支持各种语言，更彻底）
`upper()`	所有字母转大写
`swapcase()`	大写转小写，小写转大写
`title()`	每个单词首字母大写，其余小写（以空格区分单词）
`capitalize()`	只有首个字母大写，其余小写

```
print("it heima".swapcase())    # IS HEIMA
print("itcast".title())         # Itcast
print("it hei ma".title())      # It Hei Ma
print("it hei ma".capitalize()) # It hei ma   ← 注意区别
```

> **【易错】** `title()` 是"每个单词"首字母大写，`capitalize()` 是"整句"首字母大写，别用混。 **【注意】** 大小写转换只影响英文字母，中文、数字、符号一律原样保留。

例题 7-1：小写转大写（不使用内置函数）

题目：输入一行字符串，把其中的小写字母改写成大写字母，其它字符原样输出，要求**不许使用** `upper()`、`swapcase()`。

```
s = input()
t = ''
for i in range(len(s)):
    if 'a' <= s[i] <= 'z':
        t += chr(ord(s[i]) - (ord('a') - ord('A')))
```

```
    else:
        t += s[i]
print(t)
```

输入 `itCast 666`，输出：

```
ITCAST 666
```

要点：这道题把第 1 章的 `ord` / `chr`（详见 1.11 节）和第 2 章的循环结合起来，是典型的"用 ASCII 码做字符运算"。小写字母比对应的大写字母大 32，减掉 `ord('a') - ord('A')` 就完成转换；`'a' <= s[i] <= 'z'` 是第 1 章讲过的链式比较（详见 1.12 节）。 **【注意】** 这里用 `t += ch` 逐个拼接，字符串短时没问题；字符很多时更推荐先存进列表再 `join`（见 7.6 节）。

7.4 去空白与填充

方法	功能
`strip(str)`	去掉字符串**左右两侧**在参数字符串中出现过的所有字符
`lstrip(str)`	只去掉左侧
`rstrip(str)`	只去掉右侧
`ljust(len, str)`	在**右侧**补字符到指定长度
`rjust(len, str)`	在**左侧**补字符到指定长度
`center(len, str)`	在**两侧**补字符到指定长度（左补数量 ≥ 右补数量）
`zfill(len)`	在左侧补 0 到指定长度（小数点占 1 位）

三个"去空白"方法的区别（**重点**，都是返回新串）：

写法	去掉哪一侧	例子	结果
`s.strip()`	两侧的所有空白	`" hi ".strip()`	`'hi'`
`s.lstrip()`	左侧（left）	`" hi ".lstrip()`	`'hi '`
`s.rstrip()`	右侧（right）	`" hi ".rstrip()`	`' hi'`
`s.strip(chars)`	两侧，按**字符集合**删	`"xxhixx".strip("x")`	`'hi'`
`s.lstrip(chars)`	左侧，按字符集合删	`"xxhixx".lstrip("x")`	`'hixx'`
`s.rstrip(chars)`	右侧，按字符集合删	`"xxhixx".rstrip("x")`	`'xxhi'`

【核心】 不带参数时，三个方法删的是**空白字符**：空格、制表符 `\t`、换行符 `\n` 等。`line.rstrip("\n")` 是读文件时最常用的一行（详见 5.3 节）。

```
print(" itcast ".strip(" "))     # 'itcast'
print("itcast".strip("it"))      # 'cas'     ← 注意：按"字符集合"删，不是按子串
```

```
print("itcast".strip("sat"))     # 'itc'
print(" itcast ".rstrip(" "))    # ' itcast'
```

【常见错误】 `strip("it")` 不是"去掉子串 it"，而是"去掉两端出现的字符 i 和 t"，这是最容易理解错的一条。

```
# ✗ 错误理解：以为会得到 'cast'
print("itcast".strip("it"))     # 实际得到 'cas'
print("itcast".strip("sat"))    # 实际得到 'itc'
```

```
# ✓ 想删掉固定的前缀 / 后缀，用切片或 removeprefix / removesuffix
print("itcast"[2:])                 # cast
print("itcast".removeprefix("it"))  # cast   Python 3.9+ 才有
print("report.txt".removesuffix(".txt"))   # report
```

原理：参数被当成**字符集合** `{'i','t'}`，两端只要碰到集合里的字符就削掉，一直削到不是为止，跟"子串"没关系。

```
print("itcast".ljust(10, "6"))   # 'itcast6666'
print("itcast".ljust(5, "6"))    # 'itcast'  长度不够时不截断
print("itcast".rjust(10, "6"))   # '6666itcast'
print("itcast".center(10, "6"))  # '66itcast66'
print("itcast".center(11, "6"))  # '666itcast66'  左补比右补多
print("3.14".zfill(6))           # '003.14'
print("-3.14".zfill(6))          # '-03.14'  0 补在负号后面
```

【易错】 填充字符必须是**单个字符**，写成 `"66"` 会报 `TypeError: The fill character must be exactly one character long`。 **【注意】** 宽度不够时 `ljust` / `rjust` / `center` **不截断**，原样返回；想让数字右对齐补零，`zfill` 和 f-string 的 `:06.2f` 都能做（见 7.10 节）。

7.5 拆分函数

方法	功能	返回值
`partition(str)`	从左侧找到参数后，分成"左侧、参数、右侧"三段	含三个元素的元组
`rpartition(str)`	从右侧查找，其余同上	含三个元素的元组
`split(str)`	用参数作为分割线拆分成若干字符串	列表
`rsplit(str, n)`	从右侧开始拆，最多拆 n 次	列表
`splitlines()`	用换行符作为分割线拆分	列表

```
print("hello python".partition("o"))    # ('hell', 'o', ' python')
print("hello python".rpartition("o"))   # ('hello pyth', 'o', 'n')
print("hello python".partition("a"))    # ('hello python', '', '')  找不到时
print("hello python".rpartition("a"))   # ('', '', 'hello python')
```

```
print("hello python".split("o"))        # ['hell', ' pyth', 'n']
print("hello python".split("oo"))       # ['hello python']  找不到就不拆
print("hello\nitcast\npython".splitlines())   # ['hello', 'itcast', 'python']
```

【易错】 `partition` 找不到分隔符时返回"原串 + 两个空串"；`split` 找不到分隔符时返回"只含原串的列表"。这两点常被考到。

split() 与 split(" ") 的区别（重点）

写法	按什么切	空串怎么处理	例子 → 结果
`s.split()`	任意**连续空白**（空格、\t、\n）	自动丢弃	`" a b ".split()` → `['a', 'b']`
`s.split(" ")`	**一个**空格	原样保留	`" a b ".split(" ")` → `['', '', 'a', '', 'b', '', '']`
`s.split(",")`	一个逗号	原样保留	`"a,b,,c".split(",")` → `['a', 'b', '', 'c']`
全空白串 `" ".split()`	—	—	`[]`
全空白串 `" ".split(" ")`	—	—	`['', '', '', '']`

```
print("  a  b  ".split())       # ['a', 'b']          智能：按任意空白切，自动丢空串
print("  a  b  ".split(" "))    # ['', '', 'a', '', 'b', '', '']  死板：只按一个空格切
print("a,b,,c".split(","))      # ['a', 'b', '', 'c']    指定分隔符时，空串不会被丢掉
print("   ".split())            # []
```

【核心】 一句话记住：**不带参数的 `split()` 是"智能"的**（等价于按任意空白切分并丢弃空串），**带参数的 `split()` 是"死板"的**（严格按分隔符切，空串照留）。读 OJ 输入时，默认都写不带参数的版本。

```
print("a-b-c".split("-", 1))     # ['a', 'b-c']    最多切 1 刀，剩下的整块保留
print("a-b-c".rsplit("-", 1))    # ['a-b', 'c']    从右边开始，最多切 1 刀
print("2026-10-08".split("-"))   # ['2026', '10', '08']
print("name=Tom;age=18".split(";"))   # ['name=Tom', 'age=18']
```

【注意】 `split(sep, maxsplit)` 的第二个参数是"最多切几刀"，省略表示能切多少切多少；取路径里的文件名、取网址最后一段，用 `rsplit("/", 1)[-1]` 最方便。

【注意】 `"a\nb\n".splitlines()` 得到 `['a', 'b']`，而 `"a\nb\n".split("\n")` 得到 `['a', 'b', '']`——**末尾多一个空串**。按整段文本切行时优先用 `splitlines()`。

【常见错误】 用 `split(" ")` 读输入，遇到多余空格或行尾空格就会多切出空串；把 `int()` 转早了，又会踩另一个坑。

```
# ✗ 错误写法一：输入 "1  2"（中间两个空格）直接崩
a, b = input().split(" ")
# ValueError: too many values to unpack (expected 2)

# ✗ 错误写法二：字符串方法用在数字上
n = int(input())
n.split()
# AttributeError: 'int' object has no attribute 'split'
```

```
# ✓ 正确写法：不带参数 split()，并且"先 split 再 int"
a, b = map(int, input().split())
nums = list(map(int, input().split()))
```

`split(" ")` 会为连续的两个空格各切一刀，把空串留在结果里；不带参数的 `split()` 把连续空白看成一个整体。另外一定要记住：**字符串方法只能由字符串调用**，`int` 没有 `split()`。

7.6 连接函数

```
print("6".join("itcast"))    # i6t6c6a6s6t
print("66".join("itcast"))   # i66t66c66a66s66t
print("".join(['a', 'b']))   # ab
print("-".join(["2026", "10", "08"]))   # 2026-10-08
```

【核心】 `join()` 的用法是"**分隔符.join(序列)**"：分隔符在前、序列是参数，这与直觉相反，请特别记住。它是把列表拼成字符串的标准做法；参数也可以直接是字符串（会被当成字符序列，所以 `"6".join("itcast")` 能在每个字符间插 6）。

【常见错误】 `join` 里的元素必须**全是字符串**，混进数字就报错。

```
# ✗ 错误写法
nums = [1, 2, 3]
print(",".join(nums))
# TypeError: sequence item 0: expected str instance, int found
```

```
# ✓ 正确写法：先把每个元素转成字符串
nums = [1, 2, 3]
print(",".join(map(str, nums)))          # 1,2,3
print(",".join(str(x) for x in nums))    # 1,2,3  等价写法
```

记法：`join` 是"用字符串拼字符串"，数字得先"变身"。

【注意】OJ 输出技巧：题目要求"一行内用空格分隔输出所有数"时，标准写法是 `print(" ".join(map(str, nums)))`，**行尾没有多余空格**；写成 `for x in nums: print(x, end=" ")` 会在行尾留下一个空格，可能被判 PE 或 WA。

【注意】 为什么不用循环 `s += x` 拼字符串？字符串不可变，`+=` 每拼一次就新建一个对象，拼上万次会明显变慢；`join` 一次成型。字符少的时候两者没区别，养成用 `join` 的习惯即可。

7.7 查询函数

方法	功能	找不到时
`find(str, begin, end)`	从左侧查找，返回第一次出现的索引	返回 -1
`rfind(str, begin, end)`	从右侧查找	返回 -1
`index(str, begin, end)`	从左侧查找	**报错**
`rindex(str, begin, end)`	从右侧查找	**报错**
`count(str)`	查询指定字符串出现的次数	返回 0

```
str1 = "itcast"
print(str1.find("t"))         # 1
print(str1.find("t", 3, 6))   # 5
print(str1.find("t", 6, 3))   # -1  起始大于结束，直接返回 -1
print(str1.find("m"))         # -1

print(str1.rfind("t"))        # 5
print(str1.rfind("t", 0, 4))  # 1

print(str1.index("t"))        # 1
print(str1.index("m"))        # 报错 ValueError!

print(str1.count("t"))        # 2
print(str1.count("tc"))       # 1
```

【易错】 `find` 找不到返回 **-1**，`index` 找不到**直接抛异常**。写程序时如果不确定元素是否存在，用 `find` 更安全。 **【注意】** 正因为返回 -1，**不能把 `find` 的结果直接当布尔值用**：Python 里 `-1` 是"真"，下面这段代码的条件永远成立。

```
s = "itcast"
# X 明明没找到，也会打印"找到了"
if s.find("z"):
    print("找到了")
# √ 判断"在不在"就用 in，或者显式比较 -1
if "z" in s:
    print("找到了")
if s.find("z") != -1:
    print("找到了")
```

【了解】 `find` / `index` 的 `begin` / `end` 是"左闭右开"的区间；`s.count("")` 会返回 `len(s) + 1`，知道有这回事即可。

7.8 替换与其它函数

```
str1 = "itcast"
print(str1.replace("t", "T"))       # 'iTcasT'
```

```
print(str1.replace("t", "T", 1))    # 'iTcast'   只替换 1 个
print(str1)                         # 'itcast'   原串仍然没变

str2 = "itca\tst"
print(str2.expandtabs(4))           # 'itca    st'   把制表位换成空格（默认 8）
```

【注意】 `replace()` 同样是返回新串（承接 7.1 节）：执行完 `str1.replace("t", "T")` 之后，`str1` 还是 `'itcast'`。第三个参数是"最多替换几个"。 **【了解】** `"abc".replace("", "-")` 会在每两个字符之间都插一个 `-`，得到 `'-a-b-c-'`。

maketrans 与 translate：批量字符转换

```
# 用法一：一对一映射（两个字符串长度必须相同）
table = str.maketrans("123", "abc")    # 建表：1→a 2→b 3→c，实际是 {49: 97, 50: 98, 51: 99}
print("123456".translate(table))       # abc456

# 用法二：第三个参数表示"要删掉的字符"
print("a1b2c3".translate(str.maketrans("", "", "123")))   # abc

# 用法三：直接给一个字典（键是单个字符，值是替换内容）
print("hello".translate(str.maketrans({"h": "H"})))       # Hello
```

【速查】 `maketrans` 负责**建表**，`translate` 负责**查表替换**。它只遍历一遍就能完成**多条**单字符替换，比连着写好几个 `replace` 更快，而且不会出现"上一条替换的结果又被下一条规则改掉"的连锁问题，适合做敏感字符批量替换、全角转半角这类工作。

```
# 编码与解码：字符串 <-> 字节串
s = "人生苦短，我用Python。"
b = s.encode("UTF-8")      # 字符串 → 字节串
print(b)                   # b'\xe4\xba\xba\xe7\x94\x9f...'
print(b.decode("UTF-8"))   # 字节串 → 字符串，还原成原文
```

【了解】 中文乱码、文件为什么必须写 `encoding="utf-8"`，详见 1.11 节和 5.3 节。这里只要记住 `encode` / `decode` 是一对**逆操作**，两端必须用同一种字符集。

7.9 例题：三个字符串小练习

例题 7-2：打印 1~100 之间所有不含 7 的数字

题目：输出 1 到 100 之间所有十进制表示中不含数字 7 的整数，每个一行。

```
for i in range(1, 101):
    if '7' not in str(i):
        print(i)
```

输出（节选前 10 行）：

```
1
2
3
4
5
6
8
9
10
11
```

要点：`str(i)` 把整数转成字符串，再用 `in` 判断"7 这个字符在不在里面"。要判断一个数的某一位是不是某个数字，转成字符串比用 `% 10` 逐位拆更省事。

例题 7-3：去掉元音字母

题目：输入一句话，去掉其中所有元音字母（a、e、i、o、u，不分大小写）后输出。

```
VOWELS = "aeiou"
message = input("Enter a message: ")
new_message = ""
for letter in message:
    if letter.lower() not in VOWELS:
        new_message += letter
print("\nYour message without vowels is:", new_message)
```

在提示语后输入 `Hello World`，输出：

```

Your message without vowels is: Hll Wrld
```

要点：这是"遍历字符，把要保留的拼到新串"的标准套路，原串 `message` 始终没变。`letter.lower() not in VOWELS` 里的 `VOWELS` 是一个字符串，`in` 判断的是"这个字符在不在这个字符串里"，相当于在一张字符表里查（详见 4.3 节）。

【注意】OJ 上不能有提示语：`input("Enter a message: ")` 里的提示语、`print()` 里的说明文字，全都算进**标准输出**，OJ 拿去和答案一比就 WA。做 OJ 题时必须写成"只输出结果"的版本：

```
VOWELS = "aeiou"
message = input()
print("".join(ch for ch in message if ch.lower() not in VOWELS))
```

例题 7-4：统计单词并整理大小写

题目：输入一行英文句子，单词之间可能有多个空格，行首行尾也可能有空格。第一行输出单词个数；第二行把每个单词改成"首字母大写、其余字母小写"后，用**单个空格**连接输出。

输入 `hello WORLD python`，输出：

```
3
Hello World Python
```

```
words = input().split()                    # 不带参数：多余空格自动忽略
print(len(words))
print(" ".join(w.capitalize() for w in words))
```

> **要点**：三步走——**`split()` 切分**（顺便解决多余空格）、**逐个处理单词**（`capitalize()`）、**`join()` 拼回**。这就是字符串题最经典的"拆分 → 处理 → 合并"流程，OJ 里反复出现。
>
> **【注意】** 看清题目要求：要"首字母大写、其余小写"用 `capitalize()`；如果要求"只把首字母变大写，其余字母保持原样"，就得写 `w[0].upper() + w[1:]`。

7.10 字符串格式化（核心）

Python 有四种格式化方式，按出现时间排序：`%` 操作符 → `format()` 方法 → f-string → 模板字符串。

方式	写法示例	什么时候用
`%` 操作符	`"%s今年%d岁" % (name, age)`	看得懂即可，老代码里很常见
`format()` 方法	`"{}今年{}岁".format(name, age)`	模板和数据要分开保存时
f-string	`f"{name}今年{age}岁"`	**★ 推荐，日常首选**
模板字符串	见方式四	了解

> **【核心】** 日常写代码**优先用 f-string**；考试里三种都要认识。

方式一：% 操作符（最古老）

```
age = 10
name = 'Tom'
print("%s今年%d岁" % (name, age))    # Tom今年10岁
```

格式化符	意义
`%d` / `%i`	有符号十进制整数
`%o` / `%x` / `%X`	八进制 / 十六进制（小写）/ 十六进制（大写）
`%f`	浮点数
`%e` / `%E`	科学计数法
`%c`	0~255 的整数对应的 ASCII 字符
`%r` / `%s`	对象的 repr() / str() 返回值
`%%`	每两个 % 显示一个 %

```
print("字符串：%s,%s,%s" % (1, 22.22, [1, 2]))   # 字符串：1,22.22,[1, 2]
print("不足5位左边补空格%5s。" % '2')             # 不足5位左边补空格    2。
print("不足5位右边补空格%-5s。" % '2')            # 不足5位右边补空格2    。
print("宽10位截取两位%10.2s。" % "hello.world")  # 宽10位截取两位         he。
print("整数：%d,%d,%d" % (1, 22.22, 33))         # 整数：1,22,33
print("左边补0：%05d。" % 22)                     # 左边补0：00022。
print("浮点数：%f,%f" % (1, 22.22))               # 浮点数：1.000000,22.220000
print("保留两位小数：%.2f" % 22.222)              # 保留两位小数：22.22
print("保留两位小数，宽5位补0：%05.2f" % 2.222)  # 保留两位小数，宽5位补0：02.22
print("八进制 %o" % 222)                          # 八进制 336
print("十六进制 %x" % 12)                         # 十六进制 c
print("%.2f%%" % 12.3456)                         # 12.35%  文本里的百分号要写 %%
```

> **【核心】记忆口诀**：`%ns` 里的 n 决定**宽度与对齐**（n>0 左补空格右对齐，n<0 左对齐右补空格），`%.ns` 决定**截取或小数位数**。**【注意】** `%` 在格式化字符串里是"占位符开关"，想原样输出一个百分号必须写 `%%`。

方式二：format() 方法

基本格式：`<模板字符串>.format(<逗号分隔的参数>)`，模板里写 `{参数序号:格式控制标记}`。

格式控制标记有 6 个字段：`: <填充> <对齐> <宽度> <,> <.精度> <类型>`。

```
s = "PYTHON"
t = "Java"
print("{1:*^10} and {0:-^10}".format(s, t))
# ***Java*** and --PYTHON--

print("{0:30}".format(s))       # 'PYTHON' 后面补空格
print("{0:>30}".format(s))      # 右对齐（前面补空格）
print("{0:*^30}".format(s))     # 居中，两侧补 *
print("{0:3}".format(s))        # 'PYTHON'  宽度不够时不截断

print("{0:-^20,}".format(1234567890))   # '---1,234,567,890----'  逗号是千位分隔符

print("{0:.2f}".format(12345.67890))    # '12345.68'
print("{0:H^20.3f}".format(12345.67890))# 'HHHHH12345.679HHHHHH'
print("{0:.4}".format("PYTHON"))        # 'PYTH'  对字符串是截取最大长度

print("{0:e},{0:E},{0:f},{0:%}".format(3.14))
# '3.140000e+00,3.140000E+00,3.140000,314.000000%'
```

字段	取值
填充	任意单个字符，默认空格
对齐	`<` 左对齐，`>` 右对齐，`^` 居中
宽度	整数
逗号	数字的千位分隔符

字段	取值
精度	`.数字`，浮点数表示小数位数，字符串表示最大输出长度
类型	整数：`b c d o x X`；浮点数：`e E f %`

> **【注意】** 类型码必须和值的类型匹配：`"{:d}".format("a")` 会报 `ValueError: Unknown format code 'd' for object of type 'str'`。 **【注意】** `format()` 和 f-string 用的是**同一套格式说明符**，学会下面那一张表，两种写法都会了。

方式三：f-string（推荐）

f-string 在字符串前加字母 `f`，大括号里的表达式会在运行时求值并替换。

```
import datetime

name = "zings"
age = 17
date = datetime.date(2019, 7, 18)
print(f'my name is {name}, this year is {date:%Y}. Next year, I\'m {age+1}')
# my name is zings, this year is 2019. Next year, I'm 18
```

> **【核心】f-string 是现在最推荐的格式化写法**：可读性最好、写得最短，大括号里可以直接写表达式、函数调用，甚至套一层 f-string。作业题里要求"保留 2 位小数"，直接写 `print(f"{x:.2f}")` 即可。

完整格式说明符（重点 • OJ 判题最容易 WA 的地方）

完整语法（**顺序不能乱**，方括号表示可省略）：

```
f"{值:[填充字符][对齐][正负号][#][0][宽度][千分位][.精度][类型码]}"
```

表 1：每个字段管什么

字段	取值	含义	例子	结果
填充字符	任意一个字符，默认空格	宽度不够时用什么补	`f"{'ab':*<6}"`	`ab****`
对齐	`<` `>` `^` `=`	左 / 右 / 居中 / 符号后补	`f"{'ab':>6}"`	`ab`
正负号	`+` `-` 空格	正数要不要显示符号	`f"{8:+.2f}"` / `f"{8: .2f}"`	`+8.00` / `8.00`
`#`	—	进制数带前缀	`f"{255:#x}"`	`0xff`
`0`	—	用 0 补满宽度	`f"{7:03d}"` / `f"{-12:05d}"`	`007` / `-0012`
宽度	整数	最小总宽度，不够**不截断**	`f"{'PYTHON':5}"`	`PYTHON`
千分位	`,` 或 `_`	三位一组加分隔符	`f"{1234567:,}"` / `f"{1234.5678:,.2f}"`	`1,234,567` / `1,234.57`

字段	取值	含义	例子	结果
精度	`.n`	浮点=小数位数；字符串=最大长度	`f"{87.4567:.2f}"` / `f"{'PYTHON':.3}"`	`87.46` / `PYT`
类型码	见下表	按什么类型解释	`f"{0.856:.1%}"`	`85.6%`

表 2：类型码

类型码	适用值	含义	例子 → 结果
`d`	整数	十进制整数	`f"{7:03d}"` → `007`
`b` `o` `x` `X`	整数	二 / 八 / 十六进制（x 小写、X 大写）	`f"{255:#x}"` → `0xff`
`c`	整数	转成对应的 Unicode 字符	`f"{65:c}"` → `A`
`f` `F`	浮点	定点小数，默认 6 位	`f"{1/3:.2f}"` → `0.33`
`e` `E`	浮点	科学计数法	`f"{3.14159:.3e}"` → `3.142e+00`
`g` `G`	浮点	自动在定点与科学计数之间选	`f"{3.14159:.3g}"` → `3.14`
`%`	浮点	乘 100 再加百分号	`f"{0.856:.1%}"` → `85.6%`
`s`	字符串	默认类型，一般省略	`f"{'ab':>5}"` → `ab`
不写	任意	交给 `str()` 自己决定	`f"{1234567:,}"` → `1,234,567`

表 3：OJ 最常用的 8 种组合（可以直接抄）

```
x = 3.14159
n = 1234567
print(f"{x:.2f}")        # 3.14         保留 2 位小数（最常用）
print(f"{x:8.2f}")       #     3.14     宽度 8、右对齐
print(f"{x:<8.2f}|")     # 3.14    |    左对齐（用 | 标出右边界）
print(f"{x:^8.2f}|")     #   3.14  |    居中
print(f"{x:08.2f}")      # 00003.14     用 0 补满宽度
print(f"{n:,}")          # 1,234,567    千分位
print(f"{0.856:.1%}")    # 85.6%        百分比（保留 1 位小数）
print(f"{n:#x}")         # 0x12d687     十六进制
```

【易错】 三个高频陷阱：① `.2f` 名义上是四舍五入，但受二进制表示限制：`f"{2.675:.2f}"` 得到 `2.67` 而不是 `2.68`，需要精确舍入时用 `Decimal`（详见 1.9 节）；② 精度用在**字符串**上是"截断"：`f"{'PYTHON':.3}"` 得到 `PYT`；③ 宽度不够时**不截断**：`f"{'PYTHON':3}"` 还是 `PYTHON`。

【注意】 `=` 对齐只对数字有意义，它把填充补在符号之后：`f"{-3.5:=+9.2f}"` 得到 `- 3.50`。另外，宽度按"字符个数"计算，中文字符实际占两个英文宽度，所以含中文的表格用宽度对齐仍然会歪。 **【注意】** 别把 `>` 写成箭头那样的 `->`：`f"{255:->10}"` 不报错，但它的意思是"用 `-` 填充、右对齐"，得到 `--------255`，和你想的"箭头"完全是两回事。

【常见错误】 写了格式说明符却忘了字符串前面的 `f`，程序不报错，只是把占位符原样打印出来，OJ 直接 WA。

```
# ✗ 错误写法：f 忘了写
x = 3.14159
print("{x:.2f}")        # 输出 {x:.2f}

name = "Tom"
print("{name:>10}")     # 输出 {name:>10}
```

```
# ✓ 正确写法
x = 3.14159
print(f"{x:.2f}")       # 3.14
name = "Tom"
print(f"{name:>10}")    #        Tom
```

自查习惯：只要 `{}` 里出现了冒号 `:`，就回头看一眼字符串前面有没有 `f`。

方式四：模板字符串（了解）

```
from string import Template

number = 20090214
name = 'Fish'
templ_string = 'Hello $name, your number is $number.'
print(Template(templ_string).substitute(name=name, number=hex(number)))
# Hello Fish, your number is 0x1328d66.
```

模板字符串不能使用格式说明符，需要手动转换；但用来处理**用户提供的格式字符串**时更安全（不会像 `format` / f-string 那样去执行里面的表达式）。

例题 7-5：格式化输出成绩单

题目：输入一行，包含一个姓名和 3 个成绩（用空格分隔）。输出姓名（右对齐、宽度 6）、总分（保留 1 位小数）、平均分（保留 2 位小数）和得分率（百分数、保留 1 位小数）。

输入 `Tom 85 92.5 78`，输出：

```
   Tom 总分  255.5 平均 85.17 得分率 85.2%
```

```
name, *scores = input().split()        # 星号解包：第一个给 name，其余都给 scores
scores = list(map(float, scores))
total = sum(scores)
avg = total / len(scores)
print(f"{name:>6} 总分 {total:6.1f} 平均 {avg:.2f} 得分率 {avg/100:.1%}")
```

要点：`name, *scores = input().split()` 是第 4 章讲过的星号解包（详见 4.8 节），用来读"一个名字 + 若干个数字"特别方便。这一行里 `:>6`（宽度 6 右对齐）、`:6.1f`、`:.2f`、`:.1%` 覆盖了 OJ 里 90% 的格式化需求。 **【注意】** 输出里每一个空格都要和样例对得上：`{name:>6}` 后面有一个**空格**，`总分` 和 `{total:6.1f}` 之间也有一个**空格**。写完一定用样例输入跑一遍，逐字符比对。

7.11 正则表达式与 re 模块

正则表达式（Regular Expression，简称 RE 或 regex）是强大的文本处理工具，用于匹配字符串模式。它能做四件事：**查找、替换、验证、提取**。

模式的基本组成（速查）

类别	符号	含义
字面值	`abc`	直接匹配它们自身
元字符	`\d` / `\w` / `\s`	数字 / 字母数字下划线 / 空白字符
元字符	`.`	匹配任意字符（除换行符）
字符类	`[abc]`	匹配括号内任意一个字符
字符类	`[^abc]`	匹配除了括号内字符以外的任意字符
量词	`*` / `+` / `?`	前一个模式出现 0 次或多次 / 1 次或多次 / 0 次或 1 次
量词	`{n}` / `{n,}` / `{n,m}`	恰好 n 次 / 至少 n 次 / n 到 m 次
边界	`^` / `$` / `\b` / `\B`	字符串开头 / 结尾 / 单词边界 / 非单词边界
分组	`(...)` / `(?:...)`	分组并捕获 / 分组但不捕获
其它	`\` / `|`	转义字符 / 多个模式的选择（"或"）

常见正则示例（速查）

需求	正则表达式
只能输入数字	`^[0-9]*$`
只能输入 n 位数字	`^\d{n}$`
只能输入 m~n 位数字	`^\d{m,n}$`
只能输入有两位小数的正实数	`^[0-9]+(\.[0-9]{2})?$`
只能输入 26 个英文字母组成的字符串	`^[A-Za-z]+$`
用户名（字母数字下划线，3~15 位）	`^[a-zA-Z0-9_-]{3,15}$`
验证 Email 地址	`^\w+([-+.]\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*$`
验证身份证号（15 或 18 位数字）	`^\d{15}|\d{18}$`
匹配中文字符	`[\u4e00-\u9fa5]`
匹配首尾空格	`(^\s*)|(\s*$)`

> **【注意】** 表格里"两位小数"的那条原来是 `^[0-9]+(.[0-9]{2})?$`，中间的 `.` 没有转义，会把 `3x14` 也放过去，上面已经修正为 `\.`。

re 模块的 6 个常用函数（核心）

函数	作用
`re.compile(pattern)`	编译正则表达式，提高效率（同一模式反复用时才划算）
`re.match(pattern, text)`	**从头开始**匹配，不匹配返回 None，用 `group()` 取值
`re.search(pattern, text)`	在**整个字符串**中搜索第一个匹配项
`re.findall(pattern, text)`	返回所有匹配的子串列表
`re.sub(pattern, repl, text)`	替换匹配到的内容
`re.split(pattern, text)`	按匹配到的内容分割字符串

```
import re

# match：从开头匹配
result = re.match(r"hello", "hello world")
print(result.group() if result else "匹配失败")     # hello
print(re.match(r"world", "hello world"))            # None，因为开头不是 world

# search：全局搜索
result = re.search(r"world", "hello world")
print(result.group())                                # world

# findall：找出所有
text = "价格是100元，折扣20元"
print(re.findall(r"\d+", text))                      # ['100', '20']

# sub：替换
print(re.sub(r"\d+", "X", text))                     # 价格是X元，折扣X元

# split：按空白字符分割
print(re.split(r"\s+", "hello world python"))        # ['hello', 'world', 'python']

# compile：预编译，提高效率
pattern = re.compile(r"\d+")
print(pattern.findall(text))                         # ['100', '20']
```

> **【注意】** `re.match` 只认"开头"，判断"整串是否符合"应该用 `re.fullmatch()`；只要问"串里有没有"，就用 `re.search()` / `re.findall()`。

贪婪与非贪婪（重点）

默认情况下，量词 `*` `+` `{n,m}` 是**贪婪**的：能匹配多少就匹配多少。在量词后面加一个 `?` 就变成**非贪婪**（懒惰）：够用就停。

```
import re

print(re.findall(r"<.*>", "<a><b>"))     # ['<a><b>']       贪婪：一口气吃到最后一个 >
```

```
print(re.findall(r"<.*?>", "<a><b>"))    # ['<a>', '<b>']  非贪婪：吃到第一个 > 就停
print(re.search(r'".*"', '说 "你好" 和 "再见"').group())    # "你好" 和 "再见"
print(re.search(r'".*?"', '说 "你好" 和 "再见"').group())   # "你好"
```

> **【核心】** 贪婪与非贪婪只差一个 `?`：`*?` `+?` `??` `{n,m}?` 都表示非贪婪。想"只取一对引号中间的内容""只取一个标签"，就用非贪婪写法。

findall 与 finditer 的区别

函数	返回值	什么时候用
`re.findall(p, s)`	列表（**有分组时是元组列表**）	只要匹配到的内容，直接遍历或统计
`re.finditer(p, s)`	匹配对象**迭代器**，每个对象有 `.group()` `.start()` `.end()`	还要知道位置，或结果非常多、不想一次性建列表

```
import re

text = "a12b345"
print(re.findall(r"\d+", text))        # ['12', '345']
for m in re.finditer(r"\d+", text):
    print(m.group(), m.start(), m.end())
# 12 1 3
# 345 4 7

# 有分组时 findall 返回的是元组列表，这是最常见的"意外结果"
print(re.findall(r"(\d+)-(\d+)", "12-34 56-78"))   # [('12', '34'), ('56', '78')]
print(re.findall(r"\d+-\d+", "12-34 56-78"))       # ['12-34', '56-78']
```

原始字符串 r"..."（重点）

> **【核心】** 写正则一律用**原始字符串** `r"..."`。原因：普通字符串里的 `\` 会先被 Python 解释一遍，`"\b"` 是**退格符**（长度只有 1），`"\d"` 在当前版本虽然不报错（只给一个 SyntaxWarning），但语义已经和你写的字面量不是一回事。加上 `r` 之后"所见即所得"。

```
import re

print(len("\b"), len(r"\b"))                    # 1 2  普通串里的 \b 是一个退格字符
print(re.findall("\bcat", "a cat scatter"))     # []        模式被改掉了，什么都找不到
print(re.findall(r"\bcat", "a cat scatter"))    # ['cat']   \b 是单词边界
```

> **【常见错误】** 正则里的 `.` 是"任意一个字符"的通配符，想匹配真正的点号必须写 `\.`；另外别忘了给模式加 `r` 前缀。

```
# ✗ 错误写法：3x14 也被匹配上了
import re
```

```
print(re.findall(r"3.14", "3.14 和 3x14"))     # ['3.14', '3x14']
```

```
# ✓ 正确写法：元字符想当普通字符用，前面加反斜杠
import re
print(re.findall(r"3\.14", "3.14 和 3x14"))    # ['3.14']
print(re.findall(r"\d+\.\d+", "价格 3.5 元"))  # ['3.5']
```

记法：`. + * ? ( ) [ ] { } ^ $ | \` 这些是正则的"特殊符号"，想匹配它们本身都要加 `\`。拿不准时用 `re.escape("普通文本")` 自动转义。

例题 7-6：验证邮箱地址

```
import re

email_pattern = r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$"
email = "example@test.com"
if re.match(email_pattern, email):
    print("邮箱格式正确")
else:
    print("邮箱格式错误")
```

输出：

```
邮箱格式正确
```

要点：`^...$` 表示"整串必须完全符合"；`[a-zA-Z0-9_.+-]+` 是用户名字符集（`.` 在方括号里就是普通点号，不用转义）；域名里的 `\.` 必须转义，否则 `example@testXcom` 也会被判成"正确"。 **【注意】** 把它改成 OJ 题，只需要把 `email = "example@test.com"` 换成 `email = input()`，提示语一个都不要留（见 7.9 节）。 **【了解】** 常用开关：`re.I` 忽略大小写、`re.S` 让 `.` 也能匹配换行、`re.M` 让 `^` `$` 匹配每一行的头尾，用法是 `re.findall(p, s, re.I)`。 **【了解】** 正则功能强大但也容易写错。练习阶段先掌握 `\d` `\w` `\s` `.` `*` `+` `?` `[]` `()` `^` `$` 这几个符号就够用了；从网上抄来的正则，一定要用样例数据验证过再交。

7.12 本章方法总结与易错清单

一、本章知识地图

- **根**：字符串是**不可变**的序列（详见 4.2 节、4.3 节）
  - 因为它不可变 → 本章所有方法都**返回新串**，不原地修改（7.1 节）
  - 序列通性：索引 `s[0]`、切片 `s[1:3]`、`len()`、`in`、遍历，这些在第 4 章已经会了，本章是"在序列之上加方法"
- **六组方法**（按用途分）
  - 判断：`isxxx`、`startswith`、`endswith` → 返回布尔值，配合 `if` 使用
  - 转换：`lower` `upper` `title` `capitalize` `swapcase`

- 修剪填充：`strip` `lstrip` `rstrip` `ljust` `rjust` `center` `zfill`
- 切分与拼接：`split` `rsplit` `splitlines` `partition` ↔ `join`（**互为逆操作**）
- 查找替换：`find` `rfind` `index` `count` `replace`
- 批量映射：`maketrans` + `translate`

- **两条主线**（考试和 OJ 最常考）
  1. **输入解析**：`input().split()` → `map(int, ...)` → 列表；输出时 `" ".join(map(str, ...))`
  2. **格式化输出**：f-string 的"冒号宽度精度类型"，尤其是 `:.2f`、`:>8`、`:.1%`
- **一个工具**：正则 `re`（`match` `search` `findall` `finditer` `sub` `split`），用来做"查找、替换、验证、提取"四件事
- **依赖关系**：本章建立在 4.2 / 4.3 节（字符串是序列）之上；f-string 的精度问题依赖 1.7 / 1.9 节的浮点知识；读文件时的 `rstrip("\n")` 依赖 5.3 节。

二、方法总结（速查表）

方法 / 语法	作用	是否原地修改	返回值	典型例子
`s.lower()` / `s.upper()`	转小写 / 大写	否	新 str	`"AB".lower()` → `'ab'`
`s.title()` / `s.capitalize()`	每词首字母大写 / 整句首字母大写	否	新 str	`"it hei ma".title()` → `'It Hei Ma'`
`s.isdigit()` / `s.isalpha()`	是否全数字 / 全字母	否	bool	`"-1".isdigit()` → `False`
`s.startswith(t)` / `s.endswith(t)`	前缀 / 后缀判断（可传元组）	否	bool	`"a.png".endswith((".jpg", ".png"))` → `True`
`s.strip()` / `lstrip()` / `rstrip()`	去两侧 / 左 / 右的空白或字符集	否	新 str	`" x ".strip()` → `'x'`
`s.strip(chars)`	按**字符集合**削两端	否	新 str	`"xxhixx".strip("x")` → `'hi'`
`s.ljust(w, c)` / `rjust` / `center`	左 / 右 / 居中填充到宽度 w	否	新 str	`"7".rjust(3, "0")` → `'007'`
`s.zfill(w)`	左侧补 0（负号占位）	否	新 str	`"-3.14".zfill(6)` → `'-03.14'`
`s.split()`	按任意空白切，丢空串	否	list	`" a b ".split()` → `['a', 'b']`
`s.split(sep)`	按 sep 切，保留空串	否	list	`"a,,b".split(",")` → `['a', '', 'b']`
`s.split(sep, n)` / `s.rsplit(sep, n)`	最多切 n 刀 / 从右切 n 刀	否	list	`"a-b-c".rsplit("-", 1)` → `['a-b', 'c']`
`s.splitlines()`	按换行切行（不带 \n）	否	list	`"a\nb".splitlines()` → `['a', 'b']`

方法 / 语法	作用	是否原地修改	返回值	典型例子
`sep.join(seq)`	用 sep 把序列拼成串	否	新 str	`"-".join(['2026', '10'])` → `'2026-10'`
`s.replace(a, b, n)`	替换（最多 n 次）	否	新 str	`"aaa".replace("a", "b", 1)` → `'baa'`
`s.find(t)` / `s.rfind(t)`	找位置	否	int（找不到 -1）	`"itcast".find("z")` → `-1`
`s.index(t)`	找位置	否	int（找不到报错）	`"itcast".index("z")` → ValueError
`s.count(t)`	统计出现次数	否	int	`"itcast".count("t")` → `2`
`str.maketrans(a, b)` + `s.translate(t)`	建表 / 按表逐字符替换	否	dict / 新 str	`"123".translate(str.maketrans("123", "abc"))` → `'abc'`
`f"{值:[填充][对齐][符号][#][0][宽度][,][.精度][类型]}"`	格式化	否	新 str	`f"{x:>8.2f}"`
`"%s...%d" % (a, b)` / `"{}".format(v)`	老式格式化	否	新 str	`"%.2f" % 3.14159` → `'3.14'`
`re.findall(p, s)`	找出所有匹配	否	list	`re.findall(r"\d+", "a12b3")` → `['12', '3']`
`re.finditer(p, s)`	迭代匹配对象	否	迭代器	`m.group()` / `m.start()`
`re.sub(p, r, s)`	正则替换	否	新 str	`re.sub(r"\d+", "#", "a12")` → `'a#'`
`re.fullmatch(p, s)`	整串匹配判断	否	匹配对象或 None	验证邮箱、手机号

> **【核心】一句话结论**：**本章的字符串方法全部"返回新串、不原地修改"**；只有列表的 `append()` `sort()` `remove()` 之类才原地修改并返回 `None`（详见 4.5 节）。把这句话刻在脑子里，第 7 章一半的 bug 就消失了。

三、易错清单

1. **现象**：`s.strip()` 之后 `s` 没变。→ **原因**：字符串不可变，方法返回新串。→ **正确写法**：`s = s.strip()`，或者把方法直接写在表达式里用。
2. **现象**：`",".join([1, 2, 3])` 报 `TypeError: sequence item 0: expected str instance, int found`。→ **原因**：`join` 只接受字符串序列。→ **正确写法**：`",".join(map(str, nums))`。
3. **现象**：`a, b = input().split(" ")` 偶尔报 `ValueError: too many values to unpack`。→ **原因**：多余空格被切成空串。→ **正确写法**：用不带参数的 `input().split()`。
4. **现象**：`"itcast".strip("it")` 得到 `'cas'` 而不是 `'cast'`。→ **原因**：参数是**字符集合**，两端的 i、t 全被削掉。→ **正确写法**：删前缀用 `s.removeprefix("it")` 或切片 `s[2:]`。

5. **现象**：`if s.find("x"):` 永远成立。→ **原因**：找不到返回 -1，非 0 即真。→ **正确写法**：`if "x" in s:` 或 `if s.find("x") != -1:`。
6. **现象**：`print("{x:.2f}")` 原样打印出 `{x:.2f}`。→ **原因**：忘了字符串前面的 `f`。→ **正确写法**：`print(f"{x:.2f}")`。
7. **现象**：`f"{2.675:.2f}"` 得到 `2.67`，不是四舍五入应得的 `2.68`。→ **原因**：二进制浮点表示误差，不是语法错。→ **正确写法**：需要精确舍入用 `Decimal`（详见 1.9 节），或先放大成整数运算。
8. **现象**：`a, b = n.split()` 报 `AttributeError: 'int' object has no attribute 'split'`。→ **原因**：`int()` 转早了，字符串方法只能由字符串调用。→ **正确写法**：`s = input(); a, b = map(int, s.split())`。
9. **现象**：正则 `r"3.14"` 把 `3x14` 也匹配了。→ **原因**：`.` 是通配符。→ **正确写法**：`r"3\.14"`，其它元字符同理。
10. **现象**：`re.findall(r"\bcat", s)` 什么也找不到。→ **原因**：普通字符串里 `"\b"` 是退格符，模式被改掉。→ **正确写法**：正则一律用原始字符串 `r"\bcat"`。
11. **现象**：`re.findall(r"(\d+)-(\d+)", s)` 得到的是元组列表。→ **原因**：模式里有分组时，`findall` 只返回各组内容。→ **正确写法**：不要分组，或用 `[m.group() for m in re.finditer(p, s)]`。
12. **现象**：本地跑对，OJ 判 WA / PE。→ **原因**：`input()` 带了提示语、行尾多了空格、小数位数不对。→ **正确写法**：`input()` 不带参数，输出用 `" ".join(map(str, nums))`，小数用 `f"{x:.2f}"`。

四、必须能默写的 8 个片段

```
# 1. 读一行 n 个整数
nums = list(map(int, input().split()))
```

```
# 2. 输出一行、用空格分隔（行尾没有多余空格）
print(" ".join(map(str, nums)))
```

```
# 3. 去掉首尾空白 / 去掉行尾换行
s = input().strip()
line = line.rstrip("\n")
```

```
# 4. 字符串与列表互转
parts = s.split(",")          # 串 -> 列表
s = ",".join(parts)           # 列表 -> 串
```

```
# 5. 最常用的三种格式化
print(f"{avg:.2f}")               # 保留 2 位小数
print(f"{name:>8}{score:8.2f}")   # 姓名右对齐、分数宽 8 保留 2 位
print(f"{rate:.1%}")              # 百分比，保留 1 位小数
```

```
# 6. 遍历 + 条件计数（原串始终不变）
cnt = sum(1 for ch in s if ch.isupper())
```

```
# 7. 判断回文
is_pal = (s == s[::-1])
```

```
# 8. 正则三连：提取、替换、分割
import re
nums = re.findall(r"\d+", text)          # 提取所有数字串
text = re.sub(r"\s+", " ", text)         # 多个空白压成一个空格
parts = re.split(r"[,;]", line)          # 按逗号或分号切分
```

7.13 本章自测（15 道）

第 7 章 自测题 1 `"itcast".strip("sat")` 的结果是什么？为什么不是 `'itcast'`？

第 7 章 自测题 2 `find` 和 `index` 的区别是什么？

第 7 章 自测题 3 `"a,b,,c".split(",")` 和 `"a b c".split()` 的结果分别是什么？

第 7 章 自测题 4 用 `join` 把列表 `['2026', '10', '08']` 拼成 `'2026-10-08'`。

第 7 章 自测题 5 输入一个字符串，判断它是不是回文串（正读反读都一样），要求用切片实现。

第 7 章 自测题 6 用 f-string 输出：圆周率 π 保留 4 位小数、右对齐、总宽度 10。

第 7 章 自测题 7 输入一句英文，统计其中有多少个单词、有多少个大写字母。

第 7 章 自测题 8 把 `"Hello World"` 转成 `"hELLO wORLD"`。

第 7 章 自测题 9 用正则表达式从 `"我的电话是13812345678，备用电话是010-12345678"` 中提取出所有数字串。

第 7 章 自测题 10 编程：输入一个字符串，把所有出现的 `"bad"` 替换成 `"good"`，并输出替换了几次。

第 7 章 自测题 11 `" Hello ".strip()`、`" Hello ".lstrip()`、`" Hello ".rstrip()` 的结果分别是什么？为什么执行 `s.strip()` 之后 `s` 没有任何变化？

第 7 章 自测题 12 写出下列 f-string 的输出：`f"{3.14159:8.2f}"`、`f"{'py':*^6}"`、`f"{1234567:,}"`、`f"{0.1234:.1%}"`、`f"{255:#x}"`。

第 7 章 自测题 13（OJ） 输入一行若干个整数，输出它们的和与平均值。

- 输入格式：一行，若干个整数，用空格分隔（至少有 1 个）。
- 输出格式：第一行输出它们的和；第二行输出平均值，保留 2 位小数。
- 样例输入：`1 2 3 4`
- 样例输出：

```
10
2.50
```

第 7 章 自测题 14（OJ） 输入一行英文句子，统计单词个数，并把句子整理后输出。

- 输入格式：一行英文句子，单词之间可能有多个空格，行首行尾也可能有空格。

- 输出格式：第一行输出单词个数；第二行把每个单词改成"首字母大写、其余字母小写"后，用单个空格连接输出。
- 样例输入：`hello WORLD python`（行首、行尾各有两个空格）
- 样例输出：

```
3
Hello World Python
```

第 7 章 自测题 15（OJ）　输入一行文本，提取其中所有的非负整数。

- 输入格式：一行文本，由字母、数字、空格和标点组成，其中至少包含一个非负整数；整数之间至少隔着一个非数字字符。
- 输出格式：第一行按原顺序输出所有非负整数，用英文逗号连接；第二行输出其中最大的数。
- 样例输入：`price 120, count 45, total 300`
- 样例输出：

```
120,45,300
300
```

第 8 章 面向对象程序设计基础

> 对应课程讲义：《09_面向对象程序设计基础》。这一章概念比语法难：先把“类”和“对象”的关系想清楚，语法就只有几个固定套路。全章的核心只有一句话——**类负责定义，实例负责存储，方法靠 self 把两者连起来**。

8.1 类与对象

面向对象（Object Oriented）能以真实世界中的方式在程序中表示物体。

- 物体有**特征**和**行为**，对象有**属性**和**方法**；
- 对象通过**类**创建（实例化）；
- **类**是设计图，**对象**是按图造出来的房子；
- 对象也叫**实例**（instance）。

概念	说明	例子
类 Class	设计图	Circle
对象/实例 Object/Instance	按设计图造出来的具体东西	半径为 5 的那个圆
实例属性	与实例相关联的变量，表示状态	`radius`
实例方法	与实例相关联的函数，表示行为	`get_area()`
类属性	写在类里、不在 `__init__` 里的变量，所有实例共享	`count`

```
import math

class Circle:
    def __init__(self, radius=1):
        self.radius = radius

    def get_perimeter(self):
        return 2 * self.radius * math.pi

    def get_area(self):
        return self.radius * self.radius * math.pi
```

> **【核心】类名用大驼峰**（`MyCircle`、`BankAccount`），方法名和变量名用小写加下划线（`get_area`）。这是 PEP 8 的约定，也是所有 Python 代码的通用“穿衣风格”。
>
> **【注意】** 在 Python 里“类”本身就是一种“类型”。自定义的 `Circle` 和内置的 `int`、`str` 地位相同，`type(对象)` 拿到的就是它的类。

8.2 初始化方法 __init__ 与 self

> 【核心】 `__init__()` 是一个特殊方法，称为**初始化程序**。两个下划线习惯上读作 "dunder"（double underscores），所以 `__init__` 读作 "dunder init"。
>
> 【核心】**self 参数**：类的方法与普通函数只有一个特别的区别——它们必须有一个额外的第一个参数，按照惯例它的名称是 `self`。`self` 代表**类的实例**（不是类本身），是指向对象本身的参数。可以用 `self` 访问类定义中的对象成员变量和方法。

一旦一个实例变量被创建，它的作用域就是整个类。

self 到底是什么

书上说“self 代表实例”，但到底是谁把实例传进去的？答案是 **Python 自动传的**。

`c1.get_area()` 这一行，Python 实际执行的是 `Circle.get_area(c1)`——它把点号前面的 `c1` 当作**第一个实参**，塞进了第一个形参 `self` 里。

```
class Circle:
    def __init__(self, radius=1):
        self.radius = radius

    def get_area(self):
        print("我收到的 self 是：", self)
        return self.radius * self.radius * 3.14

c1 = Circle(5)
print(c1.get_area())
print("c1 自己        是：", c1)
c2 = Circle(9)
print(c2.get_area())
```

输出：

```
我收到的 self 是： <__main__.Circle object at 0x000001F2A1B0A5D0>
c1 自己        是： <__main__.Circle object at 0x000001F2A1B0A5D0>
我收到的 self 是： <__main__.Circle object at 0x000001F2A1B0A5C0>
```

第 1 行和第 3 行的地址**完全一样**：第 1 行是 `Circle(5)` 创建对象时 `__init__` 收到的 `self`，第 3 行是 `c1` 自己——它们本来就是同一个对象。最后一行地址变了，因为这次调用的是 `c2`。所以 `self` 不是什么神秘东西，它就是点号左边那个对象。

> 【核心】**三句话记住 self**： ① 定义方法时第一个参数必须留着位置，名字可以改但**一律写 self**； ② 调用时**不要传** self，Python 自动传； ③ 方法里想用实例自己的属性、或调用同一个类的其它实例方法，一律写 `self.xxx`。

【注意】 方法内部调用同一个类的另一个方法时，必须写 `self.get_perimeter()`；只写 `get_perimeter()` 会报 `NameError: name 'get_perimeter' is not defined`，因为方法名不在局部变量里。

__init__ 的三个关键性质

【核心】 ① **自动调用**：不需要写 `对象.__init__()`，写 `Circle(5)` 时自动跑一遍；② **每个实例各跑一次**：造 3 个对象就执行 3 次；③ **不能有返回值**：它只负责往 `self` 上挂属性，写 `return 0` 会报 `TypeError: __init__() should return None`。

【常见错误】 `__init__` 前后各是**两个**下划线，写成 `_init_`（各一个）或 `__init_`（尾巴少一个）都不会被自动调用，于是属性全没挂上。

```
# X 错误写法
class Circle:
    def _init_(self, radius=1):       # 前后只有一个下划线
        self.radius = radius

c = Circle(5)                          # TypeError: Circle() takes no arguments
print(c.radius)
```

```
# √ 正确写法
class Circle:
    def __init__(self, radius=1):     # 前后各两个下划线
        self.radius = radius

c = Circle(5)
print(c.radius)                        # 5
```

报错信息是 `TypeError: Circle() takes no arguments`，看着像“参数写错了”，其实是 `__init__` 根本没被认出来，Python 只好用了一个什么都不做的默认初始化方法。

【注意】 还有一种相关的错误：手写 `c = Circle.__init__(Circle, 5)` 来“创建对象”。`__init__()` 的返回值固定是 `None`（它的工作是初始化，不是创建对象），拿它当对象用会报 `AttributeError: 'NoneType' object has no attribute 'radius'`。记住：**创建对象就是写 `Circle(5)`**。

【常见错误】 定义方法时漏写 `self`；反过来，实例化时又多写了 `self`。

```
# X 错误写法一：定义时漏了 self
class Circle:
    def area():
        return 3.14

c = Circle()
print(c.area())        # TypeError: Circle.area() takes 0 positional arguments but 1 was
given
```

```
# ✗ 错误写法二：实例化时多写了 self
class Circle:
    def __init__(self, radius=1):
        self.radius = radius

c = Circle(c, 5)       # TypeError: Circle.__init__() takes from 1 to 2 positional
arguments but 3 were given
```

```
# ✓ 正确写法
class Circle:
    def __init__(self, radius=1):
        self.radius = radius

    def area(self):
        return self.radius * self.radius * 3.14

c = Circle(5)
print(c.area())        # 78.5
```

那串 "takes 0 positional arguments but 1 was given" 里的 "1"，就是 Python 自动传进去的实例。看到这句话，九成是方法定义漏了 `self`。

使用类创建对象

【了解】 实例化类在其它语言中一般用关键字 `new`，Python 中并没有这个关键字，类的实例化类似函数调用的方式。

```
import math

class Circle:
    def __init__(self, radius=1):
        self.radius = radius
    def get_perimeter(self):
        return 2 * self.radius * math.pi
    def get_area(self):
        return self.radius * self.radius * math.pi

def main():
    circle1 = Circle()          # 用默认半径 1
    circle2 = Circle(5)         # 半径为 5
    print(circle1.get_perimeter())   # 6.283185307179586
    print(circle1.get_area())        # 3.141592653589793
    print(circle2.get_perimeter())   # 31.41592653589793
    print(circle2.get_area())        # 78.53981633974483

    circle2.radius = 10              # 用点号访问对象的属性
    print(circle2.get_perimeter())   # 62.83185307179586
    print(circle2.get_area())        # 314.1592653589793
    print(circle2.radius)            # 10
```

```
main()
```

【注意】 Python 允许你**临时给对象加属性**（`circle2.color = "red"` 完全合法），但这样加出来的属性只有这一个对象有。规范做法是把所有属性都在 `__init__` 里初始化好，读代码的人一眼就知道这个类有哪些属性。

【速查】 想知道一个对象身上到底挂了哪些属性，用 `对象名.__dict__`：

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(1, 2)
print(p.__dict__)      # {'x': 1, 'y': 2}
```

8.3 封装：私有属性与 property

【核心】 直接访问对象的属性不是好方法，可能造成异常数据（比如把圆的半径设成负数）。**数据隐藏**的做法是：以两个下划线开头的是**私有属性**，以两个下划线开头的也是**私有方法**。私有属性和私有方法只能在类内部被访问，不能在类外被访问。

```
import math

class Circle:
    def __init__(self, radius=1):
        if radius < 0:
            raise RuntimeError("Negative radius")
        else:
            self.__radius = radius

    def get_radius(self):
        return self.__radius

    def set_radius(self, radius):
        if radius < 0:
            raise RuntimeError("Negative radius")
        else:
            self.__radius = radius

    def get_perimeter(self):
        return 2 * self.__radius * math.pi

    def get_area(self):
        return self.__radius * self.__radius * math.pi

def main():
```

```
    circle2 = Circle(5)
    circle2.set_radius(10)      # 正确做法：通过方法改
    # circle2.radius = 10       # 不会影响半径的值（只是新建了一个普通属性）
    # circle2.__radius = 10     # 报错：类外不能访问私有属性
    print(circle2.get_perimeter())   # 62.83185307179586
    print(circle2.get_area())        # 314.1592653589793
    print(circle2.get_radius())      # 10
    # circle2.set_radius(-10)  # 抛出 RuntimeError: Negative radius

main()
```

【易错】奇怪的知识：`__radius` 是私有属性，但 `__radius__` 不是！私有属性以两个下划线**开头**，且**不能以下划线结尾**；首尾都是两个下划线的是系统保留名称。

【易错】 私有属性并非绝对不能在类外访问，用 `实例名._类名__私有属性`（如 `circle1._Circle__radius`）其实能访问到。它并不是靠“藏起来”实现保护的，而是 Python 在编译时把 `__radius` 改名成了 `_Circle__radius`（这个动作叫**名称改写**）。名字都告诉你了，所以这只是“约定你别碰”，不是真正的安全机制。

【速查】 一个下划线和两个下划线的区别：

写法	名称	类外能访问吗	约定含义
`name`	公有属性	能	谁都能改
`_name`	受保护属性	能	“内部使用，请别碰”
`__name`	私有属性	不能（会被改名）	只能在本类内部用
`__name__`	系统保留名称	能	Python 自己用的，别自创
`name_`	避开关键字	能	想叫 `class` 就写 `class_`

property 装饰器

通过 `@property` 装饰器，可以把一个方法转换成**属性**，之后可以直接用方法名访问（不用写小括号）。修改则通过 `@属性名.setter` 转换的函数完成，且该函数必须与属性同名。

```
import math

class Circle:
    def __init__(self, radius=1):
        self.radius = radius        # 注意：这一行调用的就是下面的 setter

    @property
    def radius(self):
        return self.__radius

    @radius.setter
    def radius(self, radius):
        if radius < 0:
            raise RuntimeError("Negative radius")
```

```
        else:
            self.__radius = radius

    @property
    def area(self):
        return self.__radius * self.__radius * math.pi

    @property
    def perimeter(self):
        return 2 * self.__radius * math.pi

def main():
    circle1 = Circle()
    circle1.radius = 10            # 表面像直接赋属性，实际调用了 setter
    print(circle1.perimeter)       # 31.41592653589793，不带小括号
    print(circle1.area)            # 314.1592653589793
    print(circle1.radius)          # 10
    # circle1.radius = -1          # RuntimeError: Negative radius

main()
```

【核心】 property 的好处是“**外部用起来像属性，内部仍然受控制**”——既能写 `circle1.radius = 10` 这样简洁的代码，又能保留“半径不能为负”的检查。

【核心】@property 的三个用途（背下来，简答题常考）： ① **只读属性**：只写 `@property` 不写 setter，外部只能读不能改，赋值时报 `AttributeError: property 'radius' of 'Circle' object has no setter`； ② **写入时校验**：配上 `@属性名.setter`，把“合法性检查”藏进赋值动作里，外部代码一点都看不出来； ③ **计算属性**：像 `area`、`perimeter` 这种由别的属性算出来的值，用 property 表示最自然，还永远不会和真实数据不一致。

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    @property
    def celsius(self):
        return self.__celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("低于绝对零度")
        self.__celsius = value

    @property
    def fahrenheit(self):          # 用途三：计算属性，只读
        return self.__celsius * 9 / 5 + 32

def main():
```

```
    t = Temperature(25)
    print(t.celsius)              # 25
    print(t.fahrenheit)           # 77.0
    t.celsius = 100               # 走 setter，有校验
    print(t.fahrenheit)           # 212.0
    # t.fahrenheit = 100          # 用途一：只读，报 AttributeError

main()
```

【易错】setter 的三个硬性要求：① 装饰器必须写成 `@属性名.setter`（如 `@celsius.setter`），写成 `@setter` 或 `@property.setter` 都不行；② setter 函数必须与 getter **同名**（都叫 `celsius`，不能叫 `set_celsius`）；③ 只写 `@property` 不写 setter，属性就是只读的。

例题 8-1：银行账户 BankAccount

题目：定义一个 `BankAccount` 类，属性有户主 `owner` 和余额 `balance`（私有，不能为负）。要求提供 `deposit(amount)` 存款和 `withdraw(amount)` 取款两个方法，金额非法（存入非正数或超额取款）时抛出 `ValueError`；用 `__str__` 输出 `户主：余额`（余额保留两位小数）。把余额写成 property，赋值时自动检查非负。

```
class BankAccount:
    def __init__(self, owner, balance=0.0):
        self.owner = owner
        self.balance = balance          # 调用下面的 setter，负数会抛异常

    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("Balance cannot be negative")
        else:
            self.__balance = value

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self.__balance += amount
        return self.__balance

    def withdraw(self, amount):
        if amount > self.__balance:
            raise ValueError("Not enough balance")
        self.__balance -= amount
        return self.__balance

    def __str__(self):
```

```
        return f"{self.owner}: {self.__balance:.2f}"

def main():
    acc = BankAccount("Tom", 1000)
    print(acc)                          # Tom: 1000.00
    acc.deposit(500.5)
    print(acc)                          # Tom: 1500.50
    try:
        acc.withdraw(2000)              # 余额不足
    except ValueError as e:
        print("操作失败:", e)
    print(acc)                          # Tom: 1500.50，余额没有被改动
    print(acc.balance)                  # 1500.5

main()
```

输入 `Tom` 和 `1000`（户主与初始余额），输出：

```
Tom: 1000.00
Tom: 1500.50
操作失败: Not enough balance
Tom: 1500.50
1500.5
```

要点：`self.__balance += amount` 等价于 `self.__balance = self.__balance + amount`，读的时候走 getter、写的时候走 setter；由于 setter 里有检查，存款根本不可能把余额搞成负数。取款失败时用 `try/except` 接住异常，**对象的状态一点没变**——这就是封装带来的好处。

【注意】 `__init__` 里写 `self.balance = balance` 而不是 `self.__balance = balance`，是为了让初始化也走一遍 setter 的校验。如果直接写 `self.__balance = balance`，`BankAccount("Tom", -100)` 就能造出一个余额为负的对象，封装就白做了。

8.4 类属性、类方法与静态方法

实例属性只属于实例自己；而有的信息不希望每个对象都拥有，例如“同一类型对象的总数”，这时就要用**类属性**。

```
class Circle:
    count = 0                       # 类属性，直接定义在类中，不加 self
    def __init__(self, radius=1):
        self.__radius = radius
        Circle.count += 1

def main():
    circle1 = Circle()
    circle2 = Circle(5)
    print(Circle.count)             # 2
```

```
main()
```

【核心】类属性的几个特点：① 类属性的定义语句只执行一次；② 在第一个对象创建之前就存在；③ 用 `类名.类属性名` 访问，可以读取也可以修改；④ 用 `实例名.类属性名` **只能读取，不能修改**——如果尝试用实例名修改，会**产生一个新的实例属性**，而不是改变类属性。

```
class Circle:
    count = 0
    def __init__(self, radius=1):
        self.__radius = radius
        Circle.count += 1

def main():
    circle1 = Circle()
    circle2 = Circle(5)
    print(Circle.count, circle1.count, circle2.count)   # 2 2 2
    Circle.count = 6
    circle1.count = 8          # 这会产生一个实例属性，掩盖类属性
    print(Circle.count, circle1.count, circle2.count)   # 6 8 6

main()
```

【常见错误】 在 `__init__` 里想给计数器加一，却写成了 `self.count += 1`，结果每个实例各有一个 `count`，类属性永远停在 0。

```
# ✗ 错误写法
class Circle:
    count = 0
    def __init__(self):
        self.count += 1        # 本意是让计数器加一

a = Circle()
b = Circle()
print(Circle.count, a.count, b.count)   # 0 1 1  ← 类属性根本没动
```

```
# ✓ 正确写法
class Circle:
    count = 0
    def __init__(self):
        Circle.count += 1      # 用 类名.属性名 才是改类属性

a = Circle()
b = Circle()
print(Circle.count, a.count, b.count)   # 2 2 2
```

一句话规则：**读类属性可以写 `self.属性名`，改类属性必须写 `类名.属性名`。** 之所以会这样，是因为 `self.count += 1` 里含有一个赋值动作，而“给 `self.属性` 赋值”永远是在实例身上新建属性。

【常见错误】 可变类属性的共享陷阱：类属性如果是**列表、字典**这类可变对象，所有实例共用同一个对象；用 `self.列表.append(x)` 改的是这一份共享数据，会波及所有实例。**不可变类属性（数字、字符串、元组）不会出现这个问题**，因为对它们做 `+=` 是新建实例属性，而不是改原对象。

```
# ✗ 错误写法：可变类属性被所有实例共享
class Course:
    students = []                   # 可变类属性，所有对象共用一个列表
    def add(self, name):
        self.students.append(name)  # append 是原地修改，改的就是类属性那一份

c1 = Course()
c2 = Course()
c1.add("Tom")
print("course1:", c1.students)     # course1: ['Tom']
print("course2:", c2.students)     # course2: ['Tom']  ← 串班了
```

```
# ✓ 正确写法：每个实例自己一份，在 __init__ 里创建
class Course:
    def __init__(self, name):
        self.name = name
        self.students = []          # 实例属性，每个对象一份
    def add(self, student):
        self.students.append(student)

c1 = Course("Python")
c2 = Course("Java")
c1.add("Tom")
print("course1:", c1.students)     # course1: ['Tom']
print("course2:", c2.students)     # course2: []
```

判断口诀：**“这份数据是所有对象共同拥有一份，还是每个对象各自一份？”** 共同一份放类属性（而且最好是数字这类不可变值），各自一份一律放进 `__init__` 里用 `self.` 创建。

【速查】 分不清一个属性到底是谁的？打印 `__dict__` 一看就明白：

```
class Circle:
    count = 0
    def __init__(self, radius=1):
        self.radius = radius
        Circle.count += 1

c = Circle()
print(Circle.__dict__["count"])   # 1   ← 类属性挂在类身上
print(c.__dict__)                 # {'radius': 1}  ← 实例属性只有 radius
```

类方法与静态方法

```
class Circle:
    count = 0
```

```
    def __init__(self, radius=1):
        self.__radius = radius
        Circle.count += 1

    @classmethod
    def get_count1(cls):        # 第一个参数 cls 是类本身
        return cls.count

    @staticmethod
    def get_count2():           # 没有类参数
        return Circle.count

def main():
    circle1 = Circle()
    circle2 = Circle(5)
    print(Circle.get_count1())     # 2
    print(Circle.get_count2())     # 2
    print(circle1.get_count1())    # 2，也可以通过实例调用
    print(circle1.get_count2())    # 2

main()
```

【核心】类方法与静态方法的四条共同点：① 都只能访问类属性，不能访问实例属性（因为没有 self）；② 访问类属性时，类方法可以用 `cls` 作前缀，静态方法只能用完整类名；③ 都不能调用实例方法；④ 既可以通过类名调用，也可以通过实例对象调用。类方法与静态方法之间可以互相调用。

对比项	实例方法	类方法 `@classmethod`	静态方法 `@staticmethod`
第一个参数	`self`（实例）	`cls`（类）	没有
能访问实例属性	能	不能	不能
能访问类属性	能（`self.x` 或 `类名.x`）	能（`cls.x`）	只能写 `类名.x`
调用方式	`对象.方法()`	`类.方法()` 或 `对象.方法()`	`类.方法()` 或 `对象.方法()`
什么时候用	需要读写某个对象的数据	需要用到“类”本身（计数器、工厂方法）	纯工具函数，和谁都不搭边

【注意】 类方法里的 `cls` 是“调用它的那个类”。这一点在继承里立刻见效：子类调用继承来的类方法时，`cls` 是**子类**而不是父类。详见 8.6 节。

8.5 常用内置方法与运算符重载

Python 类中提供了很多内置方法，形式为 `__方法名__()`。与 `__init__()` 类似的还有 `__del__()`，它将在对象被销毁时自动执行（引用计数归零或程序结束时）。

__str__()

`__str__()` 在需要时将对象转换为字符串返回，应用场景如把对象与字符串相加、`print(对象)` 或者 `str(对象)`。

```
import math

class Circle:
    def __init__(self, radius=1):
        self.__radius = radius
    def get_perimeter(self):
        return 2 * self.__radius * math.pi
    def get_area(self):
        return self.__radius * self.__radius * math.pi
    def __str__(self):
        return ("Circle with radius " + str(self.__radius)
                + " has perimeter of " + str(self.get_perimeter())
                + " and area of " + str(self.get_area()))

print(Circle(5))
```

> **【注意】** `__str__` **必须返回字符串**，返回数字、元组、对象都不行。拼字符串时别忘了 `str()` 转换——私有属性存的是数字。

__str__ 与 __repr__ 的区别

这两个方法长得像，用途完全不同，考试很喜欢考。

	`__str__`	`__repr__`
面向谁	**用户**，给人看的	**开发者**，给调试和交互环境看的
什么时候被调用	`print(对象)`、`str(对象)`、f-string 里的 `{对象}`	直接敲对象名回车、`repr(对象)`、把对象放进列表再打印
没定义时	退回去用 `__repr__`，再不行打印 `<... object at 0x...>`	默认是 `<Circle object at 0x000001F2A1B0A5D0>`
建议写法	可读的描述，如 `半径 5 的圆`	长得像创建它的代码，如 `Circle(5)`

```
class Circle:
    def __init__(self, radius=1):
        self.radius = radius
    def __str__(self):
        return f"半径 {self.radius} 的圆"
    def __repr__(self):
        return f"Circle({self.radius})"

c = Circle(5)
print(c)              # 半径 5 的圆            ← 调 __str__
```

```
print([c, c])          # [Circle(5), Circle(5)]  ← 列表里调 __repr__
print(repr(c))         # Circle(5)
```

【核心】一句话记忆：`__str__` 是“给人看的”，`__repr__` 是“给 Python 看的”。**定义了 `__str__` 之后顺手写一个 `__repr__`**，调试时会舒服很多。list、dict 打印元素时只认 `__repr__`，不认 `__str__`。

__eq__()

判断两个对象是否相等。

```
class Circle:
    def __init__(self, radius=1):
        self.__radius = radius

    def __eq__(self, other):
        return self.__radius == other.__radius

circle1 = Circle(1)
circle2 = Circle(1)
print(circle1 == circle2)     # True
print(circle1 is circle2)     # False，两个不同的对象
```

【核心】 默认的 `==` 比较的是“是不是同一个对象”（效果和 `is` 一样），只有重载了 `__eq__` 才变成“按内容比较”。

【易错】 重载 `__eq__` 之后 `__hash__` 会被自动设成 `None`，于是这个类的对象**不能再放进 set、也不能当字典的键**，会报 `TypeError: unhashable type: 'Circle'`。如果确实需要，得再定义 `__hash__`：

```
class Circle:
    def __init__(self, radius=1):
        self.radius = radius
    def __eq__(self, other):
        return self.radius == other.radius
    def __hash__(self):
        return hash(self.radius)      # 内容相同的对象哈希值也相同
```

常用运算符重载

【核心】 只要在类里定义了对应的方法，这个类的对象就能支持对应的运算。**重载 `__eq__` 之后别忘了同时重载 `__ne__`**（`!=`），否则 `!=` 会沿用默认的“比较是否为同一对象”，可能出现 `a == b` 和 `a != b` 同时为 `True` 的荒唐结果。

运算符	方法	描述
`+`	`__add__(self, other)`	加法
`-`	`__sub__(self, other)`	减法

运算符	方法	描述
`*`	`__mul__(self, other)`	乘法
`/`	`__truediv__(self, other)`	算术除
`//`	`__floordiv__(self, other)`	整数除
`%`	`__mod__(self, other)`	取余
`**`	`__pow__(self, other)`	幂运算
`<` `<=` `>` `>=` `==` `!=`	`__lt__` `__le__` `__gt__` `__ge__` `__eq__` `__ne__`	比较
`[index]`	`__getitem__(self, index)`	索引
`in`	`__contains__(self, value)`	包含
`len()`	`__len__(self)`	求长度
`str()`	`__str__(self)`	转字符串
`repr()`	`__repr__(self)`	转调试字符串
`hash()`	`__hash__(self)`	哈希值，可作字典键
`print(对象)`	`__str__` 或 `__repr__`	打印

【常见错误】 `__str__` 返回了非字符串。

```
# ✗ 错误写法
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __str__(self):
        return self.x, self.y        # 返回的是元组

print(Point(1, 2))                   # TypeError: __str__ returned non-string (type tuple)
```

```
# ✓ 正确写法
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __str__(self):
        return f"({self.x}, {self.y})"   # 一定拼成字符串

print(Point(1, 2))                   # (1, 2)
```

除了 `__str__`，`__repr__` 也要求返回字符串。手懒就统一用 f-string，它总会给你一个 `str`。

【注意】 定义了 `__lt__` 之后，`sorted(对象列表)` 就能直接用了——`sorted` 内部只依赖 `<`。所以想让自定义对象能排序，**最少只需要写** `__lt__`。

例题 8-2：金额类 Money（运算符重载综合练习）

题目：定义一个 `Money` 类，内部只保存一个整数“总角数” `__total_jiao`，构造时接收“元”和“角”两个参数。要求：用 property 提供只读的 `yuan`；用 `__str__` 输出 `CNY 12.50` 形式，用 `__repr__` 输出 `Money(12, 5)` 形式；支持 `+` 相加、`==` `!=` `<` 比较以及 `m[0]` 取元、`m[1]` 取角；再提供一个静态方法 `to_jiao(yuan, jiao=0)` 做单位换算。

```
class Money:
    rate = 7.1                      # 类属性：示例汇率

    def __init__(self, yuan=0, jiao=0):
        self.__total_jiao = yuan * 10 + jiao

    @property
    def yuan(self):                 # 只读计算属性
        return self.__total_jiao / 10

    @staticmethod
    def to_jiao(yuan, jiao=0):      # 与实例无关的工具函数
        return yuan * 10 + jiao

    def __str__(self):
        return f"CNY {self.__total_jiao / 10:.2f}"

    def __repr__(self):
        return f"Money({self.__total_jiao // 10}, {self.__total_jiao % 10})"

    def __add__(self, other):
        return Money(0, self.__total_jiao + other.__total_jiao)

    def __eq__(self, other):
        return self.__total_jiao == other.__total_jiao

    def __ne__(self, other):
        return not self.__eq__(other)

    def __lt__(self, other):
        return self.__total_jiao < other.__total_jiao

    def __getitem__(self, index):
        if index == 0:
            return self.__total_jiao // 10
        elif index == 1:
            return self.__total_jiao % 10
        else:
            raise IndexError("Index out of range. 0 for yuan, 1 for jiao")

def main():
    m1 = Money(12, 5)
    m2 = Money(3, 8)
    print("m1 =", m1)
```

```
    print("repr(m1) =", repr(m1))
    print("m1 + m2 =", m1 + m2)
    print("m1 == m2 ?", m1 == m2, " m1 != m2 ?", m1 != m2)
    print("m1 > m2 ?", m1 > m2)
    print("元 =", m1[0], " 角 =", m1[1])
    print("10 元 =", Money.to_jiao(10), "角")
    print("实例读类属性 rate =", m1.rate)

main()
```

输入 `12 5` 与 `3 8`（两组金额），输出：

```
m1 = CNY 12.50
repr(m1) = Money(12, 5)
m1 + m2 = CNY 16.30
m1 == m2 ? False  m1 != m2 ? True
m1 > m2 ? True
元 = 12  角 = 5
10 元 = 100 角
实例读类属性 rate = 7.1
```

> **要点**：只写了 `__lt__` 和 `__eq__`，`>` 看起来也能用，其实是 Python 自动把 `a > b` 换成了 `b < a` 再取反。**如果自己写 `__gt__`，千万别在里面再调用自己，会造成无限递归。**另外 `m[0]`、`m[1]` 能工作完全靠 `__getitem__`，越界时自己抛 `IndexError` 比返回 `None` 友好得多。

8.6 继承与方法重写

面向对象的编程带来的主要好处之一是**代码的重用**，实现这种重用的方法之一就是**继承**。通过继承创建的新类称为**子类**或派生类，被继承的类称为**基类**、父类或超类。

```
class 派生类名(基类名):
    ...
```

Python 继承的几个要点：

1. **子类需要父类的构造方法时**：如果子类不重写 `__init__()`，实例化子类会自动调用父类的 `__init__()`；如果子类重写了 `__init__()`，就不会自动调用父类的，需要显式调用：

```
super().__init__(参数1, 参数2, ...)              # 推荐写法（Python 3）
super(子类, self).__init__(参数1, 参数2, ...)    # 老写法，等价
父类名称.__init__(self, 参数1, 参数2, ...)       # 经典写法
```

1. **调用基类方法时**需要加上基类的类名前缀，且需要带上 `self` 参数。不过更推荐用 `super().方法名(...)`，理由见下面的【注意】。
2. Python 总是**先在派生类中查找方法**，找不到才去基类中逐个查找。这条查找链叫 **MRO**（方法解析顺序），用 `类名.__mro__` 可以打印出来看。

【核心】为什么子类重写了 `__init__` 就“必须”调用 `super().__init__()`：父类的 `__init__` 负责给父类那部分属性赋值。子类重写之后，父类的初始化**不会被自动执行**，于是对象身上根本没有父类的属性，一用就报错。这不是语法强制（不写不报错），而是逻辑上必须——子类继承了父类的方法，而这些方法依赖父类的属性，属性没建好，方法就是废的。

【常见错误】 子类重写了 `__init__` 却忘了调用 `super().__init__()`。

```
# X 错误写法
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Student(Person):
    def __init__(self, name, age, school):
        self.school = school           # 只初始化了自己的属性

s = Student("Tom", 18, "THU")
print(s.name)      # AttributeError: 'Student' object has no attribute 'name'
```

```
# √ 正确写法
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Student(Person):
    def __init__(self, name, age, school):
        super().__init__(name, age)    # 第一件事：把父类的活干完
        self.school = school

s = Student("Tom", 18, "THU")
print(s.name, s.age, s.school)         # Tom 18 THU
```

位置也有讲究：`super().__init__(...)` 一般写在子类 `__init__` 的**最前面**，避免中途抛异常留下一个半成品对象。

【注意】 用 `super()` 而不是写死“父类名”的好处：以后如果继承关系改了（比如中间插进一层 `GraduateStudent(Person)`），`super()` 会自动找到 MRO 上的下一个类，写死类名的代码就全得改。

【了解】 如果在继承元组中列了一个以上的类，就称作“多重继承”：`class SubClassName(ParentClass1, ParentClass2):`。多重继承的 MRO 规则（C3 算法）比较复杂，用时记住一条：`super()` 找的是 MRO 里的下一个类，不一定是直接父类。

```
class Parent:                          # 定义父类
    parentAttr = 100
    def __init__(self):
        print("调用父类构造函数")
    def parentMethod(self):
```

```
        print('调用父类方法')
    def setAttr(self, attr):
        Parent.parentAttr = attr
    def getAttr(self):
        print("父类属性 :", Parent.parentAttr)

class Child(Parent):                # 定义子类
    def __init__(self):
        print("调用子类构造方法")
    def childMethod(self):
        print('调用子类方法')

c = Child()          # 调用子类构造方法
c.childMethod()      # 调用子类方法
c.parentMethod()     # 调用父类方法
c.setAttr(200)
c.getAttr()          # 父类属性 : 200
```

方法重写

如果父类方法的功能不能满足需求，可以在子类中**重写**父类的方法。

```
class Parent:
    def myMethod(self):
        print('调用父类方法')

class Child(Parent):
    def myMethod(self):              # 重写
        print('调用子类方法')

c = Child()
c.myMethod()                         # 调用子类方法
```

isinstance 与 type 的区别

判断“这个对象到底是什么”，有两个函数，用错了就会出 bug。

	`type(obj)`	`isinstance(obj, 类)`
返回值	对象的**确切类**	`True` / `False`
子类实例	`type(dog) == Animal` 是 `False`	`isinstance(dog, Animal)` 是 `True`
能否判断多个类	不能	能，第二个参数可传元组：`isinstance(x, (int, float))`
是否考虑继承	不考虑	**考虑继承链**
建议	只在必须“精确等于某个类”时用	**日常判断类型一律用它**

```
class Animal:
    pass

class Dog(Animal):
```

```
    pass

d = Dog()
print(type(d) == Dog)               # True
print(type(d) == Animal)            # False ← type 不认继承
print(isinstance(d, Dog))           # True
print(isinstance(d, Animal))        # True ← isinstance 认继承
print(isinstance(5, (int, float)))  # True，一次判断多种类型
```

【注意】 判断“是不是数字”要写成 `isinstance(x, (int, float))`。`bool` 是 `int` 的子类，所以 `isinstance(True, int)` 也是 `True`；用 `type(x) == int` 判断经常漏掉 `float`。也别用 `type(x) == type(y)` 比较类型，`isinstance(x, type(y))` 更稳。

例题 8-3：图形面积与周长（继承 + 多态 + OJ 风格完整程序）

题目：定义基类 `Shape`，提供 `area()` 与 `perimeter()` 两个方法（基类里直接抛 `NotImplementedError`，表示“子类必须重写”）和统一的 `__str__` 输出。派生 `Rectangle`（长、宽）、`Square`（边长，继承自 `Rectangle`）和 `Circle`（半径）。输入第一行是测试数据组数 `t`，接下来 `t` 行，每行格式为“形状代号 参数...”，`R` 表示矩形（长 宽）、`C` 表示圆（半径）、`S` 表示正方形（边长）。每种形状输出三行：`形状名 area=面积 perimeter=周长`（保留两位小数），以及它是不是 `Shape` 的实例、是不是 `Rectangle` 的实例。

```
import math

class Shape:
    def __init__(self, name="shape"):
        self._name = name           # 一个下划线：子类要改，属于“内部使用”

    def area(self):
        raise NotImplementedError("子类必须重写 area()")

    def perimeter(self):
        raise NotImplementedError("子类必须重写 perimeter()")

    def __str__(self):
        return f"{self._name} area={self.area():.2f} perimeter={self.perimeter():.2f}"

class Rectangle(Shape):
    def __init__(self, width, height):
        super().__init__("rectangle")
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

class Square(Rectangle):
```

```
    def __init__(self, side):
        super().__init__(side, side)     # 复用父类构造：长 = 宽 = 边长
        self._name = "square"

class Circle(Shape):
    def __init__(self, radius):
        super().__init__("circle")
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

    def perimeter(self):
        return 2 * math.pi * self.radius

def make_shape(kind, nums):
    if kind == "R":
        return Rectangle(nums[0], nums[1])
    elif kind == "C":
        return Circle(nums[0])
    elif kind == "S":
        return Square(nums[0])
    else:
        raise ValueError("Unknown shape")

def main():
    t = int(input())
    for _ in range(t):
        parts = input().split()
        shape = make_shape(parts[0], [float(x) for x in parts[1:]])
        print(shape)
        print("是 Shape 的实例吗？", isinstance(shape, Shape))
        print("是 Rectangle 的实例吗？", isinstance(shape, Rectangle))

if __name__ == "__main__":
    main()
```

输入：

```
3
R 3 4
C 1
S 5
```

输出：

```
rectangle area=12.00 perimeter=14.00
是 Shape 的实例吗？ True
是 Rectangle 的实例吗？ True
circle area=3.14 perimeter=6.28
是 Shape 的实例吗？ True
```

```
是 Rectangle 的实例吗？ False
square area=25.00 perimeter=20.00
是 Shape 的实例吗？ True
是 Rectangle 的实例吗？ True
```

要点：`__str__` 只在 `Shape` 里写了一次，三个子类全都能用——这叫**代码复用**；`print(shape)` 时到底算的是哪个 `area()`，由对象**运行时**的真正类型决定，这叫**多态**。`Square` 只写了三行，就完整继承了矩形的全部功能，还顺手把名字改成了 `square`。`isinstance` 沿继承链判断：正方形“是矩形”为 `True`，圆“是矩形”为 `False`。

【注意】 OJ 题里写类的完整套路：**先写数据读入的函数 `make_shape`，再写 `main()`，最后 `if __name__ == "__main__":` 保护起来**（见第 9 章）。所有子类的构造方法第一行都是 `super().__init__(...)`，这是排查“属性不存在”类错误的第一现场。

8.7 综合示例

示例 8-4：分数类 MyFraction（对应作业题 16 的进阶版）

要求：分别存储分子和分母；存储时约分为最简分数；负号标在分子上；值为 0 时分子为 0、分母为 1；值为整数时分母为 1；支持多种构造方式；为不可变类，分子分母是只读属性；支持四则运算与比较运算；支持 `str()`；支持用 `[0]`、`[1]` 取分子分母。

【注意】 这道题就是第 3 章作业题 16「两个分数的四则运算」的面向对象版本。作业题 16 用“函数 + 元组”实现，本示例用类实现，**核心算法完全一样**（约分靠最大公约数、负号放分子、0 和整数的分母取 1）。做完这个类再回头看作业题 16，你会发现 OO 版把所有细节都封在类里，主程序只剩几行。

```
class MyFraction:
    @staticmethod
    def reduce(numerator, denominator):
        if numerator * denominator < 0:
            sign = -1
        else:
            sign = 1
        gcd = 1
        numerator, denominator = abs(numerator), abs(denominator)
        for i in range(min(numerator, denominator), 0, -1):
            if numerator % i == 0 and denominator % i == 0:
                gcd = i
                break
        numerator = sign * numerator // gcd
        denominator = denominator // gcd if numerator != 0 else 1
        return numerator, denominator

    def __init__(self, numerator=0, denominator=1):
        if denominator == 0:
            raise ZeroDivisionError("division by zero")
        self.__numerator, self.__denominator = self.reduce(numerator, denominator)
```

```
    def __str__(self):
        return str(self.__numerator) + "/" + str(self.__denominator)

    def __repr__(self):
        return f"MyFraction({self.__numerator}, {self.__denominator})"

    @property
    def numerator(self):
        return self.__numerator

    @property
    def denominator(self):
        return self.__denominator

    def __eq__(self, other):
        return (self.__numerator == other.__numerator
                and self.__denominator == other.__denominator)

    def __ne__(self, other):
        return not self.__eq__(other)

    def __lt__(self, other):
        return (self.__numerator * other.__denominator
                < other.__numerator * self.__denominator)

    def __le__(self, other):
        return self.__lt__(other) or self.__eq__(other)

    def __gt__(self, other):
        return other.__lt__(self)

    def __ge__(self, other):
        return not self.__lt__(other)

    def __add__(self, other):
        x = self.__numerator * other.__denominator + other.__numerator * self.__denominator
        y = self.__denominator * other.__denominator
        return MyFraction(x, y)

    def __sub__(self, other):
        x = self.__numerator * other.__denominator - other.__numerator * self.__denominator
        y = self.__denominator * other.__denominator
        return MyFraction(x, y)

    def __mul__(self, other):
        return MyFraction(self.__numerator * other.__numerator,
                          self.__denominator * other.__denominator)

    def __truediv__(self, other):
        return MyFraction(self.__numerator * other.__denominator,
```

```
                           self.__denominator * other.__numerator)

    def __getitem__(self, index):
        if index == 0:
            return self.__numerator
        elif index == 1:
            return self.__denominator
        else:
            raise IndexError("Index out of range.")

def main():
    f1 = MyFraction(7, 8)
    f2 = MyFraction(-5, 12)
    assert f1 + f2 == MyFraction(11, 24)
    assert f1 - f2 == MyFraction(31, 24)
    assert f1 * f2 == MyFraction(-35, 96)
    assert f1 / f2 == MyFraction(-21, 10)
    assert f1 > f2
    print(f1, f2)
    print(f1 + f2, f1 - f2, f1 * f2, f1 / f2)
    print("Congratulations! All tests are passed!")

if __name__ == "__main__":
    main()
```

输入 `7 8` 与 `-5 12`（两个分数的分子分母），输出：

```
7/8 -5/12
11/24 31/24 -35/96 -21/10
Congratulations! All tests are passed!
```

> **【核心】** 这个例子把本章几乎所有知识点都用上了：封装（私有属性）、只读 property、静态方法（约分）、运算符重载、`__str__` 与 `__repr__`、`__getitem__`、异常处理。看懂它就是最好的复习。
>
> **【注意】** `reduce()` 里用 `for` 从 `min(分子, 分母)` 往下试最大公约数，写法直观但效率低；实际工程里应该用第 3 章的**辗转相除法**（`math.gcd()` 也行）更快。这里保留讲义原写法，便于和作业题 16 对照。

示例 8-5：时间类 MyTime

要求：以 00:00:00 为原点，只保存一个私有字段 `__seconds`；支持 (时, 分, 秒) 与 `seconds=` 两种构造方式；不可变，提供只读属性；支持下标取值；支持 `str()` 输出 `hh:mm:ss`；支持比较大小；支持两个实例相减得到秒数差；支持加上一个整数秒数；要有错误异常处理。

```
class MyTime:
    @staticmethod
    def time_to_seconds(hour, minute, second):
        if not 0 <= hour < 24:
            raise ValueError("Hour value is out of range")
        elif not 0 <= minute < 60:
            raise ValueError("Minute value is out of range")
```

```
        elif not 0 <= second < 60:
            raise ValueError("Second value is out of range")
        else:
            return hour * 60 * 60 + minute * 60 + second

    @staticmethod
    def seconds_to_time(seconds):
        if not 0 <= seconds < 60 * 60 * 24:
            raise ValueError("Seconds value is out of range")
        hour = seconds // (60 * 60)
        minute = seconds % (60 * 60) // 60
        second = seconds % 60
        return hour, minute, second

    def __init__(self, hour=0, minute=0, second=0, seconds=None):
        if seconds is not None:
            self.__seconds = seconds
        else:
            self.__seconds = self.time_to_seconds(hour, minute, second)

    def __str__(self):
        time = self.seconds_to_time(self.__seconds)
        return "{:0>2d}:{:0>2d}:{:0>2d}".format(time[0], time[1], time[2])

    @property
    def seconds(self):
        return self.__seconds

    def __eq__(self, other):
        return self.__seconds == other.__seconds

    def __ne__(self, other):
        return not self.__eq__(other)

    def __lt__(self, other):
        return self.__seconds < other.__seconds

    def __add__(self, seconds):
        return MyTime(seconds=self.__seconds + seconds)

    def __sub__(self, other):
        return self.__seconds - other.__seconds

    def __getitem__(self, index):
        if index > 2:
            raise IndexError("Index out of range. 0..2 for hour, minute, second")
        return self.seconds_to_time(self.__seconds)[index]

def main():
    t0 = MyTime()
    print(t0)                    # 00:00:00
```

```
    t1 = t0 + 3600
    t2 = t1 + 60 * 15
    print(t1, t2)                # 01:00:00 01:15:00
    assert t2 == MyTime(1, 15, 0)
    print("秒数差：", t2 - t0)     # 4500
    print("时 分 秒：", t2[0], t2[1], t2[2])   # 1 15 0
    print("Congratulations! All tests are passed!")

if __name__ == "__main__":
    main()
```

输入 `0 0 0`（起始时刻），输出：

```
00:00:00
01:00:00 01:15:00
秒数差： 4500
时 分 秒： 1 15 0
Congratulations! All tests are passed!
```

> **【了解】** 注意 `MyTime() + (-1)` 与 `MyTime() - 1` 的区别：前者是“加一个整数秒”，后者是“两个实例相减”。所以要减 1 秒必须写成 `MyTime() + (-1)`。这也是运算符重载的一个固有麻烦——`__sub__` 只能有一种含义。

8.8 本章方法总结与易错清单

一、本章知识地图

- **8.1 类与对象**（最底层）
  - 类 = 设计图，对象 = 实例，属性 = 状态，方法 = 行为
  - ↓ 类要能造出对象，靠的就是
- **8.2 `__init__` 与 `self`**（全章的地基，考试必考）
  - `__init__` 自动调用、每个实例各一次、不能有返回值
  - `self` = 点号前面那个对象，Python 自动当第一个实参传进去
  - ↓ 属性挂好之后，就要管“能不能随便改”
- **8.3 封装：私有属性与 `@property`**
  - `__x` 名称改写，`_x` 约定内部，`__x__` 系统保留
  - property 三用途：只读、写入校验、计算属性
  - ↓ 有些数据不该每个对象各存一份
- **8.4 类属性、类方法、静态方法**
  - 类属性全类共享，改它必须写 `类名.属性`；可变类属性是共享陷阱
  - `@classmethod` 收 `cls`，`@staticmethod` 什么都不收
  - ↓ 让对象能参与 `+`、`==`、`print`、`sorted` 这些原生写法
- **8.5 内置方法与运算符重载**

- `__str__` 给人看，`__repr__` 给 Python 看
- `__eq__` 要配 `__ne__`；重载 `__eq__` 会让 `__hash__` 失效
- ↓让两个类之间能共享代码

- **8.6 继承与方法重写**
  - 子类重写 `__init__` 必须调 `super().__init__(...)`
  - 多态：同一个 `print(shape)`，行为由运行时类型决定
  - `isinstance` 认继承，`type` 不认
  - ↓全部串起来
- **8.7 三个综合示例**：`MyFraction`（封装 + 重载）、`MyTime`（不可变 + 重载）、`Shape` 家族（继承 + 多态）

考试最常考的 5 个点：① `self` 的含义与“自动传入”；② 类属性 vs 实例属性，以及 `self.count += 1` 为什么错；③ `@property` 的写法与三个用途；④ 运算符重载（尤其 `__str__` 和 `__eq__`/`__ne__`）；⑤ 继承中 `super().__init__()` 的作用。简答题若出现在这五处，基本都是原题。

二、方法总结（速查表）

方法/语法	作用	是否原地修改	返回值	典型例子
`class 类名:`	定义类	—	类对象	`class Circle:`
`def __init__(self, ...)`	初始化实例属性，创建对象时自动调用	—	`None`（不能 return 值）	`def __init__(self, r=1):`
`对象 = 类名(参数)`	实例化	—	新实例	`c = Circle(5)`
`self.属性 = 值`	新建/修改**实例**属性	是	`None`	`self.radius = 5`
`类名.属性 = 值`	修改**类**属性	是	`None`	`Circle.count = 0`
`对象.__dict__`	查看实例身上挂了哪些属性	否	`dict`	`c.__dict__`
`__x` / `_x`	私有属性 / 受保护属性	—	—	`self.__radius = r`
`@property`	方法变属性（可只读）	—	属性值	`c.radius`
`@属性名.setter`	为属性提供带校验的赋值	—	`None`	`c.radius = 10`
`@classmethod`	类方法，第一个参数 `cls`	—	任意	`Circle.get_count()`
`@staticmethod`	静态方法，没有特殊参数	—	任意	`Circle.to_jiao(10)`
`__str__(self)`	给用户看的字符串	否	`str`（**必须是字符串**）	`print(c)`
`__repr__(self)`	给调试看的字符串	否	`str`	`repr(c)`、列表里的元素
`__eq__` / `__ne__`	`==` / `!=`	否	`bool`	`f1 == f2`

方法/语法	作用	是否原地修改	返回值	典型例子
`__lt__` `__le__` `__gt__` `__ge__`	`<` `<=` `>` `>=`	否	`bool`	`sorted(列表)` 只用 `__lt__`
`__add__` `__sub__` `__mul__` `__truediv__`	`+ - * /`	否	新对象	`f1 + f2`
`__getitem__(self, i)`	`对象[i]`	否	元素	`f[0]`、`f[1]`
`__len__(self)`	`len(对象)`	否	非负整数	`len(lst)`
`__del__(self)`	对象销毁时自动执行	—	`None`	一般不自己写
`super().__init__(...)`	调用父类构造方法	—	`None`	`super().__init__(name, age)`
`isinstance(obj, 类或元组)`	判断类型（**认继承**）	否	`bool`	`isinstance(x, (int, float))`
`type(obj)`	取对象的**确切**类（不认继承）	否	类	`type(x) == Circle`
`类名.__mro__`	查看方法查找顺序	否	元组	`Square.__mro__`

三、易错清单

1. **`__init__` 写成 `_init_` 或 `__init_`** → 下划线个数不对，Python 认不出这是初始化方法，属性全没挂上 → 记住“**前后各两个下划线**”，一共 4 个；报错通常是 `TypeError: Circle() takes no arguments`。
2. **定义方法忘写 `self`** → Python 调 `c.area()` 时会自动传一个实例进去，形参对不上 → 定义时第一个参数永远写 `self`；报错是 `TypeError: Circle.area() takes 0 positional arguments but 1 was given`。
3. **实例化时多写了 `self`** → 把 `self` 当普通参数传了 → 写 `Circle(5)`，不要写 `Circle(self, 5)`。
4. **方法里访问实例属性不写 `self.`** → 直接写了 `radius` → 一律写 `self.radius`；否则报 `NameError: name 'radius' is not defined`。
5. **方法里调用同类其它方法不写 `self.`** → 直接写方法名 → 写 `self.get_perimeter()`。
6. **在方法里用 `self.属性 += 1` 改类属性** → 赋值动作在实例身上新建了同名属性，把类属性挡在后面 → 改类属性一律写 `类名.属性名`（如 `Circle.count += 1`）。
7. **可变类属性（列表/字典）被所有实例共享** → 类属性只创建一次，`self.列表.append()` 改的是同一份数据 → 每个实例各要一份的数据，必须在 `__init__` 里用 `self.列表 = []` 创建。
8. **`__str__` 返回非字符串** → 返回了数字或元组 → 用 f-string 拼成字符串；报错 `TypeError: __str__ returned non-string (type tuple)`。
9. **以为 `print` 和 `str()` 会调用 `__repr__`** → 它俩优先用 `__str__` → 记住 `__str__` 给人看、`__repr__` 给 Python 看；列表、字典里的元素只认 `__repr__`。
10. **重载了 `__eq__` 却忘了 `__ne__`** → Python 不会从 `__eq__` 自动推导 `__ne__` → 顺手写一个 `def __ne__(self, other): return not self.__eq__(other)`。

11. **重载 `__eq__` 后对象不能放进 set 或当字典键** → `__hash__` 被自动置为 `None` → 需要的话补上 `__hash__`，让它由同一批属性算出来。
12. **子类重写 `__init__` 忘记调 `super().__init__(...)`** → 父类属性没建，继承来的方法一用就崩 → `super().__init__(...)` 写在子类 `__init__` 第一行；报错 `AttributeError: 'Student' object has no attribute 'name'`。
13. **用 `type()` 判断类型，子类实例被判成“不是”** → `type` 只看确切类，不看继承链 → 判断类型统一用 `isinstance(obj, 类名)`，多个类型用元组 `isinstance(x, (int, float))`。
14. **`@property` 没写 setter 却去赋值** → 属性只读 → 补上 `@属性名.setter`；报错 `AttributeError: property 'x' has no setter`。
15. **setter 与 getter 不同名，或装饰器写成 `@setter`** → 装饰器必须写 `@属性名.setter`，且两个函数同名 → 都叫 `radius`。
16. **`__init__` 里写 `return 值`** → 构造方法的返回值固定是 `None` → 直接删掉 return；报错 `TypeError: __init__() should return None`。
17. **`__init__` 第一个参数不是 `self`** → 比如写成 `def __init__(self, radius)` 还好，写成 `def __init__(radius)` 就全乱了 → 第一个参数永远是 `self`，真正的数据从第二个参数开始。

四、必须能默写的 6 个片段

1. **最小可用的类**（一切从它开始）：

```
class Circle:
    def __init__(self, radius=1):
        self.radius = radius

    def get_area(self):
        return 3.14159 * self.radius ** 2

c = Circle(5)
print(c.get_area())
```

1. **私有属性 + property 读写**（封装的标准模板）：

```
class Circle:
    def __init__(self, radius=1):
        self.radius = radius            # 走 setter

    @property
    def radius(self):
        return self.__radius

    @radius.setter
    def radius(self, value):
        if value < 0:
            raise ValueError("Negative radius")
        self.__radius = value
```

1. **类属性计数 + 类方法**（改类属性必须用类名）：

```
class Circle:
    count = 0
    def __init__(self, radius=1):
        self.radius = radius
        Circle.count += 1               # 不能用 self.count += 1

    @classmethod
    def get_count(cls):
        return cls.count
```

1. **一套运算符重载**（`__str__` `__repr__` `__eq__` `__ne__` `__lt__`）：

```
class Money:
    def __init__(self, jiao=0):
        self.__jiao = jiao

    def __str__(self):
        return f"CNY {self.__jiao / 10:.2f}"

    def __repr__(self):
        return f"Money({self.__jiao})"

    def __eq__(self, other):
        return self.__jiao == other.__jiao

    def __ne__(self, other):
        return not self.__eq__(other)

    def __lt__(self, other):
        return self.__jiao < other.__jiao
```

1. **继承 + `super()`**（子类构造的标准写法）：

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Student(Person):
    def __init__(self, name, age, school):
        super().__init__(name, age)     # 第一行，不能忘
        self.school = school
```

1. **类型判断**（判断类型一律用 isinstance）：

```
print(isinstance(5, int))                # True
print(isinstance(5, (int, float)))       # True
```

```
print(isinstance(True, int))              # True，bool 是 int 的子类
print(type(True) == int)                  # False，type 不认继承
```

8.9 本章自测（15 道）

第 8 章 自测题 1　类和对象是什么关系？请用“设计图”和“房子”来打比方说明。

第 8 章 自测题 2　`__init__` 方法的作用是什么？`self` 是什么？能不能把 `self` 改名叫 `this`？

第 8 章 自测题 3　为什么要把属性设成私有的？私有属性在类外部真的完全访问不到吗？

第 8 章 自测题 4　`@property` 装饰器解决了什么问题？它让外部代码的写法发生了什么变化？

第 8 章 自测题 5　类属性和实例属性有什么区别？用实例名修改类属性会发生什么？

第 8 章 自测题 6　`@classmethod` 和 `@staticmethod` 的区别是什么？各自的第一个参数有什么要求？

第 8 章 自测题 7　定义一个 `Book` 类，有书名、作者和价格三个属性，以及一个 `show()` 方法打印书本信息；再重载 `__str__` 使 `print(book)` 输出可读信息。

第 8 章 自测题 8　给第 7 题的 `Book` 类重载 `__eq__`，让“书名和作者都相同”的两本书视为同一本书。

第 8 章 自测题 9　写一个 `Student` 类继承 `Person` 类（`Person` 有 `name` 和 `age`），子类增加 `student_id` 属性，并在子类的 `__init__` 中用 `super()` 调用父类构造方法。

第 8 章 自测题 10　为什么重载了 `__eq__` 之后通常也要重载 `__ne__`？

第 8 章 自测题 11　`__str__` 与 `__repr__` 有什么区别？分别在什么情况下被调用？如果两个都不定义，`print(对象)` 会输出什么？

第 8 章 自测题 12　下面这段代码报错，请指出错在哪里并改正，并写出报错信息的原文。

```
class Circle:
    def _init_(self, radius=1):
        self.radius = radius

c = Circle(5)
print(c.radius)
```

第 8 章 自测题 13　`isinstance(obj, 类)` 与 `type(obj) == 类` 有什么区别？判断一个对象是不是数字，应该怎么写？

第 8 章 自测题 14　`@property` 有哪三个典型用途？请各写一小段代码说明，并指出“只读属性”被外部赋值时会报什么错。

第 8 章 自测题 15　下面这个类想让每个课程各有一份学生名单，但运行后发现两份名单混在一起了。请解释原因并改正。

```
class Course:
    students = []

    def __init__(self, name):
        self.name = name
```

```
    def add(self, student):
        self.students.append(student)

c1 = Course("Python")
c2 = Course("Java")
c1.add("Tom")
print(c1.students, c2.students)
```

第 9 章 模块与包

对应课程讲义：《10_模块和包》。这一章概念比语法多，考试爱出简答题。有四件事必须彻底搞清楚：**import 的五种写法**、`if __name__ == "__main__":` 到底解决了什么问题、模块搜索路径 `sys.path`、包与 `__init__.py` 的关系。

9.1 模块化编程

一个文件写到几百行还能忍，写到几千行就没法维护了。模块化编程的思路很朴素：**一个功能一个文件**，把大程序拆成一批小文件。

【核心】模块化编程的四个好处：① 简化编程，可以专注于正在处理的子任务；② 可维护性好，问题被限制在子任务中，便于排查；③ 复用性好，可以直接调用写好的模块；④ 范围性好，每个模块都有独立的命名空间，避免函数名和变量名冲突。

"把代码装进一个名字里"这件事你在第 3 章已经做过一次了，那就是函数。本章的模块是把**一批函数**装进一个文件，包是把**一批模块**装进一个目录，三级台阶是同一种思路：

层级	组织单位	解决什么问题	详见
函数	一个名字	一段代码被反复复制粘贴	3.1 节
模块	一个 `.py` 文件	函数名、变量名互相冲突	9.2 节
包	一个目录	模块名互相冲突	9.3 节

命名空间

命名空间就是"名字生活的空间"：同一个名字放在不同的空间里，互不干扰。

命名空间	范围与生命周期
局部命名空间	为每个函数创建，函数执行完毕就失效
全局命名空间	为所有模块创建，程序执行完毕失效
内置命名空间	默认的一些命名，退出程序失效

【注意】 第二行藏着一个关键信息：全局命名空间是"为每个**模块**创建"的，不是为整个程序创建。也就是说 `a.py` 里的 `x` 和 `b.py` 里的 `x` 是两个互不相干的变量，这正是"范围性好"的由来。函数内部查找变量的顺序（局部 → 闭包 → 全局 → 内置）详见 3.5 节。

9.2 模块与导入

模块就是一个扩展名为 `.py` 的 Python 源代码文件。按来源分三类：

类型	说明	例子
内置模块	直接导入即可	math、random、os
第三方模块	需要先安装再导入	numpy、matplotlib
自定义模块	需要自己写一个 .py 文件	自己写的 MyDateUtils.py

导入模块的五种写法（核心）

#	写法	调用时怎么写	什么时候用
①	`import math`	`math.sqrt(9)`	默认选择，最不容易出问题
②	`from math import sqrt`	`sqrt(9)`	只用一两个名字，代码更短
③	`from math import sqrt, pi`	`sqrt(9)` 、 `pi`	同上，一次导入多个名字
④	`from math import *`	直接用模块里所有名字	**不推荐**
⑤	`import time as tt` / `from time import sleep as sl`	`tt.sleep(2)` / `sl(2)`	名字太长，或与已有名字冲突

```
import math                     # 写法①：整个模块导入
print(math.sqrt(9))             # 3.0，要用"模块名.函数名()"调用

from math import sqrt           # 写法②：只导入某个函数
print(sqrt(9))                  # 3.0，直接写函数名

from math import sqrt, pi       # 写法③：一次导入多个函数

from math import *              # 写法④：导入所有函数（不推荐）

import time as tt               # 写法⑤：给模块起别名
tt.sleep(1)

from time import sleep as sl    # 写法⑤的另一种：给函数起别名
sl(1)
```

【核心】优先用写法① `import 模块名` 。三个理由：① 一眼能看出 `sqrt` 是从哪个模块来的；② 不会和你自己的变量、函数重名；③ 别人读代码时不用回头翻 import 语句。只有在某个名字用得特别频繁、写模块名实在太啰嗦时，才用②或⑤。

【核心】 `import` 一行只写一个模块。要导入两个模块就写两行： `import os` 和 `import sys` ，不要写成 `import os, sys` 。

【核心】不要使用 `from 模块 import *` 。它会把模块里所有不以 `_` 开头的名字都倒进当前命名空间，造成"命名空间污染"。更麻烦的是：名字一旦撞车，你调用到的是**后导入**的那个。网上流传过一个恶作剧模块，里面写着 `def print(*args): exit(1)` ，谁星号导入了它，程序就再也不输出了。

【常见错误】 星号导入会把你自己的函数悄悄换掉，而且**不报错**。

```
# ✗ 错误写法：先定义自己的 sqrt，再星号导入
def sqrt(x):
```

```
    return int(x ** 0.5)

from math import *        # math 里也有 sqrt，把上面那个覆盖了

print(sqrt(9))            # 期望 3，实际输出 3.0
```

```
# ✓ 正确写法
import math

def sqrt(x):
    return int(x ** 0.5)

print(sqrt(9))            # 3，自己的函数好好的
print(math.sqrt(9))       # 3.0，标准库的也能用
```

星号导入之后，`sqrt` 这个名字被指向了 math 里的版本，你自己写的那个再也找不回来。这种错误最难查，因为程序不报错，只是结果悄悄变了。教训：**要什么就点名要什么，不要"全都拿来"。**

【常见错误】 两种写法混着用，`math` 这个名字根本不存在。

```
# ✗ 错误写法
from math import sqrt

print(math.sqrt(9))
```

```
# ✓ 正确写法
import math

print(math.sqrt(9))       # 3.0
```

报错 `NameError: name 'math' is not defined`。`from math import sqrt` 只把 `sqrt` 这一个名字拿了进来，并没有引入 `math` 这个名字。用哪种写法，就用哪种调用方式，别混。

创建模块与使用模块

自己写一个 `mod.py`：

```
s = "If Comrade Napoleon says it, it must be right."
a = [100, 200, 300]

def foo(arg):
    print(f'arg = {arg}')

class Foo:
    pass
```

在别的文件中使用它：

```
import mod

print(mod.s)              # If Comrade Napoleon says it, it must be right.
print(mod.a)              # [100, 200, 300]
mod.foo(['quux', 'corge', 'grault'])
x = mod.Foo()
```

【核心】 `import mod` 这个动作 = **把 `mod.py` 从头到尾执行一遍**，然后把这个文件里定义的名字挂在 `mod` 这个名字下面。这是本章最值钱的一句话——9.2 节最后的 `if __name__ == "__main__":` 就是为了对付它。

【注意】 模块文件名用小写字母加下划线，比如 `my_date_utils.py`，不要写 `MyDateUtils.py` 这种驼峰名；更不要和标准库重名（原因见 9.4 节）。

例题 9-1：把函数封装成模块

题目：写一个模块 `myutils.py`，提供 `is_leap(year)`（判断闰年）和 `days_in_month(year, month)`（返回某月天数）两个函数；再写主程序 `main.py`，读入"年份 月份"，输出这个月有多少天以及这一年是否闰年。

```
# myutils.py
def is_leap(year):
    return year % 400 == 0 or (year % 100 != 0 and year % 4 == 0)

def days_in_month(year, month):
    days = (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)
    if month == 2 and is_leap(year):
        return 29
    return days[month - 1]
```

```
# main.py，与 myutils.py 放在同一个文件夹里
import myutils

year, month = map(int, input().split())
print(myutils.days_in_month(year, month))
print(myutils.is_leap(year))
```

输入 `2024 2`，输出：

```
29
True
```

要点：① 两个文件必须放在同一个文件夹里，否则 `import myutils` 找不到（下一小节讲原因）；② 调用时一律写 `myutils.函数名`，一眼就能看出这个函数不是本文件定义的；③ 在 `myutils.py` **内部**调用 `is_leap` 时不用写模块名，直接写 `is_leap(year)` 就行——同一个模块里的名字互相直接可见。

模块搜索路径（sys.path）

执行 `import mod` 时，解释器按下面的顺序去找 `mod.py`：

1. **当前执行的脚本所在的文件夹**（搜索的第一站，优先级最高）；
2. 环境变量 `PYTHONPATH` 里列出的文件夹；
3. Python 安装目录下的标准库文件夹；
4. 第三方库的安装位置 `site-packages`（`pip` 装的东西都在这里）。

```
import sys

print(sys.path)                          # 一个列表，按顺序打印上面这些位置
sys.path.append(r'C:\Users\john')        # 加到最后（优先级最低）
sys.path.insert(0, r'C:\Users\john')     # 加到最前（优先级最高）
import mod
```

> **【核心】** `import` 出问题时，第一件事就是 `print(sys.path)`，看模块文件到底在不在列表里的某个目录中。这一招能解决一大半"在我电脑上能跑"的问题。
>
> **【易错】** 搜索路径是**有顺序**的，靠前的目录先被找到。所以你在脚本旁边放一个 `random.py`，它就会盖住标准库的 `random`（见 9.4 节的常见错误）。
>
> **【核心】** 同一个模块在一次程序运行中**只会被执行一次**：第二次 `import` 直接用第一次的结果。所以模块里的顶层 `print` 只会出现一遍，模块里的初始化代码也不会重复跑。
>
> **【了解】** 模块第一次被导入后，Python 会把编译好的字节码存进同目录的 `__pycache__` 文件夹，下次导入更快。这是自动生成的，不用管它，也不用提交到作业里。

当作脚本执行：if __name__ == "__main__"

先看一个真实会遇到的场景。你写了个 `myutils.py`，写完顺手在末尾加了一句测试：

```
# myutils.py
def is_leap(year):
    return year % 400 == 0 or (year % 100 != 0 and year % 4 == 0)

print(is_leap(2024))     # 随手写的一句测试
```

第二天你在 `main.py` 里 `import myutils`，程序一运行，屏幕上先冒出一个 `True`——你根本没让它打印。原因就是上一小节那句话：`import` **会把被导入的文件从头执行一遍**，那行 `print` 自然也跑了。

`if __name__ == "__main__":` 解决的就是这个问题。每个模块都自带一个变量 `__name__`，它的值取决于这个文件**是怎么被使用的**：

运行方式	`__name__` 的值	例子
直接运行 `python mod.py`	`"__main__"`	固定就是这六个字符
被导入 `import mod`	`"mod"`	模块名

运行方式	`__name__` 的值	例子
被导入 `import pkg.a`	`"pkg.a"`	带包名的完整模块名

于是只要判断一下 `__name__` 是不是 `"__main__"`，就能知道"我是在自己跑，还是被别人导入了"。**被导入 vs 被直接运行**的完整对照：

对比项	被直接运行 `python mod.py`	被导入 `import mod`
`__name__` 的值	`"__main__"`	`"mod"`
文件顶层的代码	执行	**也执行**（这一点最容易忘）
`if __name__ == "__main__":` 里的代码	执行	**不执行**
典型用途	自己调试、提交 OJ	被别的程序复用

下面这个对照实验建议你亲手跑一遍（两个文件放同一个文件夹）：

```
# mod.py
print(f"加载 mod.py，此时 __name__ = {__name__}")

def foo(arg):
    print(f"arg = {arg}")

if __name__ == "__main__":
    print("只有直接运行才会看到这一行")
    foo("hello")
```

```
# main.py
import mod

print("主程序结束")
```

运行 `python mod.py`，输出：

```
加载 mod.py，此时 __name__ = __main__
只有直接运行才会看到这一行
arg = hello
```

运行 `python main.py`，输出：

```
加载 mod.py，此时 __name__ = mod
主程序结束
```

对比两次输出："加载 mod.py" 那行**两次都出现**（顶层代码总是执行），而 `if` 里面那两行**只在直接运行时出现**。

实际写代码时，推荐下面这个固定模板：把主流程收进 `main()` 函数，末尾统一保护。

```
def main():
    n = int(input())
```

```
    print(n * 2)

if __name__ == "__main__":
    main()
```

【核心】 三句话记住这一节：① `import 模块` = 把这个文件从头执行一遍；② 直接运行时 `__name__` 是 `"__main__"`，被导入时是模块名；③ 只有自己运行时才该做的事（读输入、打印测试、调用 `main()`）全部放进 `if __name__ == "__main__":` 里面。

【注意】 OJ 判题系统是直接运行你提交的文件，所以 `if __name__ == "__main__":` 不写也能过。但写上它是好习惯：老师导入你的模块检查时不会乱打印，同一个文件也能被别的程序复用。OJ 的输入输出模板见附录 C。

【常见错误】 模块顶层直接读输入，别人一 `import` 就卡住。

```
# ✗ 错误写法：myutils.py
def is_leap(year):
    return year % 400 == 0 or (year % 100 != 0 and year % 4 == 0)

n = int(input())          # 顶层代码，被导入时也会执行
print(is_leap(n))
```

```
# ✓ 正确写法：myutils.py
def is_leap(year):
    return year % 400 == 0 or (year % 100 != 0 and year % 4 == 0)

if __name__ == "__main__":
    n = int(input())
    print(is_leap(n))
```

✗ 的写法在 `import myutils` 时就会执行 `input()`，程序莫名其妙停下来等你输入；在 OJ 或自动化脚本里被导入时更直接——`EOFError: EOF when reading a line`。

循环导入：两个模块互相 import

两个模块互相引用，就叫**循环导入**。

【常见错误】 两个文件互相 `from ... import 名字`，谁先运行谁报错。

```
# ✗ 错误写法
# a.py
from b import fb

def fa():
    print("fa")

print(fb())

# b.py
```

```
from a import fa

def fb():
    print("fb")
```

```
# ✓ 正确写法：只导入模块，不做 from ... import 名字；或者把 import 挪进函数
# a.py
import b

def fa():
    print("fa")

def run():
    b.fb()          # 调用时才去取 b 里的东西
```

报错信息是 `ImportError: cannot import name 'fb' from partially initialized module 'b' (most likely due to a circular import)`。原因：`a.py` 执行到第 1 行就去 b 里拿 `fb`，而此时 `b.py` 才刚开始执行，`fb` 还没定义。三种解法：① 两边都只写 `import 模块名`，用的时候再 `b.fb()`；② 把两个模块都要用的部分抽到第三个模块 `common.py`，两边都只依赖它；③ 把 `import` 挪进函数内部，用到时再导入。

9.3 包 package

模块能避免变量、函数、类名重名，但**模块名重复**怎么办？两个同学都写了 `utils.py`，复制到一个项目里就撞车了。为了解决模块名冲突，Python 引入按目录组织模块的方法，称为**包（Package）**。

> **【核心】简单来说，包就是多个模块的集合，也就是一个目录（文件夹）。** 用"包名.模块名"就能访问到里面的模块。

包的初始化工作：__init__.py

包的定义很简单：**当前目录下有 `__init__.py` 文件的目录即为一个包。**

```
pkg
  |-- a.py
  |-- b.py
  `-- __init__.py
```

> **【核心】** 不管 `__init__.py` 是空的还是有内容的，这个目录都会被认为是一个包，它是一个**标识**。需要初始化配置时就往里面写指令。一个包被导入时，`__init__.py` 的代码**只会被执行一次**。
>
> **【了解】** 既然 `__init__.py` 里的可执行代码会在导入时执行，就要小心别在里面放有副作用的代码（打印、读文件、连网络），尽可能把它封装成函数或类。
>
> **【常见错误】** 包的目录里少了 `__init__.py`，在别人的电脑上就"包不见了"。

```
# ✗ 错误写法：mypkg 目录里只有 a.py，没有 __init__.py
mypkg/
```

```
  `-- a.py
```

```
# ✓ 正确写法
mypkg/
  |-- __init__.py      # 空的也行，但必须有
  `-- a.py
```

Python 3.3 以后，没有 `__init__.py` 的目录会被当成"命名空间包"，**多数情况下竟然也能导入成功**，所以很多同学一直不写它。但这属于"看起来没事"：换到老版本 Python 或打包工具上会报 `ModuleNotFoundError: No module named 'mypkg'`；在包目录里直接运行某个模块时，相对导入会报 `ImportError: attempted relative import with no known parent package`；有些 IDE 也不会把该目录识别成包。**新建包的第一件事就是补一个空的 `__init__.py`，别依赖那个特性。**

导入顺序

```
from package import something
```

`import` 语句会按下面的顺序查找，全都找不到才抛出 `ImportError`：

1. `__init__.py` 文件内的变量；
2. 是不是 package 内的 subpackage（子包）；
3. 是不是 package 内的 module（模块）。

一个简单的例子

```
# pkg/a.py
def bar():
    print("Hello, function 'bar' from module 'a' calling")

# pkg/b.py
def foo():
    print("Hello, function 'foo' from module 'b' calling")

# pkg/__init__.py 是空文件
```

```
>>> import pkg
>>> pkg
<module 'pkg' from 'D:\\pycode\\pkg\\__init__.py'>
>>> pkg.a
Traceback (most recent call last):
    ...
AttributeError: module 'pkg' has no attribute 'a'
```

【核心】 `import pkg` 只导入了包本身（也就是执行了 `__init__.py`），包里的 `a.py`、`b.py` **不会**自动被导入。想让它们自动加载，有两种办法：

```
# 办法 1：在 __init__.py 里导入
# __init__.py
import pkg.a
import pkg.b
# 之后就可以用 pkg.a.bar()

# 办法 2：使用时手动导入
import pkg.a as a
a.bar()              # Hello, function 'bar' from module 'a' calling
```

例题 9-2：把模块放进包里

题目：建一个包 `textpkg`，里面放一个模块 `wordstat.py`，提供 `count_words(s)`（单词个数）和 `longest_word(s)`（最长的单词）；主程序读入一行英文，输出单词个数和最长单词。

```
9-2/
  |-- main.py
  `-- textpkg/
        |-- __init__.py      # 空文件，只起标识作用
        `-- wordstat.py
```

```
# textpkg/wordstat.py
def count_words(s):
    return len(s.split())

def longest_word(s):
    return max(s.split(), key=len)
```

```
# main.py，与 textpkg 文件夹同级
from textpkg import wordstat

s = input()
print(wordstat.count_words(s))
print(wordstat.longest_word(s))
```

输入 `the quick brown fox jumps over the lazy dog`，输出：

```
9
quick
```

要点：① `textpkg` 文件夹必须和 `main.py` 同级，因为"脚本所在目录"是搜索的第一站；② `from textpkg import wordstat` 导入的是包里的**模块**，调用时写 `wordstat.count_words(s)`；也可以写 `from textpkg.wordstat import count_words`，之后直接调 `count_words(s)`；③ 有几个单词一样长时，`max` 返回**最先出现**的那个，所以这里输出 `quick` 而不是 `brown`。

相对导入

与命令行里的路径类似，`.` 代表当前包，`..` 代表上级包：

```
# sound/__init__.py
from . import effects            # 导入当前包下的 effects

# sound/effects/__init__.py
from .. import formats           # 导入上级包（sound）下的 formats
from ..filters import karaoke    # 从上级包的 filters 模块里取 karaoke
```

> **【注意】** 相对导入只能在**包内部被导入时**使用。直接运行包内文件（`python sound/effects/__init__.py`）会报 `ImportError: attempted relative import with no known parent package`。自己写的小程序用绝对导入（`import pkg.a`）更不容易出错，相对导入多见于较大的项目。

`__all__` 魔法变量

> **【核心】小结**：在 `from package import *` 语句中，如果 `__init__.py` 里定义了 `__all__` 魔法变量，那么 `__all__` 里的所有元素都会被作为模块自动导入（如果其中某个模块不存在，`ImportError` 依然会出现）。

```
# sound/__init__.py
__all__ = ["formats", "filters", "effects"]
```

如果 `__init__.py` 里没有 `__all__`，导出按下面的规则执行：

1. 此 package 被导入，并且执行 `__init__.py` 中可被执行的代码；
2. `__init__.py` 中定义的变量被导入；
3. `__init__.py` 中被显式导入的 module 被导入。

（模块里也可以定义 `__all__`，作用相同：只在 `from 模块 import *` 时生效，用来圈定"允许被星号导出的名字"。）

> **【了解】** `dir()` 函数返回一个排好序的字符串列表，内容是一个模块里定义过的名字（模块、变量和函数）。例如 `dir(math)` 能列出 math 里所有可用的名字，配合 `help(math)` 查文档很方便。

9.4 常用标准库与第三方库

Python 的模块和包分为**标准库**和**第三方库**：标准库随 Python 一起安装，直接 `import` 就能用；第三方库要用 `pip` 安装后才能用（见 9.6 节自测题 10）。

常用标准库（速查）

模块	一句话说明	详见
`math`	三角函数、对数、指数、常数 pi / e	6.1 节

模块	一句话说明	详见
`random`	随机整数、随机浮点数、随机取元素、洗牌	6.2 节
`time`	时间戳、格式化时间、计时、`sleep` 延时	6.3 节
`datetime`	日期时间对象、用 `timedelta` 算时间差	6.4 节
`os` / `os.path`	与操作系统打交道：路径、目录、环境变量	5.7 节
`sys`	解释器相关：`sys.path`、`stdin/stdout`、`argv`、`exit`	9.2 节
`csv`	读写 CSV 表格文件	5.6 节
`re`	正则表达式：查找、替换、切分文本	7.11 节
`decimal`	精确小数（算钱、算成绩用）	附录 A.9
`json`	字典、列表与 JSON 文本互转	下面
`collections`	`Counter` 计数、`defaultdict`、`deque`	下面
`itertools`	`combinations` 组合、`permutations` 排列、`chain` 串联	下面
`pathlib`	用 `Path` 对象操作路径，比手拼字符串省心	下面
`urllib`	访问网页、处理 URL	下面

【速查】 这张表不用背，用到时回来查。要形成条件反射的是：**数理计算想 `math`，随机想 `random`，时间想 `time` / `datetime`，文件和目录想 `os` / `pathlib`，文本匹配想 `re`，计数想 `collections`。**

```
import json

data = {"name": "Tom", "scores": [90, 85]}
s = json.dumps(data, ensure_ascii=False)     # 字典 -> JSON 字符串
print(s)                                     # {"name": "Tom", "scores": [90, 85]}
print(json.loads(s)["name"])                 # JSON 字符串 -> 字典，输出 Tom

from collections import Counter

print(Counter("hello"))                      # Counter({'l': 2, 'h': 1, 'e': 1, 'o': 1})
print(Counter("hello").most_common(1))       # [('l', 2)]，出现次数最多的一个

from itertools import combinations

print(list(combinations([1, 2, 3], 2)))      # [(1, 2), (1, 3), (2, 3)]

from pathlib import Path

p = Path("data") / "scores.txt"              # 拼路径用 / ，跨平台，不用手写反斜杠
print(p.name)                                # scores.txt
```

【了解】 `json` 是程序之间交换数据的通用格式：`dumps` 把字典变成字符串，`loads` 把字符串变回字典，配合 5.4 节的写文件就能把数据存下来下次再用（还有直接读写文件的 `dump` / `load`）。

`collections.Counter` 是"数数"神器，比手写字典计数少三行代码。`itertools` 负责排列组合，做"全排列"类 OJ 题能省一半代码。`pathlib.Path` 是新的路径写法，`Path("data") / "a.txt"` 比 `os.path.join` 更直观。`urllib` 属于【了解】级别，知道"标准库也能访问网页"即可，真正写爬虫一般用第三方库 requests。

pip 与虚拟环境

【了解】 第三方库先装后导：在命令行（不是 Python 里）敲 `pip install requests`；下载慢就加国内镜像 `-i https://pypi.tuna.tsinghua.edu.cn/simple`；`pip list` 看装了哪些包，`pip uninstall 包名` 卸载。**虚拟环境**是给每个项目单独准备一套库的文件夹：`python -m venv venv` 创建，激活后（Windows 是 `venv\Scripts\activate`）再 `pip install`，装的东西只属于这个项目，不会和别的项目打架；PyCharm 新建项目时勾选 "New environment" 就是在建虚拟环境。零基础阶段把 `pip install` 用熟即可，虚拟环境知道有这回事就行。

【核心】一句话总结本章：模块让"一个功能"可以复用，包让"很多模块"可以组织起来、避免重名。真正写项目时，你写的大部分代码最终都会变成模块和包。

例题 9-3：词频统计（collections + json）

题目：输入一行英文，统计每个单词出现的次数，输出出现次数最多的单词及次数，并把前 3 名转成 JSON 字符串打印出来。

```
import json
from collections import Counter

words = input().split()
counter = Counter(words)

top = counter.most_common(1)[0]
print(top[0], top[1])
print(json.dumps(counter.most_common(3), ensure_ascii=False))
```

输入 `the quick brown fox jumps over the lazy dog the fox`，输出：

```
the 3
[["the", 3], ["fox", 2], ["quick", 1]]
```

要点：① `Counter` 可以直接对列表计数，`most_common(n)` 按次数从多到少给出前 n 名，并列时先出现的排前面（所以 `quick` 排在 `brown` 前）；② `json.dumps` 的 `ensure_ascii=False` 能让中文原样输出，不加这个参数中文会变成 `\u4e2d\u6587` 这样的转义；③ 想把结果存成文件，用 5.4 节的写文件把字符串写进去即可。

import 失败怎么办：ModuleNotFoundError 的两个典型原因

`ModuleNotFoundError: No module named 'xxx'` 是最常见的导入报错。它说明"Python 在搜索路径里没找到这个模块"，原因基本只有两个：

报错里的模块名	原因	解决办法
`requests`、`numpy` 这类你没写过、也没装过的名字	第三方包没安装	命令行执行 `pip install requests`
`random`、`json`、`turtle` 这类标准库名字，而你自己刚好有个同名文件	你的文件名把标准库顶掉了	把自己的文件改名

排查三步：① 看模块名是不是第三方库，用 `pip list` 查装没装；② `print(sys.path)`，看目标文件在不在这些目录里；③ 在命令行单独敲 `python -c "import xxx"`，排除其它代码的干扰。

> **【常见错误】** 自己的文件名和标准库重名，`import` 进来的是你自己。
>
> ```
> # ✗ 错误写法：练习文件存成了 random.py
> # random.py
> import random            # 导入的其实是自己
>
> print(random.randint(1, 6))
> ```
>
> ```
> # ✓ 正确写法：换一个不会撞车的名字
> # my_random.py
> import random
>
> print(random.randint(1, 6))     # 6（每次不同）
> ```
>
> 报错 `AttributeError: partially initialized module 'random' has no attribute 'randint' (most likely due to a circular import)`；如果这个文件是被别的程序导入的，也可能直接报 `ModuleNotFoundError: No module named 'random'`。原因：**脚本所在目录排在标准库前面**（9.2 节搜索路径的第 1 站），你自己的 `random.py` 先把标准库的 `random` 顶掉了。`random.py`、`string.py`、`turtle.py`、`json.py`、`math.py`、`time.py` 这些名字，一律不要用。

9.5 本章方法总结与易错清单

一、本章知识地图

- **9.1 模块化编程**（思想层）
  - 三级台阶：函数（一段代码）→ 模块（一批函数）→ 包（一批模块）
  - 命名空间三级：局部 / 全局（**每个模块一个**）/ 内置
  - ↓ 落到具体文件上，就是"怎么导入"
- **9.2 模块与导入**（全章核心，简答题高发区）
  - 模块 = 一个 `.py` 文件；分内置模块、第三方模块、自定义模块
  - 五种导入写法：`import m`、`from m import f`、`from m import f1, f2`、`from m import *`（不推荐）、别名写法 `import m as x` / `from m import f as g`
  - 一句话总纲：`import 模块` **= 把这个文件从头执行一遍**（同一个模块一次运行只执行一次）
  - 搜索路径 `sys.path` 四站：脚本目录 → PYTHONPATH → 标准库 → site-packages

- `if __name__ == "__main__":`：自己跑是 `"__main__"`，被导入是模块名
- 循环导入：互相 `from ... import 名字` 会报 partially initialized module
- ↓ 模块一多就会重名，于是有了包

- **9.3 包 package**（目录层）
  - 包 = 含 `__init__.py` 的目录；`__init__.py` 在包被导入时执行一次
  - `import pkg` 只导入包本身，子模块要显式导入，否则 `AttributeError: module 'pkg' has no attribute 'a'`
  - 相对导入 `from . import x` 只能在包内被导入时使用
  - `__all__` 控制 `from pkg import *` 能拿到谁；`dir()` 查看模块里的名字
  - ↓ 包和模块里到底装什么
- **9.4 常用标准库与第三方库**（工具箱）
  - 标准库直接 `import`：math / random / time / datetime / os / sys / csv / re / json / collections / itertools / pathlib
  - 第三方库先 `pip install`；虚拟环境隔离项目
  - `ModuleNotFoundError` 两个原因：包没装 / 文件名与标准库重名

考试最常考的 5 个点：① `if __name__ == "__main__":` 的作用；② `import math` 与 `from math import sqrt` 的区别；③ 为什么不推荐 `from math import *`；④ 包与 `__init__.py` 的关系；⑤ 模块搜索路径的顺序与 `ModuleNotFoundError` 的排查。简答题若出现在这五处，基本都是原题。

二、方法总结（速查表）

写法 / 对象	作用	返回值	典型例子
`import 模块名`	导入整个模块，用"模块名.名字"访问	模块对象	`import math` → `math.sqrt(9)`
`from 模块名 import 名字`	只把指定名字拿进当前命名空间	被导入的名字	`from math import sqrt` → `sqrt(9)`
`import 模块名 as 别名`	给模块起短名	模块对象	`import numpy as np`
`from 模块名 import 名字 as 别名`	给名字起别名	被导入的名字	`from time import sleep as sl`
`from 模块名 import *`	导入所有公开名字（**不推荐**）	—	`from math import *`
`from 包 import 模块`	从包里导入模块	模块对象	`from textpkg import wordstat`
`from . import 模块`	包内相对导入（**只能包内用**）	模块对象	`from . import effects`
`if __name__ == "__main__":`	区分"直接运行"和"被导入"	—	里面放测试代码与 `main()` 调用
`__name__`	当前模块的名字	`str`	直接运行是 `"__main__"`
`sys.path`	模块搜索路径列表	`list`	`print(sys.path)`

写法 / 对象	作用	返回值	典型例子
`sys.path.insert(0, 路径)`	把自己的目录插到搜索最前面	`None`（原地修改）	`sys.path.insert(0, r"C:\mycode")`
`dir(模块)`	看模块里有哪些名字	排好序的 `list`	`dir(math)`
`__all__`	圈定 `import *` 时导出哪些名字	—	`__all__ = ["a", "b"]`
`pip install 包名`	安装第三方库（命令行，不是 Python 语句）	—	`pip install requests`

`__name__` 的取值对照：

使用方式	`__name__` 的值	`if __name__ == "__main__":` 里的代码
`python mod.py`	`"__main__"`	执行
`import mod`	`"mod"`	不执行
`import pkg.a`	`"pkg.a"`	不执行

三、易错清单

1. **`import mymod` 后程序自己跑了一遍，甚至停在 `input()` 上** → 模块的顶层代码在导入时就会执行 → 把测试、打印、读输入全部放进 `if __name__ == "__main__":` 里，主流程收进 `main()`。
2. **`from math import sqrt` 之后写 `math.sqrt(9)`** → 没有引入 `math` 这个名字 → 报 `NameError: name 'math' is not defined`；要么写 `import math` 用 `math.sqrt`，要么写 `from math import sqrt` 用 `sqrt`，别混。
3. **自己的函数被 `from m import *` 悄悄覆盖** → 星号导入把所有公开名字倒进当前命名空间，后导入的赢 → 一律用 `import m`；确实只要一两个名字就用 `from m import 名字`，点名要。
4. **`ModuleNotFoundError: No module named 'random'`（标准库却找不到）** → 你自己有个 `random.py`，脚本目录排在标准库前面，把它顶掉了 → 给自己的文件改名，如 `my_random.py`；`string.py`、`turtle.py`、`json.py` 同理。
5. **`ModuleNotFoundError: No module named 'requests'`（第三方库）** → 包没装，或者装到了另一个 Python 环境里 → 命令行 `pip install requests`；用了虚拟环境要先激活再装。
6. **建好了包目录，`import mypkg` 却报 `ModuleNotFoundError: No module named 'mypkg'`** → 目录里少了 `__init__.py`，或者主程序不在包的**上一级**目录里 → 补一个空的 `__init__.py`，并确认运行主程序时所在的位置。
7. **`import pkg` 之后写 `pkg.a.bar()` 报 `AttributeError: module 'pkg' has no attribute 'a'`** → `import pkg` 只执行了 `__init__.py`，不会自动导入子模块 → 在 `__init__.py` 里写 `import pkg.a`，或者使用时显式写 `import pkg.a`。
8. **在包里写 `from . import x`，直接运行这个文件却报 `ImportError: attempted relative import with no known parent package`** → 相对导入只在"作为包的一部分被导入"时才有意义 → 从包外的主程序运行，或者改成绝对导入 `import pkg.x`。

9. **两个模块互相导入，报 `ImportError: cannot import name 'x' from partially initialized module ... (most likely due to a circular import)`** → 循环导入，模块才执行了一半 → 改成只 `import 模块名`、把公用部分抽到第三个模块、或把 `import` 挪进函数。
10. **`sys.path.append()` 加了路径还是导不进来** → 路径拼错，或者路径加在了后面，被前面同名的文件抢先 → 用绝对路径，并用 `sys.path.insert(0, 路径)` 放到第一位；`r"C:\mycode"` 前面的 `r` 别丢（否则 `\m` 这类转义会出问题）。
11. **`__init__.py` 里写了一堆打印和计算** → 每次导入这个包都会执行一次，产生副作用 → 里面只放变量、导入语句和轻量的初始化，其余封装成函数。
12. **`from pkg import *` 之后子模块一个都用不了** → `__init__.py` 没写 `__all__`，星号导出只包含里面定义的变量和显式导入的模块 → 写 `__all__ = ["a", "b"]`；不过更好的做法是别用星号导入。
13. **`__name__` 的值记混** → 被导入时是**模块名**（`"mod"`、`"pkg.a"`），不是 `"__main__"` → 记口诀："**自己跑是 main，被导入是模块名**"。

四、必须能默写的 6 个片段

1. **五种导入写法**（考试爱考"区别"）：

```
import math                     # 推荐：math.sqrt(9)
from math import sqrt           # 直接写 sqrt(9)
from math import sqrt, pi       # 一次导入多个
from math import *              # 不推荐
import time as tt               # 别名：tt.sleep(1)
```

1. **主程序保护模板**（每个稍大的程序都这么写）：

```
def main():
    n = int(input())
    print(n * 2)

if __name__ == "__main__":
    main()
```

1. **一个规范的模块**（函数 + 自测 + 保护）：

```
def is_leap(year):
    return year % 400 == 0 or (year % 100 != 0 and year % 4 == 0)

if __name__ == "__main__":
    print(is_leap(2024))
```

1. **标准的包目录结构**：

```
mypkg/
  |-- __init__.py      # 必须有，空的也行
  |-- a.py
  `-- b.py
```

1. **查看与添加搜索路径**：

```
import sys

print(sys.path)
sys.path.insert(0, r"C:\mycode")
```

1. **导入失败时的自查三连**：

```
import sys

print(sys.path)                 # 文件在不在这些目录里
print(__name__)                 # 我是被运行还是被导入
# 第三方包还要在命令行确认：pip list | findstr 包名
```

9.6 本章自测（15 道）

第 9 章 自测题 1　什么是模块？什么是包？它们分别解决什么问题？

第 9 章 自测题 2　`import math` 和 `from math import sqrt` 在使用时有什么区别？

第 9 章 自测题 3　为什么老师不推荐 `from math import *`？请举一个可能出现的问题。

第 9 章 自测题 4　`if __name__ == "__main__":` 有什么作用？去掉这一行会发生什么？

第 9 章 自测题 5　一个目录要成为包，必须包含什么文件？这个文件可以是空的吗？

第 9 章 自测题 6　`import pkg` 之后，为什么 `pkg.a` 会报 AttributeError？怎么解决？

第 9 章 自测题 7　`__all__` 变量有什么作用？不写它会怎样？

第 9 章 自测题 8　自己写一个模块 `myutils.py`，里面有两个函数：`is_leap(year)` 和 `days_in_month(year, month)`；再写一个主程序导入它并使用。

第 9 章 自测题 9　`import` 模块时，Python 会在哪些位置查找模块文件？如果找不到怎么办？

第 9 章 自测题 10　除了本章列出的标准库，再列举三个你听说过的第三方库，并说明它们是做什么的。

第 9 章 自测题 11　写了 `from math import sqrt` 之后，又写了 `print(math.sqrt(9))`，会报什么错？为什么？应该怎么改？

第 9 章 自测题 12（编程练习）　写一个模块 `mymath.py`，提供两个函数：`is_prime(n)` 判断 `n` 是不是素数、`prime_factors(n)` 返回 `n` 的所有素因子（升序列表）。再写一个主程序读入一个整数，输出这两项结果。要求：**`import mymath` 时不能读输入、也不能有任何输出。**

第 9 章 自测题 13（编程练习）　把第 12 题的 `mymath.py` 改造成一个包 `mytools`（包里放 `__init__.py` 和 `mymath.py`），主程序用 `from mytools.mymath import is_prime, prime_factors` 调用，输入输出与第 12 题一致。请写出目录结构。

第 9 章 自测题 14（编程练习）　读入一行用空格分隔的整数，用 `collections.Counter` 统计每个数字出现的次数，输出出现次数最多的数字及次数，再用 `json.dumps` 打印完整的统计结果。

第 9 章 自测题 15　什么是循环导入？它会报什么错？请写出一个会触发它的小例子，并给出两种解决办法。

期末模拟测试

> 本套试题覆盖全书九章，题型与大学 Python 课程的期末考试接近。建议**闭卷、限时 120 分钟**独立完成，做完之后再去对照最后的答案与解析。
>
> **分值分布**：单项选择题 32 分 + 判断题 12 分 + 程序阅读题 24 分 + 编程题 32 分 = 100 分。

一、单项选择题（每题 2 分，共 32 分）

模拟题 1　以下哪个不是 Python 的合法变量名？

A. `_name`　B. `2nd_score`　C. `high_score`　D. `totalPrice`

模拟题 2　执行 `x = 7; y = 2` 后，`x // y`、`x / y`、`x % y` 的结果依次是：

A. 3、3.5、1　B. 3.5、3、1　C. 3、3.5、0.5　D. 4、3.5、1

模拟题 3　`print(bool(''))` 和 `print(bool('0'))` 的输出分别是：

A. False、False　B. False、True　C. True、True　D. True、False

模拟题 4　关于 `round(4.5)` 和 `int(4.5)`，正确的是：

A. 都是 5　B. 都是 4　C. `round(4.5)` 是 5，`int(4.5)` 是 4　D. `round(4.5)` 是 4，`int(4.5)` 是 4

模拟题 5　下面哪个表达式的值是 `2`？

A. `-7 % 3`　B. `7 % -3`　C. `-7 // 3`　D. `7 // -3`

模拟题 6　执行 `s = 'python'` 后，`s[1:5:2]` 的结果是：

A. `'yh'`　B. `'yt'`　C. `'yho'`　D. `'yhn'`

模拟题 7　下面哪个说法是**错误**的？

A. 字符串是不可变数据类型 B. 列表是可变数据类型 C. 元组可以作为字典的键 D. 集合中的元素可以重复

模拟题 8　下面代码的输出是：

```
def f(x, lst=[]):
    lst.append(x)
    return lst

print(f(1))
print(f(2))
```

A. `[1]` 和 `[2]`　B. `[1]` 和 `[1, 2]`　C. `[1, 2]` 和 `[1, 2]`　D. 报错

模拟题 9　`open('a.txt', 'w')` 的作用是：

A. 以只读方式打开，文件不存在则报错 B. 以追加方式打开，新内容加在末尾 C. 以只写方式打开，文件存在则清空原有内容 D. 以读写方式打开，指针在文件头

模拟题 10　关于类的方法，下面说法正确的是：

A. 实例方法不需要 `self` 参数 B. `@staticmethod` 修饰的方法第一个参数必须是 `cls` C. `@classmethod` 修饰的方法第一个参数 `cls` 是类本身 D. 类方法可以访问实例属性

模拟题 11 `print(f"{3.14159:.3f}")` 的输出是：

A. `3.142` B. `3.141` C. `3.14` D. `3.14159`

模拟题 12 下面代码的输出是：

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

A. `A` B. `A` 和 `C` C. `A`、`B`、`C` D. 只有 `C`

模拟题 13 在模块文件 `mymod.py` 中写 `if __name__ == "__main__":` 的作用是：

A. 定义程序的主函数，必须写 B. 其中的代码只在**直接运行**该文件时执行，被别的文件 `import` 时不执行 C. 声明这是程序的入口文件，不写就不能运行 D. 让该文件可以被其它文件导入

模拟题 14 `[x * x for x in range(5) if x % 2 == 0]` 的结果是：

A. `[0, 4, 16]` B. `[0, 1, 4, 9, 16]` C. `[4, 16]` D. `[0, 4]`

模拟题 15 下面哪个表达式的值是 `3`？

A. `len({1, 1, 2, 3, 3})` B. `len([1, 1, 2, 3, 3])` C. `len({'a': 1, 'a': 2, 'b': 3})` D. `len("1,2,3")`

模拟题 16 `" Hello, World ".strip().lower().replace("world", "python")` 的结果是：

A. `'hello, python'` B. `' hello, python '` C. `'hello, world'` D. `'Hello, Python'`

二、判断题（每题 2 分，共 12 分。正确的写 T，错误的写 F）

模拟题 17 `=` 是赋值运算符，`==` 是等于比较运算符，二者完全不同。

模拟题 18 `input()` 函数返回的永远是字符串类型。

模拟题 19 `0.1 + 0.2 == 0.3` 的结果是 True。

模拟题 20 `for...else` 中的 else 子句在循环被 break 中断时也会执行。

模拟题 21 `list1.sort()` 会返回一个新的排序后的列表，原列表不变。

模拟题 22 在 Windows 上用 `open("a.txt")` 读一个 UTF-8 编码的中文文本文件，不写 `encoding="utf-8"` 也不会出错。

三、程序阅读题（每题 3 分，共 24 分。写出程序的运行结果）

模拟题 23

```
x = 10
x = x + 5
print(x, x / 4, x // 4, x % 4)
```

模拟题 24

```
for i in range(1, 6):
    if i == 3:
        continue
    if i == 5:
        break
    print(i, end=' ')
print()
```

模拟题 25

```
s = "It was the best of times"
print(s[3:6])
print(s.split(' ')[3])
print(s[::-1][:5])
```

模拟题 26

```
lst = [3, 1, 4, 1, 5, 9, 2, 6]
print(sorted(set(lst)))
print(lst[2:5])
print(lst.count(1), lst.index(4))
```

模拟题 27

```
d = {'a': 1, 'b': 2, 'c': 3}
d['d'] = 4
del d['b']
print(len(d), list(d.keys()))
print(sum(d.values()))
for k in sorted(d):
    print(k, end='')
```

模拟题 28

```
def fib(n):
    a, b = 0, 1
    for i in range(2, n + 1):
        a, b = b, a + b
    return b
```

```
print(fib(1), fib(2), fib(5), fib(10))
```

模拟题 29

```
d = {}
for ch in "banana":
    d[ch] = d.get(ch, 0) + 1
print(sorted(d.items(), key=lambda kv: (-kv[1], kv[0])))
```

模拟题 30

```
def f(n):
    if n <= 1:
        return 1
    return n * f(n - 1)

print(f(4))
print([f(i) for i in range(4)])
```

四、编程题（共 32 分）

模拟题 31（5 分） 输入三角形的三条边长 a、b、c（保证能构成三角形），输出它的周长和面积，都保留 2 位小数。面积使用海伦公式：`p = (a + b + c) / 2`，`面积 = sqrt(p(p-a)(p-b)(p-c))`。

模拟题 32（5 分） 输入一个正整数 n，输出 1~n 之间所有**既不能被 3 整除、也不能被 5 整除**的数，每个一行；最后再输出这些数的个数。

模拟题 33（6 分） 输入一个字符串，统计其中每个字符出现的次数，按**出现次数从多到少**输出；出现次数相同时按字符的字典序排列。每行输出「字符 次数」。

模拟题 34（5 分） 输入若干个整数（以 0 结束，0 不计入），找出其中的最大值和最小值，并输出它们相差多少。要求用**列表**保存所有输入的数字。

模拟题 35（6 分） 定义一个 `Rectangle` 类，包含宽和高两个**私有**属性（都必须是正数，否则抛出异常），提供只读属性 `width`、`height`，以及求面积的方法 `area()`、求周长的方法 `perimeter()`。再重载 `__str__`，使 `print(矩形对象)` 输出形如 `Rectangle 3 x 4` 的信息。

模拟题 36（5 分） 输入一行若干个整数（用空格分隔），先**去重**并按**升序**输出（用空格分隔，**行尾不能有多余空格**），再输出这些数的平均值，保留 2 位小数。

答案与解析

第 1 章 自测题答案

> 题目见 1.16 节。第 13~16 题是本次新增的 4 道，重点覆盖负数除法、一行读入多个数、链式比较和银行家舍入。

第 1 章 自测题 1　`x` 的值是 **11**。赋值号先算右侧 `5*2+1=11`，再把 11 赋给 x，覆盖原来的 5。

第 1 章 自测题 2　分别是 **3**、**3.5**、**1**。注意 `/` 的结果一定是浮点数。

第 1 章 自测题 3　结果是 **3**。因为 `-9 // 4 = -3`（地板数），代入 `x % y = x - y*(x//y)` 得 `-9 - 4*(-3) = 3`。

第 1 章 自测题 4　只有 `b = int("12.5")` 会报错（`int()` 不接受形为浮点数的字符串，报 `ValueError`）。`float("12")` 得到 12.0，`eval("3+4")` 得到 7。

第 1 章 自测题 5　分别是 **False** 和 **True**。空字符串视同 False；`'0'` 是非空字符串，视同 True。

第 1 章 自测题 6　都是 **'hello'**。逻辑运算返回的是"最早确定表达式值的操作数"，而不是布尔值。

第 1 章 自测题 7　三种写法：`round(x, 2)`、`int(x * 100) / 100`、`f"{x:.2f}"`。OJ 上最省事、最不容易出错的是第三种。

第 1 章 自测题 8　因为 `math.sin()` 接受的参数是**弧度**，90 弧度是另一个角。应写成 `math.sin(math.radians(90))`，结果为 1.0。

第 1 章 自测题 9　合法：`_name`、`totalPrice`、`high_score`。不合法：`2nd_score`（不能以数字开头）、`class`（关键字）、`总价`（不允许汉字）、`run-year`（横线不是合法字符，会被当成减法）。

第 1 章 自测题 10　依次输出 `0.30000000000000004`、`False`、`True`。

第 1 章 自测题 11　依次是 **4**、**-2**、**-3**、**-4**。（`round` 是 .5 取偶的银行家舍入，`int` 是向零截断，`floor` 是向下取整。）

第 1 章 自测题 12

```
total = int(input())
h = total // 3600
m = total % 3600 // 60
s = total % 60
print(h, ":", m, ":", s, sep="")   # 输入 3661 得到 1:1:1
```

第 1 章 自测题 13　依次输出 **-4**、**-1**、**1**。

- `-7 // 2 = -4`：-3.5 的地板数是 -4，向下取整而不是向零截断。
- `7 % -2 = -1`：余数的符号跟**除数**（-2），所以是负数。
- `-7 % 2 = 1`：除数 +2 为正，余数也为正。

验算 `-7 % 2`：由 `-7 // 2 = -4`，代入 `x % y = x - y*(x//y)` 得 `-7 - 2*(-4) = 1`，与输出一致；再代回恒等式 `(-4) * 2 + 1 = -7`，成立。

第 1 章 自测题 14

```
a, b = map(int, input().split())
print(a + b)
```

输入 `3 5`，输出 `8`。要点：`input()` 读进来的是一整行字符串，`.split()` 按空白切成两段，`map(int, ...)` 把两段都转成整数再分别赋给 a 和 b。个数不确定时可以写 `nums = list(map(int, input().split()))`，再用 `sum(nums)` 求和。

第 1 章 自测题 15 链式比较写 `0 <= x < 100`；用 `and` 写 `0 <= x and x < 100`（也可以写成 `x >= 0 and x < 100`）。

两者**完全等价**，结果都是布尔值。区别只有两点：链式比较更接近数学写法；链式比较中 x 只求值一次（x 是函数调用时更快也更安全）。注意端点：条件是"含 0、不含 100"，所以左边用 `<=`、右边必须用 `<`。

第 1 章 自测题 16 `print(round(2.675, 2))` 输出 **2.67**，`print(f"{2.675:.2f}")` 也输出 **2.67**。

原因有两层：① 2.675 在二进制里存不下来，实际存的是 2.67499999999999982...，比 2.675 小，所以不管用哪种舍入规则都进不到 2.68；② 即使数值正好落在"一半"上，`round()` 和 `f"{...:.2f}"` 用的也是**银行家舍入**（.5 取偶），例如 `f"{2.5:.0f}"` 得到 `'2'`。

想精确得到 2.68，要用 `decimal` 并显式指定四舍五入：

```
from decimal import Decimal, ROUND_HALF_UP

print(Decimal("2.675").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))   # 2.68
```

注意 `Decimal()` 的参数要写成字符串 `"2.675"`，写成数字 `2.675` 又会把误差带进去。

第 2 章 自测题答案

第 2 章 自测题 1 输出 **D**。因为 `score >= 60` 在 85 分时已经成立，后面的分支不再判断。这正说明多分支的条件必须**从严格到宽松**排列。

第 2 章 自测题 2 序列是 **2, 5, 8**，循环执行 **3 次**。

第 2 章 自测题 3 输出 `1 2 4`（中间跳过 3）。

第 2 章 自测题 4 代码 A **无输出**（遇到 break，else 不执行）；代码 B 输出 **正常结束**（循环正常走完，没碰到 break）。

第 2 章 自测题 5 因为浮点误差，`item` 会变成 `1.38e-16`、`-0.0999…` 这类值，永远不会**恰好**等于 0，循环无法退出。改法一：**把循环变量换成整数计数**（`for count in range(100)`）；改法二：**不用精确相等判断**，改成 `while abs(item) > 1e-9:` 或者 `while item > eps:`。

第 2 章 自测题 6

```
s = "偶" if x % 2 == 0 else "奇"
```

第 2 章 自测题 7 后两个正确。第一个写法是错的：`month == 4 or 6 or 9 or 11` 实际上是 `(month == 4) or 6 or 9 or 11`，而 6、9、11 都是非零数（视同 True），整个表达式永远为真。推荐第三种 `month in`

`(4, 6, 9, 11)`，最短也最不容易错。

第 2 章 自测题 8　`try` 总是执行；`except` 只在发生匹配的异常时执行；`else` 只在**没有发生异常**时执行；`finally` **无论是否发生异常都执行**。

第 2 章 自测题 9

```
for n in range(2, 1001):
    s = 0
    for i in range(1, n):
        if n % i == 0:
            s += i
    if s == n:
        print(n)
# 输出 6、28、496
```

优化：内层循环只要试到 `n // 2` 就够了（大于 n/2 的数不可能是 n 的真因子），可以写成 `for i in range(1, n // 2 + 1):`。

第 2 章 自测题 10

```
n = int(input())
for i in range(1, n + 1):
    print("*" * i)
```

第 2 章 自测题 11　输出 `4 10`。`range(1, 5)` 产生 1、2、3、4，所以 `s = 1 + 2 + 3 + 4 = 10`；循环结束后循环变量 `i` 仍然存在，保留最后一次取到的值 4。它不会是 5（5 根本不在序列里），也不会报错（Python 的 `for` 不新建作用域，详见 2.9 节易错清单第 4 条）。

第 2 章 自测题 12　输出三行：

```
1 1
2 1
3 1
```

`break` 只跳出**内层** j 循环，外层 i 循环照常走完 3 轮；每一轮内层都在 `j == 2` 时被中断，所以每行只打印了 `j = 1`。（要一次跳出两层，得用标志变量，见 2.6 节【常见错误】。）

第 2 章 自测题 13　循环体里没有更新 `i`，`i` 永远是 1，条件 `i <= 100` 永远成立，所以是死循环。改正（把 `i += 1` 加进循环体，缩进与 `if` 同层）：

```
i = 1
count = 0
while i <= 100:
    if i % 7 == 0:
        count += 1
    i += 1          # 补上这一行
print(count)        # 14
```

第 2 章 自测题 14

```
nums = list(map(int, input().split()))
max_num = min_num = nums[0]      # 用第一个数当擂主
for x in nums:
    if x > max_num:
        max_num = x
    if x < min_num:
        min_num = x
print(max_num)
print(min_num)
```

输入 `3 7 -2 9 4`，输出 `9` 和 `-2`。注意不能写 `max_num = 0`，否则输入全是负数时会输出 0。

第 2 章 自测题 15

```
n = int(input())
total = 0
fact = 1
for i in range(1, n + 1):
    fact *= i        # 先把 i! 算出来（上一轮的 (i-1)! 再乘 i）
    total += fact    # 再累加，不必每轮从 1 重算阶乘
print(total)
```

输入 5，输出 **153**（1 + 2 + 6 + 24 + 120）。

> **自查提示**：自测题 11 对应 2.9 节易错清单第 4 条，第 12 题对应第 9 条，第 13 题对应第 5 条，第 14 题对应第 14 条。做错的题回去把对应条目读一遍。

第 3 章 自测题答案

第 3 章 自测题 1　输出 **1**。数字是不可变类型，函数内的 `n += 1` 只是让局部变量 n 指向了新值 2，没有改变外部的 x（见 3.4 节的速查表）。

第 3 章 自测题 2　`f(1, y=2, 3)` 会报错，提示 `SyntaxError: positional argument follows keyword argument`，因为位置参数不能出现在关键字参数之后。其余三行都正确。

第 3 章 自测题 3　输出两行：**20** 和 **10**。函数内的 `x = 20` 创建了一个局部变量，屏蔽了全局变量（3.5 节）。

第 3 章 自测题 4　报 `UnboundLocalError`。因为函数体里出现了 `x = 2`，Python 就把 x 当成局部变量，于是在赋值之前读取它就报错；Python 3.12 的完整提示是 `UnboundLocalError: cannot access local variable 'x' where it is not associated with a value`，老版本写作 `local variable 'x' referenced before assignment`。三种改法：把 `print(x)` 移到 `x = 2` 之后；加 `global x`（不推荐）；或者干脆不在函数里给 x 赋值，改用参数和返回值（推荐）。

第 3 章 自测题 5

```
def count_vowels(s):
    n = 0
    for c in s.lower():
```

```
        if c in "aeiou":
            n += 1
    return n

print(count_vowels("Hello World"))    # 3
```

要点：先用 `s.lower()` 统一成小写，就不用分别判断大小写；`if c in "aeiou"` 是成员运算，比写五个 `c == 'a' or ...` 短得多。

第 3 章 自测题 6

```
def maximum(*nums):
    if not nums:          # 空元组为假
        return None
    m = nums[0]
    for x in nums:
        if x > m:
            m = x
    return m

print(maximum())          # None
print(maximum(3, 9, 2))   # 9
```

要点：`*nums` 收到的是一个元组，所以能遍历、能取下标。也可以偷懒写成 `return max(nums) if nums else None`——这里调用的 `max()` 是内置函数，和我们自己定义的 `maximum()` 名字不同（再次说明函数名千万不要和内置名重复）。

第 3 章 自测题 7

```
# 递归写法：三要素齐全
def sum_rec(n):
    if n == 1:                # ① 基准情形
        return 1
    return n + sum_rec(n - 1)  # ② 递推关系 ③ if 控制是否继续

# 循环写法
def sum_loop(n):
    s = 0
    for i in range(1, n + 1):
        s += i
    return s
```

比较：递归版贴着数学定义，三行就写完；循环版多一个累加变量，但速度快、不吃栈空间。`sum_rec(2000)` 会抛 `RecursionError: maximum recursion depth exceeded`（默认上限 1000），`sum_loop(2000)` 毫无压力——这就是 3.6 节“能用循环就别用递归”的意思。

第 3 章 自测题 8 因为 `fib(40)` 的朴素递归会重复计算大量相同的子问题，调用次数达到 3 亿多次（见 3.6 节的表），所以非常慢；而循环写法每个 n 只算一次，只有约 40 步。给递归加上缓存（记忆化，见 3.11 节例题 3-10）也能瞬间出结果，但调用栈深度仍是 O(n)，所以作业题 15 明确要求用循环实现。

第 3 章 自测题 9　输出 **2 3 3**。第一次调用 `inner()` 得 2，第二次得 3；`nonlocal x` 让内层的赋值作用在外层函数的 x 上，所以最后打印的 x 也已经是 3。

第 3 章 自测题 10

```
def stat(nums):
    return max(nums), min(nums), sum(nums) / len(nums)

hi, lo, avg = stat([92, 85, 78, 96, 88])
print(hi, lo, f"{avg:.2f}")     # 96 78 87.80
```

要点：返回值其实是一个元组，用三个变量解包即可（3.3 节）。`stat([])` 会报 `ValueError: max() iterable argument is empty`（老版本提示是 `max() arg is an empty sequence`），正式代码里要先判断列表是否为空。

第 3 章 自测题 11　输出三行：`[1]`、`[1, 2]`、`[1, 2, 3]`。

原因：默认值 `[]` 只在 `def` 执行的那一次创建，三次调用共用同一个列表对象，前一次追加的元素会留到下一次（3.4 节的【常见错误】）。正确写法：

```
def add(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst

print(add(1))    # [1]
print(add(2))    # [2]
```

第 3 章 自测题 12　先打印 `hello`（这是 f 内部的 print 干的），再打印 `None`。变量 x 的值是 `None`：函数里没有 `return`，返回值就是 `None`；`x = f()` 只是把返回值赋给 x，屏幕上打印出来的内容不会进入 x。想让 x 拿到东西，必须写 `return`（3.3 节）。

第 3 章 自测题 13

```
scores = [("Zhang", 92), ("Li", 85), ("Wang", 92)]
print(sorted(scores, key=lambda t: (-t[1], t[0])))
# [('Wang', 92), ('Zhang', 92), ('Li', 85)]
```

要点：`sorted()` 返回新列表，不改变 scores（原地排序要用 `scores.sort(...)`）；`key` 返回元组，先比 `-分数`（等价于分数降序），再比姓名（升序）。如果只按分数降序，也可以写 `sorted(scores, key=lambda t: t[1], reverse=True)`。

第 3 章 自测题 14

```
def to_bin(n):
    if n < 2:                           # 基准情形：只剩一位
        return str(n)
    return to_bin(n // 2) + str(n % 2)   # 先处理高位，再补上末位
```

```
print(to_bin(13))    # 1101
```

要点：每次递归 n 都减半，递归深度约为 log2(n)。`n = 10 ** 5000` 时深度约为 16600 层，远超默认上限 1000，会抛 `RecursionError: maximum recursion depth exceeded`。两种改法：改成循环逐位取余，或者直接用内置的 `bin(n)[2:]`；调大 `sys.setrecursionlimit()` 只是权宜之计，实际可用的栈空间并没有变多（3.6 节）。

第 4 章 自测题答案

第 4 章 自测题 1　依次是 `'t'`、`'n'`、`'yth'`、`'nohtyp'`。`s[10]` 会报 **IndexError**；`s[10:20]` **不报错**，返回空字符串 `''`（切片越界是安全的）。

第 4 章 自测题 2　`t = (1)` 中 t 是**整数 1**；`t = (1,)` 中 t 是**只含一个元素的元组**。区别就在那个逗号。

第 4 章 自测题 3　① `sort()` 是列表的方法，`sorted()` 是内置函数；② `sort()` 原地修改列表，`sorted()` 返回新列表、不改原对象；③ `sort()` 没有返回值（得到 None），`sorted()` 返回排序后的新列表。另外 `sorted()` 能作用在任何可迭代对象上，`sort()` 只能用于列表。

第 4 章 自测题 4　输出 `[1, 0, 9]`。切片赋值把下标 1~3 的三个元素替换成了一个元素，列表长度可以因此改变。

第 4 章 自测题 5　因为默认参数 `lst=[]` 只在函数**定义时**创建一次，所有不传 lst 的调用共用同一个列表，所以一直在往同一个列表里加元素。正确写法：

```
def add(x, lst=None):
    if lst is None:
        lst = []
    lst.append(x)
    return lst
```

第 4 章 自测题 6

```
squares = [x * x for x in range(1, 101) if x % 3 == 0]
```

第 4 章 自测题 7　输出 `{'a': 9, 'b': 2, 'c': 3}`、`True`、`False`。最后一个是 False，因为 `in` 判断的是**键**，9 是值不是键（判值要写 `9 in d.values()`）。

第 4 章 自测题 8　因为字典的键必须是不可变（可哈希）类型，而列表是可变类型，不能保证哈希值稳定。例如 `d = {[1, 2]: 'x'}` 会报 `TypeError: unhashable type: 'list'`。想用"一串值"当键，可以先转成元组：`d = {(1, 2): 'x'}`。

第 4 章 自测题 9

```
a = [1, 2, 3, 4, 5]
b = [4, 5, 6, 7]
print(set(a) & set(b))    # {4, 5}                共同元素
print(set(a) - set(b))    # {1, 2, 3}             只在 a 中
print(set(a) | set(b))    # {1, 2, 3, 4, 5, 6, 7}  去重后的全部
```

第 4 章 自测题 10

```
print([len(w) for w in "hello world".split()])    # [5, 5]
```

第 4 章 自测题 11

```
nums = list(map(int, input().split()))
for x in sorted(set(nums)):
    print(x)
```

也可以一行输出：`print(*sorted(set(nums)))`（默认用空格分隔）。

第 4 章 自测题 12　输出 `[1, 2, 3] [1, 2, 3, 4]`。因为 `lst + [4]` 会创建一个**新列表**并让局部变量 lst 指向它，原列表 a 没有变。如果写成 `lst.append(4)` 就会改变 a。

第 4 章 自测题 13

```
print(list(enumerate(['a', 'b'])))           # [(0, 'a'), (1, 'b')]
print(list(enumerate(['a', 'b'], start=1)))  # [(1, 'a'), (2, 'b')]
print(list(zip([1, 2, 3], 'abcd')))           # [(1, 'a'), (2, 'b'), (3, 'c')]
```

`enumerate` 的编号默认从 0 开始，写 `start=1` 就从 1 开始；`zip` **按最短的序列截断**，字符串 `'abcd'` 里多出来的 `'d'` 被丢掉了，所以两行都不会报错。

第 4 章 自测题 14

```
result = sorted(students, key=lambda t: (-t[1], t[0]))
print(result)   # [('Ann', 95), ('Bob', 88), ('Tom', 88), ('Cindy', 70)]
```

key 返回一个元组 `(-分数, 姓名)`：先按 `-分数` 升序，也就是分数**降序**；分数相同时再按姓名升序。注意 Bob 和 Tom 都是 88 分，姓名升序让 Bob 排在前面。这是 4.13 节必背片段第 8 条。

第 4 章 自测题 15　两行都输出 `[[1, 2, 9], [3, 4]]`。因为 `a.copy()` 只是**浅拷贝**，`b[0]` 和 `a[0]` 仍然是同一个列表，改 `b[0]` 就是改 `a[0]`。要让两者完全独立，改成深拷贝：

```
import copy
a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)
b[0].append(9)
print(a)      # [[1, 2], [3, 4]]   没变
print(b)      # [[1, 2, 9], [3, 4]]
```

第 4 章 自测题 16

```
print(len(set(nums)) != len(nums))    # True 表示有重复元素
```

思路：集合会自动去重，去重后长度变短就说明原来有重复。注意不要写 `any(nums.count(x) > 1 for x in nums)`，虽然结果对，但每个元素都要数一遍，效率是 $O(n^2)$。

第 4 章 自测题 17 实际输出 `[3, 2, 1]`，中间那个 2 没被删掉。因为 `remove(x)` 删的是**第一个**值为 x 的元素，删除后后面的元素整体前移，而 for 循环的下标仍继续往前走，于是刚好跳过一个元素。两种正确写法：

```
# 写法一：遍历副本
a = [2, 2, 3, 2, 1]
for x in a[:]:           # a[:] 是拷贝，删的是原列表 a
    if x == 2:
        a.remove(x)
print(a)                 # [3, 1]

# 写法二：用推导式重建（推荐）
a = [2, 2, 3, 2, 1]
a = [x for x in a if x != 2]
print(a)                 # [3, 1]
```

第 5 章 自测题答案

第 5 章 自测题 1 `'w'` 是只写模式，文件存在时会**清空原有内容**再从头写；`'a'` 是追加模式，文件指针在末尾，新内容**追加**在原有内容之后。所以用 `'w'` 打开一个有内容的文件，原来的内容立刻没了。

第 5 章 自测题 2 因为第一次 `read()` 会把文件指针从开头一直推到**文件末尾**，第二次再调用时指针已经在末尾，所以读到空字符串。想重新读要先用 `seek(0)` 把指针移回开头。（`for line in f:` 遍历完之后再 `read()`，也是同样的道理。）

第 5 章 自测题 3 `read()` 返回整个文件内容的字符串；`readline()` 返回一行的字符串（**带行尾的** `\n`）；`readlines()` 返回一个列表，每个元素是一行（**也带** `\n`）。

第 5 章 自测题 4

```
with open("num.txt", "w", encoding="utf-8") as f:
    for i in range(1, 101):
        f.write(str(i) + "\n")
```

第 5 章 自测题 5

```
total = 0
with open("num.txt", "r", encoding="utf-8") as f:
    for line in f:
        total += int(line)      # int() 会自动忽略行尾的换行和空格
print(total)                    # 5050
```

第 5 章 自测题 6 `with` 在代码块结束时（无论是否发生异常）会**自动关闭文件**，等效于 `try...finally`，避免忘记 `close()` 导致数据留在缓冲区没写进磁盘、或者文件句柄一直被占用。此外写成 `with` 之后代码也更短，不容易漏掉关闭。

第 5 章 自测题 7 三个问题：① 不要用 `split(',')` 手工解析 CSV，因为 CSV 里的逗号可能是值的一部分（例如 `"Hello, world!"`），应该用 `csv` 模块；② 没有指定 `encoding="utf-8"`，Windows 上读含中文的

文件会报 `UnicodeDecodeError`；③ 文件没有被关闭，应改成 `with open('data.csv', encoding="utf-8", newline="") as f:`。

第 5 章 自测题 8

```
import os

count = 0
total = 0
for root, dirs, files in os.walk("."):
    for name in files:
        if os.path.splitext(name)[1] == ".py":
            count += 1
            total += os.path.getsize(os.path.join(root, name))
print(count, total)
```

> 说明：`os.splitext(name)[1]` 取出扩展名（含点）。`os.walk` 给出的只是文件名，取大小和路径都要先用 `os.path.join(root, name)` 拼回完整路径。

第 5 章 自测题 9

```
nums = []

with open("numbers.txt", "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()          # 去掉换行和空行里的空白
        if line == "":               # 空行跳过
            continue
        nums.append(int(line))

print(len(nums))                     # 4
print(max(nums))                     # 12
print(min(nums))                     # -3
print(f"{sum(nums) / len(nums):.2f}")   # 5.50
```

输出：

```
4
12
-3
5.50
```

第 5 章 自测题 10

```
best_name = ""
best_score = -1

with open("score.txt", "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
```

```
        if line == "":
            continue
        name, score = line.split()      # 按空白切分，得到两个字符串
        score = int(score)              # 别忘了解包出来的是字符串
        if score > best_score:          # 严格大于，并列时保留先出现的
            best_score = score
            best_name = name

print(best_name)                        # 李四
```

> 说明：因为用的是 `>` 而不是 `>=`，分数并列时先出现的那一位不会被后来的替换掉，正好满足"输出最先出现的那一位"。

第 5 章 自测题 11

```
n = int(input())

# 第一步：写入
with open("nums.txt", "w", encoding="utf-8") as f:
    for i in range(1, n + 1):
        f.write(str(i) + "\n")

# 第二步：读回并求平均。必须另起一个 with，写完关闭后才保证数据落盘
total = 0
count = 0
with open("nums.txt", "r", encoding="utf-8") as f:
    for line in f:
        total += int(line)
        count += 1

print(f"{total / count:.1f}")       # 输入 5 时输出 3.0
```

第 5 章 自测题 12 原因：Windows 中文环境下 `open()` 不写 `encoding` 时用的默认编码是 GBK，程序按 GBK 去解码一个 UTF-8 文件，遇到非 ASCII 字节就报 `UnicodeDecodeError: 'gbk' codec can't decode byte ...`。改法：打开时显式写 `open("a.txt", "r", encoding="utf-8")`，这样任何平台、任何默认编码下行为都一致。写文件时**同样要写**，否则写出的文件在 Windows 上可能是 GBK 编码，发给别人（或传到 Linux 服务器、OJ 平台）用 UTF-8 打开就是乱码。

第 5 章 自测题 13 原因是**没有关闭文件**：`write()` 只是把 `"hello"` 写进内存缓冲区，只有 `close()`（或者 `flush()`）才会把缓冲区刷到磁盘。程序正常结束前如果崩溃、或者一直被别的引用持有，文件里就什么都没有。

改法一：手动关闭。

```
f = open("out.txt", "w", encoding="utf-8")
f.write("hello")
f.close()
```

改法二（推荐）：用 `with`，退出代码块时自动关闭。

```
with open("out.txt", "w", encoding="utf-8") as f:
    f.write("hello")
```

第 6 章 自测题答案

第 6 章 自测题 1 `randint(1, 10)` 的取值范围是 **1~10（含 10）**；`randrange(1, 10)` 的取值范围是 **1~9（不含 10）**。想用 `randrange` 达到 `randint(1, 10)` 的效果，要写成 `randrange(1, 11)`。详见 6.2.1 节。

第 6 章 自测题 2

```
import random

random.seed(1)                  # 想每次结果一样就加这一行，不加也可以
heads = tails = 0
for i in range(100):
    if random.randint(0, 1) == 1:     # 1 当正面，0 当反面
        heads += 1
    else:
        tails += 1
print("正面", heads, "次，反面", tails, "次")
```

要点：抛硬币就是"在两个结果里等概率取一个"，所以 `random.choice(["正面", "反面"])` 也完全等价，可读性更好。

第 6 章 自测题 3 `strftime` 把 struct_time 对象**格式化成字符串**，例如 `time.strftime("%Y-%m-%d", time.localtime())` 得到 `'2026-10-08'`；`strptime` 把字符串**解析成 struct_time 对象**，例如 `time.strptime("2021-09-10", "%Y-%m-%d")` 得到 `time.struct_time(...)`。记忆：`f` 在前是"产出字符串"，`p` 在前是"解析字符串"，格式串永远放第二个参数。

第 6 章 自测题 4

```
from datetime import date

y, m, d = map(int, input().split())
print((date.today() - date(y, m, d)).days)
```

两个 `date` 相减得到 `timedelta`，`.days` 取整数天数。不用 `abs()`，因为出生日期一定早于今天，结果天然是正数。

第 6 章 自测题 5 `math.floor(-2.5)` = **-3**（向下取整），`math.ceil(-2.5)` = **-2**（向上取整），`math.trunc(-2.5)` = **-2**（向 0 截断），`int(-2.5)` = **-2**（也是向 0 截断）。再看 `math.floor(-7 / 2)` = **-4**，而 `int(-7 / 2)` = **-3**：对负数来说，`floor` 比 `trunc` / `int` 小 1，而 `//` 与 `floor` 结果一致（`-7 // 2` = -4）。

第 6 章 自测题 6 `random.seed(1)` 之后随机数序列就**固定**了，每次运行都得到相同的一串随机数。调试时非常有用：可以让"随机出错"的场景稳定重现，也方便对照同学的结果。注意**种子必须写在取随机数之前**；如果把 `seed` 写进循环，每次取数前都把序列拨回开头，同一个数会被反复取到（例如循环里每次都 `seed(1)` 再 `randint(1, 6)`，会连出三个 2）。

第 6 章 自测题 7

```
import time

start = time.perf_counter()
total = sum(range(1000000))
elapsed = time.perf_counter() - start
print(f"耗时 {elapsed:.3f} 秒")     # 例如 耗时 0.016 秒（依机器而不同）
```

不推荐用 `time.time()` 计时，原因有两条：① 它读的是系统时钟，可能被校时（NTP）或用户改时间影响，测出来的差值可能变小甚至为负；② 在 Windows 上它的精度只有毫秒级。`time.perf_counter()` 是单调递增的高精度计数器，只能相减、不能当年月日使用，正是为计时设计的。要更高精度可用 `time.perf_counter_ns()`。

第 6 章 自测题 8 `-7 // 2` = **-4**（向下取整，不是 -3）；`-7 % 3` = **2**（`%` 的结果符号跟**除数**一致）；`math.fmod(-7, 3)` = **-1.0**（`fmod` 的结果符号跟**被除数**一致，且返回浮点数）。所以 C / Java 里的 `-7 % 3` 是 -1，而 Python 里是 2，从别的语言转过来时这里最容易出错。

第 6 章 自测题 9

```
import random

random.seed(1)
count = 0
for i in range(20):
    if 10 <= random.randint(1, 20) < 20:    # 只有 10~19 才算
        count += 1
print(count)                                 # 9
```

要点：`randint(1, 20)` 含 20，而题目要求的是 [10, 20)，所以判断条件写成 `10 <= x < 20`（Python 支持这种链式比较，等价于 `x >= 10 and x < 20`）。用 `sum(1 for _ in range(20) if ...)` 也可以，效果一样。

第 6 章 自测题 10

```
from datetime import datetime, timedelta

s = input()                                  # 例如 2026-10-08
n = int(input())                             # 例如 100
d = datetime.strptime(s, "%Y-%m-%d")         # 字符串 -> datetime
print((d + timedelta(days=n)).strftime("%Y-%m-%d"))
```

输入：

```
2026-10-08
100
```

输出：

```
2027-01-16
```

要点：`datetime.strptime` 才能解析字符串（`date` 类没有 `strptime`），解析出来再 `+ timedelta(days=n)`，最后用 `strftime` 格式化回去；n 也可以写成负数，表示往前推 n 天。

第 7 章 自测题答案

> 对应第 7 章"7.13 本章自测"的 15 道题。第 13、14、15 题是 OJ 风格，答案里给出了完整代码、样例运行结果和易错提醒。

第 7 章 自测题 1　结果是 `'itc'`。因为 `strip("sat")` 不是去掉子串 "sat"，而是去掉**字符集合 {'s','a','t'} 中的任意字符**：右侧的 't' 被去掉，接着 's' 也被去掉，再接着 'a' 也被去掉，遇到 'c' 停止；左侧 'i'、't' 都不在集合里，所以保留。

第 7 章 自测题 2　找不到时 `find` 返回 **-1**，而 `index` 会**抛出 ValueError 异常**。不确定元素是否存在时用 `find`，或者直接用 `in` 判断存在性；另外 `find` 返回的 -1 在条件里是"真"，不能当布尔值用。

第 7 章 自测题 3　`"a,b,,c".split(",")` 得到 `['a', 'b', '', 'c']`（保留空串）；`"a b c".split()` 不带参数时按**任意多个空白字符**分割，并忽略首尾空白，得到 `['a', 'b', 'c']`。

第 7 章 自测题 4

```
print("-".join(['2026', '10', '08']))    # 2026-10-08
```

第 7 章 自测题 5

```
s = input()
print(s == s[::-1])
```

输入 `level` 输出 `True`；输入 `Python` 输出 `False`。如果输入可能带首尾空格，先写 `s = input().strip()`。`s[::-1]` 是切片反转（详见 4.3 节），它生成新串，原串不变。

第 7 章 自测题 6

```
import math

print(f"{math.pi:>10.4f}")
```

输出（前面有 4 个空格）：

```
    3.1416
```

第 7 章 自测题 7

```
s = input()
words = s.split()
upper_count = sum(1 for c in s if c.isupper())
print(len(words), upper_count)
```

输入 `Hello Python World`，输出 `3 3`。`split()` 不带参数，单词之间有多少个空格都不影响结果。

第 7 章 自测题 8

```
print("Hello World".swapcase())    # hELLO wORLD
```

第 7 章 自测题 9

```
import re

text = "我的电话是13812345678，备用电话是010-12345678"
print(re.findall(r"\d+", text))
# ['13812345678', '010', '12345678']
```

注意模式写成了原始字符串 `r"\d+"`；如果写成 `"\d+"`，虽然现在多数版本还能跑（只会给一个 SyntaxWarning），但语义已经不可靠了。

第 7 章 自测题 10

```
s = input()
print(s.replace("bad", "good"))
print(s.count("bad"))
```

输入 `a bad day, bad luck`，输出：

```
a good day, good luck
2
```

`replace()` 返回新串，原串没变；要拿到次数就再调一次 `count()`，或者写成 `print(f"{s.replace('bad', 'good')} {s.count('bad')}")`。

第 7 章 自测题 11

`" Hello ".strip()` 得到 `'Hello'`；`" Hello ".lstrip()` 得到 `'Hello '`（只去左边）；`" Hello ".rstrip()` 得到 `' Hello'`（只去右边）。执行 `s.strip()` 之后 `s` 没变，是因为**字符串是不可变对象**：`strip()` 把结果当作**新字符串返回**，原串不动。想改变必须写 `s = s.strip()`。

第 7 章 自测题 12

写法	输出	说明
`f"{3.14159:8.2f}"`	`3.14`	宽度 8、右对齐，前面补 4 个空格
`f"{'py':*^6}"`	`**py**`	宽度 6、居中，两侧各补 2 个 `*`
`f"{1234567:,}"`	`1,234,567`	千分位分隔符
`f"{0.1234:.1%}"`	`12.3%`	乘 100 加百分号，保留 1 位小数
`f"{255:#x}"`	`0xff`	十六进制并带 `0x` 前缀

第 7 章 自测题 13（OJ：求和与平均值）

```
nums = list(map(int, input().split()))
print(sum(nums))
print(f"{sum(nums) / len(nums):.2f}")
```

输入 `1 2 3 4`，输出：

```
10
2.50
```

易错提醒：① 平均值必须用 `/`（浮点除法），用 `//` 会得到 `2` 这样对不上样例的结果；② 输出必须保留 2 位小数，写 `f"{avg:.2f}"`，直接 `print(avg)` 会输出 `2.5` 导致 WA；③ `input().split()` 不带参数，多空格、行尾空格都能扛。

第 7 章 自测题 14（OJ：统计单词并整理句子）

```
words = input().split()
print(len(words))
print(" ".join(w.capitalize() for w in words))
```

输入 `hello WORLD python`（行首、行尾各两个空格），输出：

```
3
Hello World Python
```

易错提醒：① 必须用不带参数的 `split()`——它把连续空格看成一个分隔符并丢掉首尾空白，用 `split(" ")` 会切出空串，单词个数就不再是 3；② `capitalize()` 会把单词其余字母变小写（`WORLD` → `World`），如果题目要求"其余字母保持原样"，要写 `w[0].upper() + w[1:]`；③ 拼接用 `" ".join(...)`，行尾没有多余空格。

第 7 章 自测题 15（OJ：提取所有非负整数）

```
import re

text = input()
nums = [int(x) for x in re.findall(r"\d+", text)]
print(",".join(map(str, nums)))
print(max(nums))
```

输入 `price 120, count 45, total 300`，输出：

```
120,45,300
300
```

易错提醒：① `re.findall(r"\d+", text)` 返回的是**字符串**列表，求最大值前必须 `int()` 转换，否则会按字典序比较（字符串 `'45'` 比 `'300'` 大）；② `join` 的元素必须是字符串，所以要写 `map(str, nums)`；③ 模式要用原始字符串 `r"\d+"`。

第 8 章 自测题答案

第 8 章 自测题 1 类是**设计图**，对象是照着设计图造出来的**房子**。同一个类可以造出很多个对象，每个对象的属性值可以不同，但属性名和方法都由类统一定义。

第 8 章 自测题 2 `__init__` 是初始化方法，在**创建对象时被 Python 自动调用**，用来给实例属性赋初值；每个实例各调用一次，而且它**不能有返回值**。`self` 代表**正在被操作的那个实例本身**：调用 `c1.get_area()` 时，Python 自动把 `c1` 当作第一个实参传给方法，方法里的 `self` 就是它（所以 `self.radius` 等价于 `c1.radius`）。可以改名叫 `this`（Python 不强制名字），但**强烈建议按惯例用 self**，否则别人读代码会很困惑。

第 8 章 自测题 3 因为直接改属性可能造成异常数据（比如半径为负数），而对象一旦处在非法状态，后面所有基于它的计算都会出错。设成私有后，外部只能通过提供的方法修改，方法里可以做合法性检查。但它并非绝对访问不到——Python 只是把 `__radius` 改名成了 `_Circle__radius`（名称改写），用 `c._Circle__radius` 仍能访问，只是不建议这么做。

第 8 章 自测题 4 `@property` 把一个方法变成“可以像属性一样访问”的东西：外部写 `c.radius` 而不是 `c.radius()`，写法更自然；同时还能配合 `@radius.setter` 在赋值时做合法性检查，做到了“**用起来像属性，内部仍然受控制**”。它的三个典型用途是：只读属性、写入时校验、计算属性（详见 8.3 节）。

第 8 章 自测题 5 类属性属于类本身，所有实例共享一份，用 `类名.属性名` 读写；实例属性属于每个实例自己，在 `__init__` 里用 `self.属性名` 创建。用**实例名**去“修改”类属性时，不会改变类属性，而是**新建了一个同名实例属性**，把类属性屏蔽掉（例如 `Circle.count` 是 6，而 `circle1.count` 变成 8，`circle2.count` 仍是 6）。所以改类属性必须写 `类名.属性名`。

第 8 章 自测题 6 `@classmethod` 的第一个参数是 `cls`（**类本身**），访问类属性时可以写 `cls.属性名`；`@staticmethod` 没有任何特殊参数，访问类属性时必须写完整类名。两者都只能访问类属性、不能访问实例属性，都不能调用实例方法，都可以通过类名或实例调用。

第 8 章 自测题 7

```
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price

    def show(self):
        print(f"《{self.title}》 {self.author} {self.price}元")

    def __str__(self):
        return f"《{self.title}》 作者：{self.author} 价格：{self.price}"

b = Book("Python 入门", "张三", 45.0)
b.show()          # 《Python 入门》 张三 45.0元
print(b)          # 《Python 入门》 作者：张三 价格：45.0
```

第 8 章 自测题 8

```
class Book:
    # 构造方法与 show、__str__ 同第 7 题
    def __eq__(self, other):
        return self.title == other.title and self.author == other.author

    def __ne__(self, other):          # 别忘了一起写
        return not self.__eq__(other)

b1 = Book("Python 入门", "张三", 45.0)
b2 = Book("Python 入门", "张三", 60.0)
b3 = Book("Python 入门", "李四", 45.0)
print(b1 == b2)    # True，书名作者相同，价格不同也算同一本书
print(b1 == b3)    # False
```

第 8 章 自测题 9

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age)     # 第一行先初始化父类部分
        self.student_id = student_id

s = Student("李四", 20, "2026001")
print(s.name, s.age, s.student_id)      # 李四 20 2026001
```

第 8 章 自测题 10 因为 Python 不会自动由 `__eq__` 推导出 `__ne__`。如果不重载 `__ne__`，`!=` 会沿用默认的“比较两个对象是否为同一对象”的行为，可能出现 `a == b` 是 `True` 而 `a != b` 也是 `True` 这种自相矛盾的结果。标准写法是 `def __ne__(self, other): return not self.__eq__(other)`。

第 8 章 自测题 11 `__str__` 面向**用户**，要可读，被 `print(对象)`、`str(对象)`、f-string 的 `{对象}` 调用；`__repr__` 面向**开发者**，理想形式是“能重新创建这个对象的代码”，被直接敲对象名回车、`repr(对象)`、以及把对象放进 list/dict 打印时调用。两个都不定义时，`print(对象)` 会调用继承自 `object` 的 `__repr__`，输出形如 `<__main__.Circle object at 0x000001F2A1B0A5D0>` 的地址串，几乎没有可读性。

第 8 章 自测题 12 错在方法名：`__init__` **前后各是两个下划线**，这里写成了 `_init_`（前后各一个），所以它只是一条普通方法，Python 创建对象时不会调用它，属性 `radius` 从未被挂到对象上——`Circle` 于是退回到默认的无参初始化方法。报错信息原文是：

```
TypeError: Circle() takes no arguments
```

正确写法：把 `_init_` 改成 `__init__`（也可以顺手把 `self.radius` 改成 `self.__radius` 做成封装）。

```
class Circle:
    def __init__(self, radius=1):        # 前后各两个下划线，共 4 个
        self.radius = radius
```

```
c = Circle(5)
print(c.radius)      # 5
```

第 8 章 自测题 13 `type(obj)` 返回对象的**确切类**，**不考虑继承**：`Dog` 的实例满足 `type(d) == Dog`，但 `type(d) == Animal` 是 `False`。`isinstance(obj, 类)` 会**沿继承链查找**，所以 `isinstance(d, Animal)` 是 `True`；它的第二个参数还可以传元组，一次判断多个类型。判断一个对象是不是数字，应该写 `isinstance(x, (int, float))`——`bool` 是 `int` 的子类，所以 `isinstance(True, int)` 也是 `True`，用 `type(x) == int` 会漏掉 `float`、也容易判断出意料之外的结果。

第 8 章 自测题 14 三个用途是：**只读属性**、**写入时校验**、**计算属性**。

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius      # 走 setter，做校验

    @property
    def celsius(self):              # getter：只负责读
        return self.__celsius

    @celsius.setter
    def celsius(self, value):       # 用途二：写入时校验
        if value < -273.15:
            raise ValueError("低于绝对零度")
        self.__celsius = value

    @property
    def fahrenheit(self):           # 用途三：计算属性（由 celsius 算出来，不需要单独存）
        return self.__celsius * 9 / 5 + 32

t = Temperature(25)
print(t.celsius, t.fahrenheit)      # 25 77.0
t.celsius = 100                     # 走 setter
print(t.fahrenheit)                 # 212.0
```

而第**一**个用途“只读属性”的写法是：只写 `@property`、**不写 setter**，这时外部一赋值就报

```
AttributeError: property 'fahrenheit' of 'Temperature' object has no setter
```

（把属性名换成实际的只读属性名即可，例如 `fahrenheit`。）

第 8 章 自测题 15 原因是 `students` 定义在类里、又不是在 `__init__` 里创建的，所以它是**类属性**，全部实例共享同一个列表对象。`self.students.append(student)` 虽然写的是 `self`，但 `append` 是**原地修改**，改的正是那一份共享数据，于是 `c2` 也“看到”了 Tom。运行结果是 `['Tom'] ['Tom']`。

改正办法：把可变数据放进 `__init__`，用 `self.students = []` 让每个实例各有一份。

```
class Course:
    def __init__(self, name):
        self.name = name
```

```
        self.students = []          # 实例属性，每个对象一份

    def add(self, student):
        self.students.append(student)

c1 = Course("Python")
c2 = Course("Java")
c1.add("Tom")
print(c1.students, c2.students)     # ['Tom'] []
```

> 补充一句：如果这份数据**本来就想让所有对象共享**（比如计数器），那才写成类属性，而且最好用数字这类**不可变**类型，修改时写 `类名.属性名`。

第 9 章 自测题答案

> 对应正文 9.6 节，共 15 题。第 12、13、14 题是编程练习，给出完整可运行代码和实际运行结果。

第 9 章 自测题 1 模块就是一个 `.py` 源文件，解决"函数和变量重名"的问题，让功能可以复用；包就是一个含 `__init__.py` 的目录，解决"模块名重名"的问题，让大量模块可以组织管理。详见 9.1、9.2、9.3 节。

第 9 章 自测题 2 `import math` 之后必须写 `math.sqrt(9)`；`from math import sqrt` 之后直接写 `sqrt(9)`。前者更安全：一眼能看出这个名字的出处，也不会和自己的变量、函数重名；后者写起来短，但导入的名字多了容易撞车。不确定时一律用 `import math`。

第 9 章 自测题 3 因为它把模块里所有不以 `_` 开头的名字都塞进当前命名空间，可能覆盖你已有的同名变量或函数，而且**不报错**。例如你自己写了 `def sqrt(x): return int(x ** 0.5)`，再来一句 `from math import *`，`sqrt(9)` 就从 `3` 变成了 `3.0`。更极端的例子是某个模块里定义了 `print` 函数，一旦被星号导入，你的 `print` 就不再输出任何东西。

第 9 章 自测题 4 它用来区分"这个文件是独立运行的脚本"还是"被别的文件当作模块导入"。直接运行时 `__name__` 等于 `"__main__"`，条件成立；被导入时 `__name__` 等于模块名，条件不成立。去掉这一行，文件被 `import` 时里面的代码（读输入、打印测试）也会执行，产生额外输出甚至副作用；在 OJ 或自动化脚本里被导入时，还会直接报 `EOFError: EOF when reading a line`。

第 9 章 自测题 5 必须包含 `__init__.py`。这个文件**可以是空的**，空文件同样能起到"标识这是一个包"的作用；需要初始化配置时再往里写代码，它只在包被导入时执行一次。（Python 3.3 以后，没有它的目录会被当成"命名空间包"，多数情况下也能导入，但换到老环境、打包、直接运行包内文件时都会出问题，所以照规矩写。）

第 9 章 自测题 6 因为 `import pkg` 只导入了包本身（执行了 `pkg/__init__.py`），`a.py` 不会被自动导入，`pkg` 里没有 `a` 这个属性，报 `AttributeError: module 'pkg' has no attribute 'a'`。解决办法：① 在 `pkg/__init__.py` 里写 `import pkg.a`；② 使用时显式写 `import pkg.a` 或 `import pkg.a as a`。

第 9 章 自测题 7 `__all__` 控制 `from package import *` 时哪些子模块会被自动导入，写成列表：`__all__ = ["formats", "filters", "effects"]`。不写 `__all__` 时，只会导入 `__init__.py` 里定义的变量和被它显式导入的模块；`__all__` 里列了但实际不存在的模块，导入时照样报 `ImportError`。

第 9 章 自测题 8

```
# myutils.py
def is_leap(year):
    return year % 400 == 0 or (year % 100 != 0 and year % 4 == 0)

def days_in_month(year, month):
    days = (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)
    if month == 2 and is_leap(year):
        return 29
    return days[month - 1]
```

```
# main.py，与 myutils.py 放在同一个文件夹里
import myutils

print(myutils.is_leap(2000))            # True
print(myutils.days_in_month(2023, 2))   # 28
```

输出：

```
True
28
```

要点：两个文件必须放在同一个文件夹里；调用时写 `myutils.函数名`；`myutils.py` 内部调用 `is_leap` 不用写模块名。

第 9 章 自测题 9　按顺序在：① 当前执行的脚本所在的文件夹；② 环境变量 `PYTHONPATH` 里列出的文件夹；③ Python 安装目录下的标准库文件夹；④ 第三方库的 `site-packages` 文件夹。这些路径都在 `sys.path` 里，可以 `print(sys.path)` 查看。如果找不到，可以用 `sys.path.insert(0, r"自己的目录")` 把模块目录加进去；同时注意不要把自己的文件取成标准库的名字（如 `random.py`），否则它会盖住标准库。

第 9 章 自测题 10　例如：`numpy`（科学计算、数组运算）、`pandas`（数据分析、表格处理）、`matplotlib`（绘图）、`requests`（发送 HTTP 请求）、`flask` / `django`（Web 开发）、`pyinstaller`（打包成可执行文件）。答出任意三个并说清用途即可，安装方式是 `pip install 包名`。

第 9 章 自测题 11　报 `NameError: name 'math' is not defined`。因为 `from math import sqrt` 只把 `sqrt` 这一个名字拿进了当前命名空间，并没有引入 `math` 这个名字。改法有两种：写成 `import math` 再用 `math.sqrt(9)`，或者保留 `from math import sqrt` 直接写 `sqrt(9)`，两种写法不能混用。

第 9 章 自测题 12

```
# mymath.py
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
```

```
def prime_factors(n):
    factors = []
    i = 2
    while i * i <= n:
        while n % i == 0:
            factors.append(i)
            n //= i
        i += 1
    if n > 1:
        factors.append(n)
    return factors

if __name__ == "__main__":
    num = int(input())
    print(is_prime(num))
    print(prime_factors(num))
```

```
# main.py，与 mymath.py 放在同一个文件夹里
import mymath

num = int(input())
print(mymath.is_prime(num))
print(mymath.prime_factors(num))
```

输入 `84`，输出：

```
False
[2, 2, 3, 7]
```

输入 `97`，输出 `True` 和 `[97]`。

要点：读输入和打印都放在 `if __name__ == "__main__":` 里面，所以 `import mymath` 时既不读输入也不产生任何输出——这正是题目要求的"可被复用"。

第 9 章 自测题 13　目录结构：

```
mytools/
  |-- __init__.py      # 空文件
  `-- mymath.py        # 内容与第 12 题的 mymath.py 完全相同
main.py
```

```
# main.py，与 mytools 文件夹同级
from mytools.mymath import is_prime, prime_factors

num = int(input())
print(is_prime(num))
print(prime_factors(num))
```

输入 `84`，输出与第 12 题一致：

```
False
[2, 2, 3, 7]
```

要点：`main.py` 必须和 `mytools` 文件夹同级（脚本所在目录是搜索的第一站）；也可以写 `from mytools import mymath`，之后用 `mymath.is_prime(num)` 调用。

第 9 章 自测题 14

```
import json
from collections import Counter

nums = list(map(int, input().split()))
counter = Counter(nums)

top = counter.most_common(1)[0]
print(top[0], top[1])
print(json.dumps(dict(counter), ensure_ascii=False))
```

输入 `1 2 2 3 3 3`，输出：

```
3 3
{"1": 1, "2": 2, "3": 3}
```

要点：`Counter` 的键是整数，转成 JSON 后键会变成字符串（这是 JSON 格式的规定）；并列最多时 `most_common(1)` 取先出现的那个；不写 `ensure_ascii=False` 时中文会变成 `\uXXXX` 转义。

第 9 章 自测题 15 循环导入指两个模块互相 `import`：`a.py` 里写 `from b import fb`，`b.py` 里写 `from a import fa`，谁先运行谁报错：

```
ImportError: cannot import name 'fb' from partially initialized module 'b' (most likely due to
a circular import)
```

原因：`a.py` 执行第一行就去 `b` 里取 `fb`，而此时 `b.py` 才刚开始执行、`fb` 还没定义。两种解决办法：① 两边都只写 `import 模块名`，用到时再 `b.fb()` 这样取属性；② 把两个模块都需要的部分抽到第三个模块 `common.py`，`a`、`b` 都只依赖它（也可以把 `import` 挪进函数内部，用到时再导入）。

作业题答案

> **怎么用这一节**：先自己写，写完再对照。每道题都给了 **思路 → 代码 → 测试数据 → 易错点** 四块。 下面的代码全部在 Python 3 上实测通过，测试数据里的输出都是真实运行结果，可以直接拿去自测。
>
> **提交 OJ 之前必查三件事**：① 输入读了几行、每行几个数，和你写的 `input()` 对不对得上；② 有没有多打印东西（比如 `请输入` 这样的提示语）；③ 输出的行数、顺序、小数位数和样例是否完全一致。

作业题 1 答案｜三角形的周长和面积

> **考点**：第 1 章 1.8 算术运算、1.9 常用内置函数与 math 库、第 7 章 7.10 字符串格式化

思路

1. 三个数一行一个，所以写三次 `float(input())`。要用 `float` 而不是 `int`，因为边长"不超过 2 位小数"。
2. 海伦公式：先求半周长 `p = (a + b + c) / 2`，再求面积 `sqrt(p(p-a)(p-b)(p-c))`。开方用 `math.sqrt()`。
3. 输出都保留 2 位小数，用 `f"{值:.2f}"`。题目提示里的 `"{0:.2f}".format(x)` 是同一个东西的老写法，现在一律用 f-string。

参考代码

```
import math

a = float(input())
b = float(input())
c = float(input())

p = (a + b + c) / 2                          # 半周长
area = math.sqrt(p * (p - a) * (p - b) * (p - c))

print(f"{a + b + c:.2f}")                    # 先输出周长
print(f"{area:.2f}")                         # 再输出面积
```

测试数据

输入	输出	说明
`3` `4` `5`	`12.00` `6.00`	题目样例
`1` `1` `1`	`3.00` `0.43`	等边三角形
`6` `8` `10`	`24.00` `24.00`	周长和面积数值刚好相同，别以为程序写错了

易错点

- ✗ `print(a + b + c:.2f)` —— 语法错误，冒号只能出现在 f-string 的花括号里。
- ✗ 写成 `f"{area:2f}"` —— 少了小数点，`2f` 会被当成"宽度 2"。
- ✗ 用 `round(area, 2)` —— `round` 会把末尾的 0 丢掉，`6.0` 输出成 `6.0` 而不是 `6.00`，判 **WA**。
- ✗ 只在面积上保留两位小数，周长直接 `print(a + b + c)` —— 样例要求两行都是两位小数。

作业题 2 答案｜经过一段时间后的时刻

> **考点**：第 1 章 1.8 整数除法 `//` 与取余 `%`

思路

时和分是 60 进制，不能直接加减。做法是**统一换算成分钟**再算，最后换回来：

1. 起始总分钟数 `total = h * 60 + m`；
2. 加上（可能是负的）时长 `d`；
3. `total // 60` 是小时，`total % 60` 是分钟。

题目保证结果不会早于 0 点、也不会超过 24 点，所以不用处理跨天。

参考代码

```
h = int(input())
m = int(input())
d = int(input())

total = h * 60 + m + d          # 全部换成分钟

print(total // 60)              # 小时
print(total % 60)               # 分钟
```

测试数据

输入	输出	说明
`9` `5` `170`	`11` `55`	题目样例
`0` `0` `0`	`0` `0`	边界：整点
`23` `59` `-1439`	`0` `0`	时长为负，退到当天 0 点
`0` `0` `90`	`1` `30`	从 0 点往后 90 分钟

易错点

- ✗ 写成 `h + d // 60` 和 `m + d % 60` —— 分钟超过 60 不会进位。`9:05 + 170` 碰巧对，换成 `9:50 + 20` 就错成 `9:70`。
- ✗ 用 `/` 代替 `//` —— `total / 60` 得到 `11.9166...`。
- ✗ 担心负数？Python 的 `//` 是**向下取整**，`%` 的结果**符号永远跟除数一致**，正好符合"时刻"的语义（第 1 章 1.8 讲过）。本题结果非负用不到，但知道没坏处。

作业题 3 答案 | 水仙花数

> **考点**：第 1 章 1.8 取余 `%` 与整除 `//`、1.12 比较运算

思路

把一个三位数拆成百位、十位、个位，这是**数字拆分**的基本功：

- 百位：`n // 100`
- 十位：`n // 10 % 10`（先去掉个位，再取末位）
- 个位：`n % 10`

然后直接 `print(判断表达式)`。Python 打印布尔值时输出 `True` / `False`（首字母大写），正好符合要求。

参考代码

```
n = int(input())

a = n // 100            # 百位
b = n // 10 % 10        # 十位
c = n % 10              # 个位

print(a ** 3 + b ** 3 + c ** 3 == n)
```

测试数据

输入	输出	说明
`153`	`True`	题目样例，$1^3+5^3+3^3 = 153$
`255`	`False`	题目样例
`370`	`True`	370 = 27 + 343 + 0
`100`	`False`	最小的三位数

易错点

- ✗ `b = n // 10` —— 那是前两位，不是十位，要再 `% 10`。
- ✗ `print(a ** 3 + b ** 3 + c ** 3 = n)` —— 判断相等要用 `==`，单个 `=` 是赋值。
- ✗ `print("True")` 写死字符串 —— 那 `255` 也会输出 `True`。
- ✗ `print(true)` —— Python 的布尔值首字母必须大写。
- 顺带一提：`**` 的优先级比 `+` 高，所以 `a ** 3 + b ** 3` 不用加括号。

作业题 4 答案 | 四位回文数

> **考点**：第 1 章 1.8 数字拆分、1.12 逻辑运算 `and`

思路

四位回文数的定义是"千位 = 个位 **并且** 百位 = 十位"。拆位的方法和上一题一样，只是多一层：`n // 1000`、`n // 100 % 10`、`n // 10 % 10`、`n % 10`。

参考代码

```
n = int(input())

a = n // 1000           # 千位
b = n // 100 % 10       # 百位
c = n // 10 % 10        # 十位
d = n % 10              # 个位
```

```
print(a == d and b == c)
```

测试数据

输入	输出	说明
`4386`	`False`	题目样例
`1331`	`True`	题目样例
`1001`	`True`	中间两位是 0 也算
`9999`	`True`	四位全同

易错点

- ✗ `a == d and b == c == d` —— 多写一个条件，逻辑就错了。
- ✗ 用 `or` 代替 `and` —— 只要有一位对上就返回 True，`4386` 会被误判。
- **进阶写法**：`print(str(n) == str(n)[::-1])` 更短，而且对任意位数都适用（第 4 章 4.3 切片）。本题在第 2 章之前布置，所以答案用拆位法，两种都要会。

作业题 5 答案 | 判断两个直角三角形是否相似

> **考点**：第 1 章 1.12 逻辑运算、**"能用乘法就别用除法"的数值思维**

思路

题目给的条件是 `a : b = c : d` 或 `a : b = d : c`。

如果真按比例写就是 `a / b == c / d`。**不要这么写**，因为除法会产生浮点数，`1/3` 在计算机里存的是近似值，比较时可能不相等。

把比例式**交叉相乘**变成乘法：`a / b == c / d` 等价于 `a * d == b * c`。整数乘法永远精确。

参考代码

```
a = int(input())
b = int(input())
c = int(input())
d = int(input())

print(a * d == b * c or a * c == b * d)
```

测试数据

输入	输出	说明
`1` `2` `3` `6`	`True`	题目样例，1:2 = 3:6
`1` `2` `6` `3`	`True`	题目样例，边的顺序颠倒也算相似
`1` `2` `3` `4`	`False`	题目样例

输入	输出	说明
`2 3 4 6`	`True`	2:3 = 4:6

易错点

- ✗ `print(a / b == c / d or a / b == d / c)` —— 多数情况能过，遇到除不尽的比例就会因浮点误差判错。这是本题最大的坑。
- ✗ 只写 `a * d == b * c` —— 漏掉"两条边顺序可以颠倒"的要求，样例 2 会输出 False。
- `or` 的优先级比 `==` **低**，所以不用加括号也对；不过写成 `print((a * d == b * c) or (a * c == b * d))` 可读性更好。

作业题 6 答案 | 符号交替求和

> **考点**：第 2 章 2.5 `for` 循环与 `range()`、2.3 分支结构

思路

式子是 `1 + 2 - 3 + 4 - 5 + 6 - 7 ...`，规律是：

- 第 1 项 `+1`，第 2 项 `+2`；
- 从第 3 项开始，**奇数项减、偶数项加**。

所以先把 `total` 初始化为 1，再把 2 加上（前提是 `n >= 2`），剩下的从 3 循环到 n，用 `i % 2` 判断加减。

参考代码

```
n = int(input())

total = 1
if n >= 2:
    total += 2

for i in range(3, n + 1):
    if i % 2 == 1:      # 3、5、7…… 做减法
        total -= i
    else:               # 4、6、8…… 做加法
        total += i

print(total)
```

等价写法（更好想）

```
n = int(input())
total = 0
for i in range(1, n + 1):
    if i == 1 or i % 2 == 0:
        total += i
    else:
```

```
        total -= i
print(total)
```

测试数据

输入	输出	说明
`1`	`1`	边界：只有第 1 项
`2`	`3`	题目样例，1 + 2
`3`	`0`	1 + 2 − 3
`4`	`4`	1 + 2 − 3 + 4
`5`	`-1`	题目样例
`10`	`7`	验证循环部分

易错点

- ✗ `for i in range(1, n + 1)` 从头开始，又把第 1、2 项重复算了一遍。
- ✗ 用 `i % 2 == 0` 判断奇数 —— 正好搞反，`i % 2 == 1` 才是奇数。
- ✗ 忘了 `n = 1` 或 `n = 2` 的边界：`for i in range(3, n + 1)` 一次都不执行，若 `total` 又没赋初值就会 `NameError`。

作业题 7 答案 | a + aa + aaa + ...

> **考点**：第 2 章 2.5 `for` 循环、**"边生成边累加"的循环套路**

思路

关键是**每一项怎么由上一项得到**：`2 → 22 → 222 → 2222`，即"上一项 × 10 + a"。

所以用两个变量：`t` 存当前这一项，`s` 存累加和。循环 n 次，每次先更新 `t`，再把它加到 `s` 上。

参考代码

```
a = int(input())
n = int(input())

t = 0       # 当前这一项，如 2、22、222
s = 0       # 累加和

for i in range(n):
    t = t * 10 + a
    s += t

print(s)
```

测试数据

输入	输出	说明
`2` `5`	`24690`	题目样例
`1` `1`	`1`	最小情况
`9` `3`	`1107`	9 + 99 + 999

易错点

- ✗ 用字符串拼接 `t = t + str(a)` 再 `int()` —— 结果对但绕远路，忘了转回整数就变成字符串相加。
- ✗ `s = s + t` 写在 `t = t * 10 + a` 前面 —— 顺序反了，第一项会加 0。
- ✗ 循环里忘了先更新 `t` 就直接累加。

作业题 8 答案｜显示一个整数的所有素因子

> **考点**：第 2 章 2.5 `while` 循环、2.9 算法优化、**分解质因数模板**

思路

从最小的素数 2 开始试除：

1. 如果 `n % i == 0`，说明 `i` 是一个素因子，**打印它并把 n 除掉这个因子**；
2. 用 `while` 反复除，直到除不动为止（这样 120 会连续打印三个 2）；
3. `i` 加 1，继续试。

为什么只需要试到 `sqrt(n)`：如果 `i * i > n`，说明 n 已经没有不超过 `sqrt(n)` 的因子了，剩下的 n 本身一定是素数。循环结束后如果 `n > 1`，把它打印出来即可。这个"边除边缩小范围"的技巧叫**试除法**，是很多数论题的基础。

参考代码

```
n = int(input())

i = 2
while i * i <= n:
    while n % i == 0:       # 把因子 i 除干净
        print(i)
        n //= i
    i += 1

if n > 1:                   # 剩下的 n 是最后一个素因子
    print(n)
```

测试数据

输入	输出	说明
`120`	`2` `2` `2` `3` `5`	题目样例，120 = 2³ × 3 × 5
`7`	`7`	素数只输出自己

输入	输出	说明
`2`	`2`	最小素数
`100`	`2 2 5 5`	100 = $2^2 \times 5^2$
`97`	`97`	素数

易错点

- ✗ 内层用 `if n % i == 0` 而不是 `while` —— 120 只会输出一个 2。
- ✗ 忘了循环后面那句 `if n > 1: print(n)` —— 输入 `7` 时什么都不输出。
- ✗ 循环条件写成 `i <= n` —— 结果对，但输入一个大素数会非常慢，OJ 判 TLE（超时）。
- 想再快一点：试完 2 之后可以直接从 3、5、7…… 开始（跳过偶数）。本题数据规模小，不优化也能过。

作业题 9 答案 | 反素数

> **考点**：第 3 章 3.2 函数的定义与调用、第 4 章 4.3 切片 `[::-1]`

思路

题目说"非回文数"才算反素数，所以要分三步：

1. 写一个 `is_prime(x)` 判断素数（就是第 3 章例题 3-5，直接拿来用）；
2. 把数字转成字符串，用切片 `s[::-1]` 反转；
3. 如果 `s == s[::-1]`（本身是回文）就跳过；否则判断原数和反转后的数是否都是素数。

参考代码

```
def is_prime(x):
    """判断 x 是否为素数。"""
    if x < 2:
        return False
    i = 2
    while i * i <= x:       # 只需试到 sqrt(x)
        if x % i == 0:
            return False
        i += 1
    return True


a = int(input())
b = int(input())

for n in range(a, b + 1):
    s = str(n)
    if s == s[::-1]:        # 本身是回文数，跳过
        continue
    if is_prime(n) and is_prime(int(s[::-1])):
        print(n)
```

测试数据

输入	输出	说明
`2` `50`	`13` `17` `31` `37`	题目样例
`2` `100`	`13` `17` `31` `37` `71` `73` `79` `97`	扩大范围

为什么 11 不在结果里：11 本身是素数，反转后还是 11，也是素数 —— 但 11 是回文数，题目明确排除。

易错点

- ✗ 忘了排除回文数 —— 结果会多出 11、101 这类数。
- ✗ `is_prime(s[::-1])` 忘了 `int()` —— 字符串传进去会在 `x < 2` 处报 `TypeError`。
- ✗ `range(a, b + 1)` 写成 `range(a, b)` —— 漏掉上界 b。
- ✗ 把 `is_prime` 定义在循环里面 —— 能跑，但每次循环都重新定义函数，浪费且不优雅。

作业题 10 答案｜点是否在圆内

> **考点**：第 2 章 2.3 多分支 `if/elif/else`、第 6 章 6.1 `math` 模块、**浮点数比较**

思路

圆心在原点，点到圆心的距离就是 `d = sqrt(x*x + y*y)`（勾股定理）。

- `d < r`：在圆内，输出 1；
- `d = r`：在圆上，输出 0；
- `d > r`：在圆外，输出 -1。

关键点：判断"相等"**绝不能用 `==`**。`math.sqrt` 算出来的是小数，题目提示也说了：**两个浮点数之差的绝对值小于 0.000001 就算相等**。所以写成 `abs(d - r) < eps`，其中 `eps = 1e-6`。

参考代码

```
import math

x = float(input())
y = float(input())
r = float(input())

d = math.sqrt(x * x + y * y)
eps = 1e-6                          # 0.000001

if d < r - eps:
    print(1)                        # 圆内
elif abs(d - r) < eps:
    print(0)                        # 圆上
else:
    print(-1)                       # 圆外
```

> **更简洁的写法**：`math.hypot(x, y)` 就是"求直角三角形斜边"，比手写 `sqrt(x*x + y*y)` 更稳（中间不会溢出）：
>
> ```
> d = math.hypot(x, y)
> ```

测试数据

输入	输出	说明
`4 5 10`	`1`	题目样例，d ≈ 6.40 < 10
`9 9 10`	`-1`	题目样例，d ≈ 12.73 > 10
`3 4 5`	`0`	正好在圆上，专门验证 eps 判断
`0 0 5`	`1`	圆心一定在圆内

易错点

- ✗ `elif d == r:` —— 浮点数用 `==` 十有八九判不出来，测试数据第 3 行就会输出 `-1`。
- ✗ 第一个条件只写 `d < r`，没减 eps —— 边界情况的归属会变得模糊。
- ✗ 写成 `print("1")` —— 输出字符串和数字在样例里看起来一样，但严格判题可能出问题，养成输出数字的习惯。

作业题 11 答案 | 距当月月底还有几天

> **考点**：第 2 章 2.3 分支、2.7 闰年判断、第 4 章 4.4 元组

思路

1. 用元组存下每个月的"平年天数"；
2. 只有 2 月需要按闰年修正：闰年 29 天，平年 28 天；
3. `该月总天数 - 输入日` 就是距月底的天数（题目说不含输入这一天，正好不用再加 1）。

闰年判断规则（第 2 章例题 2-3 讲过）：能被 400 整除，**或者**能被 4 整除但不能被 100 整除。

参考代码

```
year = int(input())
month = int(input())
day = int(input())

days_of_month = (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)

if month == 2 and (year % 400 == 0 or (year % 100 != 0 and year % 4 == 0)):
    total = 29
else:
    total = days_of_month[month - 1]        # 注意减 1，元组下标从 0 开始
```

```
print(total - day)
```

测试数据

输入	输出	说明
`2023 2 26`	`2`	题目样例，平年 2 月共 28 天
`2022 3 3`	`28`	题目样例，3 月共 31 天
`2023 1 31`	`0`	边界：当月最后一天
`2024 2 1`	`28`	2024 是闰年，2 月有 29 天
`2000 2 1`	`28`	2000 是世纪闰年（能被 400 整除）

易错点

- ✗ `days_of_month[month]` —— 元组下标从 0 开始，必须减 1。
- ✗ 闰年判断只写 `year % 4 == 0` —— 1900 年会误判成闰年。完整规则是 `y % 400 == 0 or (y % 100 != 0 and y % 4 == 0)`。
- ✗ 最后写成 `print(total - day + 1)` —— 题目明确说"计算时不含输入的这一天"，加 1 就错了。
- ✗ `if month == 2:` 后面忘了判断闰年，直接 `total = 29`。

作业题 12 答案｜这一天是当年的第几天

> **考点**：第 2 章 循环累加、闰年判断、第 4 章 4.3 切片

思路

第几天 = 本月的日 + **前面所有月份的天数之和** + 闰日。

1. 先把 `day` 作为初值；
2. 把 1 月到上个月的天数累加起来；
3. 如果月份大于 2 而且是闰年，再加 1 天。

为什么是"月份大于 2"：闰年多出来的那天是 2 月 29 日。输入的是一月或二月时，这一天还没到，不该加。

参考代码

```
year = int(input())
month = int(input())
day = int(input())

days_of_month = (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)

days = day
for i in range(month - 1):          # 累加前 month-1 个月
    days += days_of_month[i]
```

```
if month > 2 and (year % 400 == 0 or (year % 100 != 0 and year % 4 == 0)):
    days += 1                       # 闰年补一天

print(days)
```

> **更简洁的写法**：用切片 + `sum` 一行代替那个循环。
>
> ```
> days = day + sum(days_of_month[:month - 1])
> ```

测试数据

输入	输出	说明
`2000 8 31`	`244`	题目样例，世纪闰年
`2022 3 31`	`90`	题目样例，31 + 28 + 31
`2023 1 1`	`1`	边界：第一天
`2024 12 31`	`366`	闰年最后一天，验证全年 366 天

易错点

- ✗ `for i in range(month)` —— 多算一个月（把本月也算进去了）。
- ✗ 忘了 `if month > 2` —— 输入 `2024 1 1` 会输出 2。
- ✗ 闰年判断漏掉 `year % 100 != 0` —— 1900 年会多算一天。
- 直接查表 `[0,31,59,90,...][month-1] + day` 也是对的，但闰年还是要单独处理，反而更容易乱。用循环或切片更不容易错。

作业题 13 答案 | 三个整数升序输出

> **考点**：第 2 章 2.3 分支、第 1 章 1.4 同时赋值

思路

三个数排序，用**三次两两比较**就够了：

1. 比较 a 和 b，让 a 成为较小的；
2. 比较 a 和 c，此时 a 已经是三者中最小的；
3. 比较 b 和 c，把剩下两个排好。

交换两个变量用 Python 特有的同时赋值 `a, b = b, a`，**不需要中间变量**。

参考代码

```
a = int(input())
b = int(input())
c = int(input())
```

```
if a > b:
    a, b = b, a
if a > c:
    a, c = c, a
if b > c:
    b, c = c, b

print(a)
print(b)
print(c)
```

测试数据

输入	输出	说明
`99 -8 0`	`-8 0 99`	题目样例
`1 1 1`	`1 1 1`	三个相同
`5 4 3`	`3 4 5`	完全逆序

易错点

- ✗ `a, b = b, a` 写成 `a = b` 然后 `b = a` —— 第一个赋值就把 a 冲掉了，结果两个都变成 b。
- ✗ 用 `t = a; a = b; b = t` —— 对，但太啰嗦，Python 里请用同时赋值。
- ✗ 只顾着比较 a、b，忘了第三步 `if b > c`。
- **进阶写法**：学了第 4 章的列表之后，这题一行就能搞定：`print(*sorted([a, b, c]), sep='\n')`。不过考试要求你用分支写。

作业题 14 答案｜最大值及其出现次数

> **考点**：第 2 章 2.4 `while` 循环、2.6 `break`

思路

题目没说一共几个数，只说"以 0 结束"，所以要用 `while True` + `break` 的**哨兵循环**：

1. 读到 0 就 `break`；
2. 如果比当前最大值还大，**更新最大值并把计数重置为 1**；
3. 如果正好等于当前最大值，计数加 1。

"计数重置为 1"是本题的核心：出现更大的数时，之前那个最大值的计数就作废了。

参考代码

```
mx = None           # 还没读到任何数
count = 0

while True:
    x = int(input())
```

```
    if x == 0:
        break
    if mx is None or x > mx:
        mx = x
        count = 1       # 换了新最大值，重新计数
    elif x == mx:
        count += 1

print(mx)
print(count)
```

测试数据

输入	输出	说明
`3 4 -2 5 2 5 0`	`5 2`	题目样例；注意两个 5 中间夹着别的数
`7 0`	`7 1`	只有一个数
`-5 -5 0`	`-5 2`	全是负数

易错点

- ✗ `count += 1` 写在 `if x > mx` 里面却忘了先重置 —— 结果会变成"一共读了多少个数"。
- ✗ `mx = 0` 作为初值 —— 输入全是负数时会输出 0。应该用 `None`，或者把读到的第一个数当初值。
- ✗ 用 `if x >= mx: count += 1` —— 第一个数进来时 `count` 还是 0，会漏算。

作业题 15 答案 | 斐波那契数列

> **考点**：第 3 章 3.6 递归（以及为什么这里**不能**用递归）、第 2 章 循环

思路

斐波那契数列：`F(1)=1, F(2)=1, F(n)=F(n-1)+F(n-2)`，即 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

题目要求输出第 m 项到第 n 项。做法是**用两个变量滚动向前**：

- `a` 是当前项，`b` 是下一项；
- 每轮先判断要不要输出 `a`（轮次 ≥ m），然后 `a, b = b, a + b`。

为什么不许用递归：递归版每算一项都要重复计算一大堆子问题，时间复杂度是指数级的。算到第 40 项就要几秒，第 50 项要几百年。题目提示说得很清楚：可能"递归过深、时间超时或内存溢出"。

参考代码

```
m = int(input())
n = int(input())

a, b = 1, 1                 # a 是当前项 F(i)，b 是下一项

for i in range(1, n + 1):
    if i >= m:
```

```
        print(a)
    a, b = b, a + b          # 两个变量一起往前滚
```

测试数据

输入	输出	说明
`6` `10`	`8` `13` `21` `34` `55`	题目样例
`3` `7`	`2` `3` `5` `8` `13`	从第 3 项开始
`2` `2`	`1`	只输出一项

易错点

- ✗ 真的写了递归 —— 样例能过，一提交就 TLE（超时）。
- ✗ `for i in range(m, n + 1)` 直接从 m 开始循环 —— 此时 `a`、`b` 还没滚到第 m 项，输出的全是前几项。
- ✗ 把 `a = b` 和 `b = a + b` 分成两行 —— 第一行执行完 `a` 就变了，第二行算出来是错的。**必须**写成 `a, b = b, a + b`（右边先全部算完，再一起赋值）。
- ✗ 用列表存下所有项再切片输出 —— 结果对，但 n 很大时占内存。

作业题 16 答案 | 两个分数的四则运算

> **考点**：第 3 章 3.2 函数、第 2 章 辗转相除法、**约分与符号规范**

思路

分数四则运算的公式：

运算	分子	分母
加法	`n1*d2 + n2*d1`	`d1*d2`
减法	`n1*d2 - n2*d1`	`d1*d2`
乘法	`n1*n2`	`d1*d2`
除法	`n1*d2`	`d1*n2`

算完之后按题目要求"规范化"：

1. **符号**：分母是负数时，分子分母同时取反（`n, d = -n, -d`），保证负号只在分子上；
2. **约分**：分子分母同时除以最大公约数（用辗转相除法求）；
3. **特例**：结果为 0 时，分母输出 1。

参考代码

```
def gcd(a, b):
    """辗转相除法求最大公约数。"""
    a, b = abs(a), abs(b)
```

```
    while b != 0:
        a, b = b, a % b
    return a


n1 = int(input())       # 第一个分数的分子
d1 = int(input())       # 第一个分数的分母
op = int(input())       # 1 加、2 减、3 乘、4 除
n2 = int(input())       # 第二个分数的分子
d2 = int(input())       # 第二个分数的分母

if op == 1:
    n, d = n1 * d2 + n2 * d1, d1 * d2
elif op == 2:
    n, d = n1 * d2 - n2 * d1, d1 * d2
elif op == 3:
    n, d = n1 * n2, d1 * d2
else:
    n, d = n1 * d2, d1 * n2

if d < 0:               # 保证分母为正、负号在分子
    n, d = -n, -d

g = gcd(n, d)           # 约分
n, d = n // g, d // g

if n == 0:              # 结果为 0 时分母输出 1
    d = 1

print(n)
print(d)
```

测试数据

输入	输出	说明
`2 3 2 1 4`	`5 12`	题目样例：2/3 − 1/4 = 5/12
`1 2 1 1 3`	`5 6`	1/2 + 1/3 = 5/6
`1 2 4 2 3`	`3 4`	(1/2) ÷ (2/3) = 3/4
`1 2 2 1 2`	`0 1`	结果为 0，分母必须是 1
`2 3 3 3 2`	`1 1`	结果为整数 1，分母是 1

易错点

- ✗ 忘了 `if d < 0` 那一步 —— 比如 1/2 − 3/2 会输出 `-2` 和 `4`，而题目要求 `-1` 和 `2`。
- ✗ `gcd` 里没取绝对值 —— `n` 或 `d` 为负时辗转相除的结果可能为负，约分就乱套了。
- ✗ 结果为 0 时忘了把分母改成 1 —— 会输出 `0` 和 `12` 之类。
- ✗ 用 `fractions.Fraction` —— 标准库确实有这个类，但本题是练习手写，考试不给用。

- 提示：除法分支里如果 `n2` 是 0 会 `ZeroDivisionError`。题目数据保证是合法分数，不用担心。

作业题 17 答案 | 判断序列是否平衡

> **考点**：第 4 章 4.5 列表、**前缀和**技巧、多组测试数据的读法

思路

序列平衡的意思是：存在一个位置 i，使**前 i 个数的和**等于**剩下的数的和**。

暴力做法是对每个 i 都重新求一遍两边的和，是 $O(n^2)$。更好的做法是**前缀和**：

1. 先求出总和 `total`；
2. 从左往右累加 `prefix`，每加一个数就判断 `prefix == total - prefix`，也就是 `prefix * 2 == total`；
3. 有一个满足就输出 `Yes`。

用乘法代替除法，既避免浮点误差，也更快。整体时间复杂度 $O(n)$。

输入格式要注意，这是本题最容易读错的地方：

- 第一行是测试组数 `t`；
- 接下来每组**两行**：第一行是 `n`，第二行是**一行里的 n 个整数**（用空格隔开）。

所以读序列必须用 `list(map(int, input().split()))`，不能写 n 次 `input()`。

参考代码

```
t = int(input())

for _ in range(t):
    n = int(input())
    a = list(map(int, input().split()))     # 一行读完 n 个数

    total = sum(a)
    prefix = 0
    found = False

    for i in range(n - 1):                  # 最多切到倒数第二个数
        prefix += a[i]
        if prefix * 2 == total:
            found = True
            break

    print("Yes" if found else "No")
```

测试数据

输入	输出	说明
`3` / `3` / `1 2 3` / `4` / `2 3 1 4` / `5` / `1 2 3 4 5`	`Yes` `Yes` `No`	题目样例

输入	输出	说明
`1` / `2` / `5 5`	`Yes`	两项相等

易错点

- ✗ 用 `[int(input()) for _ in range(n)]` 读序列 —— 题目说这 n 个数在**一行**里，这样读会 `EOFError`。
- ✗ 循环写成 `for i in range(n)` —— i 最大只能到 n−2（后面至少留一个数）。写成 n 会让 `prefix` 等于总和，逻辑上就错了。
- ✗ 每组忘了重新初始化 `prefix` 和 `found` —— 会串到下一组。
- ✗ `print` 放在了 `for _ in range(t)` 循环**外面** —— 要输出 t 行，必须写在循环里面。
- ✗ 用 `prefix == total / 2` —— 浮点比较，本题数据下一般不出错，但习惯不好。

作业题 18 答案｜最大值及其出现位置

> **考点**：第 4 章 4.5 列表、4.3 索引

思路

1. 第一个数是数据个数 `n`，后面跟着 n 行、每行一个数；
2. 把 n 个数读进列表，求最大值；
3. **把所有**等于最大值的位置都打印出来：位置从 1 开始数，所以下标 `i` 要输出 `i + 1`。

要特别注意：样例 1 输出的是 `321`、`1`、`4` 三行 —— 最大值 321 出现在第 1 位和第 4 位，**两个位置都要输出**；样例 2 输出 `8`、`2`、`5`、`7`，也是所有位置。所以不能"找到一个就退出"。

参考代码

```
n = int(input())
nums = [int(input()) for _ in range(n)]

mx = max(nums)

print(mx)
for i in range(n):
    if nums[i] == mx:
        print(i + 1)            # 位置从 1 开始数
```

测试数据

输入	输出	说明
`4` `321` `-58` `89` `321`	`321` `1` `4`	题目样例 1
`7` `5` `8` `2` `4` `8` `2` `8`	`8` `2` `5` `7`	题目样例 2
`1` `7`	`7` `1`	只有一个数

易错点

- ✗ `print(i)` —— 下标从 0 开始，位置要从 1 开始，必须 `i + 1`。
- ✗ 找到一个就用 `break` 退出 —— 样例 1 只会输出 `1`，少一行。
- ✗ 用 `nums.index(mx) + 1` —— `index` 只返回**第一个**位置，样例 1 会少输出 `4`。
- ✗ 忘了先 `print(mx)` —— 题目要求"先输出最大值，再输出位置"。

作业题 19 答案｜最高分和最低分

> **考点**：第 4 章 4.5 列表、4.6 内置函数 `max` / `min`

思路

先把所有分数读进一个列表，然后直接调用内置的 `max()` 和 `min()`。内置函数是 C 语言实现的，比手写循环快得多，也不容易出错。

参考代码

```
n = int(input())
scores = [int(input()) for _ in range(n)]

print(max(scores))      # 最高分
print(min(scores))      # 最低分
```

测试数据

输入	输出	说明
`3` `88` `96` `65`	`96` `65`	题目样例
`1` `50`	`50` `50`	只有一个学生，最高最低分相同

易错点

- ✗ 边读边比较，但把两个初值都设成 0 —— 分数范围是 0~100，遇到全 0 分会出错。
- ✗ `print(min(scores), max(scores))` —— 顺序反了，题目要求"先最高分，再最低分"。
- ✗ `print(max(scores), min(scores))` —— 一行两个数，题目要求**一行一个**。

作业题 20 答案｜校门外的树

> **考点**：第 4 章 4.5 列表、4.12 函数与可变类型、**模拟法**

思路

这是一道经典的**模拟题**：不需要想数学公式，直接照着题目的意思"把树标出来"。

1. 马路上从 0 到 L 一共有 **L + 1** 棵树（这是最容易数错的地方）；
2. 建一个长度 `L + 1` 的列表，初始全是 1，表示"有树"；

3. 每读一个区间 `[u, v]`，就把这个区间里的树全标成 0（区间**包含两端**，所以是 `range(u, v + 1)`）；
4. 最后数一数还剩几个 1 —— 也就是 `sum(trees)`。

为什么用 0/1 列表而不是集合：区间可能重叠，用集合要额外处理，而 0/1 数组"重复置 0"完全无害，最简单也最稳。

参考代码

```
L, m = map(int, input().split())        # 第一行两个数

trees = [1] * (L + 1)                   # 0 到 L 一共 L+1 棵树

for _ in range(m):
    u, v = map(int, input().split())    # 每个区间一行
    for i in range(u, v + 1):           # 闭区间，含两端
        trees[i] = 0

print(sum(trees))                       # 剩下 1 的个数就是树的数量
```

测试数据

输入	输出	说明
`500 3` / `150 300` / `100 200` / `470 471`	`298`	题目样例（区间有重叠）
`5 1` / `1 3`	`3`	移走 3 棵，剩 3 棵

易错点

- ✗ `trees = [1] * L` —— 少了 1 棵。0 到 L 是 **L+1** 个整数点。
- ✗ `range(u, v)` —— 少移走了终点那棵树。题目说"包括区域端点处的两棵树"，必须 `v + 1`。
- 数据范围是 `L <= 10^4`、`m <= 100`，双重循环最多约 10^6 次操作，Python 轻松跑过。想更快可以用**差分数组（超纲，了解即可）**。

作业题小结：这 20 道题到底在考什么

题号	核心考点	一句话方法
1	海伦公式、格式化输出	`f"{x:.2f}"` 保留两位小数
2	单位换算	全部换成分钟再算
3	数字拆位	`n // 10 % 10` 取中间位
4	数字拆位 + 逻辑运算	千位=个位 **and** 百位=十位
5	比例判断	交叉相乘，别用除法
6	循环 + 分支	用 `i % 2` 决定加减
7	递推	每一项 = 上一项 × 10 + a
8	试除法	`while n % i == 0` 除干净

题号	核心考点	一句话方法
9	函数 + 字符串反转	`s[::-1]` 判断回文与反素数
10	浮点数比较	`abs(d - r) < 1e-6`
11	查表 + 闰年	月份天数放元组，只修 2 月
12	累加 + 闰年	前几个月的天数求和
13	三数排序	三次两两比较 + 同时赋值
14	哨兵循环	换了最大值要重置计数
15	递推 / 迭代	两变量滚动，不用递归
16	函数 + 辗转相除	先算、再规范符号、再约分
17	前缀和	`prefix * 2 == total`
18	列表遍历	位置从 1 开始，**所有**位置都要输出
19	内置函数	`max()` / `min()`
20	模拟	0/1 数组标记，区间是闭区间

如果只让你记 5 句话

1. 输入永远先想清楚"几行、每行几个数"，再决定用 `input()` 还是 `input().split()`。
2. 涉及浮点数的"相等判断"，一律用 `abs(a - b) < 1e-6`。
3. 涉及比例的判断，一律交叉相乘变成整数乘法。
4. 循环里出现"新的最大值/最小值"，记得**重置计数**。
5. 输出格式（行数、顺序、小数位数）和样例不一致，算法再对也是 0 分。

模拟题答案与解析

> 做完整套题再来看。每道题都给了**答案 + 为什么**，程序阅读题的运行结果都是在 Python 3 上实跑出来的。

一、单项选择题（第 1~16 题）

模拟题 1 答案：B　变量名不能以数字开头，所以 `2nd_score` 不合法。

模拟题 2 答案：A　`7 // 2 = 3`（地板除法，结果是整数），`7 / 2 = 3.5`（`/` 的结果**总是**浮点数），`7 % 2 = 1`。

模拟题 3 答案：B　空字符串视同 False；`'0'` 是**非空**字符串，视同 True。记住：`bool()` 只看"空不空"，不看内容写了什么。

模拟题 4 答案：D　`round(4.5)` 在"正好居中"时返回**偶数**（银行家舍入），所以是 4；`int(4.5)` 是向零截断，也是 4。顺手验证一下：`round(3.5)` 是 4、`round(5.5)` 是 6。

模拟题 5 答案：A `-7 % 3`：因为 `-7 // 3 = -3`，代入恒等式 `x == (x // y) * y + (x % y)` 得 `-7 - 3 * (-3) = 2`。而 `7 % -3 = -2`、`-7 // 3 = -3`、`7 // -3 = -3`。**结论：取余结果的符号跟除数一致，整除是向下取整。**

模拟题 6 答案：A `s[1:5:2]` 从下标 1 开始、到下标 4 结束（不含 5）、步长 2，取下标 1 和 3 的字符，即 `'y'` 和 `'h'`，结果是 `'yh'`。

模拟题 7 答案：D 集合中的元素是**不重复**的，所以 D 的说法错误。A、B、C 都正确（元组不可变，所以可以作为字典的键；列表不能）。

模拟题 8 答案：B 默认参数列表只在**函数定义时**创建一次，两次调用共享同一个列表，所以第一次输出 `[1]`，第二次输出 `[1, 2]`。这是"可变默认参数"的经典陷阱，正确写法是用 `None` 作默认值。

模拟题 9 答案：C `'w'` 是只写模式，文件存在时**清空原有内容**。想追加要用 `'a'`。

模拟题 10 答案：C `@classmethod` 的第一个参数 `cls` 是类本身；实例方法必须有 `self`；静态方法没有特殊参数；类方法不能访问实例属性。

模拟题 11 答案：A `:.3f` 表示"保留 3 位小数"，`3.14159` → `3.142`（第 4 位是 5，进位）。

模拟题 12 答案：B `1 / 0` 抛出 `ZeroDivisionError`，被 `except` 捕获 → 打印 `A`；`else` 子句只在 **try 没有异常**时执行，这里不执行；`finally` 子句**无论是否发生异常都会执行** → 打印 `C`。所以输出是两行 `A` 和 `C`。

模拟题 13 答案：B `__name__` 是 Python 自动设置的变量：直接运行文件时它的值是 `"__main__"`，被 `import` 时它的值是模块名。所以这句话的意思是"只有直接运行我时才执行下面的代码"。

模拟题 14 答案：A `range(5)` 是 0~4，其中偶数只有 0、2、4，平方后是 0、4、16。

模拟题 15 答案：A `{1, 1, 2, 3, 3}` 是集合，会自动去重得到 `{1, 2, 3}`，长度 3。B 是列表，不去重，长度 5。C 的字典字面量里 `'a'` 出现了两次，后者覆盖前者，最终是 `{'a': 2, 'b': 3}`，长度 2。D 的字符串长度是 5。

模拟题 16 答案：A 三个方法依次作用：`strip()` 去掉两端空格 → `'Hello, World'`；`lower()` 转小写 → `'hello, world'`；`replace("world", "python")` 替换 → `'hello, python'`。注意字符串方法**不会原地修改**，必须一环扣一环地接住返回值（这里靠方法链做到了）。

二、判断题（第 17~22 题）

模拟题 17 答案：T `=` 是赋值运算符（把右边放进左边），`==` 是等于比较运算符。

模拟题 18 答案：T `input()` 返回的永远是字符串，参与运算前必须用 `int()` / `float()` / `eval()` 转换。

模拟题 19 答案：F `0.1 + 0.2` 得到 `0.30000000000000004`，与 0.3 不相等。比较浮点数要用 `abs(a - b) < eps`。

模拟题 20 答案：F `for...else` 中的 else 只在循环**正常结束**（没有遇到 break）时才执行；一旦被 break 中断，else 就不执行。

模拟题 21 答案：F `list1.sort()` 是**原地修改**列表，返回的是 `None`。返回新列表的是 `sorted(list1)`。

模拟题 22 答案：F Windows 上不写 `encoding` 时，Python 会按系统默认编码（简体中文环境是 GBK）打开文件。用 GBK 去解码 UTF-8 的中文内容会报 `UnicodeDecodeError`。读写文本文件**一律写**

`encoding="utf-8"`。

三、程序阅读题（第 23~30 题）

模拟题 23 答案

```
15 3.75 3 3
```

`x = 10 + 5 = 15`；`15 / 4 = 3.75`（浮点除）；`15 // 4 = 3`（整除）；`15 % 4 = 3`（余数）。

模拟题 24 答案

```
1 2 4
```

`i = 3` 时 `continue` 跳过本次循环；`i = 5` 时 `break` 提前结束。最后那个 `print()` 输出一个换行。**注意行末有一个空格**（`end=' '` 留下的），考试时别漏写。

模拟题 25 答案

```
was
best
semit
```

`s[3:6]` 取下标 3~5，即 `was`；`s.split(' ')[3]` 是第 4 个单词 `best`；`s[::-1]` 反转整个字符串得 `"semit fo tseb eht saw tI"`，再取前 5 个字符得 `semit`。

模拟题 26 答案

```
[1, 2, 3, 4, 5, 6, 9]
[4, 1, 5]
2 2
```

`set` 先去重（两个 1 只剩一个），`sorted` 再升序；`lst[2:5]` 取下标 2、3、4，即 `[4, 1, 5]`；`lst.count(1)` 是 2，`lst.index(4)` 返回 4 首次出现的下标 2。

模拟题 27 答案

```
3 ['a', 'c', 'd']
8
acd
```

新增 `'d'`、删除 `'b'` 之后剩下三个键 `a`、`c`、`d`；值的和是 1 + 3 + 4 = 8；`for k in sorted(d)` 按键排序输出，`end=''` 让它们连在一起不换行，最后没有 `print()` 补换行。

模拟题 28 答案

```
1 1 5 55
```

这是**循环版**斐波那契：`a, b` 从 `(0, 1)` 开始滚动。`fib(1)` 循环体不执行，直接返回 `b = 1`；`fib(2)` 执行一次得到 `(1, 1)`，返回 1；`fib(5)` 返回 5；`fib(10)` 返回 55。

模拟题 29 答案

```
[('a', 3), ('n', 2), ('b', 1)]
```

`"banana"` 里 `a` 出现 3 次、`n` 出现 2 次、`b` 出现 1 次，`d` 是 `{'b': 1, 'a': 3, 'n': 2}`。排序键是 `(-次数, 字符)`：先按次数**降序**（用负数实现），次数相同时按字符**升序**。

模拟题 30 答案

```
24
[1, 1, 2, 6]
```

`f` 是递归求阶乘：`f(4) = 4 × 3 × 2 × 1 = 24`。列表推导式里 `range(4)` 是 0~3；`f(0)` 和 `f(1)` 都命中终止条件返回 1，`f(2) = 2`，`f(3) = 6`。

四、编程题参考答案（第 31~36 题）

模拟题 31 答案

```
import math

a = float(input())
b = float(input())
c = float(input())

p = (a + b + c) / 2
area = math.sqrt(p * (p - a) * (p - b) * (p - c))

print(f"{a + b + c:.2f}")
print(f"{area:.2f}")
```

要点：用 `float` 读；两行输出都要保留 2 位小数，用 `f"{x:.2f}"` 而不是 `round`（`round` 会丢掉末尾的 0）。

模拟题 32 答案

```
n = int(input())

count = 0
for i in range(1, n + 1):
    if i % 3 != 0 and i % 5 != 0:
        print(i)
        count += 1
print(count)
```

要点：注意 `n` 本身也要判断，所以 `range` 的终点是 `n + 1`。"既不能被 3 整除、也不能被 5 整除"用 `and` 连接两个"不等于 0"的条件。

模拟题 33 答案

```
s = input()

counts = {}
for ch in s:
    counts[ch] = counts.get(ch, 0) + 1

items = sorted(counts.items(), key=lambda kv: (-kv[1], kv[0]))
for ch, c in items:
    print(ch, c)
```

要点：`counts.get(ch, 0)` 是"没出现过就当 0 次"的简洁写法，比先 `if ch in counts` 判断更省事。排序键用 `(-次数, 字符)` 就同时实现了"次数从多到少"和"次数相同按字典序"。（`collections.Counter` 有现成写法，但考试要会手写。）

模拟题 34 答案

```
nums = []
while True:
    x = int(input())
    if x == 0:
        break
    nums.append(x)

print(max(nums))
print(min(nums))
print(max(nums) - min(nums))
```

要点：题目要求用列表保存所有数字。因为保证至少输入一个非零整数，列表一定非空，直接调用 `max` / `min` 是安全的。

模拟题 35 答案

```
class Rectangle:
    def __init__(self, width, height):
        if width <= 0 or height <= 0:
            raise ValueError("width and height must be positive")
        self.__width = width
        self.__height = height

    @property
    def width(self):
        return self.__width

    @property
    def height(self):
        return self.__height

    def area(self):
        return self.__width * self.__height
```

```
    def perimeter(self):
        return 2 * (self.__width + self.__height)

    def __str__(self):
        return f"Rectangle {self.__width} x {self.__height}"


r = Rectangle(3, 4)
print(r)                        # Rectangle 3 x 4
print(r.area(), r.perimeter())  # 12 14
```

要点：私有属性用两个下划线开头（`self.__width`）；`@property` 让外部可以用 `r.width` 读取但**不能直接赋值**，这就是"只读属性"；构造函数里做合法性检查并抛异常，这正是封装的实际意义。另外 `__str__` **必须返回字符串**，返回别的类型会报 `TypeError`。

模拟题 36 答案

```
a = list(map(int, input().split()))

b = sorted(set(a))                          # 去重 + 升序
print(" ".join(map(str, b)))                # 行尾无多余空格

print(f"{sum(a) / len(a):.2f}")             # 平均值保留 2 位小数
```

输入 `5 3 5 1 3 2`，输出：

```
1 2 3 5
3.17
```

要点：

- `set(a)` 去重，`sorted()` 升序，两步合成 `sorted(set(a))`。
- `" ".join(map(str, b))` 是"一行输出多个数且行尾没有多余空格"的标准写法。**不要**写成循环 `print(x, end=' ')`，那样行尾会多一个空格，严格判题会判 PE。
- 平均值用 `sum(a) / len(a)`，**注意要用原列表** `a`（原数据的个数才是分母），保留 2 位小数用 `f"{x:.2f}"`。

模拟卷失分自查表

分数段	说明	下一步做什么
90~100	基础很扎实	直接去刷 OJ 上的练习题，重点练速度和边界
75~89	概念基本清楚，个别细节丢分	把错题对应的**章节方法总结**再看一遍，尤其是易错清单
60~74	会写程序，但概念题失分多	回头重做各章自测题（答案在书末），重点看"程序阅读题"类型

分数段	说明	下一步做什么
60 以下	需要系统性重学	从第 1 章开始逐节重读，**每段代码都亲手敲一遍**；配合一门视频课同步学习

最常失分的五个点（统计规律）：① 浮点数比较与 `round`；② `//` 与 `%` 遇负数；③ 可变默认参数与列表别名；④ 输出格式（多空格、少小数位）；⑤ 递归与循环的选择。

最后的建议

到这里，整份讲义就结束了。最后送你四句话：

1. **讲义看两遍，不如代码敲一遍。** 每个章节的例子都自己敲一次。报错是好事，报错的过程就是学习的过程。
2. **错题比新题更值钱。** 做错的题一定要回头找到讲义里对应的那一小节，弄清楚为什么错。
3. **先把作业题做扎实。** 这 20 道题覆盖了基础、分支、循环、函数、列表几大块最常考的题型，全部弄懂，期末考试的基础分就稳了。
4. **动手写小程序。** 学完一章就写一个自己的小工具（记账、猜数字、单词统计……），比做十道题库题更有用。

考前一周怎么复习

时间	做什么	用讲义的哪一部分
第 1 天	把九章的「本章方法总结与易错清单」通读一遍，标记自己没把握的条目	每章最后一节
第 2~3 天	只做各章的**自测题**，做完对答案，错题回到对应小节	各章自测 + 书末《答案与解析》
第 4~5 天	把 20 道作业题**重写一遍**（不看答案），写完再对	各章作业题 + 书末《作业题答案》
第 6 天	限时 120 分钟做完整套期末模拟测试	《期末模拟测试》
第 7 天	只看错题和附录 B 的报错速查表，不再做新题	附录 B、自己的错题本

上机考试/在线判题当天的检查清单

1. 输入读了几行、每行几个数？和题目对得上吗？
2. 输出有没有多打印提示语？
3. 输出的行数、顺序、小数位数和样例完全一致吗？
4. 边界情况试了吗（最小值、最大值、只有一个元素、负数、全相同）？
5. 浮点数比较用了 `abs(a - b) < 1e-6` 吗？
6. 比例判断用了交叉相乘（乘法）而不是除法吗？

> 最后提醒一句：**考试时写不出来，往往不是不会算法，而是输入输出格式没读清楚。** 拿到题目先把"输入

> 几行、输出几行、每行什么格式"用笔在草稿纸上写下来，再动手。

祝你顺利通过考试，也祝你真的喜欢上写程序。

附录 A　Python 常用速查表

> 这一部分是**查阅用**的，不需要背。做题时对照着看，用多了自然记住。

A.1 运算符优先级（从高到低）

优先级	运算符	说明
1	`( )`	括号，最高，想先算就加括号
2	`**`	幂，**右结合**：`2 ** 3 ** 2` = `2 ** 9` = 512
3	`+x` `-x` `~x`	正负号、按位取反
4	`*` `/` `//` `%`	乘、除、整除、取余
5	`+` `-`	加、减
6	`<<` `>>`	移位
7	`&`	按位与
8	`^`	按位异或
9	`|`	按位或
10	`<` `<=` `>` `>=` `!=` `==`	比较（可以链式写：`0 <= x < 100`）
11	`not`	逻辑非
12	`and`	逻辑与
13	`or`	逻辑或
14	`=` `+=` `-=` `*=` `/=` …	赋值（最低）

> **【易错】** 记不住优先级时的万能解法：**加括号**。括号不花钱，猜错了要花时间调试。

A.2 数据类型

类型	关键字	可变?	字面量示例	典型用途
整数	`int`	否	`42`，`-7`，`0b1010`，`0o17`，`0xFF`	计数、下标
浮点数	`float`	否	`3.14`，`1e-6`，`2.0`	小数计算
复数	`complex`	否	`3+4j`	工程计算（本课几乎不用）
布尔	`bool`	否	`True`，`False`	判断结果
字符串	`str`	否	`"abc"`，`'abc'`，`f"{x}"`	文本
列表	`list`	**是**	`[1, 2, 3]`	有序可变集合
元组	`tuple`	否	`(1, 2, 3)`	有序不可变集合

类型	关键字	可变?	字面量示例	典型用途
字典	`dict`	**是**	`{"a": 1}`	键值对查找
集合	`set`	**是**	`{1, 2, 3}`	去重、交并差
空值	`NoneType`	否	`None`	表示"什么都没有"

> **【核心】 可变 / 不可变** 是全书最重要的一条分界线：
>
> - 不可变（int、float、str、tuple）：任何"修改"其实都是造了一个新对象。
> - 可变（list、dict、set）：方法会**原地改动**，且多个变量可能指向同一个对象。
>
> 第 4 章 4.12 节专门讲这件事。

A.3 内置函数速查（常用 30 个）

函数	作用	例子 → 结果
`print(x)`	输出	`print("a", "b")` → `a b`
`input(s)`	读一行字符串	`input()` → `"123"`
`int(x)`	转整数	`int("12")` → `12`
`float(x)`	转浮点	`float("1.5")` → `1.5`
`str(x)`	转字符串	`str(12)` → `"12"`
`bool(x)`	转布尔	`bool("")` → `False`
`list(x)`	转列表	`list("ab")` → `['a','b']`
`len(x)`	长度	`len("abc")` → `3`
`max/min(x)`	最大 / 最小	`max([3,1,2])` → `3`
`sum(x)`	求和	`sum([1,2,3])` → `6`
`abs(x)`	绝对值	`abs(-3)` → `3`
`round(x, n)`	四舍六入五取偶	`round(2.5)` → `2`
`pow(a, b)`	幂	`pow(2, 10)` → `1024`
`divmod(a, b)`	商和余	`divmod(7, 2)` → `(3, 1)`
`sorted(x)`	排序（返回新列表）	`sorted([3,1])` → `[1,3]`
`reversed(x)`	反向迭代器	`list(reversed([1,2]))` → `[2,1]`
`range(a, b, s)`	整数序列	`list(range(0,5,2))` → `[0,2,4]`
`enumerate(x, s)`	带下标遍历	`list(enumerate("ab",1))` → `[(1,'a'),(2,'b')]`
`zip(a, b)`	并行配对	`list(zip([1,2],"ab"))` → `[(1,'a'),(2,'b')]`
`map(f, x)`	逐元素映射	`list(map(int, ["1","2"]))` → `[1,2]`

函数	作用	例子 → 结果
`filter(f, x)`	逐元素过滤	`list(filter(None, [0,1,2]))` → `[1,2]`
`all(x)` / `any(x)`	全真 / 有真	`all([1,1])` → `True`
`type(x)`	类型	`type(1)` → `<class 'int'>`
`isinstance(x, T)`	是否某类型	`isinstance(1, int)` → `True`
`chr(i)` / `ord(c)`	编码互转	`ord('A')` → `65`
`hex/oct/bin(i)`	转各进制字符串	`hex(255)` → `'0xff'`
`open(...)`	打开文件	`open("a.txt", encoding="utf-8")`
`eval(s)`	把字符串当表达式算	`eval("1+2")` → `3`
`id(x)`	对象身份	用来验证"是不是同一个对象"

A.4 数字格式化（f-string 格式说明符）

完整语法：`{值:[填充][对齐][符号][#][0][宽度][,][.精度][类型]}`

写法	效果	例子
`f"{x:.2f}"`	保留 2 位小数	`3.14159` → `3.14`
`f"{x:8.2f}"`	宽度 8，右对齐	`" 3.14"`
`f"{x:<8.2f}"`	左对齐	`"3.14 "`
`f"{x:^8.2f}"`	居中	`" 3.14 "`
`f"{x:*^8.2f}"`	用 `*` 填充并居中	`"**3.14**"`
`f"{x:08.2f}"`	用 0 补足宽度	`"00003.14"`
`f"{n:,}"`	千分位分隔	`1234567` → `1,234,567`
`f"{p:.1%}"`	百分比	`0.856` → `85.6%`
`f"{n:#x}"`	十六进制带前缀	`255` → `0xff`
`f"{n:b}"`	二进制	`10` → `1010`
`f"{x:.3e}"`	科学计数	`3141.59` → `3.142e+03`
`f"{x:+.2f}"`	强制显示正负号	`3.1` → `+3.10`

> **【易错】** `f"{2.675:.2f}"` 得到的是 `2.67` 而不是 `2.68`。因为 2.675 在二进制里存的是 2.67499999…，格式化时直接截断了。要精确舍入请用 `decimal` 模块。

A.5 字符串方法（按用途分组）

用途	方法	说明
大小写	`s.upper()` `s.lower()`	转大写 / 小写
	`s.title()` `s.capitalize()`	每词首字母大写 / 首字母大写
	`s.swapcase()`	大小写互换
去空白	`s.strip()`	去两端空白（含 \n、\t）
	`s.lstrip()` `s.rstrip()`	只去左 / 只去右
	`s.strip("0")`	去两端的 `"0"` 字符
判断	`s.isdigit()` `s.isalpha()`	全是数字 / 全是字母
	`s.isalnum()` `s.isspace()`	数字或字母 / 全是空白
	`s.islower()` `s.isupper()`	全小写 / 全大写
查找	`s.find(t)`	返回下标，找不到返回 `-1`
	`s.index(t)`	返回下标，找不到**报错**
	`s.count(t)`	出现次数
	`t in s`	是否包含（推荐）
	`s.startswith(t)` `s.endswith(t)`	是否以…开头 / 结尾
拆分	`s.split()`	按**任意空白**切，丢弃空串
	`s.split(",")`	按 `","` 切，保留空串
	`s.split(",", 2)`	最多切 2 次，得到 3 段
	`s.splitlines()`	按行切
连接	`sep.join(列表)`	用 sep 连接（列表元素必须都是字符串）
替换	`s.replace(a, b)`	全部替换
	`s.replace(a, b, 2)`	只替换前 2 个
对齐	`s.center(10)` `s.ljust(10)` `s.rjust(10)`	居中 / 左 / 右对齐
	`s.zfill(5)`	左侧补 0 到宽度 5
编码	`s.encode("utf-8")`	字符串 → 字节
	`b.decode("utf-8")`	字节 → 字符串

> **【核心】** 字符串是**不可变**的：上面每一个方法都**返回新字符串**，原字符串不会变。忘了赋值回去，是新手最常见的 bug 之一。

```
s = "  hello  "
s.strip()          # ✗ 白写了，s 没变
s = s.strip()      # ✓ 必须接住返回值
```

A.6 列表方法（★ 表示原地修改，返回 None）

方法	作用	返回
`lst.append(x)` ★	尾部加一个元素	None
`lst.extend(seq)` ★	尾部批量追加	None
`lst.insert(i, x)` ★	在下标 i 处插入	None
`lst.remove(x)` ★	删除**第一个**值为 x 的元素	None
`lst.pop()` ★	弹出并返回**最后一个**	被删元素
`lst.pop(i)` ★	弹出并返回下标 i	被删元素
`del lst[i]` ★	删除下标 i（语句，不是方法）	—
`lst.clear()` ★	清空	None
`lst.sort()` ★	**原地**排序	None
`lst.reverse()` ★	**原地**反转	None
`lst.copy()`	浅拷贝	新列表
`lst.count(x)`	数 x 出现几次	int
`lst.index(x)`	x 首次出现的下标	int
`sorted(lst)`	排序	**新列表**
`reversed(lst)`	反转	迭代器

【常见错误】

```
# ✗ 错误写法
a = [3, 1, 2]
b = a.sort()       # b 是 None！a 被原地排好了
print(b)           # None
```

```
# ✓ 正确写法
a = [3, 1, 2]
a.sort()           # 原地排序，a 变成 [1, 2, 3]
b = sorted(a)      # 想要新列表用 sorted()
```

判断口诀：**名字里带 `sort`、`append`、`remove`、`insert`、`extend`、`reverse`、`clear` 的都是原地修改、返回 None。**

A.7 字典方法

方法	作用
`d[k]`	取值，键不存在会 `KeyError`
`d.get(k)`	取值，不存在返回 `None`
`d.get(k, 默认值)`	取值，不存在返回默认值（**最常用**）
`d[k] = v`	新增或修改
`del d[k]`	删除键
`d.pop(k)`	弹出并返回值
`k in d`	判断**键**是否存在
`d.keys()` `d.values()` `d.items()`	三个视图
`d.setdefault(k, v)`	不存在就设为 v 并返回 v
`d.update(d2)`	用 d2 覆盖更新
`len(d)`	键的个数

> 【易错】 `in` 测的是**键**，不是值。想判断值是否存在要先 `d.values()`。 遍历时增删键会 `RuntimeError: dictionary changed size during iteration`，要改成 `for k in list(d):`。

A.8 文件操作

模式	含义	文件不存在时	是否清空原内容
`"r"`	只读（默认）	报 `FileNotFoundError`	—
`"w"`	只写	创建新文件	**是**
`"a"`	追加写	创建新文件	否
`"r+"`	读写	报错	否
`"rb"` `"wb"`	二进制读写	—	—

```
# 读：逐行迭代，最省内存
with open("data.txt", "r", encoding="utf-8") as f:
    for line in f:
        line = line.rstrip("\n")     # 每行末尾都带 \n，记得去掉
        print(line)

# 写
with open("out.txt", "w", encoding="utf-8") as f:
    f.write("第一行\n")
    f.write(str(123) + "\n")          # 数字必须先转字符串
    f.writelines(["a\n", "b\n"])      # 注意：不会自动加换行
```

> **【核心】** 在 Windows 上读写中文文件，**永远写** `encoding="utf-8"`。不写会按系统默认的 GBK 打开，遇到 UTF-8 文件就 `UnicodeDecodeError`。

A.9 常用标准库

模块	常用内容
`math`	`pi` `e` `sqrt` `ceil` `floor` `fabs` `gcd` `lcm` `hypot` `isclose` `inf`
`random`	`random` `randint` `randrange` `choice` `sample` `shuffle` `seed`
`time`	`time` `localtime` `strftime` `strptime` `sleep` `perf_counter`
`datetime`	`datetime` `date` `timedelta` `strftime` `strptime`
`os` / `os.path`	`getcwd` `listdir` `mkdir` `remove` `exists` `join` `basename`
`sys`	`stdin` `stdout` `argv` `path` `exit` `setrecursionlimit`
`csv`	`reader` `writer` `DictReader` `DictWriter`
`re`	`match` `search` `findall` `sub` `split` `compile`
`collections`	`Counter` `defaultdict` `deque` `namedtuple`
`decimal`	`Decimal`（精确小数，处理钱）

附录 B　30 个新手报错速查：报错 → 原因 → 处方

> 看到报错先做两件事：① **读最后一行**（那里写着异常类型和原因）；② **看它给的行号**（问题通常就在那一行或上一行）。

B.1 语法类（程序根本跑不起来）

#	报错 / 现象	原因	处方
1	`SyntaxError: invalid character ', '`	用了中文全角逗号、引号、括号	标点切英文输入法；中文只出现在字符串和注释里
2	`IndentationError: unexpected indent`	多缩进了一格，或 Tab 与空格混用	统一用 4 个空格；编辑器里打开"Tab 转空格"
3	`IndentationError: expected an indented block`	`if` / `for` 后面没缩进	冒号后面必须有缩进的内容
4	`SyntaxError: expected ':'`	`if` / `for` / `while` / `def` / `class` 行尾漏冒号	补上 `:`
5	`SyntaxError: invalid syntax`（在条件处）	把 `==` 写成了 `=`	判断相等用 `==`，赋值才用 `=`
6	`SyntaxError: unmatched ')'`	括号不配对	从最内层往外数一遍

B.2 类型类

#	报错	原因	处方
7	`TypeError: can only concatenate str (not "int") to str`	字符串直接和数字相加	`f"年龄：{18}"` 或 `"年龄：" + str(18)`
8	`TypeError: unsupported operand type(s) for +: 'int' and 'str'`	忘了把 `input()` 转成数字	`n = int(input())`
9	输入 `5 6` 后结果变成 `56`	`input()` 返回的是字符串	转换类型，或用 `map(int, input().split())`
10	`ValueError: invalid literal for int() with base 10: '3.5'`	`int()` 不接受带小数点的字符串	用 `float("3.5")` 或 `int(float("3.5"))`
11	`AttributeError: 'int' object has no attribute 'split'`	对数字调用了字符串方法	先 `str(x)`，或者检查变量到底读进来的是什么
12	`TypeError: 'list' object is not callable`	变量名把内置函数覆盖了（比如 `list = [...]`）	换个变量名，别用 `list`、`sum`、`str` 当变量

B.3 容器类

#	报错	原因	处方
13	`IndexError: list index out of range`	下标越界；以为下标从 1 开始	下标从 0 开始；最后一个用 `a[-1]`
14	`KeyError: 'name'`	字典里没有这个键	`d.get("name", 0)` 或先 `if "name" in d:`
15	`b = a.append(1)` 之后 `b` 是 `None`	`append` 等原地方法返回 None	先 `a.append(1)`，再 `b = a`
16	`b = a.sort()` 之后 `b` 是 `None`	同上	原地用 `a.sort()`；要新列表用 `b = sorted(a)`
17	`s.strip()` 之后 `s` 没变	字符串不可变，方法返回新串	`s = s.strip()`
18	删列表元素时"跳着删"	遍历中修改了列表长度	`for x in a[:]:` 或 `a = [x for x in a if 条件]`
19	`RuntimeError: dictionary changed size during iteration`	遍历字典时增删键	`for k in list(d):`
20	改了 `b`，`a` 也跟着变	`b = a` 只是起了个别名	要独立副本用 `b = a.copy()`；嵌套结构用 `copy.deepcopy(a)`
21	函数多次调用后列表越来越长	可变默认参数 `def f(lst=[])` 只求值一次	`def f(lst=None):` 内部再 `if lst is None: lst = []`
22	`TypeError: sequence item 0: expected str instance, int found`	`join` 里混了数字	`",".join(map(str, nums))`

B.4 函数与作用域

#	报错 / 现象	原因	处方
23	函数返回 `None`	忘了写 `return`	`def add(a, b): return a + b`
24	`UnboundLocalError: local variable 'x' referenced before assignment`	函数里给 x 赋值，它就变成局部变量	用参数 / 返回值传递；必要时 `global x`（不推荐）
25	`return` 后面的代码不执行	`return` 立即结束函数	把要先做的事移到 `return` 之前
26	函数里 `lst = []` 之后外面的列表没变	重新绑定形参不影响实参	原地 `lst.clear()` / `lst.extend(...)`，或用 `return` 返回
27	`RecursionError: maximum recursion depth exceeded`	递归没有终止条件，或层数超过 1000	检查终止条件；能改成循环就改循环

B.5 运行时与环境

#	报错	原因	处方
28	`ZeroDivisionError: division by zero`	除数为 0	`if b != 0:` 或 `try/except`
29	`UnicodeDecodeError: 'gbk' codec can't decode ...`	Windows 默认 GBK 打开了 UTF-8 文件	`open(path, encoding="utf-8")`
30	`ModuleNotFoundError: No module named 'xxx'`	包没装，或自己的文件名和标准库重名	`pip install xxx`；把 `random.py`、`string.py` 改个名

B.6 最容易被忽略的一类："程序没错，但判题说 WA"

现象	原因	处方
本地跑对，OJ 判 WA	输出多了提示语（`请输入：`）	OJ 只看结果，**不要**打印任何提示
差一个空行 / 少一行	`print` 的位置放错（循环里/外）	数一数样例输出有几行
小数位数不符	用 `round` 丢掉了末尾的 0	用 `f"{x:.2f}"`
行尾多了空格	手动拼了空格	用 `print(*ans)` 或 `" ".join(...)`
只过了第一组测试数据	多组数据的循环没写对	见附录 C
大输入超时	用了 $O(n^2)$ 的暴力算法	想优化：前缀和、试除到 sqrt(n)、提前 break

附录 C　在线判题（OJ）输入输出模板

> 课程作业几乎全部在这类平台上提交。**算法对但格式错 = 0 分**，所以这一页值得反复看。

C.1 四种常见的输入格式

类型 1：一行一个数（最常见）

题目说"输入三个整数，一行一个"：

```
a = int(input())
b = int(input())
c = int(input())
```

三行都要写三次 `input()`，**不能**写 `a, b, c = int(input())`（那会 `ValueError`）。

类型 2：一行多个数，用空格隔开

题目说"第一行有两个整数 l 和 m"：

```
L, m = map(int, input().split())        # 恰好两个
```

题目说"第二行包含 n 个正整数"：

```
a = list(map(int, input().split()))     # 个数不定，存进列表
```

题目说"一行里若干个整数，个数未知"：

```
a = list(map(int, input().split()))
n = len(a)                              # 个数就是列表长度
```

为什么用 `split()` 而不是 `split(" ")`：`split()` 会按**任意长度的空白**切分并丢弃空串。如果数据里有连续空格或行尾空格，`split(" ")` 会切出空字符串，然后 `int("")` 报错。

类型 3：先给个数，再给数据

```
n = int(input())                        # 第一个数是数据个数
nums = [int(input()) for _ in range(n)] # 后面 n 行，每行一个
```

或者数据在同一行：

```
n = int(input())
nums = list(map(int, input().split()))
```

类型 4：多组测试数据

```
t = int(input())                # 第一行是组数
for _ in range(t):
    n = int(input())
    a = list(map(int, input().split()))
    # 在这里处理这一组，并且 print 结果
    print("Yes" if ok else "No")
```

关键点：`print` 必须在 `for` 循环**里面**，因为每组都要输出一行；每组的累加变量必须在循环**里面**重新初始化，否则会串到下一组。

类型 5：读到文件结束（EOF），组数未知

```
import sys

for line in sys.stdin:
    if not line.strip():        # 跳过空行
        continue
    a, b = map(int, line.split())
    print(a + b)
```

等价的 `input()` 写法：

```
while True:
    try:
        a, b = map(int, input().split())
    except EOFError:
        break
    print(a + b)
```

C.2 转成"整数列表"的四种写法

写法	说明
`list(map(int, input().split()))`	**推荐**，最快最直观
`[int(x) for x in input().split()]`	等价，可读性也好
`list(map(float, ...))`	读实数
`a, b = map(int, input().split())`	已知恰好两个时最方便

C.3 输出格式的四种写法

```
print(ans)                              # 一个值
print(x)
print(y)                                # 两个值，分两行
print(a, b)                             # 一行两个值，中间空格
print(" ".join(map(str, ans)))          # 一行多个值，行尾无空格（推荐）
```

```
print(*ans)                    # 同上，简写
print(f"{x:.2f}")              # 保留两位小数
```

> **【易错】** `print(a, b)` 会在两个值之间自动加**一个空格**；如果题目样例里两个数之间没有空格，就要用 `print(f"{a}{b}")` 或者 f-string 拼起来。

C.4 提交前的自检清单

1. **输入**：把样例的输入**原样**贴进去，程序能不能读完、不报错？
2. **输出**：和样例逐字符对比，行数、顺序、空格、小数位数是否完全一样？（在记事本里比最保险）
3. **提示语**：有没有多余的 `input("请输入：")` 或 `print("结果是：")`？——删掉，OJ 只认结果。
4. **边界**：最小值、最大值、只有一个元素、全部相同、负数，这几个情况试过没有？
5. **多组数据**：每组的变量都在循环里重新初始化了吗？`print` 在循环里吗？
6. **时间**：数据范围是 10^4 还是 10^9？如果是后者，$O(n^2)$ 一定会超时。

C.5 常见坑一览

坑	说明
`input()` 永远是字符串	`'10' < '9'` 是 True，因为按字典序比较
`split()` ≠ `split(" ")`	后者遇到连续空格会切出空串
`range(a, b)` 不含 b	要包含 b 就写 `range(a, b + 1)`
下标从 0 开始	第 1 个数是 `a[0]`
输出行尾多空格	严格判题会判 PE（格式错误）
`round` 会丢末尾的 0	保留小数位数一律用 `f"{x:.2f}"`
浮点数不能 `==`	用 `abs(a - b) < 1e-6`
海量输出用 `print` 太慢	攒进列表，最后 `sys.stdout.write("\n".join(...))`